Privacy-Preserving Machine Learning for Practitioners

Manuel V. Loureiro

Version 2026-10-27

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **2**
What each chapter is for . . . 3
Notation . . . 3
How to use the chapters . . . 3

1. Probability distributions **4**
1.1. What it is . . . 4
1.2. When you meet it . . . 4
1.3. How to use it . . . 4
1.4. What goes wrong . . . 6

2. Concentration inequalities **7**
2.1. What it is . . . 7
2.2. When you meet it . . . 7
2.3. How to use it . . . 7
2.4. What goes wrong . . . 9

3. Divergences **10**
3.1. What it is . . . 10
3.2. When you meet it . . . 10
3.3. How to use it . . . 11
3.4. What goes wrong . . . 12

4. Hypothesis testing **13**
4.1. Why this matters . . . 13
4.2. Tests and their two errors . . . 13
4.3. The Neyman–Pearson lemma . . . 13
4.4. Trade-off curves . . . 14
4.5. The Gaussian trade-off curve . . . 14
4.6. The Laplace trade-off curve . . . 14
4.7. Best advantage and total variation . . . 14
4.8. ROC summaries: why AUC hides the low-FPR regime . . . 15
4.9. The Bayesian view . . . 15
4.10. Measuring rates from finite trials . . . 16
4.11. Evaluating a detector at a low false positive rate . . . 16
4.12. Cheat sheet . . . 17
4.13. References . . . 18

5. Information theory **19**
5.1. What it is . . . 19
5.2. When you meet it . . . 19
5.3. How to use it . . . 19
5.4. What goes wrong . . . 20
5.5. References . . . 21

6. Optimization **22**
6.1. What it is . . . 22
6.2. When you meet it . . . 22
6.3. How to use it . . . 22
6.4. What goes wrong . . . 23

7. Algorithmic stability and generalization **25**
7.1. What it is . . . 25
7.2. When you meet it . . . 25
7.3. How to use it . . . 25
7.4. What goes wrong . . . 26

8. Cryptography basics **27**
8.1. What it is . . . 27
8.2. When you meet it . . . 27
8.3. How to use it . . . 28
8.4. What goes wrong . . . 29

II. Threat models and attacks **30**

9. Threat models and privacy games **32**
9.1. Why this matters . . . 32
9.2. Writing down the threat model . . . 32
9.3. Scoring an attack against its baseline . . . 33
9.4. Reporting: TPR at a low FPR . . . 34
9.5. Average case and worst case . . . 35
9.6. Cheat sheet . . . 36
9.7. References . . . 36

10. Membership inference **37**
10.1. Why this matters . . . 37
10.2. Membership as a test about one record . . . 37
10.3. The quick check: the generalization gap . . . 37
10.4. LiRA: a likelihood-ratio test per record . . . 38
10.5. Running LiRA on your model . . . 39
10.6. Reading the result . . . 40
10.7. Who is exposed, and what helps . . . 40
10.8. Cheat sheet . . . 41
10.9. References . . . 41

11. Attribute inference and model inversion **42**
11.1. What it is . . . 42
11.2. When you meet it . . . 42
11.3. How to use it . . . 42
11.4. What goes wrong . . . 43
11.5. References . . . 44

12. Property inference **45**
12.1. What it is . . . 45
12.2. When you meet it . . . 45
12.3. How to use it . . . 45
12.4. What goes wrong . . . 46
12.5. References . . . 46

13. Memorization and training-data extraction **47**
13.1. Why this matters . . . 47
13.2. Which memorization to measure . . . 47
13.3. What gets memorized, and the first defences . . . 48
13.4. Auditing with canaries and exposure . . . 48
13.5. Extraction without the prefix: generate, rank, verify . . . 50
13.6. Cheat sheet . . . 50
13.7. References . . . 51

14. Gradient inversion **52**
14.1. Why this matters . . . 52
14.2. The attacker and what it sees . . . 52
14.3. Exact recovery from a fully connected layer . . . 52
14.4. Label recovery from the last layer . . . 53
14.5. Deep Leakage from Gradients . . . 54
14.6. Batches and averaging . . . 54
14.7. What to do as a deployer . . . 55
14.8. Cheat sheet . . . 56
14.9. References . . . 56

15. Linkage and re-identification **57**
15.1. Why this matters . . . 57
15.2. Quasi-identifiers and the linkage attack . . . 57
15.3. Pseudonyms and hashed identifiers . . . 58
15.4. How many people are unique? . . . 59
15.5. Sparse data: the Netflix attack . . . 59
15.6. Does releasing a sample protect? . . . 60
15.7. A pre-release risk check . . . 60
15.8. Cheat sheet . . . 60
15.9. References . . . 61

16. Syntactic anonymity and how it breaks **62**
16.1. Why this matters . . . 62
16.2. What the adversary believes about a class . . . 62

16.3. k-anonymity and what it costs . . . 62
16.4. Homogeneity, background knowledge and l-diversity . . . 63
16.5. Measuring what a release teaches . . . 64
16.6. Skewness, similarity and t-closeness . . . 64
16.7. The composition attack . . . 65
16.8. Choosing and running an anonymization . . . 65
16.9. Cheat sheet . . . 65
16.10. References . . . 66

17. Reconstruction attacks **67**
17.1. What it is . . . 67
17.2. When you meet it . . . 67
17.3. How to use it . . . 68
17.4. What goes wrong . . . 68
17.5. References . . . 69

18. Pre-release privacy risk assessment **70**
18.1. Why this matters . . . 70
18.2. Fix the threat model and the thresholds first . . . 70
18.3. Classifier: the free check . . . 72
18.4. Classifier: LiRA with shadow models . . . 72
18.5. Fine-tuned language model: canaries and extraction . . . 74
18.6. Table: re-identification on quasi-identifiers . . . 75
18.7. Write the decision . . . 76
18.8. Checklist . . . 76
18.9. References . . . 77

III. Differential privacy foundations **78**

19. The definition of differential privacy **80**
19.1. Why this matters . . . 80
19.2. Neighbouring datasets . . . 80
19.3. The definition . . . 80
19.4. The privacy loss and δ . . . 81
19.5. Randomized response . . . 81
19.6. What ε means for an attacker . . . 82
19.7. How small δ must be . . . 83
19.8. What differential privacy does not protect . . . 83
19.9. Writing the guarantee down . . . 84
19.10. Code . . . 84
19.11. Cheat sheet . . . 84
19.12. References . . . 85

20. Post-processing, group privacy and composition **86**
20.1. Why this matters . . . 86
20.2. Post-processing . . . 86
20.3. Group privacy . . . 86

20.4. Sequential composition . . . 87
20.5. Parallel composition . . . 88
20.6. Keeping a budget . . . 88
20.7. Cheat sheet . . . 90
20.8. References . . . 90

21. The Laplace and Gaussian mechanisms **91**
21.1. Why this matters . . . 91
21.2. Sensitivity . . . 91
21.3. The Laplace mechanism . . . 92
21.4. The Gaussian mechanism . . . 93
21.5. Laplace or Gaussian? . . . 93
21.6. Calibrating a release . . . 94
21.7. Code . . . 94
21.8. Cheat sheet . . . 95
21.9. References . . . 95

22. Advanced and optimal composition **97**
22.1. What it is . . . 97
22.2. When you meet it . . . 97
22.3. How to use it . . . 98
22.4. What goes wrong . . . 99

23. Selection: exponential mechanism, report-noisy-max, sparse vector **100**
23.1. Why this matters . . . 100
23.2. Which mechanism to use . . . 100
23.3. The exponential mechanism . . . 100
23.4. Accuracy, and the cost of many candidates . . . 101
23.5. A private quantile or clipping norm . . . 101
23.6. Report-noisy-max and Gumbel noise . . . 103
23.7. The sparse vector technique . . . 103
23.8. When the sparse vector technique goes wrong . . . 104
23.9. Cheat sheet . . . 104
23.10. References . . . 105

24. Query release and private multiplicative weights **106**
24.1. Why this matters . . . 106
24.2. Which method to use . . . 106
24.3. Noise on every cell . . . 107
24.4. Releasing a marginal with OpenDP . . . 107
24.5. Synthetic data . . . 108
24.6. Private multiplicative weights . . . 109
24.7. Cheat sheet . . . 110
24.8. References . . . 110

25. Beyond global sensitivity **111**
25.1. What it is . . . 111
25.2. When you meet it . . . 111

25.3. How to use it . . . 112
25.4. What goes wrong . . . 112

26. Discrete and floating-point-safe mechanisms **113**
26.1. What it is . . . 113
26.2. When you meet it . . . 113
26.3. How to use it . . . 114
26.4. What goes wrong . . . 114

27. Local, shuffle and distributed trust models **116**
27.1. Why this matters . . . 116
27.2. Choosing a trust model . . . 116
27.3. Central and local differential privacy . . . 116
27.4. Frequency estimation: k-ary randomized response and unary encoding . . . 118
27.5. Numbers in one bit: local mean estimation . . . 118
27.6. Collecting every day: memoization . . . 118
27.7. The shuffle model and amplification by shuffling . . . 119
27.8. Distributed DP: splitting the noise . . . 119
27.9. Cheat sheet . . . 120
27.10. References . . . 120

28. Lower bounds: what privacy costs **121**
28.1. What it is . . . 121
28.2. When you meet it . . . 121
28.3. How to use it . . . 121
28.4. What goes wrong . . . 122

29. Private mean estimation **123**
29.1. Why this matters . . . 123
29.2. Bounded data: privacy adds a lower-order term . . . 123
29.3. The cost of an unknown range . . . 124
29.4. Many dimensions: clip the norm . . . 125
29.5. Heavy tails: clip into the data . . . 125
29.6. Code . . . 126
29.7. Cheat sheet . . . 127
29.8. References . . . 127

30. Release statistics with a library **129**
30.1. Why this matters . . . 129
30.2. Choose the library . . . 129
30.3. Decide the unit and the budget . . . 130
30.4. Bound each customer's contribution . . . 131
30.5. Set up the context . . . 131
30.6. Release overall statistics with error bars . . . 132
30.7. Group by a public key list . . . 133
30.8. Group by unknown keys with partition selection . . . 134
30.9. Check the release before publishing . . . 135
30.10. Write the guarantee down . . . 136

30.11. Checklist . . . 136
30.12. References . . . 137

IV. Privacy accounting **138**

31. Choosing an accountant **140**
31.1. Why this matters . . . 140
31.2. Which accountant . . . 140
31.3. The inverse problem: noise for a target ε . . . 141
31.4. Choosing δ . . . 142
31.5. The statement . . . 142
31.6. Matching the accountant to the code . . . 143
31.7. Reviewing someone else's claim . . . 144
31.8. Checking an accountant . . . 144
31.9. Cheat sheet . . . 145
31.10. References . . . 145

32. Calibrate and account a DP-SGD run **147**
32.1. Why this matters . . . 147
32.2. Fix the privacy unit, n and δ . . . 147
32.3. Choose the sampler . . . 148
32.4. Plan the noise . . . 148
32.5. Check ε with a second accountant . . . 150
32.6. If the loader is not Poisson . . . 151
32.7. Train, stop early and resume . . . 151
32.8. Write the statement . . . 152
32.9. Checklist . . . 153
32.10. References . . . 154

33. Amplification by subsampling **155**
33.1. Why this matters . . . 155
33.2. One step under Poisson sampling . . . 155
33.3. Match the sampler to the accountant . . . 156
33.4. Accounting each sampler . . . 157
33.5. Batch size, epochs and noise . . . 158
33.6. Cheat sheet . . . 158

34. Rényi differential privacy **160**
34.1. Why this matters . . . 160
34.2. The RDP curve . . . 160
34.3. RDP curves of the basic mechanisms . . . 160
34.4. Composition adds curves . . . 161
34.5. Post-processing and group privacy . . . 162
34.6. Compose, then convert once . . . 162
34.7. Check the order grid . . . 163
34.8. Accounting with a library . . . 163
34.9. Cheat sheet . . . 164

34.10. References 164

35. From Rényi DP to (ε, δ) and to tests **165**
35.1. What it is 165
35.2. When you meet it 165
35.3. How to use it 166
35.4. What goes wrong 166
35.5. References 166

36. Numerical accounting with privacy loss distributions **168**
36.1. What it is 168
36.2. When you meet it 168
36.3. How to use it 169
36.4. What goes wrong 169
36.5. References 170

37. f-DP and Gaussian differential privacy **171**
37.1. What it is 171
37.2. When you meet it 171
37.3. How to use it 172
37.4. What goes wrong 172
37.5. References 173

38. Adaptive composition: filters, odometers and individual accounting **174**
38.1. What it is 174
38.2. When you meet it 174
38.3. How to use it 174
38.4. What goes wrong 175
38.5. References 176

39. Concentrated differential privacy **177**
39.1. What it is 177
39.2. When you meet it 177
39.3. How to use it 177
39.4. What goes wrong 178
39.5. References 179

40. Amplification by shuffling and by iteration **180**
40.1. What it is 180
40.2. When you meet it 180
40.3. How to use it 180
40.4. What goes wrong 181
40.5. References 182

V. Differential privacy in machine learning **183**

41. Private empirical risk minimization **185**
41.1. What it is 185

41.2. When you meet it . . . 185
41.3. How to use it . . . 186
41.4. What goes wrong . . . 187

42. DP-SGD **188**
42.1. Why this matters . . . 188
42.2. The algorithm and its guarantee . . . 188
42.3. The implementation checklist . . . 189
42.4. Clipping and the learning rate . . . 190
42.5. Batch size at a fixed budget . . . 190
42.6. Making it fast . . . 190
42.7. What privacy costs in accuracy . . . 191
42.8. Cheat sheet . . . 191
42.9. References . . . 192

43. Train a model with DP-SGD **193**
43.1. Why this matters . . . 193
43.2. Fix the privacy unit, n and δ . . . 193
43.3. Preprocess from public facts . . . 194
43.4. Train the baselines . . . 194
43.5. Check the model . . . 195
43.6. Choose the clipping norm, batch and epochs . . . 195
43.7. Calibrate the noise . . . 196
43.8. Train . . . 196
43.9. Cross-check ε . . . 197
43.10. Tune honestly . . . 197
43.11. Measure utility and groups . . . 198
43.12. When it goes wrong . . . 199
43.13. Record the run . . . 200
43.14. Checklist . . . 201
43.15. References . . . 201

44. Hyperparameter tuning and private selection **202**
44.1. Why this matters . . . 202
44.2. The procedure and the report . . . 202
44.3. Paying for every run . . . 203
44.4. A random number of runs . . . 203
44.5. Selecting by a noisy validation score . . . 204
44.6. Tuning on Adult . . . 204
44.7. Cheat sheet . . . 205
44.8. References . . . 206

45. User-level differential privacy **207**
45.1. Why this matters . . . 207
45.2. The privacy unit and δ . . . 207
45.3. What an example-level model gives a user . . . 208
45.4. User-level DP-SGD . . . 208
45.5. Contribution bounding . . . 209

45.6. Heterogeneous users . . . 210
45.7. Which strategy . . . 210
45.8. Code . . . 210
45.9. Cheat sheet . . . 211
45.10. References . . . 211

46. Choosing ε **212**
46.1. Why this matters . . . 212
46.2. Justifying a choice . . . 212
46.3. The pair and the whole curve . . . 213
46.4. What an attacker can believe . . . 214
46.5. How small δ must be . . . 215
46.6. Comparing with deployed values . . . 215
46.7. Cheat sheet . . . 216
46.8. References . . . 216

47. Public data and private fine-tuning **217**
47.1. Why this matters . . . 217
47.2. How many weights to train . . . 217
47.3. Linear probing first . . . 218
47.4. Low-rank adapters and full fine-tuning . . . 218
47.5. Which method at which budget . . . 219
47.6. What public data does not buy . . . 220
47.7. Cheat sheet . . . 220
47.8. References . . . 221

48. Fine-tune a language model privately with LoRA **222**
48.1. Why this matters . . . 222
48.2. Fix the privacy unit, n and δ . . . 222
48.3. Plant canaries . . . 223
48.4. Load a public model and tokenizer . . . 223
48.5. Choose what to train . . . 224
48.6. Make the model Opacus-compatible . . . 224
48.7. Calibrate and train . . . 224
48.8. Evaluate . . . 226
48.9. Check memorization . . . 226
48.10. Variants . . . 228
48.11. Report . . . 228
48.12. Checklist . . . 229
48.13. References . . . 229

49. Private prediction: PATE and in-context learning **231**
49.1. What it is . . . 231
49.2. When you meet it . . . 231
49.3. How to use it . . . 232
49.4. What goes wrong . . . 232
49.5. References . . . 233

50. DP-FTRL and matrix factorization **234**
50.1. What it is . . . 234
50.2. When you meet it . . . 234
50.3. How to use it . . . 235
50.4. What goes wrong . . . 235
50.5. References . . . 236

51. Disparate impact **237**
51.1. Why this matters . . . 237
51.2. Evaluating a private model per group . . . 237
51.3. Disparate impact, measured . . . 238
51.4. Why small groups pay: clipping and noise . . . 239
51.5. Mitigations and what they cost . . . 240
51.6. Cheat sheet . . . 241
51.7. References . . . 242

52. Auditing private training **243**
52.1. Why this matters . . . 243
52.2. Known bugs, and which audit catches them . . . 243
52.3. A CI test for the noise and the clipping . . . 243
52.4. From an attack to a lower bound . . . 245
52.5. A one-run audit of the trained model . . . 246
52.6. Cheat sheet . . . 247
52.7. References . . . 247

53. Adaptive data analysis **248**
53.1. What it is . . . 248
53.2. When you meet it . . . 248
53.3. How to use it . . . 249
53.4. What goes wrong . . . 249
53.5. References . . . 249

54. Release and governance **251**
54.1. Why this matters . . . 251
54.2. Assemble the run record . . . 251
54.3. Recompute the guarantee . . . 252
54.4. Attach the evidence . . . 253
54.5. Write the statement . . . 254
54.6. Map the claim to regulation . . . 255
54.7. Review someone else's claim . . . 256
54.8. After release . . . 257
54.9. Checklist . . . 258
54.10. References . . . 259

Appendices **260**

A. Glossary **260**

B. Index **263**

Preface

This is the practitioner edition of *Privacy-Preserving Machine Learning*. It is for engineers, data scientists and auditors who build or review machine-learning systems that handle personal data, and it tells them what each result says, when it applies, how to use it and what goes wrong, without proofs.

It keeps the parts and the chapter files of the full edition, so that every page here has a twin there that derives its results step by step, but it is organized around the work rather than the theory. Three kinds of page do that. A **workflow chapter** does one job end to end with real libraries on a real public dataset: assessing a model's privacy risk before release, releasing statistics with OpenDP and diffprivlib, calibrating and reporting a run of differentially private stochastic gradient descent, training and fine-tuning with Opacus, and governing a release. Each part reads in the order of the work, so a workflow comes early in its part and the pages it relies on follow. A **full chapter** covers a topic a practitioner computes with: the result and its formula, why it matters, a recipe with a worked example, what goes wrong, and a cheat sheet. A **reference page** covers a theory-heavy topic a practitioner meets but rarely computes with, in a few hundred words: what it is, when you meet it, how to use it and what goes wrong. Every page ends with a link to the full edition's chapter.

You need some Python and NumPy, basic probability, and enough machine learning to know what a loss and stochastic gradient descent are.

How this book is organized

1. **Prerequisites** is a mathematical reference to dip into when a later chapter cites it.
2. **Threat models and attacks** shows how models and data leak, and assesses a model's risk before release.
3. **Differential privacy foundations** defines the guarantee, builds its mechanisms, and releases statistics with them.
4. **Privacy accounting** tracks the guarantee through private training, and calibrates and reports a run.
5. **Differential privacy in machine learning** trains, fine-tunes, audits and releases private models.

You do not need to read Part 1 first. With a background in statistics or machine learning, start at Part 2 or Part 3 and follow the links back when you need a fact.

Every piece of code on a page has been run: a snippet runs on its own, and an excerpt is a marked region of a workflow script in the repository's `code/` directory, which a test runs and checks against the numbers the page quotes. To rerun a workflow, run `uv run python code/<part>/practitioner_<topic>.py`.

Part I.

Prerequisites

This part is a mathematical toolkit with no privacy in it yet. You do not need to read it first, because later chapters link back to each fact they use.

What each chapter is for

Two ideas run through the part. The first is the **likelihood ratio**, how many times more likely an observation is under one explanation than under another, and divergences and tests are built on it. The second is the **effect of one input**, how far a single record can move a result, which concentration and stability bound.

Hypothesis testing is the one full chapter, because the rest of the book leans on it most. It shows how to read a detector's trade-off curve at a low false positive rate, and how to put an honest interval on a measured rate.

The other seven are short reference pages. Probability distributions and Concentration inequalities give the noise families and the error bars and sample sizes for a measured number. Divergences compares two distributions with one number, and Information theory bounds what an output reveals about a hidden value. Optimization gives the learning rates of gradient descent, and Algorithmic stability and generalization explains why a reused holdout set misleads. Cryptography basics shows how to read a security claim stated as a game.

Notation

Table 1.: Notation used throughout the book.

Symbol	Meaning
Pr, $\mathbb{E}$, Var	Probability, expectation, variance
$\mathcal{N}(\mu, \sigma^2)$, Lap($b$)	Normal and Laplace distributions
ln, $\log_2$	Natural logarithm (nats) and base-2 logarithm (bits)
$\|x\|$	Euclidean length of a vector x
D, θ, $\ell(\theta; z)$	A dataset, the parameters of a model, and its loss on one record z

How to use the chapters

Each page states its results, works one example with real numbers, and links to its derivations in the full edition. Every number comes from a snippet on the page or from the full edition's seeded script for the chapter, such as `uv run python code/part01-prerequisites/optimization.py`, so reruns give the same numbers.

1. Probability distributions

1.1. What it is

Most random numbers in a machine-learning system are counts or draws of noise. A count of independent yes-or-no outcomes, each with the same probability, is **binomial**. When the outcomes are many and each is rare, the count becomes **Poisson**, whose variance equals its mean. Noise usually comes from one of two continuous families. The **Laplace distribution** with scale b is an exponential density mirrored around zero, and the normal distribution is the bell curve, set by its mean and its standard deviation σ.

1.2. When you meet it

You meet these families when you size a test set, check which argument of a sampler sets its spread, or put an error bar on a noisy number.

1.3. How to use it

An error rate p measured on a test set of n records is a binomial count of errors divided by n, so its standard deviation is

$$\sqrt{\frac{p(1-p)}{n}}. \tag{1.1}$$

With 2,000 records and a true error rate of 5%, this is about half a percentage point. Divided by p, it becomes the relative standard deviation $\sqrt{(1-p)/(np)}$, which shrinks only as the expected number of errors np grows. A relative target r thus needs $n = (1-p)/(p\,r^2)$ records. At $r = 10\%$, a 5% error rate needs 1,900 records, and a rate fifty times smaller needs 99,900.

A sampled batch is the Poisson case. If each of 60,000 records joins it with probability 0.01, its size is nearly Poisson with mean 600. So its variance is also 600, and its standard deviation about 24.5.

For noise, you pick the spread so that a draw stays within t of zero except with a failure probability β. A Laplace draw with scale b lands at least t from zero with probability

$$\Pr[|X| \geq t] = e^{-t/b}. \tag{1.2}$$

Setting this tail, or the normal one, equal to β gives the largest spreads that meet the target,

$$b = \frac{t}{\ln(1/\beta)}, \qquad \sigma = \frac{t}{z_{1-\beta/2}}. \tag{1.3}$$

Here $z_{1-\beta/2}$ is the standard normal quantile. Both formulas split β between the two sides, so to bound one side only, replace β by 2β. For example, Laplace noise of scale 2 stays within $2 \ln 20 = 5.99$ of zero 95% of the time. As β shrinks, the Laplace multiplier grows much faster than the normal one (Table 1.1).

Table 1.1.: How many standard deviations or scales a draw stays within, except with probability β.

Failure probability β	0.05	10^{-3}	10^{-6}
Normal, $z_{1-\beta/2}$ standard deviations	1.96	3.29	4.89
Laplace, $\ln(1/\beta)$ scales	3.00	6.91	13.82

The log scale of Figure 1.1 shows why: the Laplace tails fall along straight lines, far above the normal density's parabola.

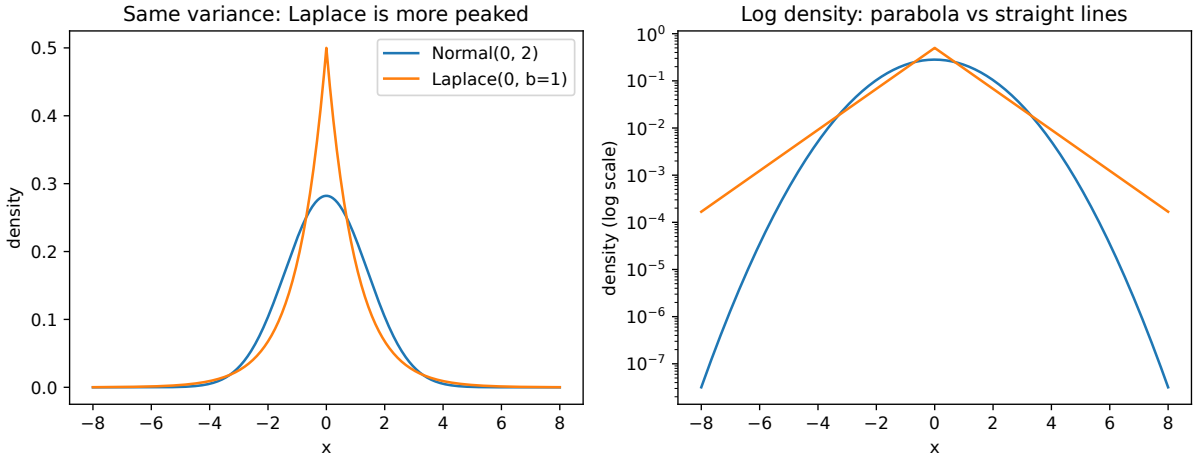


Figure 1.1.: Normal and Laplace densities with variance 2 (Laplace scale 1), on linear and log scales.

Noise on a vector adds up over its coordinates. With d independent normal coordinates of standard deviation σ, the length is close to $\sigma\sqrt{d}$. A million coordinates at $\sigma = 0.01$ give a length of about 10. The snippet checks these numbers and the integer case below.

```
import math

import numpy as np
from scipy import optimize, stats

rng = np.random.default_rng(0)
draws = rng.laplace(loc=0.0, scale=2.0, size=1_000_000)  # scale is b, not the standard devi
print(f"{draws.std():.3f} {stats.laplace(scale=2.0).std():.4f}")  # 2.832 2.8284
print(f"{stats.laplace(scale=2.0).isf(0.025):.2f}")  # 5.99
print(f"{stats.norm.isf(0.05 / 2):.2f} {stats.norm.isf(1e-3 / 2):.2f} {stats.norm.isf(1e-6 /
print(f"{math.log(1 / 0.05):.2f} {math.log(1 / 1e-3):.2f} {math.log(1 / 1e-6):.2f}")  # 3.00
```

```
print(math.ceil((1 - 0.05) / (0.05 * 0.1**2)), math.ceil((1 - 0.001) / (0.001 * 0.1**2)))  #
b = 10 / math.log(1 / 0.01)  # t = 10, beta = 0.01
tail = lambda a: 2 * stats.dlaplace(a).sf(9)  # integer version: P[|K| >= 10]
a = optimize.brentq(lambda a: tail(a) - 0.01, 0.01, 5)
print(f"{b:.4f} {tail(1 / b):.4f} {a:.4f} {tail(a):.4f}")  # 2.1715 0.0123 0.4817 0.0100
```

1.4. What goes wrong

The commonest bug is a mismatched spread. NumPy's `rng.laplace(0, b)` and SciPy's `stats.laplace(scale=b)` take the scale b, whose standard deviation is $\sqrt{2}\,b$. But `rng.normal(0, sigma)` and `stats.norm(scale=sigma)` take the standard deviation, never the variance. So compare the standard deviation of many draws with the intended one. For tiny tail probabilities, call the tail function `sf` or its inverse `isf`, because `1 - cdf` rounds to zero.

For integer data, draw from an integer distribution rather than rounding. In SciPy's `stats.dlaplace(a)`, the probability of an integer falls by a factor e^{-a} per step away from zero, so a plays the part of $1/b$. Its tail is heavier, so $a = 1/b$ gives 0.0123 where the snippet targets 0.01. Solve for a numerically instead.

These formulas assume independent records, so when a user has several rows, take the test-set size n to be the number of users. The normal approximation to a binomial count needs $np(1-p)$ of about 10 or more, so it fails for rare events.

Derivations. Every result here is derived step by step in the full edition's chapter.

2. Concentration inequalities

2.1. What it is

A concentration inequality bounds the chance that a random quantity lands more than some distance from its mean. Call that event a failure, its chance the **failure probability** β, and the distance the **width**. Fix β and solve for the width to get an error bar, or fix the width and solve for the number of samples.

Markov's inequality needs only the mean of a quantity that cannot be negative, and caps the chance of reaching a threshold at the mean divided by the threshold. **Chebyshev's inequality** adds the variance σ^2. With μ for the mean, the chance of landing at least a distance t from it is at most

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.1}$$

Setting this to β gives the width $\sigma/\sqrt{\beta}$ for any distribution, but at a steep price. At one in a million it asks for 1,000 standard deviations, where a normal-shaped tail needs 5.3.

2.2. When you meet it

You meet these bounds when you put an error bar on a mean or a rate, plan a sample size, or report many numbers at once.

2.3. How to use it

Pick the bound by what you know about the quantity (Table 2.1).

Table 2.1.: Which bound to use, and the two-sided width it gives.

What you know	Use	Width at failure probability β
The variance σ^2	Chebyshev	$\sigma/\sqrt{\beta}$
A mean, with standard deviation s estimated from the data	Normal approximation	$z_{1-\beta/2}\, s/\sqrt{n}$, smaller but not a guarantee; solve for n to plan
The exact noise distribution (Probability distributions)	Its quantile	Laplace $b \ln(1/\beta)$; normal $\sigma z_{1-\beta/2}$

What you know	Use	Width at failure probability β
A count with mean μ	Chernoff	a miss by a fraction r has chance $\le e^{-r^2\mu/(2+r)}$ above, $e^{-r^2\mu/2}$ below
The mean of n values in a range of length R	Hoeffding	$R\sqrt{\ln(2/\beta)/(2n)}$
A rate: k errors out of n	Clopper–Pearson	exact binomial bound

Hoeffding's inequality is the planning tool, because it needs only the range of the values. Solving its width for n gives the sample size for a width t:

$$n \ge \frac{R^2 \ln(2/\beta)}{2t^2}. \tag{2.2}$$

Accuracy averages zeros and ones, so its range R is 1, and measuring it to within one percentage point at 95% takes 18,445 test records.

For a small rate, Hoeffding is too wide, because it assumes the largest variance the range allows. For one false positive in 1,000 negatives, Hoeffding's one-sided 95% bound adds 0.039 to the measured rate. **Bernstein's inequality** uses the variance and adds only 0.0045, but a variance estimated from the same data makes it a heuristic. The exact **Clopper–Pearson** bound, from the binomial distribution, gives 0.0047, and 0.0030 with no errors.

For noise of a known distribution, use its quantile: Laplace noise of scale 2 lies within 5.99 of zero 95% of the time. For many draws at once, the **union bound** caps the chance of any failure at the sum of the chances, so give each of k draws β/k. The Laplace width grows only with $\ln(k/\beta)$, so a thousand draws widen 5.99 only to 19.81. The snippet prints these numbers, with the one-sided bound for k errors in n as `stats.beta.ppf(0.95, k + 1, n - k)`, and a two-sided interval for 10 errors in 10,000.

```
import math

from scipy import stats

print(math.ceil(math.log(2 / 0.05) / (2 * 0.01**2)))  # 18445
print(f"{stats.beta.ppf(0.95, 1, 1000):.4f} {stats.beta.ppf(0.95, 2, 999):.4f}")  # 0.0030 0
ci = stats.binomtest(10, 10_000).proportion_ci(confidence_level=0.95, method="exact")
print(f"{ci.low:.5f} {ci.high:.5f}")  # 0.00048 0.00184
print(f"{stats.laplace(scale=2).isf(0.05 / 2):.2f} {stats.laplace(scale=2).isf(0.05 / 1000 /
```

By the table's two Chernoff tails, a sampled batch whose size averages 600 strays more than a fifth from it with probability at most 2.4×10^{-5}.

McDiarmid's inequality extends Hoeffding's from means to any function of independent inputs that no single input moves much. If changing the i-th input moves the function by at most c_i, the chance of landing t above its mean, or t below, is at most

$$\exp\Big(-\frac{2t^2}{\sum_i c_i^2}\Big). \tag{2.3}$$

Azuma's inequality gives the same tail for a running total of steps that average zero given the past.

2.4. What goes wrong

Hoeffding needs bounded values, so clip unbounded data and use the clipping range as R. The bounds for sums, means and rates also need independent values, so with several rows per person, count people. And a two-sided error bar must put $\beta/2$ on each side, not β.

The normal approximation is not a tail bound. For a sum of ten Laplace draws of scale 1, it understates the chance of a total above 20 about ninefold. And picking the best of m models on one validation set needs β/m for each, because the winner is the one most likely to look too good.

Derivations. Every result here except Bernstein's inequality is derived step by step in the full edition's chapter.

3. Divergences

3.1. What it is

A **divergence** puts a number on how different two distributions are. It is zero for equal distributions and grows as they become easier to tell apart. For distributions P and Q, write $p(x)$ and $q(x)$ for the probabilities they give an outcome x. Every divergence summarizes the **likelihood ratio** $p(x)/q(x)$.

The **Kullback–Leibler (KL) divergence** is the average log ratio when the data come from P:

$$D_{\mathrm{KL}}(P\|Q) = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.1}$$

It is not symmetric, and it adds up over independent observations.

The **Rényi divergence** is a dial set by an order α above 1:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha}. \tag{3.2}$$

It equals the KL divergence as the order approaches 1. Higher orders weigh the outcomes with large ratios more, up to the **max-divergence** D_∞, the largest log ratio of any outcome.

The **total variation (TV)** is the mass of P that does not overlap with Q:

$$\mathrm{TV}(P, Q) = \frac{1}{2} \sum_x |p(x) - q(x)|. \tag{3.3}$$

The **hockey-stick divergence** scales q up by a factor γ of at least 1 and measures the mass of p that still sticks out above it. Writing $(y)_+$ for the positive part of y, it is

$$E_\gamma(P\|Q) = \sum_x \big(p(x) - \gamma q(x)\big)_+. \tag{3.4}$$

At $\gamma = 1$ it is the total variation, and it reaches zero once γ exceeds the largest likelihood ratio.

3.2. When you meet it

You meet divergences when a feature drifts between training and serving, or when a test-data shift may move a classifier's accuracy. Papers state results in one divergence that you may need in another.

3.3. How to use it

To measure drift, bin both samples on the same edges and call `scipy.stats.entropy(p, q)`. It normalizes both histograms and returns natural-log units (nats), or bits with `base=2`. Take a feature whose last bin, empty last month, holds a tenth of this month's data. With last month as P, the divergence is 0.1116 nats, but the reverse is infinite, because this month has mass where last month had none.

```
import math

from scipy import stats

before, after = [0.5, 0.3, 0.2, 0.0], [0.4, 0.3, 0.2, 0.1]
print(f"{stats.entropy(before, after):.4f} {stats.entropy(before, after, base=2):.4f}")  # 0
print(stats.entropy(after, before))  # inf
print(math.ceil(2 * 0.9**2 / 0.005), math.ceil((2 * stats.norm.ppf(0.95) / 0.1) ** 2))  # 32
```

For a copy shifted by s, closed forms replace histograms. For normal noise, write σ for the standard deviation and $\mu = s/\sigma$ for the shift measured in it. Then

$$D_{\mathrm{KL}} = \frac{\mu^2}{2}, \qquad D_\alpha = \frac{\alpha\mu^2}{2}, \qquad D_\infty = \infty, \tag{3.5}$$

the last because the ratio of normal densities is unbounded. The shift enters squared, so a drift of 0.1 standard deviations costs a KL divergence of only $0.1^2/2 = 0.005$. Laplace noise of scale b is the opposite case, because its log ratio is bounded: its max-divergence is the shift in scales, s/b.

The TV says how well any test can tell P from Q. If the two are equally likely in advance, the best test from one observation has an **advantage**, its true-positive rate minus its false-positive rate, equal to the TV. So it is right with probability $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$. For two normal curves one standard deviation apart, the TV is 0.3829, so no classifier is right more than 0.6915 of the time. Likewise, when the test distribution moves, a fixed classifier's accuracy moves by at most the TV. For the same normal pair, the hockey-stick divergence is

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big), \tag{3.6}$$

where Φ is the standard normal distribution function. At $\mu = 1$ and $\gamma = 2$ it is 0.1906.

Pinsker's inequality converts a KL divergence into a bound on the TV:

$$\mathrm{TV}(P, Q) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(P\|Q)}. \tag{3.7}$$

It is quick but loose. For the drift of 0.1 standard deviations, a test right 95% of the time needs a TV of 0.9. KL adds up over samples, so a sample of size n has a KL divergence of $0.005n$, and Pinsker asks for at least 324 samples. The exact answer is 1,083 (the snippet's last call).

3.4. What goes wrong

The KL divergence has a direction, so say which one you computed, and never mix nats with bits. An empty bin makes it infinite, so add a pseudocount, such as half a count per bin, or merge bins, and say which you did. Estimates also change with the bins and the sample size, so compare only like with like.

The **data processing inequality** says that binning, thresholding or any other processing of an observation alone cannot make two distributions easier to tell apart. It can make them harder. For normals one standard deviation apart, keeping only which side of the midpoint an observation falls on cuts the KL divergence from 0.5 to 0.3090. It keeps all of the TV, though, because that threshold is already the best test.

A small KL divergence is an average, so it can hide rare outputs that are overwhelming evidence. When every output matters, bound the worst case with the max-divergence, a high Rényi order or the hockey-stick divergence.

Derivations. Every result here is derived step by step in the full edition's chapter.

4. Hypothesis testing

4.1. Why this matters

A fraud detector must call a transaction normal or fraudulent. The observation came from one of two distributions, and it must decide which. So every detector and every binary classifier is a hypothesis test, and evaluating one means measuring its two kinds of error, false alarms and missed cases. What matters throughout is how a detector does when false alarms must be rare, which accuracy hides.

4.2. Tests and their two errors

The default assumption, called the **null hypothesis** H_0, is that the observation comes from a distribution P, such as a fraud model's scores for normal transactions. The **alternative hypothesis** H_1 is that it comes from a distribution Q, such as its scores for fraudulent ones. A **test** decides whether to reject the null, which here means flagging the transaction.

A test can make two kinds of mistake. Flagging a normal case is a false positive, and its probability is the **false positive rate (FPR)**, written α. Missing a real case is a false negative, with probability β, the **false negative rate (FNR)**. The share of real cases caught, $1 - \beta$, is the **true positive rate (TPR)**. Lowering a test's threshold raises the TPR but also the FPR. So you fix a budget for the FPR first, called the **level**, and then look for the test with the highest TPR, or **power**, within it.

4.3. The Neyman–Pearson lemma

Write $p(x)$ and $q(x)$ for the probabilities, or densities, of an observation x under the two hypotheses. Their quotient is the **likelihood ratio**

$$\Lambda(x) = \frac{q(x)}{p(x)}, \tag{4.1}$$

and a **likelihood ratio (LR) test** rejects wherever it exceeds a threshold τ, set so that the test's FPR equals the budget. The **Neyman–Pearson lemma** says that no other test at that level catches more real cases.

So the best test looks at an observation only through its likelihood ratio. A threshold on a raw score is that test only when a higher score always means a higher ratio. To estimate the ratio from labelled scores, fit a logistic regression of the label on the score, whose log-odds, less the log prior odds, estimate $\ln \Lambda$. If a known group shifts the scores, the ratio depends on the group, so the best test thresholds within each group.

4.4. Trade-off curves

How hard P is to tell from Q is captured by the best FNR at every level, the **trade-off function** $T(P, Q)$, which you compute by sweeping the LR threshold. The **receiver operating characteristic (ROC)** curve holds the same information upside down, as TPR against FPR. Any particular detector traces its own ROC curve, which lies on or below the best one.

4.5. The Gaussian trade-off curve

A common pair is a score that is normal under both hypotheses with the same spread, scaled so that normal cases have mean 0 and standard deviation 1. Write μ for the distance between the two centres in standard deviations, and Φ for the standard normal cumulative distribution function. The best test thresholds at $\Phi^{-1}(1-\alpha)$, and its FNR at level α is the Gaussian trade-off function

$$G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{4.2}$$

In SciPy this is `norm.cdf(norm.isf(alpha) - mu)`, with `isf` because `norm.ppf(1 - alpha)` loses the digits that matter at a tiny α. So the single number μ fixes the best TPR at every FPR. At $\mu = 2$, for example, the best TPR is 0.6388 at an FPR of 5%. At an FPR of 10^{-3} it drops to 0.1378. Conversely, a measured pair gives $\mu = \Phi^{-1}(1-\alpha) + \Phi^{-1}(\text{TPR})$, `norm.isf(alpha) + norm.ppf(tpr)`. If that estimate changes with α, the scores are not of this shape.

4.6. The Laplace trade-off curve

For a Laplace distribution and a copy shifted by κ times its scale, the likelihood ratio is capped between $e^{-\kappa}$ and e^{κ} (Divergences). Every outcome a test rejects is then at most e^{κ} times likelier under Q than under P, so every test has

$$\text{TPR} \le e^{\kappa}\,\text{FPR}. \tag{4.3}$$

At a low FPR the best test reaches this cap. At $\kappa = 1$ and an FPR of 0.05, its TPR is $e \times 0.05 = 0.1359$.

The cap is what sets the Laplace pair apart from the normal one. A Laplace pair at $\kappa = 1$ and a normal pair at $\mu = 1.0301$ have the same best advantage, the largest TPR minus FPR of any test (Section 4.7). But at a level of 10^{-6} the normal pair allows 36 times the TPR, because its unbounded ratio lets a test find rare, highly telling observations. So judge a pair at the level you will use.

4.7. Best advantage and total variation

The **advantage** of a test is its TPR minus its FPR. The best advantage over all tests is the **total variation (TV)** distance of Divergences:

$$\max(\text{TPR} - \text{FPR}) = \text{TV}(P, Q) = \frac{1}{2}\sum_x |q(x) - p(x)|. \tag{4.4}$$

For the normal pair it is $2\Phi(\mu/2) - 1$, or 0.3829 at a shift of one standard deviation. Accuracy on a balanced set is one half plus half the advantage, so it measures the same single point of the curve and says nothing about its low-FPR end.

4.8. ROC summaries: why AUC hides the low-FPR regime

Detectors are often compared by the **area under the curve (AUC)** of their ROC curves, the chance that a random positive scores higher than a random negative. But the area weighs every FPR from 0 to 1 equally, so the low FPRs where a strict detector operates add almost nothing to it. Take two detectors with standard normal scores on negatives. Detector A's positives score normal with mean 1 and standard deviation 1. B's are more spread out, with standard deviation 2, and their mean of 1.5811 gives both detectors an AUC of 0.7602. Yet at an FPR of 10^{-3}, A catches 0.0183 of the positives and B about 12 times as many (Figure 4.1). A few of B's positives score far above every negative, and a strict threshold catches exactly those.

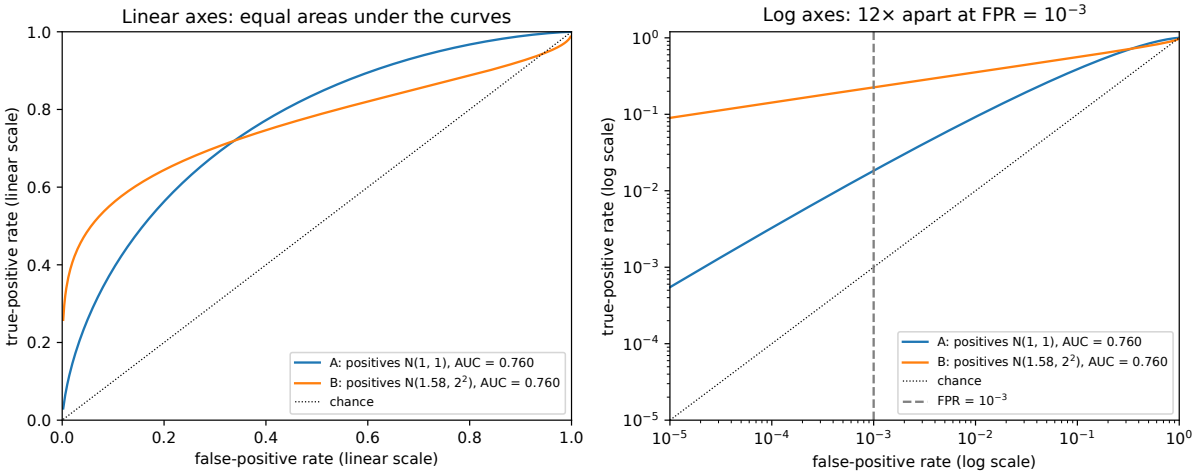


Figure 4.1.: Two detectors with the same AUC of 0.760, on linear axes (left) and log axes (right).

So plot the ROC curve on log–log axes, dropping the first point of `roc_curve`, whose FPR is 0, and report the TPR at fixed low FPRs. Pass `roc_curve(y_true, y_score)` scores, not the labels from `predict`. At a target FPR, take the highest TPR among the points at or below it, since a value interpolated with `np.interp` is one no single threshold reaches. And don't read a TPR at an FPR below one over the number of negatives, because there the curve rests on one false positive or none.

4.9. The Bayesian view

Often you know a base rate π, such as one transaction in a thousand being fraud. Bayes' rule says that the odds of H_1 after seeing x are the odds before, multiplied by the likelihood ratio:

$$\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]} = \frac{\pi}{1-\pi} \times \Lambda(x). \tag{4.5}$$

So the decision with the fewest errors, which says H_1 when these odds exceed 1, is again an LR test.

The same rule applies to an alarm, whose likelihood ratio is the TPR divided by the FPR. Take the test with $\mu = 2$ in a population where 1% are positive. At an FPR of 0.05, a flagged case is positive with probability only 0.114, so most alarms are false. At an FPR of 10^{-3} the alarm's ratio is far larger, and that probability rises to 0.582. So when positives are rare, the FPR decides whether an alarm is worth believing.

4.10. Measuring rates from finite trials

You measure a detector's FPR by counting false positives among n test negatives. If each negative is flagged independently with the same small probability, the count k is binomial and fluctuates by about $\sqrt{k}$.

The **Clopper–Pearson interval** gives the range of true rates that make the observed count unsurprising. Write $1 - \alpha$ for the confidence level, so that α is the probability that the interval misses the true rate. Its lower end l and upper end u solve

$$\Pr[\mathrm{Bin}(n, l) \geq k] = \frac{\alpha}{2}, \qquad \Pr[\mathrm{Bin}(n, u) \leq k] = \frac{\alpha}{2}. \tag{4.6}$$

It is exact, because it uses the binomial itself, and conservative, covering the true rate at least $1 - \alpha$ of the time. SciPy computes it as `binomtest(k, n).proportion_ci(method="exact")`. To claim that an FPR is below a target, you need only the upper end, which `alternative="less"` gives. With no events at all, the one-sided 95% bound is about $3/n$, the **rule of three**. Computed exactly, certifying an FPR below 10^{-3} takes 2,995 negatives with no false positive.

These intervals are wide. Even 10 false positives in 10,000 negatives give the interval $[0.00048, 0.00184]$, from about half to almost twice the observed rate. About 100 events pin a rate to within about 20%, so measuring an FPR of p takes on the order of $100/p$ negatives. The **Wald interval**, which treats the count as normal, fails here, because it is too narrow and collapses to $[0, 0]$ with no events.

4.11. Evaluating a detector at a low false positive rate

1. Fix the target FPRs, such as 10^{-2} and 10^{-3}.
2. Plan the trials with about $100/p$ negatives for a target FPR p.
3. Choose the threshold on the negatives of one split and measure the rates on another, because a threshold tuned on the evaluation set looks better there than it is. The calibration split needs as many negatives, since below one over their number `np.quantile` only interpolates between the top scores.
4. Report the TPR and FPR measured at that threshold, with Clopper–Pearson intervals on both, and the full ROC curve on log–log axes. A TPR read off the test split's ROC curve is optimistic, because it picks its threshold on the test data.
5. Compare two detectors at the same FPR, never by AUC or accuracy alone. On a shared evaluation set, check the gap with McNemar's test, `binomtest(m1, m1 + m2, 0.5)`, where m_1 and m_2 count the positives that only one of them catches.

The snippet runs steps 3 and 4 on scikit-learn's breast cancer dataset, with a logistic regression as the detector and malignant tumours as positives. It holds out 30% of the data for calibration and 30% for testing. For a target FPR of 1%, it thresholds at the 99th percentile of the calibration split's benign scores.

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y  # make malignant (coded 0) the positive class
X_tr, X_ev, y_tr, y_ev = train_test_split(X, y, test_size=0.6, stratify=y, random_state=0)
X_cal, X_te, y_cal, y_te = train_test_split(X_ev, y_ev, test_size=0.5, stratify=y_ev, random
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr, y_tr)
threshold = np.quantile(model.decision_function(X_cal[y_cal == 0]), 0.99)  # target FPR 1%,
s = model.decision_function(X_te)
pos, neg = s[y_te == 1] > threshold, s[y_te == 0] > threshold  # flagged positives, negative
tpr = binomtest(int(pos.sum()), pos.size).proportion_ci(method="exact")
fpr_upper = binomtest(int(neg.sum()), neg.size, alternative="less").proportion_ci(method="ex
print(pos.sum(), pos.size, f"{pos.mean():.3f} [{tpr.low:.3f}, {tpr.high:.3f}]")  # 56 63 0.8
print(neg.sum(), neg.size, f"{neg.mean():.4f} <= {fpr_upper:.4f}")  # 2 108 0.0185 <= 0.0571
```

The detector catches 56 of the 63 malignant test cases, a TPR of 0.889 with a 95% interval from 0.784 to 0.954. But the threshold aimed at 1% flags 2 of the 108 benign test cases, and the one-sided 95% bound from those counts certifies only an FPR below 0.0571. Step 2 asks for about 10,000 negatives at this target, so on a dataset this small, report the interval and don't claim the target.

4.12. Cheat sheet

Table 4.1.: The questions of this chapter and their answers.

Question	Answer
What is the best test?	Threshold the likelihood ratio, per group if groups score differently (Neyman–Pearson)
How good can any test be?	The trade-off curve; normal pair $G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$
What does a capped ratio allow?	$\text{TPR} \le e^\kappa \text{FPR}$
What do accuracy and advantage measure?	Best advantage = TV, one point of the curve
Why is AUC not enough?	Equal AUC can hide a 12-fold gap in TPR at an FPR of 10^{-3}

Question	Answer
What does a base rate do?	Posterior odds = prior odds × TPR/FPR
How sure is a measured rate?	Clopper–Pearson; zero events give about $3/n$ at 95%

Rules of thumb

- Plan for $100/p$ negatives, and use Clopper–Pearson, never Wald.
- Report TPR at fixed low FPRs on log–log axes, never AUC alone.

4.13. References

- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves and their pitfalls.
- Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): why Wald fails and what to use.

Derivations. Every result here is derived step by step in the full edition's chapter.

5. Information theory

5.1. What it is

Information theory measures how much one quantity tells you about another, with two sets of measures for two questions. The first, Shannon's, asks how many bits an observation saves, on average, when you describe a hidden value. The **entropy** of a random variable X is the average number of bits per value that the best compression code needs. Writing $p(x)$ for the probability of the value x, with logarithms base 2, it is

$$H(X) = -\sum_{x} p(x) \log_2 p(x). \tag{5.1}$$

The **mutual information** $I(X;Y)$ is the drop in the entropy of X that seeing an observation Y causes on average. From a joint table it is $H(X) + H(Y) - H(X,Y)$, where $H(X,Y)$ is the entropy of the pairs. Unlike a correlation, it is zero only when the two are independent.

The second question asks how much more likely a single best guess becomes. A guesser with one try picks the most likely value, so it wins with the largest probability, called the **vulnerability** $V(X)$. Its negative logarithm is the **min-entropy**,

$$H_\infty(X) = -\log_2 \max_x p(x), \tag{5.2}$$

which is never larger than the entropy.

5.2. When you meet it

Feature selection often ranks features by mutual information, which decision trees call the **information gain**. Min-entropy matters whenever a value must be hard to guess, such as a password or an identifier. So use Shannon's measures for averages and features, and min-entropy for any value that an attacker gets one guess at.

5.3. How to use it

`scipy.stats.entropy(p, base=2)` returns the entropy of a distribution in bits. For two discrete columns, `sklearn.metrics.mutual_info_score(a, b)` plugs their observed frequencies into the definition of mutual information. Continuous values rarely repeat, so `mutual_info_classif` in `sklearn.feature_selection` estimates it from nearest-neighbour distances instead. Both return nats (natural-log units), so divide by $\ln 2$ to get bits.

The two measures can disagree sharply. A uniform 4-digit PIN has $\log_2 10{,}000 = 13.2877$ bits of entropy. Now suppose 10% of people choose 1234 and the rest choose evenly among the others. The entropy barely moves, to 12.4278 bits. But the min-entropy falls to 3.3219 bits, because the guess “1234” succeeds one time in ten.

To measure how much an output helps one guess, start from two tables. The prior $\pi(x)$ says how likely each secret is, and the channel $K[x, y]$ says how likely each output y is when the secret is x. Multiplying each row of the channel by its prior gives the joint table. The best guess after seeing y is the row with the largest entry in column y, so its success probability is the sum of the column maxima:

$$V(X \mid Y) = \sum_{y} \max_{x} \pi(x) K[x, y]. \tag{5.3}$$

The logarithm of $V(X \mid Y)/V(X)$ is the **min-entropy leakage**, and 1 bit of it doubles the chance of a correct guess. In the full edition’s example with four secrets, the output raises that chance from 0.4 to 0.66, a leakage of $\log_2(0.66/0.4) = 0.7225$ bits.

Without a known prior, use the worst case over priors, the **min-capacity**. It is the logarithm of the sum of the channel’s own column maxima, 1.2630 bits in the example.

Mutual information can also show that a task is hard, through the uncertainty $H(X \mid Y) = H(X) - I(X;Y)$ that the output leaves. Take a secret with m possible values, and a guess from the output with error probability P_e. Writing $h(P_e)$ for the entropy of a coin with that bias, Fano’s inequality says

$$H(X \mid Y) \le h(P_e) + P_e \log_2(m - 1). \tag{5.4}$$

For example, a label uniform over 8 classes has 3 bits of entropy. If the features carry at most 1 bit about it, 2 bits remain. Solving Equation 5.4 for P_e numerically, with `scipy.optimize.brentq`, then shows that every classifier errs at least 0.3730 of the time.

5.4. What goes wrong

Plug-in estimates of mutual information run high on small samples, so two independent columns rarely give exactly zero. Compare an estimate with the same estimate after shuffling one column, which shows what independence alone produces.

Low mutual information does not mean that little is revealed, just as high entropy did not make the PIN hard to guess. Take a uniform 64-bit secret, and a channel that outputs all of it one time in eight, when its top 3 bits are zero, and a fixed symbol otherwise. Its mutual information is only 8.1686 bits, yet a guesser who sees its output wins one time in eight. Min-entropy leakage catches this channel, at 61 bits, but it too averages over secrets, so it can stay small when one rare value is fully exposed.

Processing an output cannot add information about its source. If a variable Z is computed from Y alone, possibly with fresh randomness, the data-processing inequality says

$$I(X; Z) \le I(X; Y). \tag{5.5}$$

The condition matters, because a Z that also looked at X can know more.

5.5. References

- Smith, “On the Foundations of Quantitative Information Flow” (FoSSaCS 2009): why one-try guessing needs min-entropy.

Derivations. Every result here is derived step by step in the full edition’s chapter.

6. Optimization

6.1. What it is

Training a model minimizes the average loss over the n records of its training set D, usually by stochastic gradient descent (SGD). Each step moves the parameters θ against the gradient of a random minibatch, scaled by a learning rate η, also called the step size. A loss is L**-Lipschitz** when its gradient never has length more than L, and **smooth** when its gradient changes by at most a constant β per unit of distance.

It is **strongly convex** when its curvature is at least a constant λ in every direction, which pins down a single minimizer. Table 6.1 gives all three for common losses.

Table 6.1.: The constants of common losses, for labels ± 1 and features of length at most 1.

Loss	Lipschitz	Smooth	Strongly convex
Logistic, $\ln(1 + e^{-y\langle\theta,x\rangle})$	$L = 1$	$\beta = \frac{1}{4}$	No
Hinge, $\max(0, 1 - y\langle\theta, x\rangle)$	$L = 1$	No, it has a kink	No
Squared, $\frac{1}{2}(\langle\theta, x\rangle - y)^2$	No, the gradient grows with θ	$\beta = 1$	Only if the features span every direction
Any of these plus $\frac{\lambda}{2}\|\theta\|^2$	Only on a bounded set: $L + \lambda r$ within radius r	$\beta + \lambda$	λ

6.2. When you meet it

You meet these results when you choose a learning rate, a batch size and a number of steps, or set scikit-learn's regularization arguments. They also bound one training record's influence, which the next chapter turns into a bound on the generalization gap.

6.3. How to use it

For a convex loss, run SGD for a number T of steps inside a ball of radius R that contains the minimizer. Return the **averaged iterate** $\bar{\theta}_T$, the average of the parameters over all steps. Let G^2 bound the average squared length of the minibatch gradients, which is L^2 for an L-Lipschitz

loss. Then the best fixed step, and the error it guarantees on the average training loss $\mathcal{L}(\theta; D)$ above its smallest value $\mathcal{L}^*$, are

$$\eta = \frac{R}{G\sqrt{T}}, \qquad \mathbb{E}[\mathcal{L}(\bar{\theta}_T; D)] - \mathcal{L}^* \le \frac{RG}{\sqrt{T}}. \tag{6.1}$$

Logistic regression on normalized features has $L = 1$, so $G = 1$. In a ball of radius 5, $T = 10{,}000$ steps then get a learning rate of $5/\sqrt{10{,}000} = 0.05$. The formula assumes the worst case, so use it to start a grid search. On the full edition's data the best rate was 3.6 times larger.

A ridge term $\frac{\lambda}{2}\|\theta\|^2$ makes the loss strongly convex, so the error falls about like $1/T$ instead of $1/\sqrt{T}$. That needs a learning rate that shrinks like $1/(\lambda t)$ at step t, which `SGDClassifier(alpha=λ, learning_rate="optimal", average=True)` uses. Scikit-learn's `LogisticRegression` instead minimizes `C` times the summed loss plus $\frac{1}{2}\|\theta\|^2$, a ridge of strength $\lambda = 1/(n\,C)$ on the average loss, so a large `C` means a weak ridge.

If the variance of one record's gradient around the full gradient is σ_1^2, a minibatch of B records drawn without replacement has a gradient variance of

$$\mathbb{E}\|g_B - \nabla\mathcal{L}(\theta; D)\|^2 = \frac{\sigma_1^2}{B} \cdot \frac{n - B}{n - 1}. \tag{6.2}$$

The constant G^2 of Equation 6.1 is the squared length of the full gradient plus this variance. So a larger batch lowers G, which allows a larger learning rate.

Gradient-norm clipping caps the length of each per-example gradient g_i at a chosen clipping norm C, replacing it by $g_i \min(1, C/\|g_i\|)$. Suppose a training step subtracts η/B times the sum of the clipped gradients. Then adding or removing one record changes the step by at most the first amount below, and replacing one by at most the second, whatever the loss:

$$\frac{\eta C}{B}, \qquad \frac{2\eta C}{B}. \tag{6.3}$$

Here B must be fixed in advance, because dividing by the actual batch size would let a removal rescale every other gradient too. And the bound covers one step from the same starting point, so over a run the changes can accumulate.

Regularized **empirical risk minimization (ERM)** minimizes the average loss plus a ridge term of strength λ. Write $\theta^*(D)$ for the minimizer on a training set D, and D' for the same set with one record replaced. If each loss is convex and L-Lipschitz,

$$\|\theta^*(D) - \theta^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.4}$$

For logistic regression on 1,000 records, $\lambda = 0.1$ gives a bound of 0.02. It is loose, three times the largest movement observed on the full edition's data.

6.4. What goes wrong

A single unnormalized feature vector breaks $L = 1$, so normalize every one to length 1 or less. The decaying learning rate needs a value of λ no larger than the true curvature, because an

overestimate shrinks the steps too fast and training stalls. And a learning rate above $2/\beta$ can raise the loss, the usual cause of a diverging loss curve.

On a neural network's non-convex loss, SGD guarantees only a point with a small gradient, perhaps a saddle point. The bound of Equation 6.4 also needs the ridge term's strong convexity. Without it, one replaced record can move the median, the absolute loss's minimizer, from 0 to 1. So for a neural network, a statement about one record's influence must be about the training steps, as in Equation 6.3, not about the minimizer.

Derivations. Every result here is derived step by step in the full edition's chapter.

7. Algorithmic stability and generalization

7.1. What it is

A model's **generalization gap** is its average loss on new data minus its average loss on the training data. A model generalizes when it does not depend much on any single training record. To measure that dependence, replace the i-th training record by a fresh draw from the same source and retrain. How much higher the new model's loss is than the old one's, on the record taken out, is its **memorization score** mem_i. Averaged over the n training records and over random training sets, the scores give the expected gap exactly:

$$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[\text{mem}_i]. \tag{7.1}$$

To bound the scores, call a learning algorithm β**-uniformly stable** when replacing any one training record moves the loss at any test point by at most β. Every score is then at most β, so the expected gap is too:

$$\big|\mathbb{E}[\text{gap}]\big| \le \beta. \tag{7.2}$$

Regularized training and stochastic gradient descent (SGD) stopped early are stable in this sense, while a model that stores its training set is not.

7.2. When you meet it

You meet stability when you choose a regularization strength and a number of epochs, and when a training error of 0 says nothing about the test error. Memorization scores tell you which records a model has learned by heart.

7.3. How to use it

For regularized **empirical risk minimization (ERM)**, which minimizes the training loss plus a ridge penalty of strength λ, the stability is

$$\beta = \frac{2L^2}{\lambda n}. \tag{7.3}$$

It needs a convex loss whose gradients have length at most L (Optimization), and features normalized to length at most 1 give $L = 1$ for the logistic loss. So with 1,000 records and $\lambda = 0.1$, logistic regression has $\beta = 0.02$. Scikit-learn's `LogisticRegression(C=...)` sets λ to one over n

times `C`, so a large `C` means a large β. Its intercept is not penalized, so strictly the bound covers only `fit_intercept=False`.

The bound gets the trend right but not the size. On the full edition's simulated data, the measured gap falls like $1/n$, as predicted, but is only about 3% to 13% of the bound. Because Equation 7.1 is an average over training sets, average the measured gap over several random splits. Use the bound to see which direction helps, and choose λ itself on held-out data, since a strong ridge also pulls the weights away from the best fit.

SGD on a convex and smooth loss is also stable if it stops early. Hardt, Recht and Singer (2016) showed that with a constant step η, its stability after E epochs is at most $2L^2\eta E$. This grows with every epoch, so training longer trades a better fit for less stability, which is what early stopping balances. In the full edition's unregularized example, the gap does grow with the epochs, but after 40 of them the bound is 165 times the measured gap of 0.121.

To measure a memorization score, retrain without the record, or with a held-out point in its place, and compare its loss. That costs one retraining per record. So for a large model, score a sample, or train many models on random subsets and compare each record's loss when it is in and out. Expect uneven scores. A typical record is fitted anyway through similar records, while a mislabelled or rare one is fitted only when it is there itself.

7.4. What goes wrong

A perfect training score is a warning, not a result. The 1-nearest-neighbour rule, `KNeighborsClassifier(n_neighbors=1)`, has training error 0, and under the 0–1 loss its β is 1, so the bound says nothing. With noisy labels its gap never closes.

A small average gap does not mean that no record was memorized, because the average leaves room for a few large scores. So look at the largest scores, not only the mean. Some memorization of rare records is needed for accuracy on data with a long tail of rare subpopulations (Feldman, 2020), so a large score is not always a defect.

A bound for a single run, rather than for the average, is looser still. The Bousquet–Elisseeff bound holds with 95% confidence for one run. For logistic regression on 10,000 records with $\lambda = 0.1$, it adds 0.614 to the training loss, close to the 0.69 loss of a coin flip.

Finally, a held-out set is honest only for questions fixed before you look at it. Once you look at a result, change something and ask again, you have trained on the holdout. Freedman's paradox shows the size of the effect on a holdout of 1,000 records with 1,000 random features, all independent of the label. Each feature's agreement rate with the label is a question fixed in advance, and each strays from one half by less than the bound for such questions, 0.0616. But a majority vote of the features that agree most reports an accuracy of 0.8053 on the same data, against 0.5018 when they are chosen on a separate half. So split with `train_test_split` before you start, tune with `cross_val_score` on the training part, and use the test set once.

Derivations. Every result here is derived step by step in the full edition's chapter.

8. Cryptography basics

8.1. What it is

A security claim is a statement about a guessing game. A challenger secretly picks one of two worlds, such as encrypting one of two messages, and an adversary sees the result and guesses which. Write X and Y for what it sees in the two worlds. If an adversary, written $\mathcal{A}$, answers 1 for the second, its **advantage** is the gap between how often it says 1 in each:

$$\mathrm{Adv}_{\mathcal{A}}(X, Y) = \big|\Pr[\mathcal{A}(X) = 1] - \Pr[\mathcal{A}(Y) = 1]\big|. \tag{8.1}$$

If the two worlds are equally likely, it wins with probability $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}_{\mathcal{A}}$, and no advantage exceeds the total variation distance between X and Y (Divergences).

A scheme has **κ-bit security** when no attack beats trying keys, so an attacker who tries t keys gets an advantage of at most about $t/2^{\kappa}$. For example, testing 10^{18} keys per second for a year breaks a 128-bit key with probability 9.3×10^{-14}.

8.2. When you meet it

You meet these tools whenever a pipeline pseudonymizes identifiers, generates keys, encrypts data or splits a value among holders. Table 8.1 gives the construction for each job. Never implement your own.

Table 8.1.: The construction to use for each job, and the mistake to avoid.

Job	Use	In Python	What goes wrong
Encrypt data	Authenticated encryption, such as AES in Galois/Counter Mode	`AESGCM` from the `cryptography` package	A reused nonce reveals data
A keyed pseudonym or tag	The hash-based message authentication code (HMAC)	`hmac.new(key, msg, hashlib.sha256)`	A plain hash is reversed by hashing every candidate
Keys, nonces, identifiers	A cryptographic random generator	`secrets.token_bytes`	`random` and seeded NumPy generators are predictable

Job	Use	In Python	What goes wrong
Split a value among holders	Additive secret sharing modulo a prime	`secrets.randbelow(p)`	Shares not reduced modulo p leak
Compute on encrypted values	Lattice-based homomorphic encryption	SEAL, TenSEAL or OpenFHE	Every operation spends a noise budget

8.3. How to use it

The **birthday bound** says that when a number m of values are drawn uniformly from a number N of possible values, the chance of a repeat is at most

$$\Pr[\text{collision}] \le \frac{m(m-1)}{2N}. \tag{8.2}$$

A repeat becomes as likely as not after about $1.18\sqrt{N}$ draws, so random κ-bit values start to collide after about $2^{\kappa/2}$.

To split a secret among several holders, draw all shares but one uniformly modulo a prime p, and choose the last so that they add up to the secret. Adding two secrets' shares holder by holder gives shares of their sum, as the snippet shows for 42 and 100. Its birthday bounds then show why pseudonyms of 10 million identifiers keep 128 bits of the HMAC. At 64 bits, a collision has probability up to 2.7×10^{-6}.

```
import hashlib
import hmac
import secrets

p = 2**61 - 1  # a prime larger than any secret or sum of secrets
a, b = [secrets.randbelow(p) for _ in range(2)], [secrets.randbelow(p) for _ in range(2)]
a, b = [*a, (42 - sum(a)) % p], [*b, (100 - sum(b)) % p]  # three shares each of 42 and 100
print(sum(a) % p, sum(x + y for x, y in zip(a, b)) % p)  # 42 142

key = secrets.token_bytes(32)  # kept apart from the data
pseudonym = hmac.new(key, b"alice@example.com", hashlib.sha256).hexdigest()[:32]  # 128 bits
q = 10**7  # identifiers
print(f"{q * (q - 1) / (2 * 2**128):.1e} {q * (q - 1) / (2 * 2**64):.1e}")  # 1.5e-25 2.7e-0
q, n = 2**32, 2**96  # messages under one key, 96-bit nonces
print(f"{q * (q - 1) / (2 * n):.1e}")  # 1.2e-10
```

The `cryptography` package is not installed here, so this encryption sketch was not run:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
key = AESGCM.generate_key(bit_length=256)
nonce = secrets.token_bytes(12)             # 96 bits, fresh for every message
ct = AESGCM(key).encrypt(nonce, data, aad)  # aad: authenticated, not encrypted
```

HMAC is secure if SHA-256's compression function is a pseudorandom function. Encryption with AES assumes that AES is a pseudorandom permutation, and lattice schemes that learning with errors is hard. Secret sharing needs none, because a holder's shares are random numbers whatever the secret, which **simulation** makes precise.

Homomorphic encryption hides each value behind a small random error modulo a number Q, and decryption works while the total error is below $Q/4$. Adding ciphertexts adds their errors, so with each error at most a bound B, a sum of k ciphertexts is guaranteed to decrypt when $k < Q/(4B)$. A larger modulus buys a larger **noise budget**, at the cost of slower arithmetic.

8.4. What goes wrong

A repeated nonce reveals the XOR of the two messages, so rotate keys before random nonces collide. With 96-bit nonces, the snippet's last line puts that chance at 1.2×10^{-10} after 2^{32} messages.

Shares must be reduced modulo p, or their range moves with the secret. An unreduced share equal to the secret plus a random number from 0 to 15 tells the secrets 0 and 7 apart with advantage $7/16$.

Finally, "no efficient adversary gets more than a negligible advantage" is an asymptotic claim and says nothing about the key length you use.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part II.

Threat models and attacks

This part teaches you to attack your own models and releases before someone else does. You need the attacks even if you plan to use differential privacy, because they check that an implementation keeps its promise. Every attack here is a game, in which an adversary guesses a secret hidden in a model or a release. Its guess counts only by how much it beats a baseline that never looks at the model.

Read Threat models and privacy games first and Membership inference second, then the chapter whose secret matches your question. For a trained model, Memorization and extraction seeks text the model memorized. In federated learning, Gradient inversion rebuilds a client's input from its updates. Two short reference pages cover rarer attacks: attribute inference seeks a person's sensitive field, and property inference a fact about the whole training population.

For released data, read the last three chapters in order, because they tell one story. Records without names still fall to linkage. Records hidden in crowds still give away sensitive values. And accurate aggregates alone can give away the whole dataset (reconstruction attacks). Answering too many statistics too accurately reveals the data, so neither removing names nor releasing only aggregates is anonymization. The next part starts from differential privacy instead.

Two lessons drive every audit here. Report the true positive rate at a low false positive rate, such as one in a thousand, not accuracy or the area under the curve. And the worst-off records leak first, such as outliers, duplicates and people with a unique combination of attributes, so an average over all records misleads. A failed attack proves little, because it only gives a lower bound on the leak.

The part ends with a workflow, Pre-release privacy risk assessment. It fixes the blocking thresholds first, then audits a classifier with shadow models, a fine-tuned language model with planted secrets and a table with quasi-identifiers, and writes the decision down. By the end of the part, you will be able to write a threat model and run a membership audit you can defend. You will also be able to test a table for unique records and decide whether a model or a dataset is safe to release.

9. Threat models and privacy games

9.1. Why this matters

A hospital trains a model on patient records and offers it to doctors through a web service. Whether the model is private has no answer until you say against whom. One attacker reads only predicted labels, while another holds the weights, knows the other patients or slips crafted records into the training data. A **threat model** pins these assumptions down. Once it is fixed, an attack becomes a **game**, in which the adversary guesses a hidden **secret**, such as whether one patient's record was used.

9.2. Writing down the threat model

Write the threat model before anyone runs an attack, because it decides which attacks to run and what result stops the release. In the worksheet, Table 9.1, the first six rows are the questions every threat model answers, and the last two turn the answers into a test.

Table 9.1.: The threat-model worksheet.

Question	What to write down	Default when unsure
Goal: what is the secret?	Membership, an attribute of a known record, the records themselves, or a property of the whole dataset	Membership, the goal that differential privacy bounds directly
Access: what does the adversary see?	Labels only, scores, the weights, or the training transcript	The most anyone outside the training team can get
Behaviour: does it follow the rules?	Honest-but-curious, or malicious (poisoning, tampered models)	Malicious, if it runs any part of the system
Trust: who sees the raw data?	Central curator, local (data protected on each device before collection), or distributed	Central, only if the data holder is not an adversary
Side information: what does it know?	The population, public datasets, other records, the target's other fields	Every record except the target's
Unit: what is one record?	A row, a user, a household	The person, whenever one person has several rows

Question	What to write down	Default when unsure
Attack and metric	Membership: the strongest attack's true positive rate (TPR) at a low false positive rate (FPR). A table: unique share and smallest class. A language model: canary exposure and extraction	The row of the goal and artefact in the risk assessment's thresholds
Acceptable result	The number that blocks the release	Fixed before any attack is run

The access row has four levels, each enough to compute the one below it. The weakest is label-only, the model's decisions on inputs the adversary chooses. Next come black-box scores, from which the labels follow, and then white-box access to the weights, from which the scores follow. The strongest is the **transcript**, everything training exposed, such as per-step gradients, whose last entry is the weights. So in this ordering, the **access chain**, more access never weakens the best attack. That compares the *best* attacks, not the ones someone tried, so "the attack we tried failed" proves little (Membership inference).

To fill the access row, read the serving code and the release plan. A service that returns probability vectors gives black-box access, and a checkpoint shared with a partner gives it white-box access. A model in several deployments carries the risk of the most exposed one.

Behaviour and trust decide who the adversary is. An **honest-but-curious** adversary follows the protocol and analyses only what it sees. A **malicious** one may insert crafted records into the training data, called **poisoning**, or send tampered models if it runs part of the system. If whoever collects the data may be an adversary, protection must happen before the data leave the user (Local, shuffle and distributed trust models).

Side information is the row you control least, because you cannot see what else the adversary holds. So assume it knows every record except the target's. The unit row decides what one guarantee covers: a per-row guarantee covers each of a user's 200 rows, but not all 200 together.

For the hospital's classifier, served through an API that returns class probabilities, the goal is membership and the access is black-box scores. A doctor using it is honest-but-curious but may query adaptively, the side information is a public sample of the same population, and the unit is the patient. The release is blocked if the lower end of the TPR's interval at an FPR of 10^{-3} exceeds ten times that FPR. Fix that threshold before the attack runs, because one chosen after seeing the results can always be set where they pass.

9.3. Scoring an attack against its baseline

In the membership game, a challenger flips a coin to decide whether to train the model with or without the target's record. The adversary then guesses whether the target is a **member**, that is, whether its record was used.

The attack's **true positive rate** is the fraction of members it flags, and its **false positive rate** is the fraction of non-members it wrongly flags. Write π_1 for the chance that the coin makes the target a member, the prior, and $\pi_0 = 1 - \pi_1$ for the opposite. The attack wins with probability

$$\Pr[\text{win}] = \pi_1 \,\text{TPR} + \pi_0 \,(1 - \text{FPR}). \tag{9.1}$$

The two rates do not depend on the prior, but this win rate, the accuracy, does. If only one target in ten is a member, always saying "not a member" has both rates at 0 and wins $\pi_0 = 90\%$ of the games. So score every attack against a **baseline**, the best score of a guess that never looks at the model.

With a fair prior, $\pi_1 = \pi_0 = 1/2$, the baseline is one half. The attack's **advantage** is how far its TPR exceeds its FPR, and putting the fair prior into the win rate gives

$$\Pr[\text{win}] = \frac{1}{2} + \frac{1}{2}\,(\text{TPR} - \text{FPR}). \tag{9.2}$$

So an attack with 55% accuracy at a fair prior has an advantage of 0.1, and unlike the accuracy, the advantage does not change with the prior. No attack's advantage exceeds the total variation distance between the adversary's views with and without the target (Divergences), which differential privacy later bounds.

9.4. Reporting: TPR at a low FPR

An attack scores each target and flags those above a threshold. Its **receiver operating characteristic (ROC)** curve plots its TPR against its FPR as that threshold moves. The advantage is the curve's largest height above the diagonal, where TPR equals FPR. Like the area under the curve, it hides the region where false positives are rare, which is where privacy breaks. An attack that flags 1% of members with no false alarms has a tiny advantage, yet identifies those members with certainty. So report membership attacks, as Carlini et al. (2022) do, by their TPR at a low FPR such as 10^{-3}.

A flagged person cares about a third number, the chance that the verdict is right. That is the **precision**,

$$\Pr[\text{member} \mid \text{flagged}] = \frac{\pi_1 \,\text{TPR}}{\pi_1 \,\text{TPR} + \pi_0 \,\text{FPR}}. \tag{9.3}$$

When members are rare, π_0 is large, so even a small FPR applied to the many non-members swamps the true positives. Take a count of people with a condition, released with Gaussian noise of standard deviation 1, so that a member adds 1 to the count. An adversary who knows every other record flags the target when the count is high. Held to an FPR of 10^{-3}, it catches members with probability 0.0183. At a fair prior the two halves of the prior cancel, so its precision is $0.0183/(0.0183 + 0.001) = 0.948$. If only one target in a hundred is a member, it drops to 0.156, so never quote a precision without its prior.

Each rate is measured on a finite set of targets, so it needs an interval. The recipe of Hypothesis testing plans about $100/\alpha$ non-members for an FPR of α, so that about 100 false positives pin the rate down. It sets the threshold on one split, measures on another, and puts a Clopper–Pearson interval on each rate. The snippet below applies it to the black-box membership attack of the Adversarial Robustness Toolbox (ART), against a random forest trained on half of scikit-learn's

569 breast cancer records. The attacker trains its attack on every other record, whose membership it knows, and reads the TPR on the rest at an FPR of at most 0.01. To stay short, it sets the threshold on those same scores.

```
import numpy as np
from art.attacks.inference.membership_inference import MembershipInferenceBlackBox
from art.estimators.classification import SklearnClassifier
from scipy.stats import binomtest
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import auc, roc_curve

np.random.seed(0)  # ART's attack model draws from NumPy's global generator
x, y = load_breast_cancer(return_X_y=True)
member = np.random.default_rng(0).random(len(y)) < 0.5  # the target's training records
target = SklearnClassifier(RandomForestClassifier(random_state=0).fit(x[member], y[member]))
known = np.arange(len(y)) % 2 == 0  # records whose membership the attacker knows, to train
attack = MembershipInferenceBlackBox(target, attack_model_type="rf")
attack.fit(x[known & member], y[known & member], x[known & ~member], y[known & ~member])
fpr, tpr, _ = roc_curve(member[~known], attack.infer(x[~known], y[~known], probabilities=Tru
i = np.flatnonzero(fpr <= 0.01)[-1]  # the best TPR whose FPR is at most 0.01
n_in, n_out = int(member[~known].sum()), int((~member[~known]).sum())
tp, fp = round(tpr[i] * n_in), round(fpr[i] * n_out)
print(f"AUC={auc(fpr, tpr):.3f}  TP={tp} of {n_in}  FP={fp} of {n_out}")  # AUC=0.552  TP=3
t, f = (binomtest(k, n).proportion_ci(method="exact") for k, n in ((tp, n_in), (fp, n_out)))
print(f"TPR 95% CI [{t.low:.4f}, {t.high:.4f}]  FPR 95% CI [{f.low:.4f}, {f.high:.4f}]")  #
```

With only 151 non-members, the smallest nonzero FPR the attack can show is one in 151, far above one in a thousand. Even one false positive leaves the FPR's interval running from 0.0002 to 0.0363. The TPR's interval, from 0.0047 to 0.0645, overlaps it. So this test cannot show that the attack beats chance at a low FPR, and its area under the ROC curve, 0.552, adds little. At 10^{-3} the recipe asks for about 100,000 non-members, which is why audits retrain many models on many records (Membership inference).

A report you can act on therefore gives the TPR at a stated low FPR, with its counts, intervals on both rates and the ROC curve on log axes. It also states the worksheet's threat model, the baseline, the prior behind any precision, and whether the targets were random records or chosen worst cases.

9.5. Average case and worst case

An evaluation that draws the data and the target from the population measures the **average case**. One that lets the adversary know or pick the other records and the target measures the **worst case**. Audits of real models measure the first, but a guarantee must control the second, and the gap can be large. In the noisy count, the adversary who knows the other records has an advantage of 0.3829. If instead 100 of those records are unknown to it, each a fair coin, their sum

blurs the count like extra noise, and the advantage falls to 0.0779. But an adversary who planted the other records knows them, so a defence that relies on the crowd fails against the adversary it most needs to stop.

Averaging over targets hides the worst-treated person in the same way, because exposing a few outliers barely moves an average over everyone. So treat an average-case number as a lower bound on the risk. Audits close the gap by planting **canaries**, records built to be as exposed as possible (Memorization and training-data extraction), and differential privacy bounds the worst case directly.

9.6. Cheat sheet

Table 9.2.: The results of this chapter.

Quantity	Formula	Use it for
Advantage	$\mathrm{TPR} - \mathrm{FPR}$	Comparing attacks at any prior; 0 means no attack
Precision	$\pi_1 \mathrm{TPR} / (\pi_1 \mathrm{TPR} + \pi_0 \mathrm{FPR})$	What a flagged person faces; state the prior

Rules of thumb

- Fill in the worksheet, including the blocking threshold, before running or reading any attack.
- Assume the adversary knows the other records, and audit with worst-case canaries.

9.7. References

- Salem et al., "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023): every inference attack written as a game.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): the case for reporting TPR at a low FPR on log-scaled ROC curves.

Derivations. Every result here is derived step by step in the full edition's chapter.

10. Membership inference

10.1. Why this matters

A hospital publishes a model that predicts a disease, and you hold a patient's record. **Was that record in the training set?** If it was, the patient was treated there for that disease. This is **membership inference**, the standard measure of leakage, because a model that cannot hide whether a record was used hides little else. Membership is a test about one record, so the attack to use in an audit compares each record with itself.

10.2. Membership as a test about one record

In the game of Threat models and privacy games, the attacker gets a training member or a fresh record and says "member" or "not". It is scored by its **true positive rate (TPR)**, the share of members it flags, and its **false positive rate (FPR)**, the share of non-members it flags.

Fix a target record, written z, and look at the model's output on it, such as its loss. Retrain on fresh random draws of the data, and that output has one distribution over the runs that include z and another over the runs without it. Call them the **IN** and **OUT** distributions of z. The best test of membership compares their likelihoods at the observed output (the Neyman–Pearson lemma), so it does well only where the two lie far apart. A typical record's distributions almost overlap, because the rest of the data already predict it. An outlier's lie far apart, because the model gets it right only if it trained on it. So the records an attack finds at a low FPR are mostly outliers.

The numbers in this chapter come from practitioner_lira.py. It trains 64 logistic-regression classifiers, each on a random half of scikit-learn's 1,797 images of handwritten digits. Each model takes a turn as the target, while the other 63 serve as the attacker's **shadow models**, copies trained the same way whose training images the attacker knows.

10.3. The quick check: the generalization gap

Before any attack, compare the training and test errors. A model fits its training data, so it gets members right more often than fresh records, and Yeom et al. (2018) turned that into the attack "member if correctly classified". When members and non-members come from the same population, its advantage, the TPR minus the FPR, equals the gap between the two errors:

$$\text{Adv} = \text{test error} - \text{training error}. \tag{10.1}$$

On the digits, averaged over the 64 models, the gap is 0.0422.

So a large gap shows a leak at no cost. But **a small gap does not mean a private model**, because the gap averages over records and hides who is exposed. The loss attack, which flags every record whose loss falls below one threshold, shows this. At an FPR of 10^{-3} it reaches a TPR of 0.0009 on the digits, no better than a coin that says "member" one time in a thousand.

The classic attack of Shokri et al. (2017) also applies one rule to all records, learned from shadow models. The script runs it from the Adversarial Robustness Toolbox (ART), a Python library of attacks, with four shadow models.

```
# ART copies the target's recipe and trains each shadow on a random half of the images.
shadows = ShadowModels(models.first, num_shadow_models=N_ART_SHADOWS, random_state=seed)
(x_in, y_in, pred_in), (x_out, y_out, pred_out) = shadows.generate_shadow_dataset(x, onehot)
# One attack model learns "member" from a shadow's probabilities and the true label.
attack = MembershipInferenceBlackBox(models.first, attack_model_type="rf")
attack.fit(x_in, y_in, x_out, y_out, pred=pred_in, test_pred=pred_out)
# The same attack model scores every target's probabilities on every image.
scores = [attack.infer(x, onehot, pred=p, probabilities=True)[:, 0] for p in models.probs]
```

Its one rule fails at a low FPR too, finding only 0.0014 of the members at an FPR of 10^{-3}.

10.4. LiRA: a likelihood-ratio test per record

The likelihood ratio attack (LiRA) of Carlini et al. (2022) uses the shadow models to estimate each record's own IN and OUT distributions instead. Its statistic is the logit of the model's confidence p_y in the true label y,

$$\phi = \ln \frac{p_y}{1 - p_y}. \tag{10.2}$$

The logit is close to normal over retrainings, while the confidence itself piles up against 1.

For each record, the attacker fits a normal distribution to the record's logit over the shadow models that trained on it. Its mean is written μ_{in} and its standard deviation σ_{in}. A second fit, over the shadow models that did not, has mean μ_{out} and standard deviation σ_{out}.

Online LiRA scores the target model's logit ϕ_z on the record by how much more likely it is under the IN fit than under the OUT fit. Writing $\mathcal{N}(\phi; \mu, \sigma^2)$ for the normal density, the score is

$$\ln \Lambda_z = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in}}, \sigma_{\text{in}}^2) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out}}, \sigma_{\text{out}}^2). \tag{10.3}$$

Offline LiRA uses only OUT models, which need not contain the record being tested, so they can be trained once and reused for every new record. It asks how many OUT standard deviations the logit lies above its usual value,

$$\frac{\phi_z - \mu_{\text{out}}}{\sigma_{\text{out}}}. \tag{10.4}$$

Each score already compares a record with its own fits, so one threshold serves for all records. In the script, ART's `SklearnClassifier` wraps the scikit-learn recipe, trains every model and returns its probabilities.

```
# Each image is in each model's training set with probability 1/2.
member = rng.random((n_models, n)) < 0.5
for k in range(n_models):
    model = recipe()  # a fresh, untrained copy of the target's recipe
    model.fit(x[member[k]], onehot[member[k]])
    probs[k] = model.predict(x)  # the probabilities the attacker sees, for every image
    p = np.clip(probs[k, np.arange(n), y], 1e-15, 1 - 1e-15)  # confidence on the true label
    phi[k] = np.log(p) - np.log(1 - p)  # its logit, close to Gaussian over retrainings
    confidence[k] = p
```

The LiRA scores come from a helper, because no library installed with this book computes them. Its `lira_scores`, in practitioner_privacy_tests.py, fits the two normals per image and returns Equation 10.3 and Equation 10.4.

The command `uv run python code/part02-attacks/practitioner_lira.py` runs in seconds. It compares the attacks by their TPR at two low FPRs and by their area under the curve (AUC) of the TPR against the FPR.

```
  loss attack: AUC=0.518  TPR@0.001=0.0009  TPR@0.01=0.0105
  ART shadow-model attack: AUC=0.570  TPR@0.001=0.0014  TPR@0.01=0.0146
  LiRA offline: AUC=0.577  TPR@0.001=0.0087  TPR@0.01=0.0345
  LiRA online: AUC=0.623  TPR@0.001=0.0318  TPR@0.01=0.0790
  LiRA online, global variance: AUC=0.614  TPR@0.001=0.0250  TPR@0.01=0.0737
```

By AUC the attacks look close, and ART's attack nearly matches offline LiRA. At an FPR of 10^{-3} they are not, because online LiRA finds 0.0318 of the members and the loss attack only 0.0009. All of them see the same model outputs, so the whole gain comes from comparing each record's output with that record's own fits. Online LiRA beats offline LiRA because its IN fit also tells it how far training on a record moves that record's logit.

Noisy fits are the main practical failure. Each record's variance comes from only about half the shadow models. An underestimated one makes an ordinary output look like a far-tail event, a false positive at the strict thresholds that matter. With few shadow models, Carlini et al. therefore recommend one **global variance** shared by all records. It makes 16 shadows usable, while per-record variances need about 64.

10.5. Running LiRA on your model

The recipe is the same for any model. Train the target and its shadows each on a random half of the data pool, all with exactly the target's recipe. A shadow trained differently, even with another augmentation, estimates the wrong IN and OUT distributions. Fit the IN and OUT normals to each record's logits, let each model take a turn as the target, and score every pair of target model and record. Each shadow is a training run, so the shadow count is the budget decision. Pre-release privacy risk assessment runs the whole job on the digits.

For a confident model, the confidence p_y in the true label nears 1, so $1 - p_y$ rounds to 0 and the logit becomes infinite. Compute it instead from the class scores f before the softmax, as

$f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, or from `log_softmax`. Clipping the probabilities is no substitute. On a nearly separable model most confidences hit the clip, so their fits have no spread and those pairs drop out, taking the most exposed records with them. Treat such dropped pairs as a failed audit. The script clips only because no digits model comes near the clip.

The most common mistake is to draw members and non-members from different distributions. If non-members come from a later period, another site or another preprocessing pipeline, the attack measures that shift rather than the model, so split one pool at random.

Two more facts matter in practice. Returning only labels does not stop the attack, because a label-only attack measures how much an input must change before its label flips (Choquette-Choo et al. 2021). And libraries differ: TensorFlow Privacy's `run_attacks` and ART's `MembershipInferenceBlackBox` treat all records alike, while ML Privacy Meter implements LiRA.

To catch a training change that makes a model leak, rerun a cheaper LiRA on every release candidate. Train 16 models with the candidate's own recipe and score online LiRA with a global variance. On the digits, `practitioner_lira.py --models 16` runs in seconds and blocks the model under the release rule of the next section, as the 64-model run does. Compare candidates by their intervals, not their point values, which differ by seed noise alone.

10.6. Reading the result

Report the TPR at an FPR of 10^{-3} or lower, with the receiver operating characteristic (ROC) curve on log axes. Put Clopper–Pearson intervals on both rates, as in Part 1's recipe for evaluating a detector at a low FPR. That recipe also sets the thresholds on a separate split, a step the script, like the literature, skips.

```
LiRA offline: TP=502/57704  TPR=0.0087 [0.0080, 0.0095]  FP=56/57304  FPR=0.00098 [0.00074
LiRA online: TP=1833/57704  TPR=0.0318 [0.0303, 0.0332]  FP=57/57304  FPR=0.00099 [0.00075
```

The last word on each line applies the risk assessment's blocking rule. It blocks a release when, at an FPR of 10^{-3}, the lower end of the TPR's 95% interval is above ten times that FPR. Online LiRA's lower end, 0.0303, is above 0.01, so it blocks this model, which offline LiRA alone would have passed. For online LiRA, the logarithm of the ratio of the rates is 3.46. It estimates how large the privacy level ε of differential privacy (Part 3) would have to be to allow this attack. It is not a certified lower bound, because the pairs share models and so are not independent trials. A certified bound needs independent ones, such as the canaries of Auditing.

10.7. Who is exposed, and what helps

The records an attack scores highest are usually outliers, rare subgroups or mislabelled points, which the rest of the data do not predict. On the digits, the most exposed image is a 9 that the models classify with confidence 0.812 when they trained on it and 0.003 when they did not. Online LiRA flags it at an FPR of 10^{-3} in all 33 target models that trained on it.

Removing the exposed records does not help, because the next layer then becomes exposed, which Carlini et al. (2022b) call the onion effect. Closing the generalization gap is not enough either, because the worst records, not the average, set the low-FPR end.

Differential privacy does help. Private training limits how far any one record can move the model, and so how far apart every record's IN and OUT distributions can lie. Expect a trade-off, because when many subgroups are rare, memorizing them can be what makes a model accurate (Feldman 2020).

10.8. Cheat sheet

Table 10.1.: Which attack or check to use.

Situation	Use	Notes
A first look, no attack budget	Test error − training error	A lower bound on the leak; a small gap proves nothing
A release decision	Online LiRA, Equation 10.3, 64 shadows	Report TPR at FPR 10^{-3} with intervals
Few shadow models	Online LiRA with a global variance	16 shadows are usable this way

Rules of thumb

- Audit with LiRA. A failed weak attack, such as a global loss threshold, is not evidence of safety.

10.9. References

- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, the global variance, and the audit protocol.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the loss attack and its link to the generalization gap.

Derivations. Every result here is derived step by step in the full edition's chapter.

11. Attribute inference and model inversion

11.1. What it is

An **attribute inference** attack uses a trained model to recover one sensitive field of a person's record, such as smoking status, from the fields the attacker already knows. An insurer who knows your age and diagnosis could query a hospital's model that predicts the disease from age and smoking, and guess whether you smoke. But a published study on smoking would allow the same guess, so success alone is not a privacy failure. What matters is whether the model reveals more about you because your record was in its training data.

Yeom et al. (2018) turned this question into a game (Threat models and privacy games). A fair coin decides whether the target is a training record, a **member**, or a fresh draw from the same population. The attacker knows the target's public features and label, queries the model and guesses the sensitive value. Its score is the **attribute advantage**,

$$\mathrm{Adv}^{\mathrm{attr}} = \Pr[\text{guess right} \mid \text{member}] - \Pr[\text{guess right} \mid \text{non-member}]. \tag{11.1}$$

A guess that rests only on the whole population, called **population inference**, helps members and non-members equally, because both come from that population. So it cancels in the difference, and what is left is the leak about individuals.

11.2. When you meet it

You meet it when someone claims that "the model reveals" a sensitive field, or asks whether dropping a column made a model safe. Success up to what the population allows is then a question for policy and fairness, because it would be there without the person's record. Only success on members beyond non-members is a privacy leak.

11.3. How to use it

Start with the **imputation baseline**, the success of the best guess of the sensitive value from the public features and label alone, without the model. Many published attacks do no better (Jayaraman & Evans 2022), so it is the number to beat. Estimate it with a classifier that predicts the sensitive field from the public features and label, fitted on population records the model never saw.

Then run the attack. **Model inversion** (Fredrikson et al. 2014) scores each candidate value s of the sensitive field by two numbers. The first is the probability $p(s \mid x)$ that a person with the target's public features x has that value, estimated from population data. The second is $f_y(x, s)$,

the model's confidence in the known label y when it is fed x and the candidate. The inversion attack guesses the value that maximizes their product,

$$\hat{s} = \arg\max_{s} \; p(s \mid x)\, f_y(x, s). \tag{11.2}$$

Give the attacker every column it could hold, because a near-copy of the sensitive field, such as a relationship column next to marital status, moves the baseline most. Run the attack on training members and on as many held-out non-members. Report the baseline and both success rates with confidence intervals, including one on the advantage, since Pre-release privacy risk assessment blocks a release when that interval lies above zero.

The script practitioner_attribute.py, run with `uv run python code/part02-attacks/practitioner_attribut` applies the Adversarial Robustness Toolbox (ART) to Adult, census records of 45,222 people. A random forest predicts whether income exceeds $50K from every column, and sex is the sensitive field. The attacker holds 4,000 records of its own, and everyone's other columns and income.

```
def label(rows: IntArray) -> Array:  # the label the model returns for each record, as a col
    return np.argmax(target.predict(x[rows]), axis=1).reshape(-1, 1).astype(np.float64)

np.random.seed(seed)  # ART builds its random forests unseeded, from NumPy's global generato
attack = AttributeInferenceBlackBox(target, attack_model_type="rf", attack_feature=SENSITIVE
attack.fit(x[attacker])  # the attacker's own records, sensitive field included
baseline = AttributeInferenceBaselineTrueLabel(attack_model_type="rf", attack_feature=SENSIT
baseline.fit(x[attacker], y[attacker])  # the same records and the true label, without the m
hits: dict[str, list[int]] = {}
for name, rows in (("members", members), ("non-members", others)):
    known = np.delete(x[rows], SENSITIVE, axis=1)  # everything but the sensitive field
    guess = attack.infer(known, pred=label(rows), values=[0.0, 1.0])
    imputed = baseline.infer(known, y=y[rows], values=[0.0, 1.0])
    hits[name] = [int(np.count_nonzero(np.equal(g.ravel(), female[rows]))) for g in (guess,
```

```
ART black-box attack: right on members 0.8300 [0.8180, 0.8415]  on non-members 0.8423 [0.830
ART black-box attack: advantage -0.0123 [-0.0285, +0.0040]
imputation baseline: right on members 0.8307 [0.8188, 0.8423]  on non-members 0.8417 [0.8301
```

The attack is right on 0.8300 of the members, and the baseline, which never queries the model, on 0.8307. So the success comes from the population, and the advantage's interval contains zero, which passes the rule. On a synthetic population the full edition does find a small leak: inversion is right on 0.7832 of members and 0.7262 of non-members, an advantage of 0.0569.

11.4. What goes wrong

The commonest error is to report one success rate, such as that 0.7832 on members, without the baseline and the rate on non-members, which overstates the leak. Measuring only on fresh records understates it. A model is surer of the true label on its own members, the generalization gap of Membership inference, so the attack does better on them.

Dropping the sensitive column from the inputs does not stop population inference, because its correlates remain. **Confidence masking**, rounding the confidences or returning only the top label, puts no bound on the gap between members and non-members, and that gap can grow. In the full edition's experiment, returning labels only raised the advantage from 0.0569 to 0.0650, because success fell more on non-members than on members. Less overfitting shrinks the gap, and differential privacy bounds it for every attacker and every output format.

11.5. References

- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015): inversion from confidences, and rounding as a countermeasure.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022): the imputation baseline, and what it takes to beat it.

Derivations. Every result here is derived step by step in the full edition's chapter.

12. Property inference

12.1. What it is

A **property** is a fact about a training set as a whole, such as the share of a hospital's patients who are over 65. **Property inference** recovers it from a trained model. In the property game (Threat models and privacy games), the data owner flips a fair coin to pick one of two shares for a subgroup. It then trains a model on data with that share. The attacker knows both shares, can train models the way the owner does, and guesses which share was used, so a random guess is right half the time.

A property can leak when no single record does. Write n for the number of training records and g for the gap between the two shares. Switching shares changes about ng records at once, so it moves the model far more than one record can. The leak grows with n, while the risk to one record shrinks. Even a release of only the exact count of subgroup records shows this. At 10,000 records, the best attacker asking whether one patient is in the data is right with probability 0.502. The best one asking whether the share was 0.45 or 0.55 is right with probability 1.0000.

12.2. When you meet it

You meet it when you share or publish a model trained on your organization's data. First decide whether the property is a secret at all. It is harmful to reveal only when it describes the *data owner* rather than the world. One hospital's patient mix is a secret, but that smokers get more lung cancer is the point of a medical study. If it is not a secret, record that in the threat model of Pre-release privacy risk assessment and move on.

If it is a secret, differential privacy (DP) does not protect it. DP protects one record at a time, and its guarantee for a group multiplies the privacy level ε by the group's size (Post-processing, group privacy and composition). For a hospital of 1,000 patients and shares a tenth apart, the group is the $ng = 100$ patients that differ between the two datasets. So the property gets only 100ε, which promises nothing at any useful ε.

12.3. How to use it

To measure the risk, run the shadow-model attack of Ateniese et al. (2015), which treats whole models as data points. Draw many datasets of your training set's size at each of the two shares, from auxiliary data of the same population or from your own data. Train a **shadow model** on each with your architecture and training code, and record its outputs on a fixed set of probe inputs. A linear **meta-classifier**, such as scikit-learn's `LogisticRegression`, learns to tell the

two shares apart from those outputs, and its accuracy on held-out shadow models is the attack's accuracy. If the property is a secret, block the release when that accuracy's confidence interval lies above one half (Pre-release privacy risk assessment).

Use probe outputs rather than raw weights, because reordering hidden units, or flipping the signs of tanh units, changes the weights but not the network's function. On the full edition's small tanh networks, every raw weight therefore averages to zero under both shares. A linear meta-classifier on raw weights scores 0.497, which is chance, and on the probe outputs 1.000. Probes need only query access, so a model behind an application programming interface is exposed too.

12.4. What goes wrong

The commonest error is to assume that DP, or a passed membership audit, covers the property. Nor is a weak measured attack evidence of safety, because with few records and a small gap, a meta-classifier fitted to noisy shadow models finds the shift badly.

There is no cheap defence. Removing the group column leaves its correlates, and noise large enough to hide ng records destroys the model's use. If the property is a secret, the strongest option is not to release a model trained on that mix. If you must, report the attack's accuracy and state in the release notes that DP protects individuals only.

12.5. References

- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018): the practical attack on neural networks, and why raw weights fail.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021): property inference through queries alone.

Derivations. Every result here is derived step by step in the full edition's chapter.

13. Memorization and training-data extraction

13.1. Why this matters

A language model trained on scraped text or support tickets sees phone numbers, addresses, keys and private messages. Asked to continue "Anna Smith's phone number is", it may finish with the real number. This is **training-data extraction**. Membership inference needs the record in hand to ask whether it was used, while extraction hands the attacker the record itself.

13.2. Which memorization to measure

The notions of memorization in Table 13.1 give rates that differ by orders of magnitude, so a report must say which one it measures. All use greedy decoding, which writes out the most likely next token at each step.

Table 13.1.: The notions of memorization and when to use each. The first three are verbatim, needing an exact match, and each implies approximate. Discoverable implies extractable.

Definition	The string counts as memorized if	Use it for
Discoverable (Carlini et al. 2023)	The text before it in the training data makes greedy decoding produce it	Measuring memorization on a model you own
Extractable (Carlini et al. 2021)	Some prompt, any at all, makes greedy decoding produce it	The real risk, which attacks find but no test measures
k-eidetic	It is extractable and appears in at most k documents	Flagging the dangerous case, $k = 1$
Approximate (Ippolito et al. 2023)	Greedy decoding comes within a set edit distance of it	Near copies, such as a number with one digit changed

Discoverable memorization is the one you can measure, because you hold the training text it uses as the prompt. Treat any verbatim rate as a lower bound on the leak, because near copies leak too. A filter that blocks only verbatim training text is easy to bypass (Ippolito et al. 2023).

Deployed systems sample rather than decode greedily, so whether a string comes out depends on how many samples the attacker draws. A sample at temperature 1 writes out a string with

probability p, the model's probability of that string after the prompt. Write β for the chance of failure the attacker accepts. Then a number m of independent samples suffices when

$$m \geq \frac{\ln(1/\beta)}{p}. \tag{13.1}$$

For a string of probability 10^{-3} and $\beta = 0.01$, that is about 4,600 samples. So report a sampling budget alongside the greedy rate, and measure with the decoding settings you deploy.

13.3. What gets memorized, and the first defences

Memorization grows with the copies of a string. On open models, the largest reproduced at least 1% of its training data in the discoverable sense (Carlini et al. 2023). A sequence repeated ten times is generated about a thousand times more often than one seen once (Kandpal, Wallace & Raffel 2022).

So the cheapest defence is **deduplication**, which lowers the copies of every repeated string. Models trained on deduplicated data emit memorized text about ten times less often without losing quality (Lee et al. 2022). Deduplicate before you plant the canaries of the next section, so that only they carry known copy counts.

The second defence removes personal data before training. A detector of personally identifiable information (PII), such as Microsoft's Presidio, finds names, phone numbers and similar fields so you can mask them, but it misses whatever its patterns do not cover. Neither defence helps a secret seen once in a large model, where one copy is enough. Only fine-tuning with differential privacy, which limits how much any one training sequence can change the model, gives a guarantee for such singletons.

13.4. Auditing with canaries and exposure

Discoverable memorization is all or nothing, while an auditor wants a graded answer. Carlini et al. (2019) plant **canaries**, random secrets of a fixed format such as "my pin is" followed by five random digits. After training, you order all the values the secret could have taken from the most to the least likely under the model. The planted value's **rank** in that order is the number of guesses an attacker trying them in turn needs.

Write $\hat{r}$ for the planted value and $|\mathcal{R}|$ for the number of values in the set $\mathcal{R}$ it could have taken. The **exposure** is the number of bits of search the model saves an attacker,

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{13.2}$$

Each bit halves the guesses the attacker needs. At rank 1 the exposure reaches its maximum, $\log_2 |\mathcal{R}|$, and the secret can be extracted. For five digits, 10^5 values, the maximum is 16.61 bits.

Exposure works as an audit because its behaviour is known when nothing was memorized. A canary's value is drawn uniformly at random, so a model that never saw it puts it at a uniformly random rank. So for any threshold of t bits,

$$\Pr\left[\text{exposure}(\hat{r}) \geq t\right] \leq 2^{-t}. \tag{13.3}$$

So $2^{-\text{exposure}}$ bounds the p-value, the chance that an uninserted canary would look at least this memorized. The same uniform rank gives an uninserted canary an average exposure of $\log_2 e$, about 1.44 bits, which the controls should match.

To run the audit, draw canaries in the format of the secrets your data could contain, since a format unlike your data measures how the model treats unfamiliar text. Insert each one a chosen number of times, from once to many, keep some never inserted as controls, and fine-tune as usual. Then score each canary's value, and many random values of the same format, by the model's log-probability after the prefix. The share of random values that score at least as high estimates the **rank fraction**, the rank divided by $|\mathcal{R}|$, and minus its base-2 logarithm is the exposure.

Scoring is where audits most often go wrong. A candidate's log-probability is the sum over its tokens, but the mean loss that Hugging Face models return divides that sum by the number of tokens. Values of different token lengths then get scores on different scales, and the ranking is wrong. So score every candidate by its summed log-probability, in a format where every candidate has the same number of tokens. The script practitioner_lora.py fine-tunes the small language model Pythia-14m with low-rank adapters (LoRA) from `peft`. It spaces the digits of each pin, as in "the pin of Alice is 8 6 5 2 3", so that each digit is one token. The exposure functions come from the book's module practitioner_privacy_tests.py, which follows TensorFlow Privacy.

```
@torch.no_grad()
def secret_scores(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str, secrets
    """Summed log-probability of each secret's five digit tokens after the prefix."""
    prefix_ids = tokenizer(prefix)["input_ids"]
    rows = [prefix_ids + tokenizer(" " + s)["input_ids"] for s in secrets]
    ids = torch.tensor(rows)  # every secret is five tokens, so the rows have equal length
    logits = model(input_ids=ids[:, :-1], logits_to_keep=5).logits  # the predictions of the
    logprobs = logits.log_softmax(dim=-1).gather(-1, ids[:, -5:, None])
    return logprobs.sum(dim=(1, 2)).double().numpy()


def canary_exposure(
    model: PeftModel, tokenizer: PreTrainedTokenizerBase, canary: Canary, candidates: list[s
) -> tuple[float, float]:
    """The exposure of the canary's secret among the candidates, measured and extrapolated p
    candidate_scores = scores_batched(model, tokenizer, canary.prefix, candidates)
    score = float(scores_batched(model, tokenizer, canary.prefix, [canary.secret])[0])
    return exposure(score, candidate_scores), extrapolated_exposure(score, candidate_scores,
```

The model trains on 1,024 sequences of Tiny Shakespeare, a public extract of Shakespeare's plays. The script inserts three canaries at each copy count from 1 to 64, keeps three as controls, and ranks each value among 2,000 random candidates. The command `uv run python code/part05-dp-ml/practitioner_lora.py` runs it in about two and a half minutes. Each line gives a group's mean exposure, that exposure extrapolated past the cap the candidates set (explained below), and how many of its secrets greedy decoding wrote out.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
```

```
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
```

Exposure rises with the copies, and the controls sit at 1.4 bits, as the 1.44 expected for them predicts. Greedy decoding extracts only the three canaries seen 64 times. Yet two of the canaries seen 4 times reach 6.9 and 8.2 bits, which no extraction test notices, so an audit measures exposure and not only extraction. The canaries fill a quarter of the training set here, which lets so small a model memorize them in a short run. In the same script, fine-tuning with differential privacy at a privacy level $\varepsilon = 8$ extracts nothing (Fine-tune a language model privately).

The cap comes from sampling the candidates. A canary that beats all 2,000 shows only that its rank fraction is below about 1 in 2,000. So its exposure stops at $\log_2 2000 = 10.97$ bits, which the 64-copy canaries reach. Past the cap, the script fits a skew-normal distribution to the candidates' scores and reads the canary's rank off its tail, as Carlini et al. (2019) do. Ties hide memorization too, because when every candidate gets the same score, every exposure is 0.

A canary repeated 16 or more times is the positive control: if it does not reach a high exposure without privacy, the run is too short to tell you anything. Pre-release privacy risk assessment sets the exposure that blocks a release.

13.5. Extraction without the prefix: generate, rank, verify

A real attacker has only the model. Carlini et al. (2021) extracted data from GPT-2 by generating many samples, ranking them by how likely each was to be memorized, and checking the top ones. Ranking is the hard step, because a model finds a string likely either because it is ordinary text or because it memorized it. A **reference model**, trained on similar data but not on the target's training set, stands in for ordinary text. Write $p_\theta(x)$ for the target model's probability of a string x, and $p_{\text{ref}}(x)$ for the reference's. Rank by the log-likelihood ratio

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x). \tag{13.4}$$

The ratio cancels the strings that any model finds likely, such as software licences and boilerplate, which raw likelihood would put first and which reveal nothing. A crude reference, such as a smaller model of the same family, works, and to audit your own fine-tuned model, the base model before fine-tuning is a natural one.

Without the prefix, an attacker extracts less than the discoverable rate suggests, but aligned production models are not safe. Nasr et al. (2023) asked ChatGPT to repeat one word forever, until it left its aligned behaviour and wrote memorized text. For about 200 US dollars of queries, they extracted over 10,000 records.

13.6. Cheat sheet

Table 13.2.: The results of this chapter.

Question	Answer
Exposure of a canary	$\log_2\lvert\mathcal{R}\rvert - \log_2$ rank, at most 16.61 bits for five digits; p-value at most $2^{-\text{exposure}}$; controls average 1.44 bits
How to score a candidate	Summed log-probability, in a format where all candidates have the same token count
Ranking generations	$\ln p_\theta(x) - \ln p_{\text{ref}}(x)$, never raw likelihood
Defences, in order	Deduplicate (`deduplicate-text-datasets`, `datasketch`), scrub PII, then differential privacy for singletons

Rules of thumb

- A verbatim rate is a lower bound on the leak, not a measure of it.

13.7. References

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and how to estimate it.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): how memorization scales with duplication, model size and prompt length.

Derivations. Every result here is derived step by step in the full edition's chapter.

14. Gradient inversion

14.1. Why this matters

A phone keyboard learns from what its users type without the text leaving the phones. A server sends the current model to each phone, which computes the gradient of the loss on its own few records and sends back only that gradient. The server averages the gradients and takes a training step. This is **federated learning (FL)**, often called private because "the data stay on the device".

Yet one client's gradient can give back all of that client's data, because it holds one derivative for every parameter, and many of them are simple functions of the input. Averaging over a batch protects much less than people expect (Section 14.6), so treat a per-client update as being as sensitive as the data it came from.

14.2. The attacker and what it sees

The attacker is the server, which knows every parameter because it sent the model out. Since it also sees each client's own update, its access is the **transcript** of Threat models and privacy games, everything training exposes. Assume first that it is **honest-but-curious**, so it only analyses what it receives. Each client holds a batch of B records and sends the average of their gradients. Write ℓ_i for the loss on record i, which depends on all the parameters θ. The client then sends

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell_i. \tag{14.1}$$

14.3. Exact recovery from a fully connected layer

The most common first layer, a fully connected layer with a bias, gives its input back by one division. Write W_1 and b_1 for its weights and bias. Each unit j multiplies the input x by its row of weights and adds its bias. Its backward signal u_j is how fast the loss responds to the unit's output, and the chain rule turns it into the weight and bias gradients

$$\nabla_{W_1} \ell = u\, x^\top, \qquad \nabla_{b_1} \ell = u. \tag{14.2}$$

So row j of the weight gradient is the input scaled by u_j, and the bias gradient of unit j is that same scale. For a single record, dividing one by the other gives the input exactly, for every unit whose bias gradient is not zero:

$$x = \frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j}. \tag{14.3}$$

Everything above the layer, including the network's depth, the loss and the label, sits inside u_j and cancels. So the attack needs no optimization and no knowledge of the data. The snippet plays one client with one image of scikit-learn's digits in PyTorch, and its last line reads the label (Section 14.4).

```
import torch
import torch.nn.functional as F
from sklearn.datasets import load_digits

torch.manual_seed(0)
images, labels = load_digits(return_X_y=True)
x = torch.tensor(images[[7]] / 16, dtype=torch.float32)  # one client's image, pixels in [0,
y = torch.tensor(labels[[7]])
first, last = torch.nn.Linear(64, 32), torch.nn.Linear(32, 10)
F.cross_entropy(last(torch.relu(first(x))), y).backward()  # the gradient the client sends
# Row j of the weight gradient is the input times the bias gradient of unit j: divide them.
j = int(first.bias.grad.abs().argmax())
recovered = first.weight.grad[j] / first.bias.grad[j]
print(f"image recovered: {torch.allclose(recovered, x[0], atol=1e-6)}")  # image recovered:
# Only the true label's logit has a negative gradient (softmax minus one-hot).
print(f"recovered label {int(last.bias.grad.argmin())}, true label {int(y)}")  # recovered l
```

The snippet divides at the unit whose bias gradient is largest, because a switched-off ReLU unit has a zero gradient, and one active unit is enough. A convolutional first layer stops the division. Its filter is reused at every position of the image, so its gradient sums one such product per image patch, which no division can separate. The attacker then has to search (Section 14.5).

14.4. Label recovery from the last layer

The label is often the most sensitive part of an example, such as a diagnosis. It can be read off the last layer without search if that layer has a bias and feeds the softmax and cross-entropy loss (Zhao, Mopuri & Bilen 2020). Write p for the predicted probabilities and e_y for the vector with a 1 at the position of the label y. The gradient of the last layer's bias b_2 is then $p - e_y$. Every entry of p lies between 0 and 1, so only the entry at the true label is negative, and for a single record

$$y = \arg\min_k (\nabla_{b_2} \ell)_k, \tag{14.4}$$

which is the snippet's `argmin`. It fails only for a very confident model: if the true label's probability p_y rounds to 1 in floating point, that entry becomes zero, and no entry is negative.

A batch hides which record has which label, but not how many records each class has. Each entry of the bias gradient is then the class's average predicted probability minus its share of the batch. So a server that estimates the average prediction on data of its own can read off each class's count. In the full edition's experiment, this gave every count exactly in all batches of 4, and in 52.5% of batches of 64.

14.5. Deep Leakage from Gradients

When the division does not apply, the attacker searches. **Deep Leakage from Gradients (DLG)**, from Zhu, Liu & Han (2019), guesses inputs and computes the gradient they would produce. It then moves the guesses until that gradient matches the observed gradient g. Write x'_i for the guessed inputs, y_i for their labels, read first from the last layer, and $\ell(\theta; x', y)$ for the loss of one guess. The attack minimizes the **matching loss** R, the squared distance between the guesses' average gradient and g:

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \Big\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2. \tag{14.5}$$

In PyTorch, `create_graph=True` in `torch.autograd.grad` makes the guess's gradient differentiable, so an optimizer such as Adam can follow the slope of R back to the input.

Two things decide whether the search works. The activation's slope must vary, because the search differentiates a gradient, and the ReLU's slope is constant on each side of zero. So Zhu et al. used the sigmoid. And because R is not convex, the search can get stuck. So run it until R stops falling, and judge the final value against the squared size $\|g\|^2$ of the observed gradient. In double precision, a successful run in the full edition ends near 10^{-27} and a stuck one near 10^{-5}. Another input can also give a small R, so restart from several random guesses, and trust an image only when the restarts agree on it.

14.6. Batches and averaging

Clients usually send the average over a batch, or the change in their parameters after several training steps on the device (**local steps**). Averaging breaks the division and slows the search, but much of the structure survives. Each record adds one outer product to the first-layer gradient, so every row of that gradient is a combination of the B inputs. The server therefore learns their **span**, the set of all such combinations, which contains every record.

The division still recovers a record exactly when some ReLU unit is switched on for that record only, because the other records then add nothing to that unit's row. Such a unit **isolates** the record. If units switched on like independent fair coins, a unit would isolate a given record with probability 2^{-B}, so isolation would become rare as the batch grows. But on images that resemble one another, a unit fires on many records or on few, so far more units fire on just one. In the full edition's network with 1,024 hidden units and batches of 16 synthetic 8×8 images, the coin model predicts that 1.6% of the batch is exposed, but 69.1% was.

Table 14.1 gives the share exposed in the same experiment. It falls with a larger batch and rises with a wider first layer, so measure at your own width and batch size, report both, and treat any isolated record as a breach.

Table 14.1.: Isolation attack on synthetic images, from the full edition.

Hidden units	Batch size	Share of the batch recovered exactly
64	4	0.838
64	8	0.550

Hidden units	Batch size	Share of the batch recovered exactly
64	16	0.144
1,024	16	0.691

DLG suffers more from batching: on the full edition's sigmoid network with 32 hidden units, the median run on a batch of 4 gets stuck. That is a harder search, not less information, since the isolation attack recovered all four images of such a batch. Local steps do not protect a single record either. Each step's first-layer gradient is an outer product with the same input, and only the backward signal changes, so the division still works on the total change. And a **malicious** server that chooses the weights can plant units that fire only on rare inputs, each isolating one record. It then recovers most records of a batch of any size (Boenisch et al. 2023; Fowl et al. 2022).

14.7. What to do as a deployer

Table 14.2 reads off the risk of an FL deployment from who can see one client's update.

Table 14.2.: Gradient leakage by FL setting, and the response.

Setting	What leaks to the server	What to do
One record per update, fully connected first layer with bias	The input exactly, and the label	Never deploy without protection
Batches of any size, or several local steps	Label counts, the span of the batch, isolated records, and the rest by search in small batches	Treat each update as raw data
Server not trusted to choose weights	Most records of any batch	Batch size is no defence; treat each update as raw data
Updates protected by differential privacy (DP)	Bounded per client by the DP guarantee	The only protection with a guarantee

Secure aggregation is a cryptographic protocol that lets the server see only the sum over many clients, which removes the per-client view every attack here uses. But a sum over very few clients is close to a single update, so set a minimum cohort size, and make sure that clients dropping out mid-round cannot shrink a sum below it.

Secure aggregation alone is not enough. The sum is still a batch average that the attacks above can read, the final model still leaks like any model, and a malicious server can still choose its weights. For a guarantee, add DP with the client as the unit of privacy. Clipping each client's update to a fixed maximum norm and adding noise to the sum bounds what any one client's data can change (User-level privacy). Flower and TensorFlow Federated offer both protections. Heuristic defences such as gradient pruning raise the cost of the search but carry no guarantee.

To assess your deployment, find out whether the server sees single updates or only sums, and with what minimum cohort. Then run the attacks on your own architecture, batch size and number of

local steps, for example with Geiping's open-source `breaching` framework for PyTorch. If any client's update is visible on its own, block the deployment until secure aggregation or client-level DP is in place, and record the verdict in the Pre-release privacy risk assessment.

14.8. Cheat sheet

Table 14.3.: The results of this chapter.

Attack	Formula	Needs	Fails when
Exact recovery, one record	$x = (\nabla_{W_1}\ell)_{j,:}/(\nabla_{b_1}\ell)_j$	Fully connected first layer with bias	Convolutional first layer; batches, in part
Label, one record	$y = \arg\min_k(\nabla_{b_2}\ell)_k$	Softmax and cross-entropy	p_y rounds to 1
Deep Leakage from Gradients	Minimize the matching loss R	An activation whose slope varies; the labels	Search stuck (final $R \gg 0$); larger batches
Isolation, batch	Isolated records recovered exactly	Fully connected first layer, ReLU units	Rarely on real data

Rules of thumb

- Secure aggregation needs a minimum cohort, and only client-level DP bounds what any update reveals.

14.9. References

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019): the gradient-matching attack.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020): cosine matching with an image prior, on trained networks, local steps and batches.

Derivations. Every result here is derived step by step in the full edition's chapter.

15. Linkage and re-identification

15.1. Why this matters

A hospital shares its discharge records with researchers after deleting the names, but keeps the postcode, the date of birth, the sex and the diagnosis. Anyone with a voter roll, which lists names next to those same three columns, can join the two tables and put a name back on most rows. This is a **linkage attack**, and putting a name back on a "de-identified" record is **re-identification**.

Whether a table can be linked is judged by counting identifying information in bits. Each bit halves the candidates, so singling out one person in a population of n people takes $\log_2 n$ bits, 27.9 bits for the United States of 1990. Almost everyone is unique once the attributes an adversary knows carry a few bits more than that, and a unique record can be linked.

15.2. Quasi-identifiers and the linkage attack

Direct identifiers, like a name or an email address, name a person on their own. **Sensitive attributes**, such as a diagnosis, are what the release is about. **Quasi-identifiers** sit in between: none of them names anyone alone, but together they often single out one person, and they appear next to names in other datasets.

To score a release you hold, group its n records into **equivalence classes**, the sets of records with identical quasi-identifiers, and write J for the number of classes. Suppose an adversary knows the target's quasi-identifiers and that the target is in the release. If the target is alone in its class, the adversary names its record for sure, and if the class holds k look-alikes, one time in k. A class of k records holds k targets, each named with probability $1/k$, so every class adds one win whatever its size. Averaged over all n targets, the chance of naming the right record is therefore

$$\Pr[\text{win}] = \frac{J}{n}, \tag{15.1}$$

and no adversary that sees only the quasi-identifiers can do better. This is the **prosecutor model**, the worst case, because it assumes the adversary knows the target is in the data.

The script practitioner_table_risk.py shows how fast uniqueness grows on a real table, Adult, the 48,842 people of the 1994 US Current Population Survey in the UCI repository. Adult has no ZIP code or birth date, so the script starts from age, sex and race. It adds one quasi-identifier at a time, with native country as the only geography. For each set it prints the share of unique records, the share in classes smaller than 5, the smallest class size and J/n.

```
n = df.height
for qis in QI_SETS:
    # One row per equivalence class (records that agree on every quasi-identifier), with its
    sizes = df.group_by(qis).len()
    unique = sizes.filter(pl.col("len") == 1).height / n  # share of records alone in their
    small = sizes.filter(pl.col("len") < 5)["len"].sum() / n  # share in classes smaller tha
    min_k = sizes["len"].min()  # the table is k-anonymous for this k
    print(
        f"{' + '.join(qis)}: unique={unique:.2%}  in classes <5={small:.2%}  "
        f"min k={min_k}  J/n={sizes.height / n:.3f}"
    )
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`, which prints:

```
age + sex + race: unique=0.12%  in classes <5=0.75%  min k=1  J/n=0.012
age + sex + race + native-country: unique=3.07%  in classes <5=7.83%  min k=1  J/n=0.060
age + sex + race + native-country + marital-status: unique=5.88%  in classes <5=11.99%  min
age + sex + race + native-country + marital-status + education: unique=14.64%  in classes <5
age + sex + race + native-country + marital-status + education + hours-per-week: unique=36.5
```

Three columns that look harmless leave 0.12% of the records unique, and six leave 14.64%. Adding the hours worked per week, which nobody would list as an identifier, makes 36.55% unique and puts more than half the records in classes smaller than 5. So add every column an adversary could know to the test, one at a time, because the one that breaks the table may be one nobody suspected. A minimum class size of 5 is a common convention, yet every set here has classes of a single record.

15.3. Pseudonyms and hashed identifiers

Many releases replace each identifier by a **pseudonym**, a code that stays the same across a person's records, and such codes fail in two ways. First, a hash without a secret key, such as plain MD5, can be reversed whenever the possible identifiers can be listed, because the adversary can hash every candidate and look each pseudonym up. New York City hashed its taxi drivers' licences this way in its 2013 trip release. There were only about 2 million possible licences, so in 2014 Vijay Pandurangan hashed them all in minutes, and public lists of licence holders then named each driver.

The second failure survives any key. The pseudonym ties a person's records together, so linking any one of them reveals all the others. In the taxi data, a pickup minute and place single out one trip, which a passenger who knows who drove it can find. The driver's pseudonym, even a random one, then reveals every trip that driver made. This is why the General Data Protection Regulation (GDPR) still counts pseudonymized data as personal data.

If you must keep a join key, derive it with a keyed hash, a hash-based message authentication code (HMAC), and keep the key away from the data.

```
import hashlib
import hmac
import secrets

licence = "482913"  # a six-digit licence number, the identifier being pseudonymized
pseudonym = hashlib.md5(licence.encode()).hexdigest()  # what an unkeyed release prints

# Dictionary attack: hash all 10**6 possible licences once, then look the pseudonym up.
lookup = {hashlib.md5(f"{i:06d}".encode()).hexdigest(): f"{i:06d}" for i in range(10**6)}
print(lookup[pseudonym])  # 482913

# A keyed hash still gives one code per person, but without the key nobody can build the tab
key = secrets.token_bytes(32)  # store it apart from the data; draw a new one for each relea
keyed = hmac.new(key, licence.encode(), hashlib.sha256).hexdigest()
print(keyed in lookup)  # False
```

The key stops the dictionary attack but not the second failure, since each person's records still form an identifying history. Reusing the key would join that history across releases, so draw a fresh key for each release.

15.4. How many people are unique?

For a release still being designed, there is no table to count, so estimate uniqueness by adding bits. An attribute with v equally likely values carries $\log_2 v$ bits, and the bits of independent attributes add. Attributes that carry b bits in total split people into 2^b cells. If each person lands in a cell at random, every cell equally likely, the expected share of the n people who are alone in their cell is about

$$\frac{\text{unique people}}{n} \approx \exp\left(-2^{\log_2 n - b}\right). \tag{15.2}$$

The share depends only on the bits beyond $\log_2 n$, and it rises steeply with them. Half the people are unique at 0.53 bits beyond $\log_2 n$, and 99% at 6.64 bits.

For example, ZIP code, birth date and sex carry 31.1 bits, a few more than the 27.9 bits that single out one American. The formula then predicts that 0.899 of Americans are unique, close to the 87% that Latanya Sweeney estimated from the 1990 census. Real attributes are skewed and correlated, which usually lowers uniqueness, so the estimate errs on the side of caution.

15.5. Sparse data: the Netflix attack

Ratings, purchases and locations have no quasi-identifier columns, yet they identify people even better. Netflix released about 100 million movie ratings in 2006, with the subscribers' names replaced by random numbers. Narayanan and Shmatikov (2008) matched its records to public reviews with a score that adds up the items on which a record agrees with what the adversary knows. Each item is weighted by its rarity, because agreeing on an obscure film says more than

agreeing on a hit. With this score, eight ratings with dates known to within 14 days identified 99% of the records, even when two of them were wrong.

The best score always points at some record, even when the target is not in the release. So the **eccentricity test** declares a match only if the best score beats the second best by a margin, counted in standard deviations of all the scores. Calibrate that margin on people known not to be in the data: on the full edition's simulated ratings, the original attack's margin also matched 26.3% of such outsiders. So report the attack's success at a stated false positive rate (Hypothesis testing).

15.6. Does releasing a sample protect?

Agencies often release a sample, as Adult is, arguing that a record unique in the sample may share its attributes with someone left out. The argument is weaker than it sounds. Suppose the adversary knows a person and finds exactly one matching record in a release that holds a share q of the population, the **sampling rate**. Write λ for the expected number of other people in the population who share the target's cell. The record found can be a look-alike's only if the target was left out, which happens with probability $1 - q$. So for a rare cell, the probability κ that the match is right is about

$$\kappa \approx \frac{1}{1 + \lambda(1 - q)}. \tag{15.3}$$

Take a rare cell, one expected to hold $\lambda = 0.1$ other people. Even a 1% sample leaves a match right with probability 0.910. Sampling enters only through $1 - q$, which can never exceed 1, so the chance of a right match never falls below $1/(1 + \lambda)$, and shrinking the sample barely helps. Sampling protects only people whose cells are crowded anyway.

So the Adult figures above are **sample uniqueness**, exact when the target is known to be in the release. The **population uniqueness** behind them is lower, but not by much for rare cells, so estimate it with a model rather than assume the risk away. ARX and sdcMicro offer such estimates, and Rocher, Hendrickx and de Montjoye (2019) model correlated attributes for it.

15.7. A pre-release risk check

A quasi-identifier is any column an adversary could find next to a name elsewhere, such as a date, a location or a rare event, so list them all. No tool can do this for you, because no tool knows what auxiliary data exist. To lower the risk, coarsen geography and dates first, since they carry the most bits. Then suppress the records that remain unique (Syntactic anonymity and how it breaks). If the analysis needs fine-grained or sparse data, release a differentially private output instead (Release statistics with a library). Pre-release privacy risk assessment runs this check with thresholds fixed before the counts are seen.

15.8. Cheat sheet

Table 15.1.: The results of this chapter.

Question	Result	Use it for
Bits to name one person	$\log_2 n$	27.9 for the US of 1990
Risk of a release you hold	Success J/n; at most $1/k$ if every class has at least k records	Prosecutor model
Share unique, uniform cells	$\exp(-2^{\log_2 n - b})$	50% at $\log_2 n + 0.53$ bits, 99% at $\log_2 n + 6.64$
Releasing a sample	$\kappa \approx 1/(1 + \lambda(1 - q))$	0.910 at $\lambda = 0.1$ in a 1% sample

Rules of thumb

- Add quasi-identifiers one at a time, because one innocent column can more than double the unique share.
- Estimate uniqueness for the population, not just the sample.
- Hashing an identifier is not anonymization. Key the hash, and use a new key for each release.

15.9. References

- Sweeney, "Simple Demographics Often Identify People Uniquely" (2000): the 87% estimate and its variants with coarser geography.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (2008): the Netflix attack.
- Rocher, Hendrickx & de Montjoye (2019), in *Nature Communications*: population uniqueness from a sample.

Derivations. Every result here is derived step by step in the full edition's chapter.

16. Syntactic anonymity and how it breaks

16.1. Why this matters

Linkage and re-identification showed that deleting names does not anonymize a table, because quasi-identifiers such as age, postcode and sex single out many people. The classic answer is to coarsen the table until every person hides in a crowd. The definitions that make this precise are called **syntactic**, because you check each one by reading the published table. Statistics offices and tools such as ARX and sdcMicro still use them, so you will meet them in audits. But each holds for one table against an adversary whose knowledge is fixed in advance, so one unanticipated fact, or a second release of the same people, voids it.

16.2. What the adversary believes about a class

To anonymize a table, the custodian, the organization that holds the data, applies a **generalization**, which turns an age into a range or a postcode into a prefix. Records with the same generalized quasi-identifiers form an **equivalence class**. An adversary who knows the target's quasi-identifiers finds the target's class E but cannot tell its records apart. So its belief about the target's sensitive value, its **posterior**, is the share of the class with that value. Write $|E|$ for the size of the class and $n_E(v)$ for the number of its records with a sensitive value v. The posterior that the target's sensitive value, written S_{target}, equals v is

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|}. \tag{16.1}$$

Background knowledge works by elimination: ruling out a value crosses out its records, and the posterior is recomputed over those that remain. Every attack in this chapter is this calculation.

16.3. k-anonymity and what it costs

k-anonymity (Sweeney and Samarati, 1998) asks that every class hold at least k records, for a crowd size k the custodian chooses. Generalization gets there, together with **suppression**, which deletes the records that still stand out. In the previous chapter, an adversary who knows the target is in the release links it with probability J/n, the number of classes J over the number of records n. At least k records per class means at most n/k classes, so

$$\Pr[\text{win}] = \frac{J}{n} \le \frac{1}{k}. \tag{16.2}$$

The guarantee covers names only, and only against an adversary who knows nothing beyond the quasi-identifiers. The choice of k is a policy: a minimum of 5 is common, and some health-data rules ask for more.

Anonymizers such as **Mondrian** search for a generalization that loses little, but you can also generalize by hand and recheck. The script practitioner_table_risk.py takes the previous chapter's Adult census extract and its six quasi-identifiers. It puts age into 10-year bands, native country into US or other, and education into four levels. Then it counts the records alone in their class, and the different occupations in each class, with occupation standing in for a sensitive attribute. Both are Polars counts, since no maintained Python package computes k-anonymity or distinct ℓ.

```
general = df.with_columns(
    (pl.col("age") // 10 * 10).alias("age"),  # 10-year bands: 30 stands for 30-39
    pl.when(pl.col("native-country") == "United-States")
    .then(pl.lit("US"))
    .otherwise(pl.lit("other"))
    .alias("native-country"),
    # education-num runs from 1 (preschool) to 16 (doctorate); 9 is high-school graduate.
    pl.col("education-num")
    .cut([8, 9, 12], labels=["no diploma", "HS-grad", "some college", "bachelor+"])
    .alias("education"),
)
# Recheck, and count the occupations in each class (distinct l).
classes = general.group_by(QIS).agg(pl.len(), pl.col("occupation").n_unique().alias("l"))
unique = classes.filter(pl.col("len") == 1).height / general.height
one_job = classes.filter(pl.col("l") == 1)["len"].sum() / general.height
print(f"after generalizing: unique={unique:.2%}  in a class with one occupation={one_job:.2%
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`. Its first two output lines use the six generalized quasi-identifiers, and the third adds the raw hours worked per week.

```
after generalizing: unique=0.94%  in a class with one occupation=1.20%
age + sex + race + native-country + marital-status + education: unique=0.94%  in classes <5=
age + sex + race + native-country + marital-status + education + hours-per-week: unique=10.0
  records in classes of size >=5 with l=1: 0.03%
```

Generalizing cut the unique share to 0.94%, from 14.64% before. Yet the smallest class still holds one record, so the 3.31% of records in classes smaller than 5 must be suppressed, or the bands made coarser. And the result is fragile, because an adversary who also knows the hours worked finds 10.09% of the records unique again.

16.4. Homogeneity, background knowledge and l-diversity

k-anonymity hides who a record belongs to, but not what it says (Machanavajjhala and colleagues, 2006). If all three records of Alice's class say heart disease, her posterior is 1 without anyone

finding her record, which is the **homogeneity attack**. If Bob's class says cancer, cancer and flu, a neighbour who knows he did not have flu crosses out that record. The neighbour learns that Bob has cancer, which is the **background knowledge attack**.

l-diversity asks each class to hold enough different sensitive values. The distinct version asks for at least ℓ different values. It does not cap the most common value's share, so a class can be 3-diverse with one value in most of its records. The recursive version adds that cap, with a constant c the custodian picks, so that an adversary who rules out $\ell - 2$ values keeps a posterior below $c/(1 + c)$. That cap is already one half at $c = 1$, so even this version protects against certainty, not likelihood.

Suppressing the small classes also removes most homogeneous ones. On Adult, 1.20% of the generalized records sit in a class where everyone has the same occupation, but only 0.03% sit in such a class of 5 or more. On a synthetic hospital table of the full edition, distinct 3-diversity removed the certain disclosures at little cost, while the recursive version forced some classes to grow to 641 patients.

16.5. Measuring what a release teaches

Compare every posterior with the **imputation baseline** of Attribute inference and model inversion, the best guess of a sensitive value from the population alone. Only the success beyond it is about the individual. On the hospital table, guessing each patient's diagnosis from age alone is right for 53.6% of patients, and a 5-anonymous release raises that to 59.3%. Most of what the release teaches is the link between age and diagnosis, which any useful study would report. Only the excess of 5.7 percentage points is a leak.

16.6. Skewness, similarity and t-closeness

l-diversity misses two attacks (Li, Li and Venkatasubramanian, 2007). In the **skewness attack**, a class that is half HIV is 2-diverse, yet in a table where 1% have HIV it raises its members' posterior fifty-fold. In the **similarity attack**, a class of three different stomach diseases is 3-diverse but still reveals a stomach disease.

t-closeness answers both, by asking that the distribution of values in every class E be within a distance t of the whole table's. Write $P_E(A)$ and $P(A)$ for the chance that a value drawn from the class, or from the table, lands in a set A of values. With the **total variation (TV)** distance, no such chance, and so no belief, moves by more than t,

$$|P_E(A) - P(A)| \leq t. \tag{16.3}$$

The bound is a difference, not a ratio, so it protects rare values badly. At $t = 0.15$, a value held by 1% of the table may rise to 16%. Against the similarity attack, Li and colleagues used the **earth mover's distance (EMD)** instead, which counts similar values as close.

t-closeness costs a lot. At $t = 0.15$, guessing diagnoses from the hospital release was right for only 22.2% of patients, below the 53.6% of imputation from age alone. The release told less about the population than no release.

16.7. The composition attack

Ganta, Kasiviswanathan and Smith (2008) asked what happens when two hospitals each publish a k-anonymous or l-diverse table. A patient treated at both sits in a class E_A of one release and a class E_B of the other. The adversary keeps only the diagnoses that appear in both classes,

$$\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}. \tag{16.4}$$

When one candidate is left, the diagnosis is **pinned down**.

In the full edition's simulation, two hospitals share 6,649 patients. Their two 5-anonymous releases together pin down the diagnosis of 9.9% of them, over three times what either pins alone. With distinct 3-diversity neither release pins anyone, yet together they pin 18.

No single-table definition can stop this, because the intersection of two tables is a table that neither custodian checked. A definition that survives must compose across releases, hold whatever the adversary knows, and bound the ratio of beliefs, so that rare values are protected. The definition of differential privacy starts from these requirements, which is why statistics offices such as the US Census Bureau have moved towards it (Reconstruction attacks).

16.8. Choosing and running an anonymization

Start from the quasi-identifiers of Linkage and re-identification, since a list that is too short is the commonest failure. Use k-anonymity against linkage, and add distinct l-diversity when a class with one sensitive value would be a disclosure. Use t-closeness, the costliest, only when the sensitive attribute is skewed or has similar values.

Then audit the anonymizer's output yourself, in Polars as the excerpt does, or with pycanon, which reports the crowd size k, ℓ and t for a pandas table. Pre-release privacy risk assessment turns these checks into a release decision. Move to differential privacy when other releases of the same people exist, when the sensitive attribute is rare, or when the data are sparse. For tables of counts and sums, Release statistics with a library does the job end to end.

16.9. Cheat sheet

Table 16.1.: The syntactic definitions and the attacks on them.

Definition or attack	Condition	What it gives, and where it ends
k-anonymity	Every class has at least k records	Linkage at most $1/k$; says nothing about attributes
Distinct, entropy l-diversity	At least ℓ values, or $\log_2 \ell$ bits of entropy	No cap on the top value's share
Recursive (c, ℓ)-diversity	Top value below c times the values ranked ℓ-th and lower	Posterior below $c/(1+c)$ after ruling out $\ell - 2$ values

Definition or attack	Condition	What it gives, and where it ends
t-closeness (TV)	Each class within t of the table	A 1% value may reach $1\% + t$
Composition	Intersect the target's classes in two releases	No single-table definition prevents it

Rules of thumb

- k-anonymity protects names, not attributes, so check what each class's sensitive values reveal.
- Assume another release exists, because a guarantee about one table is not a guarantee about a person.

16.10. References

- Machanavajjhala and colleagues, "l-Diversity" (2006), and Li, Li & Venkatasubramanian, "t-Closeness" (2007): the attacks and the definitions that answer them.
- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (2008): the intersection attack.

Derivations. Every result here is derived step by step in the full edition's chapter.

17. Reconstruction attacks

17.1. What it is

A **reconstruction attack** rebuilds the records behind published aggregates, because every count is a linear equation in the secret values of the people counted. Suppose each of n people has one secret bit, listed in d. A **subset-sum query** asks how many people in a set have bit 1. The attacker's m queries are the rows of a matrix A of zeros and ones, and the **curator** who answers publishes the true answers plus an error vector e,

$$a = Ad + e. \tag{17.1}$$

Least squares finds the real numbers whose answers come closest to the published ones, then clips and rounds each to 0 or 1. Let each query include every person with probability one half, and let the noise be normal with standard deviation σ. Then least squares recovers about a fraction

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right) \tag{17.2}$$

of the bits, where Φ is the standard normal distribution function. The baseline is the **majority guess**, which gives everyone the commoner value: one half for balanced bits. With a few times more queries than people, noise well below $\sqrt{n}$ therefore reveals the data. Answering all 2^n subset sums needs error of order n to stop reconstruction (Dinur and Nissim 2003).

These results are the **Fundamental Law of Information Recovery**: answering too many questions too accurately destroys privacy. But stopping reconstruction is not privacy. A curator that answers from a secret random half of the people stops it, yet gives that half away exactly. And learning only whether someone took part needs far fewer statistics (Homer et al. 2008, Lower bounds).

17.2. When you meet it

Every published count, cross-tabulation, dashboard cell or query answer is such an equation. The Census Bureau attacked its own 2010 publications and rebuilt a record for every person. About 46% of them matched a real person exactly on block, sex, age, race and ethnicity. Linked to commercial data, they confirmed the identity of about 17% of the population, so the 2020 Census moved to differential privacy. Query systems fall too: the noise of the commercial system Diffix did not grow with the number of queries, so Cohen and Nissim (2020) averaged it away.

17.3. How to use it

To audit an aggregate release, first count the equations. If the cells or allowed queries outnumber the people, only the noise protects them. Put your noise level and number of queries into Equation 17.2, and if the prediction is far above the majority guess, run the attack. The snippet draws the noise from OpenDP's Gaussian mechanism, as a release would, and runs least squares. OpenDP's noise cannot be seeded, so it checks the share recovered against the prediction instead of printing it.

```
from statistics import NormalDist

import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib")
rng = np.random.default_rng(0)
n, m = 500, 2000  # people with a secret bit each, and random subset-sum queries
sigma = 0.25 * np.sqrt(n)  # noise sd added to each answer
secret = rng.integers(0, 2, n)
queries = rng.integers(0, 2, (m, n))
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.l2_distance(T=float)
gaussian = dp.m.make_gaussian(*space, scale=sigma)  # OpenDP's Gaussian mechanism on all m a
answers = np.array(gaussian((queries @ secret).astype(float).tolist()))
estimate = np.linalg.lstsq(queries, answers, rcond=None)[0]  # the textbook attack: least sq
recovered = np.mean(np.clip(estimate, 0, 1).round() == secret)
predicted = NormalDist().cdf(np.sqrt(m - n) / (4 * sigma))
print(f"predicted {predicted:.4f}")  # predicted 0.9584
print(f"recovered within 0.04 of it: {abs(recovered - predicted) < 0.04}")  # recovered with
```

The number of queries matters as much as the noise. In the full edition's simulation, with noise of twice $\sqrt{n}$, four queries per person recovered only 0.5875 of the bits. Yet solving Equation 17.2 at that noise shows that 106.1 queries per person recover 90%. So fixed noise falls to enough queries, and the noise must grow with the number of answers, as differential privacy makes it.

For queries that are not random, the linear-programming decoder of Dinur and Nissim fits unknowns between 0 and 1 with `scipy.optimize.linprog`. For published tables, write each cell as a constraint on whole-number unknowns, solve with `scipy.optimize.milp`, and count the solutions, since a single one rebuilds the records. The release fails when either decoder beats the majority guess.

17.4. What goes wrong

Rounding, household swapping and small-cell suppression are not noise. Rounding hides nothing, because the attacker can round the answers of each candidate dataset too. Swapping leaves most constraints exact, and a suppressed cell adds one, that the count is below the threshold. Sticky

noise, which repeats the same answer to a repeated query, only stops repeats, which the attack never needs.

Test the union of all releases, not one table at a time. And do not trust least squares alone, since errors chosen against it hurt it more than random noise. Reconstruct your own release with both decoders before publishing it, and record the result in Pre-release privacy risk assessment.

17.5. References

- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020): the attack on Diffix.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (2019): a census block rebuilt with a solver.

Derivations. Every result here is derived step by step in the full edition's chapter.

18. Pre-release privacy risk assessment

18.1. Why this matters

Before a model or a table leaves your hands, someone has to decide whether it leaks too much about the people in it. This chapter runs the attacks of this part as the assessment a privacy reviewer would ask for. Its three releases are a classifier behind a scoring API, a language model whose weights go to a partner, and a record-level census extract.

The order matters as much as the attacks. The results that block the release are written down before anything runs, because a threshold chosen after seeing the numbers can be bent to fit them. And a failed attack shows only that this attack failed, so the assessment reports how strong its attacks were.

`practitioner_lira.py` and `practitioner_table_risk.py` run in seconds on public data. The language-model step reuses the output of `practitioner_lora.py`, from Fine-tune a language model privately with LoRA, which takes a few minutes on an idle CPU.

```
uv run python code/part02-attacks/practitioner_lira.py
uv run python code/part02-attacks/practitioner_table_risk.py
uv run python code/part05-dp-ml/practitioner_lora.py
```

18.2. Fix the threat model and the thresholds first

Fill in the worksheet of Threat models and privacy games once per artefact, because each release can expose the same data differently. By default the privacy unit is the person, and the adversary sees scores, the confidences a model returns, since many services return them.

Then write the blocking rules of Table 18.1 into the plan. The membership rule uses the true positive rate (TPR), the share of members an attack flags, and the false positive rate (FPR), the share of non-members it wrongly flags. At an FPR of 10^{-3}, it blocks when the lower end of the TPR's 95% confidence interval exceeds ten times the FPR. Reading the lower end means a lucky run cannot trigger it, and the factor of ten is a policy choice: an attack may beat chance, but not tenfold.

Several remedies use differential privacy (DP), Part 3's guarantee that caps what any attack learns about one person by a privacy level ε. One is stochastic gradient descent (SGD) with each record's gradient clipped and noise added, called DP-SGD.

Table 18.1.: The blocking thresholds, written before any attack runs.

Artefact	The adversary sees	Attack	Blocks the release when	Then
Classifier behind a scoring API	Confidences for any input	Online LiRA, 64 models	Lower end of the TPR's 95% interval at FPR 10^{-3} is above 0.01	Regularize, or train with DP-SGD
Fine-tuned language model shared with a partner	The weights	Canaries, exposure, greedy extraction	Any secret extracted, or a one-copy canary's extrapolated exposure within 3 bits of the maximum	Deduplicate, or fine-tune with DP
Record-level table	The table and a register with the quasi-identifiers	Uniqueness on the agreed quasi-identifiers	Any record unique, or in a class smaller than 5 (11 for counts from US Medicare and Medicaid data)	Generalize and suppress, else release DP statistics
Model, sensitive field	The target's other fields and label	Model inversion	Attribute advantage's 95% interval above 0	Regularize, or train with DP
Model whose training mix is secret	Probe outputs	Shadow models, meta-classifier	Accuracy's 95% interval above one half	Do not release that mix
Published counts or queries	Every answer	Least squares, linear programming	Either beats the majority guess at the planned noise and query count	DP noise scaled to all answers
Federated training	Each client's update	Division and search	Any single update visible	Secure aggregation, client-level DP
Any model claimed to be DP	Depends on the claim	Audit with canaries	Certified lower bound on ε above the claimed ε	Block: the code or the accounting has a bug

A canary within 3 bits of the maximum exposure ranks among the 8 likeliest of all possible secrets. The rule reads one-copy canaries because a deduplicated corpus holds each secret once. If yours is not deduplicated, apply it at your data's largest repeat count.

The four middle rows block on any leak beyond the attack's own baseline, which the chapters on attribute, property and reconstruction attacks and gradient inversion define. In the last row, an audit with independent trials certifies a lower bound on a DP model's ε, and a bound above the claim proves a bug (Auditing private training).

18.3. Classifier: the free check

Compare the model's accuracy on its training data and on held-out data. Then run the loss attack, which says "member" whenever the loss on the true label is low (Membership inference). The model is a logistic regression on scikit-learn's digits, 1,797 small images of handwritten digits. The script trains 64 copies, each on a random half of the images. Each copy in turn plays the target, the model under assessment, while the others serve as shadow models in the next step.

```
target 0: train accuracy=1.0000  test accuracy=0.9650
train-test gap, mean over 64 targets=0.0422
ART rule-based attack on target 0: TPR=1.0000  FPR=0.9650  advantage=0.0350
loss attack at FPR 0.001: TPR=0.0009
loss attack at FPR 0.01: TPR=0.0105
```

The first target gets every training image right, but only 0.9650 of its test images. So even the rule "member if correctly classified" beats chance, by the mean gap of 0.0422. That is a free lower bound on the leak, though it does not say who is exposed. The Adversarial Robustness Toolbox (ART), a Python library of attacks, runs that rule as `MembershipInferenceBlackBoxRuleBased`, and its advantage of 0.0350 on the first target is that model's gap. The loss attack, at an FPR of 10^{-3}, flags no more members than non-members. A weak attack's failure proves little, so this check can never clear a model. A large gap says regularize first, and a small one only that you need the next step.

18.4. Classifier: LiRA with shadow models

The likelihood ratio attack (LiRA) of Membership inference compares the target's output on a record with the outputs of **shadow models**. These are trained like the target on random halves of the data, some with that record and some without. LiRA works with the logit ϕ of the confidence p in the true label,

$$\phi = \ln \frac{p}{1-p}, \tag{18.1}$$

which is close to normal across retrainings. For each image, it fits a normal to ϕ over the shadows that trained on it (IN) and another over those that did not (OUT). Online LiRA scores the target's ϕ by its log-likelihood ratio between the two fits. Offline LiRA uses only the OUT fit, and counts how many OUT standard deviations ϕ lies above the OUT mean. ART's `SklearnClassifier` trains every model, and the helper practitioner_privacy_tests.py computes the LiRA scores.

Four decisions set the attack's strength. Use online LiRA, the strongest, for a release decision. Train 64 shadows when training is cheap; with 16 or fewer, fit one variance shared by all images, because per-image variances from so few models are too noisy. Put each record in each model by

a fair coin, so that every record has IN and OUT shadows. And let each model serve once as the target, with the other 63 as its shadows. That gives 57,704 member and 57,304 non-member pairs of a model and an image.

At an FPR of 10^{-3}, the 57,304 non-member pairs allow only about 57 false positives, so the script reads the rate with care. It takes the best TPR among thresholds whose FPR stays at or below the target. Then it puts a Clopper–Pearson interval, the exact binomial interval of Hypothesis testing, on each rate.

```
# Every (target model, image) pair is one trial; members are the positives.
fpr, tpr, thresholds = roc_curve(member.ravel(), scores.ravel())
# The best TPR among thresholds whose FPR does not exceed the target.
i = int(np.flatnonzero(fpr <= target_fpr)[-1])
n_members, n_nonmembers = int(member.sum()), int((~member).sum())
true_pos = round(tpr[i] * n_members)
false_pos = round(fpr[i] * n_nonmembers)
# Exact 95% intervals (Clopper-Pearson) for both rates.
tpr_ci = clopper_pearson(true_pos, n_members)
fpr_ci = clopper_pearson(false_pos, n_nonmembers)
```

```
  loss attack: AUC=0.518  TPR@0.001=0.0009  TPR@0.01=0.0105
  ART shadow-model attack: AUC=0.570  TPR@0.001=0.0014  TPR@0.01=0.0146
  LiRA offline: AUC=0.577  TPR@0.001=0.0087  TPR@0.01=0.0345
  LiRA online: AUC=0.623  TPR@0.001=0.0318  TPR@0.01=0.0790
[At FPR 0.001: counts, exact 95% intervals, ln(TPR/FPR) as a point estimate, not a certified
  loss attack: TP=53/57704  TPR=0.0009 [0.0007, 0.0012]  FP=57/57304  FPR=0.00099 [0.00075,
  ART shadow-model attack: TP=82/57704  TPR=0.0014 [0.0011, 0.0018]  FP=57/57304  FPR=0.0009
  LiRA offline: TP=502/57704  TPR=0.0087 [0.0080, 0.0095]  FP=56/57304  FPR=0.00098 [0.00074
  LiRA online: TP=1833/57704  TPR=0.0318 [0.0303, 0.0332]  FP=57/57304  FPR=0.00099 [0.00075
```

By area under the curve (AUC) the four attacks look alike, but at an FPR of 10^{-3} they differ sharply. Online LiRA flags 1,833 members, a TPR of 0.0318. The lower end of its interval, 0.0303, is above 0.01, so the model fails Table 18.1. Offline LiRA would have passed it, and so would ART's attack of Shokri et al. (2017), which learns one rule for all images from four shadow models of its own. So a release decision rests on the strongest attack you can afford.

A model with privacy level ε keeps the TPR within about e^{ε} times the FPR. So the last column, ln(TPR/FPR), estimates the ε each attack implies. It is not a certified lower bound, because pairs that share a model are not independent trials (Auditing private training).

The script also prints the ten images an attacker exposes first, those whose IN and OUT fits lie furthest apart. Inspect them, since such records are usually rare or mislabelled.

Three things go wrong here. Shadows trained with another recipe describe another model, which weakens the attack. Too few non-members leave the FPR uncertain, and even here its interval runs from 0.00075 to 0.00129. The script also sets thresholds and measures on the same pairs, which flatters the attack slightly, where Hypothesis testing uses two splits. And a confidence of 1 makes ϕ infinite, so compute ϕ from the logits, the model's raw outputs. Clipping the confidence instead hides a confident model's most exposed records (Membership inference).

18.5. Fine-tuned language model: canaries and extraction

A language model can repeat a secret word for word, which a membership test does not measure. So the assessment plants **canaries**, random secrets of a fixed format, and checks how well the trained model knows them (Memorization and training-data extraction). The run adapts Pythia-14m with low-rank adaptation (LoRA) to 1,024 sequences of Tiny Shakespeare, once without privacy and once with DP at $\varepsilon = 8$ per sequence.

The canaries should look like the secrets you fear. Each reads "the pin of", a name such as Alice, "is", and five random digits, spaced so that every possible secret is exactly five tokens. Three are inserted once, and three more at each of 4, 16 and 64 copies. Three controls are drawn the same way and never inserted. The canaries make up a quarter of the training set, far denser than real secrets, so even this small model memorizes them. The non-private model is then a positive control, one known to leak.

Each canary is scored in two ways, with the functions of Fine-tune a language model privately with LoRA. Exposure ranks the secret among random secrets of the same format by the summed log-probability of its five digit tokens. It turns the rank into bits, as $\log_2$ of the number of candidates minus $\log_2$ of the rank. Sum rather than average, since the mean loss that `transformers` returns misranks secrets of different token counts. Extraction gives the model the prefix and lets it write the likeliest token five times, which is greedy decoding. Run it with the partner's decoding settings, and check whether it wrote the secret.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
```

Ranking among 2,000 random candidates caps the measured exposure at 10.97 bits, below the 16.61 the whole space of secrets allows. So the measured column can never reach the blocking line 3 bits below that maximum. Only the extrapolated column can, by fitting a skew-normal distribution to the candidates' scores and reading how far into its tail each secret lies. A random rank is worth $\log_2 e = 1.44$ bits on average, and the controls' 1.4 bits match it, so the measurement is sound. The one-copy canaries stay far below the line, at 2.0 bits. But greedy decoding writes out all three 64-copy secrets, so the non-private model fails Table 18.1 and must not be shared.

The 4-copy canaries show why you measure both, since two reach 6.9 and 8.2 bits yet neither is extracted. The DP run extracts nothing, but its most repeated canaries still average about 4 bits, against 2.2 for its controls. That is because ε protects one sequence, and a repeated secret is many sequences. So deduplicate before training, whatever else you do.

The usual failures are a canary format unlike your data, no controls, and ranking by a mean loss. For a release, also measure the share of training sequences that greedy decoding completes from their start, the discoverable rate, which the script skips (Memorization and training-data extraction).

18.6. Table: re-identification on quasi-identifiers

Quasi-identifiers are the columns an adversary can find elsewhere, such as age, sex and country in a register (Linkage and re-identification). Records that agree on all of them form an **equivalence class**, and a record alone in its class can be linked to a name. The table is Adult, a census extract from the UCI Machine Learning Repository, with 48,842 records and an agreed set of six columns, from age to education.

For each set of columns, the script prints the share of unique records and the share in classes smaller than 5. It also prints the smallest class size k, and J/n, the number of classes per record (Linkage and re-identification shows the loop).

```
age + sex + race: unique=0.12%  in classes <5=0.75%  min k=1  J/n=0.012
age + sex + race + native-country: unique=3.07%  in classes <5=7.83%  min k=1  J/n=0.060
age + sex + race + native-country + marital-status + education: unique=14.64%  in classes <5
age + sex + race + native-country + marital-status + education + hours-per-week: unique=36.5
```

On the agreed six, 14.64% of the records are unique, and hours per week, a column that looks harmless, raises that to 36.55%. So the list must include every column an adversary might hold, and when in doubt, test with the column added.

The standard remedy is to generalize: age into 10-year bands, country into United States or other, and education into four levels. The script then rechecks the classes and counts the distinct occupations in each, the distinct ℓ of Syntactic anonymity and how it breaks, with occupation as the sensitive column.

```
after generalizing: unique=0.94%  in a class with one occupation=1.20%
age + sex + race + native-country + marital-status + education: unique=0.94%  in classes <5=
age + sex + race + native-country + marital-status + education + hours-per-week: unique=10.0
```

Generalizing cuts the unique share from 14.64% to 0.94%, but the smallest class still has one record, so the table still fails Table 18.1. Suppressing the classes smaller than 5 would remove 3.31% of the records. That would hold only for these columns, since raw hours per week makes 10.09% unique again. And 1.20% of the records sit in a class with a single occupation, which reveals it without any unique match.

These shares count uniqueness in the sample, which fits the prosecutor model, where the adversary knows the target is in the table. Otherwise, estimate population uniqueness (Linkage and re-identification).

Even a table that passes after suppression can be intersected with a second release of the same people (Syntactic anonymity and how it breaks). And k-anonymity says nothing against richer side information. So when a table cannot pass without losing its use, release statistics with DP, as Release statistics with a library does. Plain aggregates are no fallback, because many accurate counts let an attacker rebuild records (Reconstruction attacks).

18.7. Write the decision

The last step compares each result with its threshold and writes a release memo, Table 18.2. A reviewer who did not run the attacks must be able to check it, so it gives each estimated rate with its interval.

Table 18.2.: A release memo for the three artefacts.

Memo entry	This assessment
Threat model	Scores from an API; weights shared with a partner; a table linked with a register. Unit: the person
Attacks and their strength	Loss attack, offline and online LiRA with 64 models; 12 canaries and 3 controls with exposure and greedy extraction; uniqueness on six quasi-identifiers plus one
Results	Online LiRA TPR 0.0318 [0.0303, 0.0332] at FPR 0.00099; all three 64-copy secrets extracted; 14.64% unique, 0.94% after generalizing
Verdict	All three blocked
Mitigation	Classifier: regularize or train with DP-SGD, then rerun. Language model: deduplicate and fine-tune with DP. Table: release DP statistics
Residual risk	Not assessed: attribute inference, property inference and reconstruction from aggregates. Not applicable, with no federated training: gradient leakage

The residual-risk entry, the one most often left out, names the attacks that were not run. Write it even when every attack fails, because a model can pass this assessment and still leak to an attack nobody tried. Only a DP guarantee bounds every attack at once. For a DP model the memo carries that guarantee and its evidence, which Release and governance assembles into the release package.

18.8. Checklist

Table 18.3.: The steps of a pre-release risk assessment, their defaults and their checks.

Step	Default	Check
Threat model and thresholds	Person as unit; adversary sees scores; rules of Table 18.1	Written and dated before any attack runs
Free check	Train and test accuracy; loss attack	Gap and loss TPR at FPR 10^{-3} reported; a pass clears nothing
LiRA	Online, 64 models, fair-coin inclusion, target role rotated	TPR's interval at FPR 10^{-3} against 0.01; enough non-members; top records inspected
Canaries	Secret format, 1 to many copies, uninserted controls	Controls near 1.44 bits; summed log-probability; deployed decoding
Table	Every column an adversary may hold; classes of at least 5	Unique share and smallest class after generalizing; distinct ℓ on the sensitive column
Decision	Memo with intervals, verdict, mitigation and residual risk	Every attack not run is named

18.9. References

- Carlini et al., "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, and why attacks are reported by their TPR at a low FPR.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and the skew-normal extrapolation.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications 2019): population uniqueness from a sample.

Derivations. This workflow has no counterpart in the full edition. It draws on the full edition's chapters on threat models in the full edition, membership inference in the full edition, memorization and extraction in the full edition, linkage and re-identification in the full edition and syntactic anonymity in the full edition.

Part III.

Differential privacy foundations

Part 2 showed that anonymization and access control do not stop releases from leaking. This part introduces **differential privacy (DP)**, the defence the rest of the book builds on. It is a property of the release process, not of a dataset. Changing one person's data may change the probability of any output by at most a factor e^{ε}, set by a privacy level ε, plus a small slack δ. That one condition caps every attack of Part 2. So a DP claim must state ε, δ and what counts as one person's data, the unit of privacy, such as a record or a user.

Table 18.4.: Notation used throughout Part 3.

Symbol	Meaning
ε, δ	Privacy level and slack
$\Delta_1 f$, $\Delta_2 f$	Sensitivity, the most one record moves the answer
$\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$	Laplace noise of scale b, normal noise of standard deviation σ

The part teaches one workflow. First fix the unit of privacy and the neighbouring relation (the definition of differential privacy). Then bound each person's contribution and calibrate a mechanism to it (The Laplace and Gaussian mechanisms). Finally, account for every release, since only post-processing is free (Post-processing, group privacy and composition). The last page, Release statistics with a library, runs this workflow with OpenDP on a real shop's invoices, from bounding each customer's lines to the written guarantee.

The full chapters in between cover common tasks. Selection picks the best of many options or a private median, and Query release publishes many counts at once. Trust models apply when nobody can be trusted with the raw data, and Private mean estimation says how many records a private mean needs.

Four short reference pages cover the rest. Advanced composition and Beyond global sensitivity matter when the simple rules are too loose. Read Discrete mechanisms before you sample noise yourself, and Lower bounds before you promise an accuracy.

Part 4 then replaces hand composition with accountants, and Part 5 applies all of it to training models. The code here runs OpenDP and diffprivlib on the Adult census data and the UCI Online Retail invoices, and every number comes from a snippet or script you can run. By the end, you can release a count, a sum, a median or a table with error bars and a guarantee you can defend.

19. The definition of differential privacy

19.1. Why this matters

Every attack in Threat models and attacks runs an algorithm on one of two datasets that differ in one person, and guesses from the output which one was used. **Differential privacy (DP)** makes the two outputs look alike against every test, including tests nobody has invented yet. It is a property of the algorithm, not of the data, so you can check it before any data are touched, and read any claim of it for what it protects and what it leaves out.

19.2. Neighbouring datasets

The first question about any claim is what "differ in one record" means, and the **neighbouring relation** answers it. Under **add/remove**, two datasets are neighbours when one has a record the other lacks, so the guarantee hides whether a person is in the data at all. Under **replace**, they have the same size and differ in the value of one record, so only what the record says is hidden. A replacement is a removal followed by an addition, two steps that each cost the privacy level ε of Section 19.3, so

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{19.1}$$

The reverse fails, because replace never compares datasets of different sizes.

You also choose the **unit of privacy**, what one record is. At the **example level**, it is one training record or one event, and at the **user level**, all the data of one person. Under example-level DP, a user who wrote 200 messages is a group of 200 records, protected only at 200ε (Post-processing, group privacy and composition). So check that the claim protects the person. The guarantee must also hide every record the data format allows, even a person aged 900, whose record could move an average a long way. So private algorithms clip each input to a fixed range before they add noise.

19.3. The definition

Call a randomized algorithm that takes a dataset and returns an output a **mechanism**, written M, and write D for a dataset and D' for one of its neighbours. The definition asks that no set of outputs become more than a factor e^{ε} more likely when one record changes, up to a small additive slack δ. Here ε is a non-negative privacy parameter. A mechanism is (ε, δ)**-differentially private** if every pair of neighbours and every set S of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{19.2}$$

The probabilities come only from the mechanism's own randomness, with the data held fixed.

When $\delta = 0$ the guarantee is called **pure DP**, and otherwise **approximate DP**. Either way, a private mechanism must be random. A deterministic one that answers differently on two neighbours gives that answer probability 1 on one and 0 on the other, which no factor e^{ε} covers. So rounding, aggregating or hiding small table cells gives no guarantee by itself.

19.4. The privacy loss and δ

For an output y, write $p_{M(D)}(y)$ for its probability, or its density, when the mechanism runs on D. The **privacy loss** of y is the logarithm of how much more likely the output is on D than on D',

$$L(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{19.3}$$

Independent outputs multiply their likelihood ratios, so their losses add, which is why Part 4's accountants work with the loss. Pure ε-DP holds exactly when every output has a loss between $-\varepsilon$ and ε. Laplace noise meets that, while the loss of Gaussian noise grows without limit in the far tail (The Laplace and Gaussian mechanisms).

Under approximate DP, δ pays for the outputs whose loss exceeds ε. It counts only the part of each one's probability on D above e^{ε} times its probability on D'. So δ is not the chance that the loss exceeds ε, and the total is the **hockey-stick divergence** (Divergences). Each ε has its own smallest δ, and the function $\delta(\varepsilon)$ is the mechanism's **privacy curve**, of which a quoted pair is one point. The noise parameters fix the whole curve, so report them with the pair and others can recompute it (Numerical accounting).

19.5. Randomized response

In **randomized response**, a survey design, each person tells the truth with some probability t and lies otherwise, so a single answer proves nothing about them. A person who tosses a coin, answers truthfully on heads and answers by a second toss on tails tells the truth with probability $\frac{3}{4}$. The true answer is then three times as likely as the false one, which gives pure DP at $\varepsilon = \ln 3$. In general, randomized response at level ε tells the truth with probability

$$t = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{19.4}$$

If the coins are independent, the guarantee is ε per person however many people answer, but only under replace, since under add/remove the number of reports reveals who answered. Write $\bar{y}$ for the share of "yes" reports among the n people. Undoing the lies with t gives the estimate of the true fraction π and its standard deviation,

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon/2}}{(e^{\varepsilon} - 1)\sqrt{n}}. \tag{19.5}$$

For 1,000 people the two-coin scheme has a standard deviation of 0.0274. When the true fraction is 0.3, that is about twice the sampling error of 0.0145 the survey already has. This doubling is

the price of the **local** model, in which nobody sees a true answer (Local, shuffle and distributed trust models).

OpenDP builds the mechanism from its truth probability, here the two-coin 0.75, and `map` turns one changed answer into its ε, $\ln 3$. The `contrib` feature switches on mechanisms not yet formally vetted.

```
import opendp.prelude as dp

dp.enable_features("contrib")
rr = dp.m.make_randomized_response_bool(prob=0.75)
print(f"{rr.map(1):.4f}")  # 1.0986
```

19.6. What ε means for an attacker

A membership attacker looks at the output and decides whether to accuse the target of being in the data, so it has two error rates (Hypothesis testing). Its **false positive rate (FPR)** is how often it accuses a target who is out, and its **true positive rate (TPR)** how often it accuses one who is in. The outputs on which it accuses form a set, which the definition bounds, so under add/remove every attack satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta. \tag{19.6}$$

This is the form to use, because Part 2 judged attacks by their TPR at a low FPR. Take $\varepsilon = 1$ and the $\delta = 10^{-5}$ that Section 19.7 recommends for 10,000 people. At an FPR of 10^{-3}, no attack reaches a TPR above $e \cdot 10^{-3} + 10^{-5} = 0.002728$. As the FPR falls towards 0, the first term vanishes, so at a very low FPR the guarantee rests almost entirely on δ.

Read backwards, the cap turns a measured attack into a lower bound on ε,

$$\varepsilon \ge \ln \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}}. \tag{19.7}$$

Suppose a model is claimed to be $(2, 10^{-5})$-DP, and an attack reaches a TPR of 0.05 at an FPR of 0.001. If those rates were exact, ε would be at least about $\ln 50 = 3.91$, and the claim false. A real audit puts confidence bounds on the rates and plants a **canary**, a record built to be as easy to detect as possible (Auditing).

Two summaries are easier to explain to a lawyer. The first, the **membership advantage**, is the TPR minus the FPR, how much better than a coin flip an attacker does. Under (ε, δ)-DP it is at most

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}. \tag{19.8}$$

The second, the posterior bound, speaks about beliefs. Under pure DP, one output can multiply an adversary's odds between two neighbours by at most e^{ε}. So a prior belief p that the data are D rises at most to

$$\Pr[D \mid y] \le \frac{e^{\varepsilon} p}{e^{\varepsilon} p + 1 - p}. \tag{19.9}$$

Table 19.1 turns a claimed ε into both numbers.

Table 19.1.: What a pure ε-DP guarantee promises against the strongest attacker.

ε	Advantage bound	Posterior from a prior of 0.01	Posterior from a prior of 0.5
0.1	0.0500	0.0110	0.5250
1	0.4621	0.0267	0.7311
3	0.9051	0.1687	0.9526
8	0.9993	0.9679	0.9997

So an ε of at most 1 is strong for any mechanism, since a 1% prior rises to under 3%. Between 1 and 10 a claim needs the mechanism's privacy curve as justification, and above 10, where the bounds promise almost nothing, an audit as well (Choosing ε). For statistics, start from the error you can accept and solve it for ε (The Laplace and Gaussian mechanisms). The table is for pure DP, so for an (ε, δ) claim quote the advantage bound and the cap on the TPR, which both include δ.

19.7. How small δ must be

A small δ barely moves the advantage bound, but the advantage averages over everyone, so it can hide total harm to a few people. Take a mechanism that publishes each record in full, independently with probability δ. Under add/remove it is $(0, \delta)$-DP, because a record changes the output only when its own coin publishes it. Yet over n records it publishes at least one with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{19.10}$$

With 10,000 records and $\delta = 1/n$, that is $1 - e^{-1} = 0.6321$.

So treat δ as a per-person budget for catastrophe, which may be spent on total failures like these. This book's default keeps $n\delta$, the expected number of people exposed this way, well below 1. Let n count people rather than records, and choose

$$\delta \leq \frac{1}{n^{1.1}}, \tag{19.11}$$

rounded down to a power of ten. For 10,000 people the rule gives 10^{-5}. The publishing mechanism then has $n\delta = 0.1$ and exposes someone with probability only 0.0952.

19.8. What differential privacy does not protect

DP compares a world with your record and a world without it, so anything that can be learned in both worlds is outside the guarantee. A study that never used Alice's record may show that smokers get cancer at a high rate, which changes what anyone believes about Alice, a smoker, yet the study is 0-DP with respect to her. So a reported harm is a DP violation only if the inference would differ materially without the person's record. Correlated records are a second gap: a household of five that always answers alike is a group of five, unless the household is the unit of privacy. The third gap is the implementation, which needs its own review (Discrete and floating-point-safe mechanisms).

19.9. Writing the guarantee down

A reviewer can check only what a claim writes down. So a guarantee names the unit of privacy with its neighbouring relation, and ε and δ for the whole release rather than per query. It also names the mechanism, the library version, the trust model and what was treated as public, since public information is not protected. This example uses the **central model**, in which a trusted curator holds the raw data and adds the noise:

> This release satisfies $(1, 10^{-5})$-DP for adding or removing one customer and all their invoice lines, at most 100 per customer, under the central model, with Gaussian noise from OpenDP 0.16.0. The market list was fixed in advance.

Release statistics with a library and Release and governance print such statements from the code that made the release.

19.10. Code

This snippet checks two numbers quoted above.

```
from math import exp, log

print(f"{(exp(3) - 1) / (exp(3) + 1):.4f}")  # 0.9051
print(f"{log((0.05 - 1e-5) / 1e-3):.2f}")  # 3.91
```

19.11. Cheat sheet

Table 19.2.: The results of this chapter.

Result	Formula	Notes
(ε, δ)-DP	$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$	All neighbours, all sets S
Attack cap	$\mathrm{TPR} \leq e^{\varepsilon} \mathrm{FPR} + \delta$	Audit: $\varepsilon \geq \ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$
Choosing δ	$\delta \leq 1/n^{1.1}$, rounded down to a power of ten	n counts people, not records

Rules of thumb

- When people contribute many rows, the unit of privacy is the person.
- An ε is never a probability of re-identification. Under pure DP it bounds how far any output can shift an attacker's odds.

19.12. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition and what DP promises.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing reading behind the attack cap.

Derivations. Every result here is derived step by step in the full edition's chapter.

20. Post-processing, group privacy and composition

20.1. Why this matters

A private mechanism is never the whole system. Its output gets rounded or fed into a model, people own many records each, and an analyst rarely asks only one question. Three properties of differential privacy (DP) say what guarantee then holds, without a new proof for every pipeline. Post-processing makes anything computed from a private output free, group privacy covers a person with several records, and composition adds up the budgets of several mechanisms. Together they turn privacy into bookkeeping: you prove each building block once, then keep a ledger of what you spend.

20.2. Post-processing

Take an (ε, δ)-DP mechanism M, with privacy level ε and slack δ. Apply to its output any function f, which may use randomness of its own but never reads the data. Then the processed release is just as private,

$$f(M(D)) \text{ is } (\varepsilon, \delta)\text{-DP.} \tag{20.1}$$

Processing can blur the evidence that an output carries about the data, but it can never sharpen it.

So rounding a noisy count, clamping it at zero, or adjusting noisy subtotals to match their noisy total costs nothing. A private system therefore adds noise at one narrow point, such as counts or gradients, and treats everything after it as public. The rule covers attackers too, because a membership-inference attack is a function of the output. So the definition's cap on how often any attack succeeds holds even for attacks invented after the release.

The condition is that f never reads the data. "Take the noisy top category, then report its exact count" breaks it, because the second step is a new and non-private mechanism.

20.3. Group privacy

The definition protects one record, but a customer of an online shop has dozens of invoice lines. For a person who owns k records, you need a guarantee for datasets that differ in k records. You can walk from one such dataset to the other by changing one record at a time, at a cost of ε per step. So under pure DP, where $\delta = 0$, ε-DP per record becomes $k\varepsilon$-DP for the group. The slack grows much faster than k. For a group of k records, an (ε, δ)-DP mechanism is

$$\Big(k\varepsilon,\ \delta_k\Big)\text{-DP}, \qquad \delta_k = \delta\,\frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1} \le k\, e^{(k-1)\varepsilon}\delta. \tag{20.2}$$

The form on the right is a simpler, looser bound. With $\varepsilon = 0.5$ and $\delta = 10^{-5}$ per record, a person with 10 records gets a slack of 2.272×10^{-3}. That hurts, because δ was chosen far below one over the number of people (How small δ must be).

OpenDP computes group guarantees from the mechanism itself. A measurement's `map(d_in=k)` is its guarantee when neighbouring datasets differ in k records, here for a count with Laplace noise and one with Gaussian noise.

```
count = dp.t.make_count(dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance())
laplace = count >> dp.m.then_laplace(scale=2.0)  # epsilon = 0.5 per record
print(f"Laplace count: epsilon {laplace.map(d_in=1):.1f} per record, {laplace.map(d_in=10):.

def gaussian(scale: float) -> dp.Measurement:  # a Gaussian count, as a privacy curve
    return dp.c.make_zCDP_to_approxDP(count >> dp.m.then_gaussian(scale=scale))

def at_1e5(scale: float) -> dp.Measurement:
    return dp.c.make_fix_delta(gaussian(scale), delta=1e-5)

scale = dp.binary_search_param(at_1e5, d_in=1, d_out=(0.5, 1e-5))  # (0.5, 1e-5) per record
curve = gaussian(scale).map(d_in=10)  # the privacy curve for a group of 10 records
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(1e-5):.2f} at delta 1e-5")
# delta_k is the slack that the group-privacy formula gives the group
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(delta_k):.2f} at delta {delta_k:
```

```
Laplace count: epsilon 0.5 per record, 5.0 for 10
Gaussian count, group of 10: epsilon 6.43 at delta 1e-5
Gaussian count, group of 10: epsilon 4.57 at delta 2.272e-03
```

For the Laplace count, `map(d_in=10)` gives $k\varepsilon = 5.0$, as the formula does. The Gaussian count, calibrated to $(0.5, 10^{-5})$ per record, does better, because OpenDP reads the group's guarantee off the mechanism's whole privacy curve. At the formula's slack of 2.272×10^{-3} the group gets $\varepsilon = 4.57$, below the formula's 5.0. So read a group guarantee off the library's map, and fall back on the formula only for a mechanism the library cannot map.

Better still, if you care about people, make the person the unit of privacy, so that neighbouring datasets differ in all of one person's records. Then bound how many records each person may contribute before any noise is added, as Release statistics with a library does on real data. Group privacy is the fallback when you cannot, and for records of different people that reveal each other, such as a household that shares an address and an income.

20.4. Sequential composition

An analyst asks several queries of the same data, each answered with fresh noise, and an adversary sees all the answers together. Say there are k mechanisms, and write $(\varepsilon_i, \delta_i)$ for the budget of the i-th. Releasing all their outputs is then

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{20.3}$$

This is **basic composition**. It needs every mechanism to draw fresh noise and to use the same neighbouring relation, add/remove or replace.

The rule also covers **adaptive** composition, where each query is chosen after reading the earlier answers, as long as each budget is fixed in advance. A budget chosen from the answers ("keep going until the estimate looks good") needs a privacy filter, a rule that decides when to stop (Adaptive composition).

Basic composition is loose for many releases. Ten Gaussian counts, each calibrated to $(0.5, 10^{-5})$, are charged $(5, 10^{-4})$, but at that slack their exact ε is only 1.504. So beyond a handful of releases, use an accountant, which computes the total from the mechanisms themselves (Advanced and optimal composition, Part 4).

20.5. Parallel composition

Per-region statistics use disjoint subsets of the data. When each person has one record, that record falls in one region only, so each region can spend the full budget. To use this, fix in advance a public rule that sends every possible record to exactly one part. Then run a mechanism M_i with independent noise on the i-th part. If each M_i is $(\varepsilon_i, \delta_i)$-DP under add/remove, the whole release is

$$\big(\max_i \varepsilon_i,\ \max_i \delta_i\big)\text{-DP under add/remove.} \tag{20.4}$$

So a histogram with Laplace noise of scale $1/\varepsilon$ on every bin is ε-DP however many bins it has. Under replace, a record can move from one bin to another, so two bins change and the cost doubles to 2ε. A person with records in several bins breaks the rule too. Then either charge each bin as a separate query, or first bound the number of bins a person may touch.

The rule fixed in advance is the condition that is easiest to break. A split into "the top 10% of earners and the rest" moves its boundary when one high earner is added, so both parts change. Partitioning on the keys that appear in the data fails too, because a key present only through one person's record reveals that person. So use fixed bins or a public key list. When the keys are not public, release only the keys whose noisy count clears a threshold, a step called **partition selection** (Release statistics with a library).

20.6. Keeping a budget

Fix a total budget for a dataset before the first release, and charge every release against it. In the Python library diffprivlib, a `BudgetAccountant` holds the total, and each function you hand it charges its own ε and δ. A call that would exceed the total raises a `BudgetError` instead of answering. The script practitioner_adult_statistics.py runs it on the Adult census data, 48,842 people with one row each, for which the default slack is 10^{-6}. It spends half the budget on the mean age and half on the number of people over 50, then asks for one more count.

```
age = adult["age"].to_numpy()
acc = BudgetAccountant(epsilon=1.0, delta=1e-6)  # the total, fixed in advance
mean_age = tools.mean(age, epsilon=0.5, bounds=(17, 90), accountant=acc, random_state=seed)
over_50 = tools.count_nonzero(age > 50, epsilon=0.5, accountant=acc, random_state=seed)
print(f"private mean age: {mean_age:.2f}  private count over 50: {over_50:,}")
print(f"spent: {acc.total()}")
try:
    tools.count_nonzero(age > 60, epsilon=0.1, accountant=acc, random_state=seed)
except BudgetError as err:
    print(f"refused: {err}")
```

Run it with `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`. With `random_state=0` it prints the following.

```
private mean age: 38.64  private count over 50: 9,813
spent: (epsilon=1.0, delta=0.0)
refused: Privacy spend of (0.1,0) not permissible; will exceed remaining privacy budget. Use
```

The true values are 38.64 and 9,808. Both calls add Laplace noise, so no δ was spent and the 10^{-6} stays available for a Gaussian query. The ledger has two limits. It applies basic composition to every call, so it overcharges calls on disjoint parts of the data. And it trusts each call's own guarantee: diffprivlib's mean divides by the exact number of rows, treating it as public. That number is public only when neighbours have the same size, so this ledger holds under replace neighbours.

Most real failures break a condition of the three properties, and the damage is usually total rather than a small loss. Table 20.1 lists the common cases.

Table 20.1.: What composes and what doesn't.

Situation	Composes?	What to do
A function of the output, with its own coins	Yes, for free	Process as much as you like
Another query on the same data, fresh noise	Yes, budgets add	Charge it to the ledger
A query chosen from earlier answers	Yes, budgets add	Fix its budget in advance
Two releases that share one noise draw	No	Draw fresh noise for every query
"Post-processing" that reads the data again	No	Charge it as a new mechanism
A DP release beside an exact statistic of the same data	No, the exact one has no guarantee	Release the statistic privately too
A split whose boundaries depend on the data	No, not in parallel	Fix the bins in advance

Situation	Composes?	What to do
Guarantees under add/remove and replace in one ledger	Only after conversion	Convert to replace: 2ε and $(1 + e^{\varepsilon})\delta$

Shared noise is the clearest case. Release the number of smokers and the number of smokers aged 40 or over, adding the same Laplace draw of scale 1 to both. Each release alone is 1-DP, but their difference cancels the noise and gives the exact number of smokers under 40, so the pair is not DP at all. In code, the bug hides in two mechanisms with the same random seed, a cached noise value re-added to a related query, or noise reused across shards. A fixed seed such as `random_state=0` above reproduces a book's numbers, but in production it makes the noise reproducible and so removable. Returning the same noisy answer to a repeated identical query is fine, because that is post-processing of one release.

20.7. Cheat sheet

Table 20.2.: The results of this chapter.

Property	Rule	Condition
Post-processing	$f(M(D))$ is (ε, δ)-DP	f never reads the data
Group privacy	$(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$ for k records	Prefer the person as the unit, with bounded contributions
Basic composition	$(\sum \varepsilon_i, \sum \delta_i)$	Fresh noise, one neighbouring relation, budgets fixed in advance
Parallel composition	$(\max \varepsilon_i, \max \delta_i)$	Add/remove, parts fixed in advance; 2ε under replace

Rules of thumb

- Add noise once, as early as possible, and everything computed from the noisy output after it is free.
- Fix the total budget before the first release, and let one accountant refuse what exceeds it.
- Never share a random seed between mechanisms, and never fix one in production.

20.8. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): post-processing, group privacy and basic composition, the standard reference.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a query system built on post-processing and composition as its only rules.

Derivations. Every result here is derived step by step in the full edition's chapter.

21. The Laplace and Gaussian mechanisms

21.1. Why this matters

Suppose a hospital wants to publish how many of its patients have diabetes, with a differential privacy (DP) guarantee. Almost every such release, and every noisy gradient of private training, is built with one of the two mechanisms of this chapter. Both compute the exact answer of a query f on the dataset D, then add fresh random noise, written Z, and publish

$$M(D) = f(D) + Z. \tag{21.1}$$

Three decisions fix such a mechanism. The **sensitivity**, how far one record can move the answer, is what the noise must hide. The noise distribution sets the kind of guarantee: Laplace noise gives pure DP, while Gaussian noise needs a small slack δ. And a calibration formula sets the amount of noise for a target ε, through the privacy loss of the definition of differential privacy.

21.2. Sensitivity

The noise must cover the largest change one record can make to the answer. A query often returns a vector of d numbers, such as the bins of a histogram, so the change needs a size. Laplace noise uses the ℓ_1 norm, the sum of the absolute changes, and Gaussian noise the ℓ_2 norm, the square root of the sum of the squared changes. Writing $D \sim D'$ for a pair of neighbours, the ℓ_p **sensitivity** of the query is the largest change between their answers,

$$\Delta_p f = \max_{D \sim D'} \left\| f(D) - f(D') \right\|_p, \qquad p \in \{1, 2\}. \tag{21.2}$$

The two are equal when one record moves a single coordinate, but when it moves all d of them, $\Delta_1 f$ can be $\sqrt{d}$ times $\Delta_2 f$.

The sensitivity also depends on the neighbouring relation. Under add/remove one dataset has one record more than the other, while under replace one record is swapped for another, so the size n is public. Write B for the largest value a record may take after clipping. Then Table 21.1 gives the standard queries.

Table 21.1.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, values in $[0, B]$	B	B	B	B

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Mean, values in $[0, B]$, n public	none (Section 21.6)	none	B/n	B/n
Histogram with k bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \geq j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

A replaced record leaves one histogram bin and joins another, so it changes two bins. The threshold counts, the numbers of records at or above each of d values, are a **dense** query, in which one record can move every coordinate.

Compute the sensitivity from the domain, before you look at the data. A sensitivity read off the data, such as the largest value present, makes the noise leak that value (Beyond global sensitivity). Unbounded data have infinite sensitivity, so clip every value to a fixed range first. With ages clipped to at most 100, the mean age of 1,000 patients has sensitivity $B/n = 0.1$ under replace.

21.3. The Laplace mechanism

The Laplace mechanism adds independent noise from the **Laplace distribution**, a two-sided exponential peaked at 0, to each coordinate. Its scale b sets its typical size, and $\mathrm{Lap}(b)$ denotes it. For an ε-DP guarantee, the scale is the sensitivity divided by ε,

$$b = \frac{\Delta_1 f}{\varepsilon}. \tag{21.3}$$

The error of the mechanism is the noise itself. Each coordinate of the noise is off by b on average, and exceeds a distance t with probability $e^{-t/b}$. Allow a small failure probability β for all d coordinates together, and give each coordinate a share β/d of it. The width that holds for all of them is then

$$\Pr\left[\max_i |Z_i| > b \ln \frac{d}{\beta}\right] \leq \beta. \tag{21.4}$$

Publish this width with the release. Read backwards, it is an error budget: for a single number, fix a target width t and solve for the privacy level,

$$\varepsilon = \frac{\Delta_1 f \, \ln(1/\beta)}{t}. \tag{21.5}$$

So a count that must be within 10 of the truth with probability 0.95 needs $\varepsilon = \ln(20)/10 = 0.2996$. For d numbers, solve each with the failure probability β/d, and add up their ε by composition.

At the hospital, 1,234 of the 10,000 patients have diabetes. A count has sensitivity 1 under add/remove, so at $\varepsilon = 0.1$ the noise is $\mathrm{Lap}(10)$. Its 95% error bar is $10 \ln 20 = 29.96$, or 2.43% of the count.

The scale does not depend on the size of the data, so the relative error of a count of size n is about $1/(n\varepsilon)$. But the sensitivity must include every number one record can move. If one record can change all of 10 released counts, noise calibrated for one count makes the release 10ε-DP, not ε-DP.

21.4. The Gaussian mechanism

The Gaussian mechanism adds independent normal noise with mean 0 and standard deviation σ to each coordinate. Its privacy loss is unbounded in the far tail, so it always carries a δ. In exchange, it is calibrated to the ℓ_2 sensitivity, which can be much smaller than the ℓ_1 one. Only the ratio of the sensitivity to the noise matters, so write $\mu = \Delta_2 f/\sigma$ for the number of noise standard deviations between the answers on two neighbours. Write Φ for the cumulative distribution function (CDF) of the standard normal distribution. The mechanism is then (ε, δ)-DP exactly when δ is at least

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{21.6}$$

This formula is how you calibrate. As μ grows, the exact δ rises steadily from 0 to 1, so exactly one value μ^* hits your target. The smallest noise that works is then

$$\sigma^* = \frac{\Delta_2 f}{\mu^*}, \qquad \text{where } \delta_{\mu^*}(\varepsilon) = \delta, \tag{21.7}$$

and libraries solve for μ^* for you (Section 21.7). For 10,000 patients the book's default slack is $\delta = 10^{-5}$ (The definition of differential privacy). At $\varepsilon = 1$ this **analytic calibration** then gives $\sigma^* = 3.7306\,\Delta_2 f$.

Many papers and libraries still use the classic calibration of Dwork and Roth (2014), proved only for ε and δ both between 0 and 1,

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{21.8}$$

At $\varepsilon = 1$ and $\delta = 10^{-5}$ it gives $4.8448\,\Delta_2 f$, 30% more noise than the analytic $3.7306\,\Delta_2 f$. Above $\varepsilon = 1$ it has no proof behind it, and beyond $\varepsilon = 8.42$ it gives too little noise for the claimed guarantee. So use the analytic calibration.

A Gaussian mechanism satisfies a whole curve of guarantees, one δ for each ε, and the accountants of Part 4 compose whole curves. So report $\sigma/\Delta_2 f$ with every Gaussian release, as well as the pair you chose.

Back at the hospital, Gaussian noise for the diabetes count at $\varepsilon = 0.1$ and $\delta = 10^{-5}$ gives a 95% error bar of 60.27, which is 1.96 standard deviations. Laplace noise gave 29.96 with no δ at all, so for a single count it wins on both error and guarantee.

21.5. Laplace or Gaussian?

The choice depends on how one record spreads its change across the coordinates. To see why, compare the root-mean-square error (RMSE) of each released number at the same ε. Laplace noise of scale $b = \Delta_1 f/\varepsilon$ has standard deviation $\sqrt{2}\,b$. Write σ_1 for the analytic Gaussian noise level at unit ℓ_2 sensitivity. The errors per coordinate are then

$$\text{Laplace: } \frac{\sqrt{2}\,\Delta_1 f}{\varepsilon}, \qquad \text{Gaussian: } \Delta_2 f\,\sigma_1. \tag{21.9}$$

In a sparse query, such as a histogram under add/remove, one record moves one coordinate, so the two sensitivities are equal. At $\varepsilon = 1$ and the default δ, $\sigma_1 = 3.7306$, so Laplace noise wins at every d, since $\sqrt{2} = 1.4142$ is smaller. In a dense query, such as the threshold counts, one record moves every coordinate, so the Laplace error grows like d and the Gaussian error only like $\sqrt{d}$. Laplace noise is then more accurate only while

$$d < \frac{(\varepsilon\sigma_1)^2}{2}, \tag{21.10}$$

so with $\sigma_1 = 3.7306$, Gaussian noise wins for threshold counts from 7 counts on.

Gaussian noise also wins under heavy composition, because several Gaussian releases compose exactly into one Gaussian mechanism, with the root of the summed squared sensitivities. Private training has both, since one record's gradient moves every coordinate of a model at every step. So it clips each gradient's ℓ_2 norm and calibrates Gaussian noise to that clip.

21.6. Calibrating a release

Every noisy release takes five steps.

1. Fix the neighbouring relation and the unit of privacy, usually one person under add/remove.
2. Clip every value to a range fixed in advance, and read the sensitivity off Table 21.1 or work it out from the worst record.
3. Choose Laplace noise for one number or a sparse vector, and Gaussian noise for a dense vector.
4. Set the scale with Equation 21.3 or Equation 21.6, and charge it to your budget (Post-processing, group privacy and composition).
5. Draw the noise with a vetted library, and publish the error bar and the noise parameters with the result.

A mean under add/remove has no simple sensitivity, because its denominator moves too. So release a noisy sum and a noisy count, each with half of the budget, and divide one by the other, which as post-processing costs nothing more. For the mean's error bar, take each one's bar at 97.5%, so that both hold together 95% of the time. The mean then lies above the sum's lower end divided by the count's upper end. It lies below the sum's upper end divided by the count's lower end, provided both lower ends are positive. OpenDP's `.dp.mean` makes the same split, and `summarize(alpha=0.025)` reports the two bars at 97.5%. When you do not know the range to clip to, Private mean estimation finds it privately.

Plain `numpy.random.laplace` is for prototypes only, because textbook floating-point samplers can leak through the low-order bits of their output (Discrete and floating-point-safe mechanisms). For a whole release with clipping, grouping and one budget, follow Release statistics with a library.

21.7. Code

The snippet checks the hospital's numbers and the analytic σ^* at $\varepsilon = 1$. In OpenDP, a mechanism's `map` takes the sensitivity and returns the ε it costs, and the accuracy helpers turn a scale into a

95% error bar or back. For a count, with sensitivity 1, one over that scale is the ε of the error budget. IBM's diffprivlib calibrates the analytic Gaussian.

```
import math

import opendp.prelude as dp
from diffprivlib.mechanisms import GaussianAnalytic

dp.enable_features("contrib")
laplace = dp.m.make_laplace(dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float
print(laplace.map(1.0))  # 0.1
print(f"{dp.laplacian_scale_to_accuracy(10.0, alpha=0.05):.2f}")  # 29.96
print(f"{1 / dp.accuracy_to_laplacian_scale(10.0, alpha=0.05):.4f}")  # 0.2996
gauss = GaussianAnalytic(epsilon=1.0, delta=1e-5, sensitivity=1.0)
print(f"{math.sqrt(gauss.variance(0.0)):.4f}")  # 3.7306
```

To add the noise, call the mechanism on the value, as in `laplace(1234.0)`.

21.8. Cheat sheet

Table 21.2.: The results of this chapter.

Result	Formula	Notes
Laplace mechanism	$\mathrm{Lap}(\Delta_1 f/\varepsilon)$ per coordinate	Pure DP; error does not depend on n
Laplace error bar	$b\ln(d/\beta)$ for all d coordinates, scale b	Solve it for ε to budget error
Exact Gaussian δ	$\Phi(\frac{\mu}{2}-\frac{\varepsilon}{\mu})-e^{\varepsilon}\Phi(-\frac{\mu}{2}-\frac{\varepsilon}{\mu})$, $\mu=\Delta_2 f/\sigma$	Solve for σ with `GaussianAnalytic`
Classic Gaussian	$\sigma=\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$	Too much noise below 1, unsafe above 8.42
Dense query	Laplace wins only for $d<(\varepsilon\sigma_1)^2/2$	Otherwise use Gaussian noise

Rules of thumb

- Calibrate Laplace noise to $\Delta_1 f$ and Gaussian noise to $\Delta_2 f$, never the other way round.
- When n is not public, never use B/n as the sensitivity of a mean.

21.9. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 and Appendix A: the Laplace mechanism and the classic Gaussian calibration.

- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): the exact privacy curve and the bisection calibration.

Derivations. Every result here is derived step by step in the full edition's chapter.

22. Advanced and optimal composition

22.1. What it is

A dashboard refreshes a hundred counts, and a training run takes thousands of noisy steps, yet you report one differential privacy (DP) guarantee for the whole run. Each step has its own privacy level ε and slack δ (The definition of differential privacy). **Basic composition** adds them, so a run of k steps costs $k\varepsilon$ and $k\delta$ (Post-processing, group privacy and composition). That is correct but pessimistic. The steps' privacy losses add up, but each depends on the noise drawn and is often negative, so their sum almost never reaches $k\varepsilon$.

Advanced composition (Dwork, Rothblum & Vadhan 2010) uses this cancellation. For an extra slack δ' that you pick for the whole run, the steps are together $(\varepsilon_{\text{total}}, k\delta + \delta')$-DP, with

$$\varepsilon_{\text{total}} = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{22.1}$$

The first term is the spread of the total loss, which grows only like $\sqrt{k}$. The second is its average, the drift.

Optimal composition (Kairouz, Oh & Viswanath 2015) is the exact best bound for steps known only by their (ε, δ). Take a hundred counts at $\varepsilon = 0.1$ each, with no slack, on 10,000 people. Basic composition gives a total of 10, still with no slack. The other two rules spend the book's default slack for 10,000 people, $\delta' = 10^{-5}$, and give 5.30 and 4.31.

22.2. When you meet it

You meet these rules in papers, privacy reviews and any library that quotes a total ε for many steps. They decide which number you plan with and which you report (Table 22.1).

Table 22.1.: Which composition rule to use, at a slack of 10^{-5} (none for basic composition).

What you know about the steps	Rule	Total ε for 100 steps at 0.1
Each step's ε, and δ must stay 0	Basic composition	10
Each step's (ε, δ), to plan by hand	Advanced composition	5.30
Each step's (ε, δ), for the number you report	Optimal composition	4.31

What you know about the steps	Rule	Total ε for 100 steps at 0.1
The mechanism itself, such as Gaussian noise	An accountant (Part 4), which composes whole privacy curves	Often far smaller
Budgets chosen during the run	A privacy filter, which enforces a budget fixed in advance	—

"Optimal" means optimal for steps described by one pair each. A real mechanism has a whole privacy curve, one δ for every ε, and a single pair forgets the rest of it. A thousand Gaussian steps with noise standard deviation 20 on a query of sensitivity 1 are exactly $(7.51, 10^{-5})$-DP when composed as curves. Summarizing each step by its best single pair and composing optimally gives 62.7, about eight times worse. So whenever you know the mechanism, compose its curve with an accountant (Privacy accounting).

22.3. How to use it

Composition does two jobs. The first is planning: to split a total budget $\varepsilon_{\text{total}}$ over k steps, run advanced composition backwards. When $\varepsilon\sqrt{k}$ is small the second term, the drift, is negligible, so each step can have about

$$\varepsilon \approx \frac{\varepsilon_{\text{total}}}{\sqrt{2k \ln(1/\delta')}}. \tag{22.2}$$

Once $\varepsilon\sqrt{k}$ nears 1, the drift takes over and this shortcut gives each step too much.

The second job is reporting. When you know the mechanism, report the accountant's number at the run's total δ, and otherwise report optimal composition. This snippet plans the hundred counts within a total of $(1, 10^{-5})$, giving the whole slack to δ'. It then reports the thousand Gaussian steps with two accountants of Google's `dp_accounting`, based on the privacy loss distribution (PLD) and on Rényi differential privacy (RDP).

```
import math

import dp_accounting

print(f"{1.0 / math.sqrt(2 * 100 * math.log(1e5)):.4f}")  # 0.0208
step = dp_accounting.GaussianDpEvent(noise_multiplier=20.0)
run = dp_accounting.SelfComposedDpEvent(step, 1000)
pld = dp_accounting.pld.PLDAccountant()
pld.compose(run)
print(f"{pld.get_epsilon(target_delta=1e-5):.2f}")  # 7.51
rdp = dp_accounting.rdp.RdpAccountant()
rdp.compose(run)
print(f"{rdp.get_epsilon(target_delta=1e-5):.2f}")  # 8.08
```

Each count gets 0.0208, twice basic composition's 1/100. The PLD accountant matches the exact 7.51 quoted above, and the RDP accountant gives a valid but looser 8.08 (Choosing an accountant).

22.4. What goes wrong

For a few steps, advanced composition can lose to basic composition, so always take the smaller of the two. Every rule also adds up the steps' slacks, so count each δ. And when the steps have different privacy levels, optimal composition is computationally hard, one more reason to use an accountant.

All these rules hold when each query depends on earlier answers, provided each step's budget is fixed in advance. Choosing the budgets from the answers needs a privacy filter (Adaptive composition), and stopping early needs care. A stopping rule that reads only the noisy answers is post-processing, so the run costs no more than its planned length. But a run that stops when the true count of some group crosses a threshold reveals that count through its length. Every answer can then be ε-DP while the whole run is not DP for any finite ε.

So, for a reviewer, a total below optimal composition is a red flag unless an accountant computed it from the mechanism's curve.

Derivations. Every result here is derived step by step in the full edition's chapter.

23. Selection: exponential mechanism, report-noisy-max, sparse vector

23.1. Why this matters

Many questions ask you to pick something rather than compute a number, such as the best model on a validation set, a median, or the first alarm among many statistics. Publishing every candidate's score with Laplace noise would charge the budget once per candidate, because the budgets of separate releases add. The mechanisms of this chapter instead **release the position, not the values**. One record can change which candidate wins only a little, however many candidates there are. So under differential privacy (DP), a selection costs as much as a single query, and the number of candidates enters only the accuracy, through its logarithm.

23.2. Which mechanism to use

Table 23.1 matches common tasks to the mechanisms below. Throughout, the **sensitivity** Δu is the most that one person's record can change any candidate's score.

Table 23.1.: Which selection mechanism to use.

Task	Mechanism	Privacy cost
Pick the best of a fixed list	Exponential mechanism or report-noisy-max	ε
Pick the top c, such as the most frequent items	Gumbel noise, keep the c largest, or noisy max c times	$c\varepsilon$
Pick a number from a range, such as a quantile or a clipping norm	Exponential mechanism over the range	ε
Find the first query in a stream above a threshold	AboveThreshold	ε, however long the stream

23.3. The exponential mechanism

The candidates form a fixed set, written $\mathcal{R}$. Each candidate r has a **score** on the dataset D, written $u(D, r)$, such as the number of validation records a model gets right. The **exponential**

mechanism of McSherry and Talwar (2007) picks each candidate with probability proportional to an exponential of its score. With a privacy budget ε, it outputs candidate r with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\varepsilon\, u(D, r)/(2\Delta u)\right)}{\sum_s \exp\left(\varepsilon\, u(D, s)/(2\Delta u)\right)}. \tag{23.1}$$

It is ε-DP because one record changes each weight, and also their sum, by at most a factor $e^{\varepsilon/2}$, which makes e^{ε} for their ratio.

Two conditions decide whether a use is correct. First, fix the list of candidates before you look at the data, because a list built from the data reveals records by itself, as a word in a corpus's vocabulary does when only one person used it. Second, drop the 2 in the exponent only when the scores are **monotone**, meaning that one record pushes all of them the same way. Counts under add/remove neighbours are monotone. A histogram under replace neighbours is not, because moving one record lowers one count and raises another, so dropping the 2 there doubles the real cost.

23.4. Accuracy, and the cost of many candidates

The number of candidates $|\mathcal{R}|$ is free for privacy but not for accuracy, because more candidates means more bad ones to pick. Call the best score minus the chosen candidate's score the **gap**. On average it is at most

$$\frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}| + 1), \tag{23.2}$$

so doubling the candidates adds only $\ln 2$ to the bracket.

Suppose you trained 50 models and want to release the best one, judged on a private validation set of 1,000 records. Model r, numbered from 1, gets $900 - r$ records right, so model 1 is the best. The score is that count, which one record moves by at most 1, so $\Delta u = 1$. At $\varepsilon = 1$ the mechanism picks the best model only 39% of the time. Yet it loses just 1.54 correct predictions on average, or 0.15% accuracy, because its mistakes are mostly near-ties. Hyperparameter tuning picks models this way.

23.5. A private quantile or clipping norm

Reading a median or a clipping norm off the data leaks it, so pick it with the exponential mechanism. First fix a range before looking at the data, and clip every record into it. The mechanism then scores each point of the range by how close the share of records below it is to the quantile you want, such as one half for the median, and one added or removed record moves any score by at most 1. The score stays the same between consecutive sorted records. So the mechanism picks one of these intervals, called **pieces**, favouring long pieces near the quantile, and returns a uniform point inside it.

diffprivlib's `tools.quantile` and `tools.median` take an `epsilon` and the range as `bounds`. OpenDP's `.dp.quantile` and `.dp.median`, which work on Polars data frames, pick one value from a finite list of candidates instead. The script practitioner_adult_statistics.py takes the

median age of the Adult census data, 48,842 people with one row each, at $\varepsilon = 1$. The range of 17 to 90 comes from the survey's documentation.

```
age = adult["age"].to_numpy()
private = tools.median(age, epsilon=1.0, bounds=(17, 90), random_state=seed)
print(f"true median age: {np.median(age):.1f}  diffprivlib median: {private:.2f}")
```

With `random_state=0` it prints a true median of 37.0 and a private one of 37.72. Ages are whole numbers, so every point between 37 and 38 has the same score, and the mechanism picked that piece and a point in it at random. OpenDP refuses to run the median without a public upper bound on the number of rows, declared as a `Margin`. Choose it generously from public facts, since it must be at least the true number of rows and a larger one costs almost nothing.

```
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),  # one row per person
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=100_000)],  # a public bound on the number of rows
)
age_col: Any = pl.col("age")  # OpenDP adds .dp to Polars expressions when it is imported
query = context.query().select(age_col.dp.median(candidates=list(range(17, 91))))
print(query.summarize(alpha=0.05))
released = query.release().collect()  # changes on every run: OpenDP's noise has no seed
```

Its summary reports `ExponentialMin` with a scale of 2.0, because OpenDP 0.16 adds exponential noise to every candidate's score and reports the best. That is **permute-and-flip** (McKenna & Sheldon 2020), with the exponential mechanism's guarantee and never a worse expected error. The accuracy is `null`, because OpenDP gives no error bar for a quantile. Its noise has no seed, so the released value changes from run to run, and `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py` prints one at the end.

The error bar you can state is about rank, not value. With probability $1 - \beta$, the exponential mechanism's gap is at most $(2\Delta u/\varepsilon)(\ln|\mathcal{R}| + \ln(1/\beta))$, the tail form of Equation 23.2. For a median, with $\Delta u = 1$, the gap is the number of records by which the output misses the middle rank, so $\beta = 0.05$ gives its 95% bar in ranks. A bar in value would need the number of records near the median, which the release does not reveal. If a dashboard needs values, spend budget on a few noisy counts below candidate values.

A generous range is cheap, because its width enters the gap only through the logarithm of the number of candidates.

Two things go wrong in practice. Where the data near the quantile are sparse, the best piece is long and the output can land anywhere in it. A small error in rank is then a large error in value. And the range or the candidates must not come from the data, since setting the upper end to the observed maximum reintroduces the leak. diffprivlib warns with a `PrivacyLeakWarning` when you leave out `bounds` (Sensitivity).

23.6. Report-noisy-max and Gumbel noise

Report-noisy-max adds independent Laplace noise to each candidate's score and outputs the candidate with the largest noisy score. It is ε-DP for any number of candidates when the noise has scale $2\Delta u/\varepsilon$, or $\Delta u/\varepsilon$ when one record pushes all scores the same way. Release only the winner's index, and pay for a fresh noisy answer if you need its score. Its accuracy is close to the exponential mechanism's, and neither is always better.

Gumbel noise of scale $2\Delta u/\varepsilon$ in place of the Laplace noise gives exactly the exponential mechanism. Libraries sample it with this **Gumbel-max trick**, because the exponential of a score in the thousands overflows, while noisy scores do not. Keeping the c largest Gumbel-noised scores picks the top c at once (Durfee & Rogers 2019).

23.7. The sparse vector technique

Sometimes queries arrive one at a time, and you want the first one that is high enough. The **sparse vector technique** (Dwork et al. 2009) compares each with one noisy threshold and pays only for the one that fires. Its basic form, **AboveThreshold**, takes a threshold τ and a stream of queries f, all with the same sensitivity Δf. It splits the budget into ε_1 for the threshold and ε_2 for the queries.

1. Draw one threshold noise, written ν, from $\mathrm{Lap}(\Delta f/\varepsilon_1)$ for the whole run.
2. For each query, draw a fresh noise from $\mathrm{Lap}(2\Delta f/\varepsilon_2)$ and add it to the query's answer. If the result is at least $\tau + \nu$, output "above" and halt. Otherwise output "below" and go on.

AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of "below" answers, even when each query is chosen after the earlier answers. All the "below" answers together pay ε_1, through the one threshold noise, and the single "above" pays ε_2.

The accuracy, unlike the privacy, depends on the stream length k, which must be fixed before the run. Write β for the chance that the accuracy guarantee fails. With an equal split, $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$, every query is classified correctly unless its true value is within a margin

$$\alpha = \frac{8\Delta f(\ln k + \ln(2/\beta))}{\varepsilon} \tag{23.3}$$

of the threshold. Monitor 1,000 counts, each with $\Delta f = 1$, against a threshold of 500 at $\varepsilon = 1$. At $\beta = 0.05$ the margin is 84.77, so with probability 0.95 only a count that close to the threshold can be misclassified.

To find the first c queries above the threshold, restart AboveThreshold with fresh threshold noise after each "above", at ε/c per run, and halt after the c-th. The margin then grows with c, so ask for as few alarms as you can. Lyu, Su and Li instead keep one threshold for all c alarms and multiply the query noise by c.

23.8. When the sparse vector technique goes wrong

Several published variants do not have the privacy they claim (Table 23.2).

Table 23.2.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Actual privacy
Algorithm 3	Outputs the noisy value of each "above"	∞
Algorithm 4	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5	No noise on the queries	∞
Algorithm 6	Never halts	∞

The no-halt variant is the one to remember. It adds exactly as much noise per query as AboveThreshold, so a reviewer who checks noise scales will miss it. Yet at $\varepsilon = 1$ its privacy loss reaches 18.1246 on 80 queries.

So review the accounting, not just the noise. An implementation must draw one threshold noise per run, since a threshold redrawn for each query makes every "below" answer cost budget that nobody counted. It must draw fresh noise for every query, since one query-noise draw reused for all of them amounts to Algorithm 5. And it must halt after the allowed number of "above" answers and never publish the noisy values it compared.

23.9. Cheat sheet

Table 23.3.: The selection mechanisms of this chapter, with $|\mathcal{R}|$ candidates or k queries.

Mechanism	Privacy	Accuracy
Exponential mechanism, Equation 23.1	ε; drop the 2 only for monotone scores	Mean gap $\leq \frac{2\Delta u}{\varepsilon}(\ln \lvert\mathcal{R}\rvert + 1)$
Report-noisy-max, noise $\mathrm{Lap}(2\Delta u/\varepsilon)$	ε, index only	Similar to the exponential mechanism
AboveThreshold	$\varepsilon_1 + \varepsilon_2$	Margin $8\Delta f(\ln k + \ln \frac{2}{\beta})/\varepsilon$, failure probability β

Rules of thumb

- Release positions, not values.
- Fix the candidate list, and the range of any number you pick, before you look at the data.

23.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3, §3.4 and §3.6: the three mechanisms in their standard form.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (VLDB 2017): the budget split and the survey of broken variants.

Derivations. Every result here is derived step by step in the full edition's chapter.

24. Query release and private multiplicative weights

24.1. Why this matters

Analysts rarely want one statistic. They ask for every two-way table over a handful of columns, or for a synthetic file they can query as often as they like, and both must satisfy differential privacy (DP). Such tables are **marginal tables**: each counts the records for every combination of values of a few attributes, and each count is a **cell**.

These releases work by adding noise to whole tables rather than to single cells, scaled to how much one person can change the table. One person sits in one cell of a table, so a whole table costs about as much as one count. Everything computed from the noisy tables afterwards is free, including repairs that make the cells add up to the total. When the queries are too many for noise on every cell, synthetic data fitted to a few noisy tables takes over.

24.2. Which method to use

The choice depends on how many queries you have, compared with the number of records, and on how many attributes a record has (Table 24.1).

Table 24.1.: Which query-release method to use.

Situation	Use
A few marginal tables, with a public list of cells	Noisy counts on every cell, one budget share per table
Hundreds of tables or thousands of cells	Gaussian noise on every cell, scaled to the whole set of tables
A table whose list of values is not public	Noisy counts, keeping only cells that clear a noisy threshold (Release statistics with a library)
Far more queries than records, or queries not known in advance	Synthetic data from AIM or MST
Queries that arrive one at a time, perhaps adaptively	Online private multiplicative weights

Where "a few" tables end follows from the dense-query rule of the Laplace and Gaussian mechanisms. One person sits in every table, so the tables form a dense query, and at a privacy level ε of 1 and a slack δ of 10^{-5}, Gaussian noise wins from 7 tables on.

24.3. Noise on every cell

Under add/remove neighbours, a person with one record changes one cell of a marginal table by 1, so the whole table has sensitivity 1, whatever its number of cells. Laplace noise of scale $1/\varepsilon$ on every cell then makes the table ε-DP, by parallel composition. Under replace neighbours a person can move from one cell to another, so two cells change and the sensitivity doubles.

Several tables of the same people do compose, because one person sits in one cell of every table. With Laplace noise the budget is split among the tables, so the error per cell grows in proportion to their number. Gaussian noise is calibrated to the square root of the sum of squared changes, which for one person in m tables is $\sqrt{m}$, so its error grows only like $\sqrt{m}$.

An example from the full edition shows the gap. A table of 10,000 records has eight yes/no attributes, so $2^8 = 256$ possible records. The release is every cell of the marginal table of every set of attributes, 6,560 cells in all, each as a fraction of the records, under replace neighbours. A uniform guess, which ignores the data, has a largest error of 0.232 over these cells. At $\varepsilon = 1$, Laplace noise on every cell does worse than that, with 0.497. Gaussian noise does much better, with 0.037, at a slack δ of 10^{-6}, stricter than the book's default.

Two mistakes are common. Scaling each cell's noise to its own sensitivity, as if it were the only query, silently spends ε once for every table. And splitting ε evenly over thousands of separately noised cells with basic composition wastes most of the budget. Calibrate to the whole set of tables instead.

24.4. Releasing a marginal with OpenDP

The script practitioner_adult_statistics.py releases the education-by-income table of the Adult census data, 48,842 people with one row each, at $\varepsilon = 1$. The survey's documentation lists 16 levels of education and two income classes, so the keys are public and the table has 32 cells. Empty cells are released too, since omitting one would reveal that nobody has those values. Map a missing value to a listed key such as "Unknown", because a row whose key is not in the list is dropped. In OpenDP, a `Margin` (a setting, not a marginal table) with `invariant="keys"` declares the list of cells public, and `with_keys` pins the release to exactly those cells.

```
keys = pl.LazyFrame({"education": EDUCATION}).join(pl.LazyFrame({"income": INCOME}), how="cr
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["education", "income"], invariant="keys")],  # the cells a
)
query = context.query().group_by("education", "income").agg(dp.len(signed=True)).with_keys(k
print(query.summarize(alpha=0.05))
released = query.release().collect()  # all 32 cells, zeros included; changes on every run
```

Run it with `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`. The summary is deterministic.

```
shape: (1, 5)
┌────────┬──────────────┬──────────────────┬───────┬──────────┐
│ column ┆ aggregate    ┆ distribution     ┆ scale ┆ accuracy │
│ ---    ┆ ---          ┆ ---              ┆ ---   ┆ ---      │
│ str    ┆ str          ┆ str              ┆ f64   ┆ f64      │
╞════════╪══════════════╪══════════════════╪═══════╪══════════╡
│ len    ┆ Frame Length ┆ Integer Laplace  ┆ 1.0   ┆ 3.375618 │
└────────┴──────────────┴──────────────────┴───────┴──────────┘
```

Each cell gets integer Laplace noise of scale 1 (Discrete and floating-point-safe mechanisms). The 95% accuracy of 3.38 means each noisy count lands that close to the truth 95% of the time. Print that error bar beside every cell, and remember that it covers the noise only, not sampling error or mistakes in the data. OpenDP's noise has no seed, so these four cells come from one run, beside the true counts.

```
  Doctorate >50K: 432  true 431  accuracy 3.38
  HS-grad <=50K: 13281  true 13281  accuracy 3.38
  Preschool <=50K: 82  true 82  accuracy 3.38
  Preschool >50K: 0  true 1  accuracy 3.38
```

The large cells barely move, but the smallest true cell, one person with a preschool education and a high income, is smaller than the noise. Noise of a fixed size is large only next to small counts, so if small cells matter, merge rare categories in advance. With `signed=True` a count may come out negative, which keeps it unbiased. Clamping it at zero, rounding, and scaling the cells to add up to a total already released are post-processing, so they cost nothing.

Three things go wrong. Taking the list of cells from the data, as `unique()` on a private column does, reveals rare values by itself. Without a codebook, use partition selection (Release statistics with a library). A second table of the same people needs its own share of the budget, so set `split_evenly_over` to the number of tables. And for many tables, give the budget as `dp.loss_of(rho=...)`, with the ρ that converts to your (ε, δ) (Release statistics with a library shows how). OpenDP then uses Gaussian noise, which composes far better, and accounts in zero-concentrated differential privacy (zCDP) (Concentrated differential privacy).

24.5. Synthetic data

When the queries far outnumber what noise on every cell can bear, or nobody knows them in advance, release synthetic data instead. The practical tools follow four steps.

1. **Select** a set of marginal tables to measure, from domain knowledge or privately with the exponential mechanism, scoring each table by how badly the current model fits it.
2. **Measure** each whole table with Gaussian noise, scaled to the whole set.
3. **Generate** a compact graphical model that fits the noisy tables.

4. **Sample** synthetic rows from the model.

Everything after the measurements is post-processing, so the synthetic rows can be queried and shared at no further privacy cost. The standard methods are MST and AIM, both by McKenna and coauthors, and AIM chooses which tables to measure as it goes. SmartNoise Synth, the `snsynth` package, wraps both behind one interface. It is not installed in this book's environment, so the call below was not run; install it in an environment of its own.

```
# smartnoise-synth 1.0.8: not installed in this book's environment, not run
from snsynth import Synthesizer

synth = Synthesizer.create("aim", epsilon=1.0, delta=1e-6)
synth.fit(adult_pandas, categorical_columns=["education", "sex", "race", "occupation", "inco
rows = synth.sample(48_842)
```

A synthetic table is accurate on the tables it measured, by construction, so evaluate it on other queries, but not freely against the private data. Keeping the best of several runs, judged against the private data, is a selection on that data, and each extra run spends the generator's budget again. So score candidate settings on a public dataset of the same shape, or charge the comparisons to the budget. And a generator trained without DP is not anonymised, however synthetic its rows look.

24.6. Private multiplicative weights

The synthetic-data tools keep the select, measure and update loop of the **multiplicative weights exponential mechanism (MWEM)** of Hardt, Ligett and McSherry (2012). MWEM is the practical form of private multiplicative weights. It keeps a guessed histogram with one weight for every possible record, 256 in the eight-attribute example, starting from uniform, and runs a fixed number of rounds. Each round selects a query that the guess answers badly, with the exponential mechanism, measures it with fresh Laplace noise, and moves the guess toward the measurement. An online-learning bound caps the number of corrections ever needed, whatever the number of queries. So the budget pays for corrections rather than queries, and the error grows only with the logarithm of the number of queries.

That pays off only for very large sets of queries. On the eight-attribute example at privacy level 1, with 38 rounds, its largest error is 0.078. That beats Laplace noise but loses to Gaussian noise by a factor of about two. Its running time also grows with the number of possible records, which limits it to a few attributes.

Three rules make an implementation correct. Fix the number of rounds before seeing the data, by minimizing the error bound, which uses only public quantities. Measure with fresh noise, because the selection step's scores are exact and never released, so reusing one would use the data without paying for it. And release the average of the guesses, not the last one, because the accuracy guarantee covers only the average. For queries that arrive one at a time, an online form answers from the guess and pays only for its corrections (Adaptive data analysis).

24.7. Cheat sheet

Table 24.2.: The query-release methods of this chapter.

Method	Privacy	Error grows with
Laplace noise on every cell	ε; one table costs ε under add/remove	The number of tables
Gaussian noise on every cell	(ε, δ) or zCDP	The square root of the number of tables
Private multiplicative weights	ε or (ε, δ)	The logarithms of the numbers of queries and possible records
Select, measure, generate (MST, AIM)	(ε, δ)	No closed form; the practical choice for many attributes

Rules of thumb

- Take the list of cells from a codebook, never from the private data.
- Add noise to whole tables, scaled to the set of tables, and post-process freely afterwards.
- Fit synthetic data once, and never pick the best of several runs on the private data.

24.8. References

- McKenna, Mullins, Sheldon & Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (VLDB 2022): the select, measure and generate loop as used in practice.
- Hardt, Ligett & McSherry, "A Simple and Practical Algorithm for Differentially Private Data Release" (NeurIPS 2012): MWEM, with experiments on real tables.

Derivations. Every result here is derived step by step in the full edition's chapter.

25. Beyond global sensitivity

25.1. What it is

The Laplace and Gaussian mechanisms set their noise by the **global sensitivity**, the most one record can move the answer over every pair of neighbouring datasets that could exist. The median of ages between 0 and 100 has global sensitivity 100. So under differential privacy (DP) at a privacy level of $\varepsilon = 1$, the Laplace mechanism's median error is $100 \ln 2 = 69.31$ years. Yet on a simulated sample of 1,001 ages, one person moves the median by only 0.1386 years. That smaller number is the **local sensitivity**, the most one record can move the answer f starting from the dataset D you hold:

$$\mathrm{LS}_f(D) = \max_{D' \text{ a neighbour of } D} |f(D) - f(D')|. \tag{25.1}$$

The techniques of this page add noise on the scale of the data you hold, under one rule. **A noise scale, bound or threshold computed from the data is itself a statistic of the data, so it must be private too.** Throughout, datasets have a fixed size n, and neighbours replace one record.

25.2. When you meet it

You meet these techniques whenever a statistic has no useful worst case (Table 25.1).

Table 25.1.: Which technique releases what, and where you meet it.

You release	Technique	Where you meet it
A sum, mean or gradient of unbounded values	Clipping to a bound fixed in advance	Every library's `bounds`, and the clipping norm of private training
A median or quantile	Inverse sensitivity	`diffprivlib.tools.quantile`, OpenDP's `dp.quantile`
A discrete answer with a clear winner	Stability-based release	The threshold of private partition selection
A black-box model or estimator	Subsample-and-aggregate	Teacher ensembles for private prediction

A **stability-based release** publishes a discrete answer, such as the most common value, exactly, but only when a noisy test shows that it leads the runner-up by about $2\ln(1/\delta)/\varepsilon$ records, for a

slack δ. Otherwise it refuses, so plan for refusals downstream. The same test decides which keys found in the data a group-by may release (Release statistics with a library).

Subsample-and-aggregate runs any estimator on disjoint blocks fixed in advance, clips each block's answer and averages them with noise. Private aggregation of teacher ensembles (PATE) is its voting form (Private prediction).

Propose-test-release proposes a bound on the local sensitivity, checks with noise that the data are many records away from breaking it, and refuses otherwise. **Smooth sensitivity** scales heavy-tailed noise to a bound that cannot jump between neighbours. PATE uses it to publish the ε of its votes, but on the 1,001 ages it is about twelve times less accurate than inverse sensitivity (below).

25.3. How to use it

Most of the time you bound the inputs. **Clipping** replaces each value x by $\min(x, C)$, for a clip bound C fixed before seeing the data. For values that cannot be negative, one record then moves the sum by at most C and the mean of n records by at most C/n. Laplace noise of scale $C/(n\varepsilon)$ on the clipped mean gives ε-DP, and raising C shrinks the bias but grows the noise.

Choose C from domain knowledge, such as a legal maximum or a sensor range. Failing that, assume a distribution in advance and take the C that minimizes bias plus noise under it, or spend a small slice of the budget, charged by composition, on a private high quantile and clip there. If you omit the bound, diffprivlib computes it from the data and issues a `PrivacyLeakWarning`. So turn that warning into an error with `warnings.simplefilter("error", PrivacyLeakWarning)`, as practitioner_adult_statistics.py does. Bounding the rows each person contributes is the same move for a whole person (Release statistics with a library).

For a median or quantile, use **inverse sensitivity**, which favours each candidate output by how few records must change before the median equals it. That is the exponential mechanism, with pure ε-DP, so pass it bounds or a candidate grid fixed in advance. On the 1,001 ages at $\varepsilon = 1$, its median error is 0.0783 years, below even the local sensitivity of 0.1386.

25.4. What goes wrong

Noise scaled to the local sensitivity is not private. The five ages $(0, 0, 0, 0, 100)$ have median 0, and no single change moves it, so they get no noise. Their neighbour $(0, 0, 0, 100, 100)$ has a positive local sensitivity, so it gets continuous noise and its output is never exactly 0. An output of exactly 0 therefore names the first dataset. The same leak hides in a clip bound set to the observed maximum, a candidate grid built from the data's values, or keys read off the data. In review, look for a `max()`, `quantile()` or `unique()` on the private data that feeds a parameter.

Clipping from above always biases a mean downward, and a two-sided clip cancels the bias only when the range is centred on the data (Private mean estimation). If neighbours add or remove a record, n is not public, so release a noisy clipped sum and a noisy count and divide them.

Derivations. Every result here is derived step by step in the full edition's chapter.

26. Discrete and floating-point-safe mechanisms

26.1. What it is

The proof of the Laplace mechanism is about densities on the real line, but a computer samples floating-point numbers, a finite set with uneven gaps. The textbook sampler adds $\pm b \ln(1/U)$ to the true answer, for a noise scale b and a uniform draw U. Mironov (2012) showed that in double precision it leaks the true answer through the last bits of its output. Floats are dense near 0 and sparse far from it, so many outputs can come from a true answer of 0 or of 1, but not from both, and such an output names its input with certainty.

Differential privacy (DP) cannot cover that. It caps how much one record can change the chance of any output, through a privacy level ε and a slack δ (The definition of differential privacy). Take a set S of outputs that a dataset D can reach but its neighbour never can. No factor e^{ε} can lift the neighbour's probability of S above 0, so for a mechanism M the slack must cover

$$\delta \geq \Pr[M(D) \in S]. \tag{26.1}$$

In a toy sampler with 12-bit floats, such outputs carry probability 0.4791, and more bits do not shrink it.

So **DP is a property of the distribution the code actually produces.** There are two repairs. The **snapping mechanism** keeps floats, but clamps the answer, adds noise, rounds to a coarse power-of-two grid and clamps again. The other makes the query integer-valued and adds exact discrete Laplace or discrete Gaussian noise.

26.2. When you meet it

You meet this when you review code that adds noise itself, such as the unsafe `x + np.random.laplace(0, b)`, and when you choose a library. OpenDP's samplers are exact, and on integer data they give discrete Laplace and discrete Gaussian noise. diffprivlib offers `mechanisms.Geometric`, its name for the discrete Laplace, `mechanisms.GaussianDiscrete` and `mechanisms.Snapping`. Its continuous `Laplace`, used by its `tools` functions, resists this attack by drawing four uniform variates (Holohan & Braghin 2021), a mitigation rather than an exact sampler. Secure aggregation needs integer noise too (Local, shuffle and distributed trust models).

In private training, Opacus adds continuous Gaussian noise in floating point, so the proof of Private stochastic gradient descent covers only idealized code. Its `secure_mode=True` draws the noise from a cryptographic generator to blunt these attacks. Check that a released model used it and states its guarantee for the idealized mechanism.

26.3. How to use it

Use a library on integer data. Its **discrete Laplace** noise gives each integer z a probability proportional to $e^{-|z|/b}$, and you calibrate it like continuous noise. For an integer query whose sensitivity is Δf, it gives $\varepsilon = \Delta f/b$, and at $b = 1$ a count comes out exact with probability 0.4621.

The **discrete Gaussian** gives each integer x a probability proportional to $e^{-x^2/(2\sigma^2)}$, for a width σ. Its exact privacy curve differs slightly from the continuous one: at sensitivity 1 and $\sigma = 1$, its δ at $\varepsilon = 1$ is 0.1414, against 0.1269. So either report the discrete curve, or account as OpenDP does in zero-concentrated DP. There the discrete Gaussian keeps the continuous one's privacy parameter, $\rho = (\Delta f)^2/(2\sigma^2)$. In the snippet, `map(1)` gives each guarantee at sensitivity 1.

```
import opendp.prelude as dp

dp.enable_features("contrib")
integers, distance = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace = dp.m.make_laplace(integers, distance, scale=1.0)
print(laplace.map(1))  # 1.0
gauss = dp.m.make_gaussian(integers, distance, scale=2.0)
print(gauss.map(1))  # 0.125
print(gauss.output_measure)  # ZeroConcentratedDivergence
```

A real-valued query, such as a sum of clipped values, becomes integer in three steps. Divide by a step much smaller than the noise and round randomly, up with probability equal to the fractional part, so that the result stays unbiased. Recompute the sensitivity in those units, since rounding can push neighbours slightly apart. Then add discrete noise and multiply back.

26.4. What goes wrong

Ask these questions of any implementation.

- Is every sensitivity computed from bounds fixed in advance, with no `max()`, `quantile()` or `unique()` on the private data (Beyond global sensitivity)?
- Under add/remove, is a mean a noisy sum over a noisy count, not over the true n (The Laplace and Gaussian mechanisms)?
- Do all releases about the same people draw on one budget (Post-processing, group privacy and composition)?
- Is an exact statistic of the same data, such as a maximum, published beside the private ones, with no guarantee?
- Is the noise drawn by a vetted library, or by an exact sampler with no floating-point `exp`, `log` or `random()` inside? Search for `np.random`, `random.`, `scipy.stats`, `default_rng`, `.laplace(` and `.normal(`.
- Does integer data get integer noise, and is every floating-point output snapped and clamped to a fixed range?
- Does each release draw fresh noise from a cryptographic source, such as `secrets.SystemRandom()`, rather than a fixed seed that makes it removable?

- Is the running time fixed or padded? A rejection sampler's rounds depend on the value drawn, which Jin et al. (2022) exploit.
- Does the stated guarantee match the mechanism, including the discrete Gaussian's own curve and the sensitivity after rounding?

Derivations. Every result here is derived step by step in the full edition's chapter.

27. Local, shuffle and distributed trust models

27.1. Why this matters

Every mechanism so far in this part assumed a **trusted curator**, which holds the raw data and releases only the mechanism's output. In this **central model**, the differential privacy (DP) guarantee protects what the curator publishes, but not against a breach of its database, a curious engineer or a court order. A browser vendor that collects usage statistics is often not trusted that far.

Three other models trust less. In the **local model** nobody is trusted, and each person randomizes their own record before it leaves their device. The **shuffle model** trusts a shuffler only to cut the link between each report and its sender. The **distributed model** trusts cryptography that reveals only a sum, run by enough honest participants. The closer to each person the noise is added, the fewer parties you trust, and the more noise reaches the answer.

27.2. Choosing a trust model

That extra noise is the price of trusting less. As the population grows, the noise on a private count stays fixed in the central model but grows with its square root in the local model. On the full edition's eight-value histogram over a hundred thousand people, the local model's squared error is about 42,000 times the central one at the same guarantee. The shuffle and distributed models close most of that gap.

So use the central model whenever a curator can be trusted with the raw data, as Release statistics with a library does end to end. Choose local only when nobody may see a raw record and the population is large enough, which you check by solving the standard deviation of Section 27.3 for the number of people. Either way, state the trust model next to the privacy parameters, because the same ε protects against a different adversary in each model.

27.3. Central and local differential privacy

In the local model, each person sends only a **report**, the output of a **local randomizer** R run on their own record. Write ε_0 for the privacy level of one report. The randomizer is ε_0-DP when no set of possible reports becomes more than e^{ε_0} times as likely when the record changes. In symbols, every set of reports S and every two records x and x' satisfy

$$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{27.1}$$

Changing one person's record changes only their own report, so the whole collection is then ε_0-DP in the central sense, with neighbours that differ in one replaced record. The guarantee covers only the randomized value, so a network address stored with a report links it back to the person.

The basic local randomizer is randomized response on one bit: each person reports their true bit with a probability $p = e^{\varepsilon_0}/(1 + e^{\varepsilon_0})$, and the flipped bit otherwise. The analyst undoes the expected flips by subtracting $1 - p$ from the share of reported ones and dividing by $2p - 1$. The estimate's standard deviation then depends only on the level and the number of people n:

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0} - 1)\sqrt{n}}. \tag{27.2}$$

The script practitioner_adult_statistics.py runs both models at $\varepsilon_0 = \ln 3$ on the 48,842 people of the Adult census data, where each bit says whether a person's income is over \$50K. In the local model, each bit goes through diffprivlib's `Binary` mechanism, and the analyst sees only the reports.

```
answers = (adult["income"] == ">50K").cast(pl.Int8).cast(pl.String).to_list()  # "1" or "0"
n, eps = len(answers), log(3)
coin = Binary(epsilon=eps, value0="0", value1="1", random_state=seed)
reports = [coin.randomise(a) for a in answers]  # each person randomizes their own answer
p = exp(eps) / (1 + exp(eps))  # chance a report is truthful: 0.75
reported = reports.count("1") / n
local = (reported - (1 - p)) / (2 * p - 1)  # undo the expected flips
local_sd = math.sqrt(p * (1 - p) / n) / (2 * p - 1)  # the randomization noise alone
print(f"true share over 50K: {answers.count('1') / n:.4f}")
print(f"local estimate: {local:.4f}  sd {local_sd:.4f}")
```

In the central model, a trusted curator counts the true bits with diffprivlib's `tools.count_nonzero` at the same level, which adds Laplace noise with standard deviation $\sqrt{2}/\varepsilon$ to the count.

```
rich = (adult["income"] == ">50K").to_numpy()
central = tools.count_nonzero(rich, epsilon=log(3), random_state=seed) / len(rich)
central_sd = math.sqrt(Laplace(epsilon=log(3), sensitivity=1).variance(0)) / len(rich)  # th
print(f"central estimate: {central:.4f}  sd {central_sd:.6f}")
```

Its seed-0 run, `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`, prints this.

```
true share over 50K: 0.2393
local estimate: 0.2359  sd 0.0039
central estimate: 0.2393  sd 0.000026
local sd / central sd: 149
```

So at the same guarantee, the local estimate is 149 times noisier, and the factor grows like the square root of the population. It belongs to the model, not to randomized response: no local protocol can estimate a fraction with an error below order $1/((e^{\varepsilon_0} - 1)\sqrt{n})$, and randomized response reaches that order. That is why Apple, Google and Microsoft use local DP only for the most frequent items in very large populations.

27.4. Frequency estimation: k-ary randomized response and unary encoding

The workhorse of deployed local DP is **frequency estimation**. Each person holds one value from a known list, such as an emoji, and the analyst wants the share holding each one. A report votes for a value with some probability p_1 if the person holds it, and with a smaller probability p_0 otherwise. If c_v of the n reports vote for a value v, removing the false votes on average gives the unbiased estimate

$$\hat{f}_v = \frac{c_v/n - p_0}{p_1 - p_0}. \tag{27.3}$$

Two designs set p_1 and p_0 for a list of k values. In **k-ary randomized response**, each person reports their true value with probability $e^{\varepsilon_0}/(e^{\varepsilon_0}+k-1)$, and another value otherwise. **Optimized unary encoding** sends one bit per value, keeping the 1 of the value held only half the time but almost never turning a 0 into a 1. Most values of a long list are rare, and a rare value's variance sets the smallest share that can be seen. Comparing those variances, use k-ary randomized response when $k < 3e^{\varepsilon_0} + 2$, and unary encoding otherwise, at the price of a k-bit report. The pure-ldp Python package implements both.

Take a keyboard app that wants the share of each of 128 emoji from a million users at $\varepsilon_0 = 1$. A rare emoji's estimate has a standard deviation of 0.0019 under unary encoding, against 0.0066 under k-ary randomized response. By convention, an estimate must stand three standard deviations above zero to be told from noise. So even the better design loses every emoji used by fewer than about 3×0.0019, or 0.6%, of users. Size the population from that floor before collecting. For domains too large to list, local hashing and heavy-hitter protocols send a few bits per report instead of one per value (Wang et al. 2017; Bassily et al. 2017).

27.5. Numbers in one bit: local mean estimation

For numbers between 0 and a known bound, such as minutes in an app, one bit per person is enough (Ding, Kulkarni & Yekhanin 2017). The chance of sending a 1 rises in a straight line with the value, so the analyst can invert that line to get an unbiased mean. For a million users, a bound of 1,440 minutes and $\varepsilon_0 = 1$, the mean has a standard deviation of 1.420 minutes. Fix the bound before collection and clip to it on the device, because a bound computed from the reports would let one person's value change everyone's randomizer.

27.6. Collecting every day: memoization

Telemetry asks the same question every day. An analyst who averages the daily reports of an unchanged value averages away their fresh noise, and composition counts that leak by adding each report's ε_0. The fix is **memoization**: the device draws one permanent randomized response per value and reuses it, so an unchanged value costs only that response's level, written ε_∞. RAPPOR, Google's collection of Chrome users' homepages, has $\varepsilon_\infty = 2.043$.

Memoization fails when the value changes, since each new value costs another ε_∞. A user who changes their homepage every month reaches $12 \times 2.043 = 24.5$ in a year.

27.7. The shuffle model and amplification by shuffling

In the **shuffle model** (Cheu et al. 2019), each person sends a local report to a **shuffler**, which strips identifiers and timing and forwards the reports in random order. Each report then hides among the others, so the collection meets central (ε, δ)-DP at an ε far below ε_0 (Amplification by shuffling and by iteration). The closed-form bound of Feldman, McMillan & Talwar (2021) gives that ε only when there are enough people to hide among:

$$\varepsilon_0 \le \ln \frac{n}{16 \ln(2/\delta)}. \tag{27.4}$$

In practice you fix the central target and solve for the largest ε_0 it allows. For a central $(1, 10^{-6})$, ten thousand people may each randomize at $\varepsilon_0 = 3.504$, while at a thousand people the condition above binds first and caps it at 1.460. Both use a δ below the book's default $1/n^{1.1}$. The closed form overstates ε by a factor of 2.6 to 4.6 at ten thousand people, so plan with it, and deploy with the paper's published numerical bound.

Three things undo the amplification. A shuffler that colludes with the analyst leaves each report at ε_0, so choose an ε_0 you could defend on its own, within the tiers of the definition. Fake senders give no cover, so count only honest ones in n. And real shufflers batch and pad their traffic, so that timing and sizes do not leak.

27.8. Distributed DP: splitting the noise

In the **distributed model**, each client adds a small share of the central noise, and **secure aggregation** reveals only the sum to the server (Cryptography basics). Gaussian noise splits cleanly, because variances of independent shares add. If each of n clients adds normal noise of variance σ^2/n, the shares sum to central noise with standard deviation σ, as private as the central Gaussian mechanism.

The small shares are also the weakness. A client that colludes with the server can reveal its share, and one that drops out adds none. With t such clients, only the honest clients' noise is left, with standard deviation

$$\sigma \sqrt{\frac{n-t}{n}}. \tag{27.5}$$

So fix in advance the largest fraction ρ of clients that may collude or drop out, and have each client add variance $\sigma^2/(n(1-\rho))$. The honest clients alone then supply the full noise.

For example, a server sums a histogram of about a hundred thousand users through 100 clients and targets $(1, 10^{-6})$, the default δ for that many users. A user who changes value moves one count down and another up, so the ℓ_2 sensitivity is $\sqrt{2}$. The analytic Gaussian calibration of the Laplace and Gaussian mechanisms gives a total σ of 5.975, or $5.975/\sqrt{100} = 0.597$ per client. Without over-provisioning, a fifth of the clients colluding weakens the guarantee to $\varepsilon = 1.128$ at the same δ, and nine-tenths to 3.525.

Secure aggregation works on integers, so clients add integer noise, Skellam or the distributed discrete Gaussian of Discrete mechanisms, as TensorFlow Federated's `tff.aggregators.DistributedDpSumFactory` does. And it does not check inputs, so a malicious client can still send any vector.

27.9. Cheat sheet

Table 27.1.: Which trust model and protocol to use, by situation.

Situation	Use	Rule
A curator can be trusted	Central Laplace or Gaussian	The default
Nobody may see raw records	Local randomized response	Frequent items, very large populations
Frequencies over k values	k-ary randomized response or unary encoding	k-ary if $k < 3e^{\varepsilon_0} + 2$
The same question daily	Memoization	Each new value costs ε_∞
An anonymizing relay	Shuffle model	Deploy with a numerical bound
Federated sums	Distributed Gaussian or Skellam	Over-provision for colluders

Rules of thumb

- Local noise exceeds central noise by a factor that grows like $\sqrt{n}$, 149 on Adult at $\ln 3$.

27.10. References

- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): frequency oracles and local hashing, compared.
- Feldman, McMillan & Talwar, "Hiding Among the Clones" (FOCS 2021): the shuffling bounds, closed form and numerical.

Derivations. Every result here is derived step by step in the full edition's chapter.

28. Lower bounds: what privacy costs

28.1. What it is

A **lower bound** says that no mechanism satisfying differential privacy (DP) can be more accurate than some floor. It holds for every mechanism at once, so it works as a feasibility check. Accuracy has two numbers here. The **error** α is how far the output may land from the true answer, and the **failure probability** β is the chance that it lands further away.

The floor you meet most is for a histogram over N bins under pure DP, with a privacy level ε and no slack. If such a histogram is within α records of every count with probability at least $1 - \beta$, then

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{28.1}$$

The floor grows only with the logarithm of the number of bins, but under pure DP it keeps growing. Under approximate DP, which allows a small slack δ, it stops at about $\ln(1/\delta)/\varepsilon$ records, however many bins there are.

28.2. When you meet it

You meet a lower bound when a design looks too good. Take a request for a histogram over a million locations at $\varepsilon = 1$ with no slack, accurate to five records in every bin with 95% probability. First ask whether one person moves one of the statistics, as in the disjoint cells of a histogram, or all of them, as in the means of many attributes. The histogram floor governs the first case, and the fingerprinting and reconstruction floors below the second. The floor then decides whether to keep tuning. A target below it needs more data, a coarser answer or a weaker guarantee. A mechanism within a small factor of it is done, because no algorithm gains more than that factor.

28.3. How to use it

For the million locations, every pure-DP histogram has an error of at least 6.38 records in some bin, so the five-record promise is impossible. The Laplace histogram of the Laplace and Gaussian mechanisms has an error of 16.79, about 2.6 times the floor, so it is already near the best. With $\delta = 10^{-6}$, the floor never passes about $\ln(10^6) = 13.82$ records. A stability-based histogram, which releases only non-empty bins, has an error that does not grow with N at all (Beyond global sensitivity). So when a huge, mostly empty universe puts the floor too high, switch to approximate DP.

The same floor sizes a dataset, because dividing an error in records by the number of records n gives an error in frequencies. A pure-DP histogram whose frequencies must all be within α needs

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{28.2}$$

If the data are a sample from a population, sampling costs records too, about $\ln(2N/\beta)/(2\alpha^2)$ by Hoeffding's inequality, and you need the larger of the two. At $\varepsilon = 1$ and 95% probability, a million cells at an error of 0.1 need at least 64 records for privacy and at most 875 for sampling. At an error of 0.01, privacy needs 638 and sampling 87,522. So at $\varepsilon = 1$ it is sampling, which no mechanism fixes, that makes fine answers short of records.

When one person moves many statistics, the rate changes. Releasing the means of d yes/no attributes costs records in proportion to d under pure DP. Under approximate DP, the **fingerprinting** bound puts the cost at about $\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon)$ records, and the Gaussian mechanism matches it to within $\sqrt{\ln d}$. So use approximate DP for outputs with many coordinates, such as a gradient, and once the Gaussian noise is calibrated, look for more records, not a better mechanism. For about n counting queries on n records, the reconstruction attack of Dinur and Nissim sets the floor at an error of order $\sqrt{n}$ per answer (Reconstruction attacks). With far more queries than records, answer them from one private synthetic dataset, whose error grows only slowly with their number (Query release).

28.4. What goes wrong

Aggregates are not a defence. The attack behind the fingerprinting bound, from Homer and colleagues (2008), found from a genome study's published frequencies of genetic variants whether a given person took part. It scores how far the released means lean, attribute by attribute, towards the target's record and away from the population means. Members score higher, because each pulls the means toward their own record. On the means of 50 people over 2,000 attributes, it reaches a true positive rate (TPR) of 0.242 at a false positive rate (FPR) of 1%. A $(1, 10^{-5})$-DP Gaussian mechanism cuts that TPR to 0.014.

So use the score, with population means from a separate sample, to audit any release of many averages. Compare its TPR at a low FPR with the cap $\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta$ from the definition. An audit that beats the cap has found a bug in the mechanism or its accounting. Passing proves nothing, because an attack only bounds ε from below (Auditing private training).

Every approximate-DP floor here assumes a meaningful δ, such as the book's default $\delta \leq 1/n^{1.1}$ (How small δ must be). Otherwise they mean little: a mechanism that publishes each record outright with probability δ is $(0, \delta)$-DP, so a claim with δ near $1/n$ allows publishing about one record.

Derivations. Every result here is derived step by step in the full edition's chapter.

29. Private mean estimation

29.1. Why this matters

The mean is the statistic that private machine learning computes most often. Each step of private stochastic gradient descent averages clipped per-example gradients, and even a survey share is a mean of 0/1 answers. The planning questions are how many records a private estimate needs, and how to choose a clipping range without looking at the data.

Differential privacy (DP) must hold for every dataset, including datasets your assumptions say cannot happen (The definition of differential privacy). So an assumption such as "the records are normal" may make a mechanism more accurate, but it must never enter the privacy argument. Every mechanism below therefore clips each record to a range, which caps one record's influence whatever the data are. Its noise grows with the width of that range, and a narrow range must not be read off the data. So spend a small part of the budget finding where the data are, then clip there and spend the rest on the mean.

29.2. Bounded data: privacy adds a lower-order term

Throughout the chapter, datasets have a fixed size n, and neighbours differ in one replaced record. An estimate is judged by its **root-mean-square error** (RMSE), the typical distance between the estimate and the mean of the population the records came from.

Start with records in a known interval of width w, such as yes/no answers coded 0 and 1, and write σ^2 for the variance of a record. Release the average plus Laplace noise with scale $w/(n\varepsilon)$, which is ε-DP because one record moves the average by at most w/n (The Laplace and Gaussian mechanisms). Laplace noise has variance twice its scale squared, and it is independent of the sampling error, so the two variances add:

$$\mathrm{RMSE} = \sqrt{\frac{\sigma^2}{n} + \frac{2w^2}{n^2\varepsilon^2}}. \tag{29.1}$$

To plan a study, set the RMSE equal to a target error α and solve for n, as the snippet in the Code section does. The number of records needed is at most

$$\frac{\sigma^2}{\alpha^2} + \frac{\sqrt{2}\,w}{\alpha\varepsilon}. \tag{29.2}$$

So sampling needs about σ^2/α^2 records whatever you do, and privacy adds at most about $w/(\alpha\varepsilon)$ more. For a yes/no survey of a population that is half yes, the worst case for sampling, an RMSE of 0.05 needs 100 people. Privacy at $\varepsilon = 1$ raises that to 108. The privacy term grows only like

$1/\alpha$, against $1/\alpha^2$ for sampling, so it matters most at a coarse target and a small ε. It also grows with the width w, so the rest of the chapter is about narrowing the range.

Libraries make you state the range. In diffprivlib, `tools.mean(x, epsilon=..., bounds=(lower, upper))` clips to the bounds and adds noise. Without `bounds`, it computes them from the observed minimum and maximum, which leaks them, and raises a `PrivacyLeakWarning`. OpenDP likewise needs clamping bounds, its word for clipping bounds (Release statistics with a library).

Bounds that are too tight fail in another way, because they clip real records. The script practitioner_adult_statistics.py releases the mean hours worked per week by the 48,842 people of the Adult census data at $\varepsilon = 1$, once with loose bounds and once with tight ones.

```
hours = adult["hours-per-week"].to_numpy()
loose = tools.mean(hours, epsilon=1.0, bounds=(1, 99), random_state=seed)
tight = tools.mean(hours, epsilon=1.0, bounds=(20, 60), random_state=seed)
clipped = np.clip(hours, 20, 60).mean()  # what the tight bounds can at best recover
print(f"true mean hours: {hours.mean():.3f}")
print(f"bounds (1, 99): {loose:.3f}  bounds (20, 60): {tight:.3f}")
print(f"bias of (20, 60): {clipped - hours.mean():+.3f}")
```

Its seed-0 run, `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`, prints these lines, the last two from code next to the excerpt.

```
true mean hours: 40.422
bounds (1, 99): 40.421  bounds (20, 60): 40.361
bias of (20, 60): -0.061
people outside (20, 60): 8.7%
noise scale (1, 99): 0.0020  (20, 60): 0.0008
```

The tight bounds shrink the noise scale from 0.0020 to 0.0008 hours. But they clip the 8.7% of people outside them, which moves the mean by 0.061 hours, far more than the noise they save. No budget removes that bias, so report the error in three parts: sampling, clipping bias and noise. Choose the bounds from the schema or public data, and check the share they clip on data you may look at.

The formulas also treat n as public, which holds only because neighbours replace a record. If the size is itself private, divide a noisy sum by a noisy count, with the budget split between the two.

29.3. The cost of an unknown range

Now let the records be normal with variance 1, and suppose you know only that their mean lies within some distance R of zero. Normal records rarely stray far from their mean, so write τ for a distance from the mean that holds them all, except with a small failure probability β. For 1,000 records at $\beta = 0.05$, it is 4.604.

The **naive estimator** clips every record to the widest place the data could be, $R + \tau$ on either side of zero, and adds Laplace noise for that width. So its error grows in step with R. With 1,000

records at $\varepsilon = 1$, its RMSE reaches 2828.44 when R is a million. Yet Lower bounds shows that an unknown range need cost only about $\ln R/\varepsilon$ extra records.

Coarse then fine (Karwa & Vadhan 2018) comes close to that floor in two steps. The coarse step cuts the range into bins of width 2τ and picks the fullest bin with the exponential mechanism, spending a part ε_1 of the budget. The records it needs grow only with the logarithm of the number of bins. The fine step clips every record to the chosen bin and its two neighbours, since the data can straddle two bins. It releases their average with Laplace noise for that window. It spends the rest of the budget, so by composition the two steps together are ε-DP.

With a quarter of the budget on the coarse step, the same 1,000 records give an RMSE of 0.0609 for every R from 10 to a million. The fine step has only three quarters of the budget and a window three bins wide, though, so for small ranges the naive estimator wins, here up to $R = 13.81$. Beyond that, use coarse then fine, or locate the data with a private quantile such as diffprivlib's `tools.quantile`. Under approximate DP, with its slack δ, a stability-based histogram does the coarse step with about $\ln(1/\delta)/\varepsilon$ records whatever the number of bins, so R drops out (Beyond global sensitivity).

Two things go wrong. A budget split, bin width or β chosen after looking at the data leaks it. And a coarse step with too few records can pick an empty bin and clip everything to the wrong place.

29.4. Many dimensions: clip the norm

Vectors in d dimensions, such as embeddings or gradients, follow the same recipe with a different measure of size. Clip each record's ℓ_2 norm, its length, to a radius C, so that one replaced record moves the average by at most $2C/n$. Then add Gaussian noise calibrated to that sensitivity (The Laplace and Gaussian mechanisms). This is exactly the step of private gradient descent, and private training gives the rules for choosing the clipping norm there. With unit variance per coordinate, records lie about $\sqrt{d}$ from their mean, so the radius must grow like $\sqrt{d}$. Noise at that scale in every coordinate then grows like d in total, against $\sqrt{d}$ for the sampling error. So in high dimension privacy is the larger cost, and the fingerprinting lower bound shows that no mechanism avoids it.

When you do not know where the mean is, the range problem returns, and a grid of bins would pay for the range once per dimension. The **shrinking balls** of CoinPress (Biswas, Dong, Kamath & Ullman 2020) avoid the grid: each round clips to a ball around a noisy guess of the mean and then shrinks it, so fix the number of rounds and their budget split in advance.

29.5. Heavy tails: clip into the data

Incomes, session lengths and per-example gradient norms have heavy tails. When all you know is that the variance is at most σ^2, a range wide enough to touch no record is enormous. So clip into the data on purpose, and accept a bias.

Now let τ be a clipping half-width: clip every record to within τ of a centre near the mean, found by a coarse step. Only the rare records beyond τ move, so with the centre at the mean the bias is

at most σ^2/τ, while the noise grows in proportion to τ. The bound on the total error is smallest at

$$\tau_b = \sigma\sqrt{\frac{n\varepsilon}{2\sqrt{2}}}, \tag{29.3}$$

which uses only n, ε and the variance bound. There, an error α needs about $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ records. So privacy is no longer a lower-order cost, since both terms grow like $1/\alpha^2$. No ε-DP mechanism does better for every distribution with that variance, though a bound on a higher moment lowers the cost.

Table 29.1 shows the trade-off on Pareto records, a heavy-tailed distribution scaled to variance 1. Its first width clips no record except with probability 0.05, and its noise dominates. The worst-case width τ_b cuts the error to 0.0607, and the best width for this distribution does better still.

Table 29.1.: Clipped private means of Pareto records with variance 1, 1,000 records, $\varepsilon = 1$, for three clipping half-widths.

Clipping half-width τ	Bias	Sampling sd	Noise sd	RMSE
141.42 (no record clipped)	0.0000	0.0313	0.4000	0.4012
18.80 (worst-case best, τ_b)	−0.0018	0.0293	0.0532	0.0607
5.36 (best for this distribution)	−0.0153	0.0249	0.0152	0.0329

So treat τ_b as a safe default, not as the best width. A narrower range usually does better, but choose it without looking at the records, from domain knowledge or with a private quantile (Beyond global sensitivity).

29.6. Code

The snippet computes the records a bounded mean needs, checks its noise with OpenDP, and computes the default clipping width for heavy tails. Setting the RMSE equal to α and multiplying by n^2 gives a quadratic in n, whose positive root the first function rounds up.

```
from math import ceil, sqrt

import opendp.prelude as dp


def records_for_bounded_mean(alpha: float, sd: float, width: float, eps: float) -> int:
    """Smallest n with sd^2/n + 2 (width/(n eps))^2 <= alpha^2."""
    a, c = sd**2, 2 * (width / eps) ** 2
    return ceil((a + sqrt(a * a + 4 * c * alpha**2)) / (2 * alpha**2))
```

```
def heavy_tail_width(n: int, eps: float, sd: float) -> float:
    """The clipping half-width that balances the bias bound sd^2/tau against the noise."""
    return sd * sqrt(n * eps / (2 * sqrt(2)))


n = records_for_bounded_mean(0.05, 0.5, 1.0, 1.0)
print(n)  # 108
print(f"{dp.laplacian_scale_to_accuracy(1.0 / (n * 1.0), alpha=0.05):.4f}")  # 0.0277
print(f"{heavy_tail_width(1000, 1.0, 1.0):.2f}")  # 18.80
```

OpenDP states accuracy as a half-width that the noise stays within with a chosen probability. Its `laplacian_scale_to_accuracy` turns the scale $w/(n\varepsilon)$ at the planned 108 records into that half-width at 95%.

29.7. Cheat sheet

Table 29.2.: Private mean estimation at a glance, with $\tau_b = \sigma\sqrt{n\varepsilon/(2\sqrt{2})}$.

What you know	Mechanism	Records for error α (up to log factors)
Bounded, width w, variance σ^2	Mean plus $\mathrm{Lap}(w/(n\varepsilon))$	$\sigma^2/\alpha^2 + \sqrt{2}\,w/(\alpha\varepsilon)$
Normal, mean within R of zero	Coarse histogram, then clip to three bins	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$
Same, approximate DP	Stability-based coarse step	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln(1/\delta)/\varepsilon$, no R
d dimensions, mean length at most R	Clip the norm, Gaussian noise; shrinking balls to find the mean	$d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$
Variance at most σ^2 only	Clip to within τ_b of the centre	$\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$

Rules of thumb

- Never clip to the observed minimum and maximum. Take bounds from the schema or public data, or spend a quarter of the budget or less, a simple default, finding the data.
- Bounds that clip real records bias the mean whatever ε is, so put the clipping bias into every reported interval.

29.8. References

- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): coarse then fine for normal means and variances, under pure and approximate DP.

- Biswas, Dong, Kamath & Ullman, "CoinPress: Practical Private Mean and Covariance Estimation" (NeurIPS 2020): shrinking balls in practice, with code.

Derivations. Every result here is derived step by step in the full edition's chapter.

30. Release statistics with a library

30.1. Why this matters

Suppose you are the analytics engineer of an online shop. The business wants a dashboard of purchase lines, revenue and the typical sale, overall and per country, and of how many customers bought each product. The figures describe customers, so they should come with a differential privacy (DP) guarantee. The difficulty is that one customer appears in many rows, and a guarantee for one row would protect a single purchase, not the person behind a hundred. This chapter does the whole job with OpenDP, from the unit and the budget to the written guarantee.

The data are the Online Retail set from the University of California, Irvine (UCI) Machine Learning Repository, with 541,909 invoice lines of a British online shop. Each line records one product on one invoice, with its quantity, unit price, customer and country. The script practitioner_release_statistics.py runs every step in under a minute, with OpenDP 0.16.0 and Polars 1.36.1.

```
uv run python code/part03-dp-foundations/practitioner_release_statistics.py
```

OpenDP draws its noise from a cryptographic generator, which cannot be seeded, so the released values change on every run. This page therefore quotes scales, error bars and thresholds, which involve no noise, and shows released values only in a block marked as one run.

30.2. Choose the library

The main question is whether a library can bound the contribution of a person who owns many rows (Table 30.1).

Table 30.1.: Libraries for releasing statistics, and what each can express.

Library	Runs on	Person with many rows	Groups not known in advance	Use it for
OpenDP	Polars, one machine	Yes, with an identifier column	Yes, by a noisy threshold	Statistics on one machine; the default here
diffprivlib	NumPy, scikit-learn	No, one row per person	No	Quick experiments and simple models

Library	Runs on	Person with many rows	Groups not known in advance	Use it for
PipelineDP	Apache Beam, Spark	Yes, built in	Yes, built in	Pipelines that already run on Beam or Spark
Tumult Analytics	Spark	Yes, with an identifier column	Yes, in a separate query	Large releases on Spark

PipelineDP and Tumult were not run here, so their rows follow their documentation. Whatever you pick, write no mechanism yourself, and keep one accountant, which totals the budget spent, for the whole release.

30.3. Decide the unit and the budget

The **unit of privacy** is what the guarantee protects, and for this dashboard it is a customer. The neighbouring datasets therefore differ by adding or removing one customer with all their lines (The definition of differential privacy). The script drops lines with no customer identifier, which belong to nobody, as well as returns and lines with a zero price. Each filter judges a line by that line alone, so adding or removing a customer still changes only that customer's lines.

```
raw = practitioner_datasets.online_retail()
lines, stats = clean(raw)
n_customers = stats.customers
delta = delta_for(n_customers)  # 1/n^1.1, rounded down to a power of ten
per_line = per_customer(lines)
print(f"customers: {n_customers:,}  purchase lines: {lines.height:,}")
print(f"delta: {delta:.0e}")
p50, p90, p99, top = quantiles(per_line, [0.5, 0.9, 0.99, 1.0])
print(f"lines per customer: p50={p50} p90={p90} p99={p99} max={top}")
```

```
customers: 4,372  purchase lines: 397,884
delta: 1e-05
lines per customer: p50=41 p90=208 p99=662 max=7847
```

The slack δ follows the book's default for n people, $\delta \leq 1/n^{1.1}$ rounded down to a power of ten, which gives 10^{-5}. The 4,372 customers are counted before returns are dropped, which leaves 4,338, but both counts give the same slack. OpenDP logs a general warning that δ should be at most 10^{-6}, while this book ties δ to the number of people. The privacy level is $\varepsilon = 1$ for the whole release, because all its tables describe the same customers and their costs add up (Post-processing, group privacy and composition).

The median customer has 41 lines, but under a guarantee per line the largest customer would be a group of 7,847 records. Group privacy multiplies ε by the size of the group. Printing that maximum is fine only because this set is public, since a bound read off private data leaks.

30.4. Bound each customer's contribution

Write k for the largest number of lines a customer may contribute, so that every query keeps at most k lines of each customer. A small k means little noise but throws data away, while a large k keeps the data but needs more noise, as the script measures on this public set.

```
k=10: lines kept 10.2%  customers truncated 84.8%
k=25: lines kept 22.4%  customers truncated 64.6%
k=50: lines kept 36.9%  customers truncated 44.0%
k=100: lines kept 55.1%  customers truncated 24.9%
k=200: lines kept 72.7%  customers truncated 10.4%
k=500: lines kept 88.2%  customers truncated 1.8%
```

The default is a high percentile of lines per customer, read from public or historical data, never from the data you release. With neither, spend a small share of the budget on a private high quantile of the lines per customer, over candidates fixed in advance (Selection). This chapter takes $k = 100$, below the 90th percentile of 208 because the noise grows with k. It keeps 55.1% of the lines and truncates a quarter of the customers.

Keeping the first k lines by date would bias every statistic towards each customer's early purchases, so the script keeps a random sample of k lines per customer. The filter below does what OpenDP's `truncate_per_group(k, keep="sample")` does, but with a seed, so that the true values of the check stay fixed.

```
def sample_per_customer(k: int, seed: int, *by: str) -> pl.Expr:
    """True for at most k randomly chosen lines of each customer (within each value of `by`)
    return pl.int_range(pl.len()).shuffle(seed=seed).over("CustomerID", *by) < k
```

Values need bounds as well. A line's value is its quantity times its unit price, which the script clips to between £0 and £200. The lower end holds because returns were dropped, and the upper end is a bound the business sets, which clips 1.02% of the lines. So one customer can move the revenue by at most 100 lines of £200 each. Take value bounds from the domain, never from the data's minimum or maximum (Beyond global sensitivity).

A group-by needs two more bounds: L_0, the number of groups one customer may touch, and L_∞, the lines a customer may put in any one group. Countries need neither, because however a customer spreads their k lines over countries, the country counts change by at most k in all, which the noise already covers. When customers spread over many groups, as with country by month, bounding L_0 and L_∞ can beat a total k. The noise then scales with $L_0 L_\infty$ for Laplace and $\sqrt{L_0}\, L_\infty$ for Gaussian. Products need both bounds for partition selection (Section 30.8).

30.5. Set up the context

OpenDP keeps the whole release in a **context**, which holds the data, the unit, the total budget and its split between queries. Contexts need `dp.enable_features("contrib")`, which switches on components not yet formally vetted, so a reviewer who needs vetted code must check each call in OpenDP's documentation.

```
context = dp.Context.compositor(
    data=lines.lazy(),
    privacy_unit=dp.unit_of(contributions=1, identifier="CustomerID"),
    privacy_loss=dp.loss_of(rho=rho, delta=delta / 2),  # Gaussian noise; delta/2 for thresh
    split_by_weights=WEIGHTS,  # scalars, countries, products
    margins=[dp.polars.Margin(by=[], max_length=MAX_LINES)],
)
```

The identifier column makes the unit a customer, so `contributions=1` means adding or removing one customer with all their lines, which the truncation filters then bound. For replace, OpenDP takes `changes=` instead. The budget is in **zero-concentrated differential privacy (zCDP)** (Concentrated differential privacy). It states privacy as one number, written ρ, that adds up across queries and converts to (ε, δ) at the end.

With a zCDP budget, the context adds Gaussian noise to every count and sum. That suits the product counts, where one customer touches many groups (The Laplace and Gaussian mechanisms). The script's `rho_for` finds $\rho = 0.0284$, the largest value that OpenDP's own conversion turns into $\varepsilon \leq 1$ with half of δ. The other half pays for the threshold of Section 30.8. Finally, the margin states a public upper bound on the number of lines, which OpenDP needs before it sums floating-point values.

The weights give a quarter of the budget to the overall statistics, a quarter to the countries and half to the products, which need it most. The script prints the split and converts the whole release back to the (ε, δ) the statement will claim.

```
budget scalars: rho=0.0071 delta=1.25e-06
budget countries: rho=0.0071 delta=1.25e-06
budget products: rho=0.0142 delta=2.50e-06
whole release as (epsilon, delta): (1.0000, 1e-05)
```

A query takes its share of the budget when it is built, but spends it only when it is released. Each new query is built with the next unspent share. So if you build all the queries first, every one gets the first share, and the product query gets a quarter's noise but is charged half. So build, check and release each query in turn, as the script does.

30.6. Release overall statistics with error bars

The first query releases the count of lines, the revenue and the median line value. The context holds the raw lines, so each query truncates them itself.

```
value = clipped_value()  # the line value, clipped to [low, high]
scalars = (
    context.query()
    .filter(sample_per_customer(K, seed))  # at most K lines per customer
    .select(
        dp.len(signed=True),
```

```
        value.dp.sum((low, high)).alias("revenue"),
        value.dp.median(candidates=MEDIAN_CANDIDATES).alias("median"),
    )
)
scalar_summary = scalars.summarize(alpha=0.05)
scalars_hat = scalars.release().collect()
```

`signed=True` keeps negative noisy counts, and `summarize` reports, before anything is released, each statistic's noise and 95% error bar.

```
scalars len: Frame Length, Integer Gaussian, scale=1,453.38 95% accuracy=2,850.0
scalars revenue: Sum, Float Gaussian, scale=290,804.33 95% accuracy=569,966.0
scalars median: 0.5-Quantile, GumbelMin, scale=1,453.38 95% accuracy=none
```

With probability 0.95, the released count of lines is within 2,850 of the truth, and the revenue within £569,966. These bars depend only on the bounds and the budget, so you can read them before the release and decide whether the budget is enough.

The output also shows how OpenDP splits a query's share among its statistics: each gets the same noise scale relative to its sensitivity. The count and the median both get 1,453.38, and the revenue, whose sensitivity is 200 times the count's, about 200 times that. A mean, `.dp.mean`, is a sum and a count, so it takes two shares. Its error bar follows from theirs (The Laplace and Gaussian mechanisms). To give one statistic more budget, put it in a query of its own with its own weight. The median is chosen by the exponential mechanism, sampled by `GumbelMin`, among candidates in steps of £0.50 across the value bounds. So its error bar is in ranks, and `summarize` gives none (Selection).

The error bars cover the noise only, not the bias of truncation and clipping, which together keep only 48.5% of the revenue. So the private revenue estimates a truncated total, not the shop's takings. Say so, and choose a larger k if the full total matters more than the noise.

30.7. Group by a public key list

Lines and revenue per country come next, and the key list must not come from the data. The distinct values of the Country column would publish which countries appear in the private data, which no noise covers. So the script fixes its keys in advance. They are the United Kingdom, the 27 EU member states, the 4 of the European Free Trade Association, the United States, Canada, Australia, Japan, China and "Other". A rule renames three of the source's spellings, such as `EIRE` to Ireland, and maps every other value to Other.

```
by_country = (
    context.query()
    .filter(sample_per_customer(K, seed))
    .group_by("Market")
    .agg(dp.len(signed=True), value.dp.sum((low, high)).alias("revenue"))
    .with_keys(pl.LazyFrame({"Market": MARKETS}))  # every key is released, zeros included
```

```
)
country_summary = by_country.summarize(alpha=0.05)
countries_hat = by_country.release().collect()
```

```
per country len: Frame Length, Integer Gaussian, scale=1,185.25 95% accuracy=2,324.0
per country revenue: Sum, Float Gaussian, scale=237,698.48 95% accuracy=465,880.5
```

Every key is released, including the 11 with no purchase lines, whose counts are pure noise, so the release says nothing about which markets have customers. Each market's count has a 95% error bar of 2,324, and its revenue one of about £466,000. Germany's true count of 5,379 is still useful, but its revenue of £131,902 is smaller than its error bar. So publish the error bar with every cell, or merge small markets into Other before the release.

30.8. Group by unknown keys with partition selection

The list of products is not public, and a product bought by one customer would reveal that purchase simply by appearing in the table. So the release decides privately which products to show, by **partition selection**. It releases only the products whose noisy count clears a threshold, so a product with few customers almost never appears. The threshold is what costs δ (Beyond global sensitivity).

To count customers rather than lines, the query first keeps one line per customer and product, so $L_\infty = 1$. It then keeps at most m products per customer, so $L_0 = m$, and the script measures the trade-off that m sets on this public set.

```
m=1: pairs kept 1.6%  customers truncated 97.9%  scale=5.93  threshold=29  products above it
m=5: pairs kept 7.9%  customers truncated 92.4%  scale=13.27  threshold=66  products above i
m=10: pairs kept 15.1%  customers truncated 83.6%  scale=18.77  threshold=96  products above
m=20: pairs kept 27.5%  customers truncated 68.3%  scale=26.54  threshold=138  products abov
m=50: pairs kept 52.7%  customers truncated 38.6%  scale=41.96  threshold=225  products abov
```

The threshold grows with m, from 29 to 225, because each customer can push more groups over it, but the counts grow faster. So the number of products whose true count clears the threshold rises from 1 to 50, and the script takes $m = 50$ (`M` in the code), again chosen on public data.

```
by_product = (
    context.query()
    .filter(sample_per_customer(1, seed, "StockCode"))  # one line per customer and product
    .filter(sample_per_customer(M, seed))  # at most M products per customer
    .group_by("StockCode")
    .agg(dp.len(signed=True).alias("customers"))
)
product_summary = by_product.summarize(alpha=0.05)
products_hat = by_product.release().collect()
```

```
per product customers: Frame Length, Integer Gaussian, scale=41.96 95% accuracy=83.0 thresho
```

A product is released only if its noisy count reaches 225, and a released count has a 95% error bar of 83. Never publish how many products were dropped, or which, since both come from the private data. Products just below the threshold get in when their noise is positive, so treat a count near 225 as a sign of popularity, not as a measurement.

OpenDP's `truncate_num_groups` should bound the groups per customer, but in version 0.16.0 it does not work. With the default `keep="sample"`, `summarize` accepts the query, but `release` fails after spending the query's share, so run each query end to end on public data first. With `keep="first"` it keeps one group fewer than asked, the first in sort order, which biases the counts. The filters above avoid both problems only because $L_\infty = 1$ turns groups into lines. For $L_\infty > 1$ this book knows of no unbiased group bound in this version.

30.9. Check the release before publishing

To check that the code does what the plan says, the script compares the private values with the true ones, after the same truncation and clipping.

```
true lines after truncation: 219,291 (all purchase lines: 397,884)
true revenue after truncation and clipping: 4,318,947 (all lines, unclipped: 8,911,408)
true median line value after truncation: 15.00 (all lines: 11.80)
```

The release itself is random. One run printed the following.

```
released (one run):
  lines 218,972  revenue 4,397,410  median 14.50
  United Kingdom: lines 196,311  revenue 3,927,959
  Germany: lines 7,151  revenue 643,094
  Spain: lines -546  revenue -37,048
  country line counts within their 95% accuracy: 34 of 38
  products released: 63
```

Read it against the error bars: about 95% of the market counts should fall inside theirs. Negative values are noise on small markets, and rounding them up to zero is post-processing, which costs no budget. Expect more than the 50 products whose true count clears the threshold, since products just below it get in when their noise is positive. Compare the private median with the truncated £15.00, not the £11.80 of all lines.

The check fails when a value lies far outside its error bar, which points to a wrong sensitivity. It also fails when far more or far fewer than 95% of the cells fall inside, or when a released product's true count lies far below the threshold. In production you cannot compare against the truth, because a decision to publish that depends on the true values is itself a release of them. So run this check on historical or public data while you develop, and release the real data once, with the code you checked.

30.10. Write the guarantee down

The script prints the claim from the variables that set the bounds and the budget, so it cannot drift from the code.

```
This release satisfies (epsilon = 1, delta = 1e-05)-differential privacy for adding or
removing one customer and all their invoice lines, with at most 100 lines per customer
(sampled at random), at most 50 random products per customer in the product counts, and
line values clipped to [0, 200] pounds, under the central model, using OpenDP 0.16.0 with
Gaussian noise (accounted as 0.0284-zCDP plus 5e-06 for thresholds, converted to epsilon
and delta). The market list was fixed in advance; the product list is not public.
```

The statement names everything a reviewer needs to recompute the claim (Writing the guarantee down). Next to it, say what it does not cover. Another release about the same customers adds its own budget. If that one is pure ε-DP, you can add the (ε, δ) pairs. Or convert it to $\rho = \varepsilon^2/2$, add that to this release's ρ and then convert, whichever total is smaller (Concentrated differential privacy). And the figures describe truncated data, which the dashboard should say. Release and governance maps such a claim to data-protection law.

30.11. Checklist

Table 30.2.: The release checklist: each step, its default and its check.

Step	Default	Check
Unit and budget	One person, add/remove; one ε; $\delta \leq 1/n^{1.1}$	n counts people, not rows
Contribution bound	k from a high percentile of public or historical data; random sample	No bound read from the release data
Value bounds	From the domain or the business	Share clipped and bias stated
Context	zCDP with Gaussian noise; weights by need	ρ converted to (ε, δ) equals the claim
Queries	Build, summarize and release one at a time	Scales match the plan; each query run end to end on public data
Public keys	Fixed in advance, values mapped by a rule; every key released	No key list built from the data
Unknown keys	Partition selection with L_0 and L_∞	Number dropped never published
Check	On historical or public data	About 95% inside their error bars

Step	Default	Check
Statement	Printed by the releasing code	Unit, bounds, version and public information named

30.12. References

- The OpenDP documentation, “Polars” and “Grouping” in its user guide: contexts, margins and partition selection.
- Wilson et al., “Differentially Private SQL with Bounded User Contribution” (PETS 2020): contribution bounding in a production SQL engine.
- Desfontaines, Voss, Gipson & Mandayam, “Differentially private partition selection” (PETS 2022): the threshold and why it needs δ.

Derivations. This workflow has no counterpart in the full edition. The results it uses are derived in the full edition’s chapters on the definition in the full edition, its properties in the full edition, the Laplace and Gaussian mechanisms in the full edition, selection in the full edition and sensitivity in the full edition.

Part IV.

Privacy accounting

A model trained with stochastic gradient descent (SGD) under differential privacy (DP), the method called DP-SGD, takes thousands of noisy steps. Yet you report one guarantee for the run, a privacy level ε with a small slack δ. The composition theorems of Part 3 are far too loose for that. An **accountant** does better, because it tracks a summary of each step's cost that composes tightly, and converts the total to (ε, δ) once, at the end.

The running example is a DP-SGD run of 10,000 steps. Each step samples every record with probability 0.01 and adds noise with multiplier 1. At $\delta = 10^{-5}$, advanced composition certifies $\varepsilon = 24{,}379.55$, while the privacy loss distribution accountant of Google's `dp_accounting` certifies 6.19. So an ε means nothing until you name the accountant behind it, and the sampler and neighbouring relation it assumed.

The part starts with two jobs. Report and review an `ε` claim says what a privacy statement must contain and how to check one. Calibrate and account a DP-SGD run turns a target ε into a noise multiplier for the Adult run of Part 5.

Two full chapters explain what those jobs rely on. Amplification by subsampling shows how much a random batch helps, provided the data loader samples the way the accountant assumes. Rényi differential privacy defines the curve most libraries track.

Short reference pages follow. Three finish the DP-SGD story: the conversion from Rényi DP, numerical accounting, and Gaussian DP, which reads a guarantee as a limit on attacks. Then come adaptive composition for budgets decided during training, concentrated DP for analytics ledgers, and shuffling and iteration.

Every number comes from code you can run with Opacus or `dp_accounting`. By the end, you will be able to calibrate a DP-SGD run, account for it with two libraries, and report ε so that others can reproduce it.

31. Choosing an accountant

31.1. Why this matters

A model trained with differential privacy (DP) is reported with a privacy level ε and a small slack δ. The procedure that computes ε is an **accountant**, and the same run can get 6.24 or 24,379.55 depending on which one you use. Table 31.1 shows three, advanced composition, Rényi differential privacy (RDP) and the privacy loss distribution (PLD), on one run of private stochastic gradient descent (SGD), called DP-SGD. Each record joins each batch with probability $q = 0.01$, the sampling rate, and the noise multiplier σ, the noise as a multiple of the clipping norm, is 1.

Table 31.1.: One DP-SGD run, $q = 0.01$ and $\sigma = 1$, under three accountants: ε at $\delta = 10^{-5}$. The PLD columns round the privacy loss up and down to a grid of width 10^{-5}, so they bracket the true value.

Steps	Advanced composition	RDP, improved	PLD upper	PLD lower
1,000	1258.64	2.1014	1.8332	1.8232
10,000	24379.55	6.7127	6.2377	6.1377

Only the PLD's lower column is a lower bound, so the true ε lies between the two PLD columns. Advanced composition, which adds up each step's guarantee, is useless here, and RDP still overstates ε, by 14.6% on the shorter run and 7.6% on the longer one. Hence the chapter's rule. **Report the tightest upper bound whose assumptions hold for the code you ran, and give enough detail that someone else can recompute it.**

31.2. Which accountant

An accountant is tight only when the summary it keeps of each step fits the mechanism, so the mechanism decides (Table 31.2).

Table 31.2.: Which accountant to report for which mechanism.

Mechanism	Report with	Avoid
Gaussian noise, no subsampling	Gaussian differential privacy (GDP), converted exactly (f-DP and Gaussian DP)	Per-step composition

Mechanism	Report with	Avoid
DP-SGD with Poisson sampling	PLD, or the privacy loss random variable (PRV) accountant (Numerical accounting), with the PLD's lower value as the error bar (its code)	Zero-concentrated DP (zCDP); the GDP central limit theorem (CLT)
Pure-DP steps (Laplace, randomized response)	PLD, exact when the privacy loss takes few values; Laplace with scale b is `LaplaceDpEvent(b/Δ)`, Δ its ℓ_1 sensitivity	Advanced composition
Many statistical releases	A zCDP ledger (Concentrated DP) or PLD	Adding up each release's ε
Noise or budget chosen during the run	A Rényi filter (Adaptive composition)	Plugging realized values into a fixed theorem
Correlated noise across steps: matrix factorization, or the follow the regularized leader (FTRL) optimizer on tree sums, DP-FTRL	The mechanism's own accountant (Matrix mechanisms)	The Poisson DP-SGD accountant
A shuffler of locally randomized reports, or only the last model released	Shuffling and iteration, with its conditions; a loader that shuffles is a sampler (Table 31.4)	The Poisson accountant for a shuffler; a last-model claim for non-convex training

The GDP CLT is an approximation, not a bound. After 1,000 steps it gives 1.6177, below the PLD's certified lower value of 1.8232, a guarantee the run does not have. Opacus's `"gdp"` accountant is this CLT, so never report it; its default PRV accountant is safe.

31.3. The inverse problem: noise for a target ε

Usually you fix the target ε, batch size and epochs, and ask for the smallest σ that meets them. A looser accountant overstates ε, so it asks for more noise. For a target of 3 on the 10,000-step run, RDP needs 1.662 and the PLD only 1.584. So calibrate with the accountant you will report, passing `accountant="prv"` to Opacus's `get_noise_multiplier`, whose default is `"rdp"` (Calibrate and account a DP-SGD run).

31.4. Choosing δ

Treat δ roughly as the chance that the guarantee fails for someone, possibly badly. On a dataset of n people, publishing one random record in full meets a δ of about $1/n$ with ε zero, yet exposes someone completely. So δ must sit well below $1/n$. This book follows Ponomareva et al. (2023) and chooses

$$\delta \leq \frac{1}{n^{1.1}}, \quad \text{rounded down to a power of ten.} \tag{31.1}$$

Count n in the unit the claim protects, which at the user level means users, not records.

For example, run the 10,000 steps of Table 31.1 on 50,000 records, which at $q = 0.01$ is 100 epochs. Then $1/n^{1.1}$ is 6.8×10^{-6}, which rounds down to 10^{-6}. The table's 10^{-5} is half of $1/n$, so it fails the rule. The US National Institute of Standards and Technology (NIST) recommends the stricter $1/n^2$, so report ε there too.

31.5. The statement

Table 31.3 fills in the items that Ponomareva et al. (2023) and NIST ask for, for the run on 50,000 records.

Table 31.3.: A filled privacy statement.

Item	The run on 50,000 records
Setting	DP-SGD in the central model, covering every checkpoint and the final model
Unit and neighbours	One record per person; neighbours add or remove one record
Training	Poisson sampling at $q = 0.01$, 10,000 steps (100 epochs), clipping norm 1, noise multiplier 1
Accountant	PLD, upper value, on the grid 10^{-5}, with the library's version
Guarantee	$(6.96, 10^{-6})$ and, at $1/n^2$, $(9.06, 4.0 \times 10^{-10})$; the PLD's lower values, 6.8574 and 8.9378, show how far each could still fall
Other data	No pretrained weights; no preprocessing fitted on private rows; validation rows held out and not protected
Not covered	Nothing: hyperparameters were tuned on a public proxy dataset, so no tuning run touched the private data

Round an upper bound up, because a lower value is one the accountant has not certified. So the PLD's upper value at 10^{-6}, 6.9574, becomes 6.96. And publish the value of the accountant you

name: with `dp_accounting`'s PLD, which gives 6.9074 (Section 31.8), the claim is 6.91. Calibrate and account a DP-SGD run prints such a statement from code.

31.6. Matching the accountant to the code

An accountant's number holds only if the code does what its theorem assumes. Many papers have reported the Poisson-sampling number while training with shuffled fixed-size batches, a different algorithm (Ponomareva et al. 2023). So check Table 31.4 against the code before you report.

Table 31.4.: A checklist for matching the accountant to the training code.

Item	The accountant assumes	Check in the code	If it does not match
Sampler	Poisson sampling at rate q	Do batch sizes vary from step to step?	The sampler's row of Subsampling: fixed-size batches at $\sigma/2$; shuffle and partition unamplified, for zero-out neighbours
Neighbours	Add or remove one record	Which relation does the claim state?	Replace-one costs more per step; for Poisson sampling, compose the same steps in `PLDAccountant(NeighboringRelati`
Unit	One record	Can one person give several records?	Clip and sample per user, or pay for the group (User-level DP)
Tuning, adaptivity and steps	Settings and steps fixed in advance	Tuning on private data, early stopping, restarts?	Account for the tuning runs (Hyperparameter tuning); report the planned maximum; keep the accountant's state across restarts
Other data	Only the training rows are private	Pretraining, fitted preprocessing, validation rows?	Name them; preprocessing fitted on private rows needs its own budget

The sampler costs the most. At 10^{-5}, the 6.19 that `dp_accounting`'s PLD gives under Poisson sampling becomes 43.37 with fixed-size batches, and 91.82, for zero-out neighbours only, with shuffle and partition (Subsampling).

31.7. Reviewing someone else's claim

A reviewer works in this order.

1. Ask for the inputs of Table 31.3, and reject a claim that cannot be recomputed.
2. Check δ against Equation 31.1, with n counted in the claim's unit.
3. Recompute ε with a different library, and expect agreement within a few percent. A PLD accountant's `get_delta` at the claimed ε gives the δ at which it holds. A claim well above your value is valid but loose.
4. Read the training code against Table 31.4, starting with the sampler.
5. Ask what the claim leaves out, such as tuning runs, pretraining data and preprocessing fitted on the private rows.

For example, a team claims $(2, 10^{-5})$-DP for a model trained on one million users with noise multiplier 0.65. Batches of 1,000 users make the sampling rate q 0.001, and 10 epochs 10,000 steps, but the loader shuffles and takes fixed batches. This snippet does steps 2 and 3, then accounts for the loader as it ran.

```
from math import floor, log10

from dp_accounting import GaussianDpEvent, PoissonSampledDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant

n, q, sigma, steps, epochs = 1_000_000, 0.001, 0.65, 10_000, 10  # the claim's parameters
delta = 10.0 ** floor(log10(n**-1.1))  # the book's rule for one million users
print(f"{delta:.0e}")  # 1e-07
poisson = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
pld = PLDAccountant(value_discretization_interval=1e-4).compose(poisson)
print(f"{pld.get_epsilon(1e-5):.2f}")  # 1.72
print(f"{pld.get_delta(2.0):.1e}")  # 3.0e-06
print(f"{pld.get_epsilon(delta):.2f}")  # 2.84
# What the loader ran: shuffle and partition, zero-out neighbours, no amplification
shuffled = SelfComposedDpEvent(GaussianDpEvent(sigma), epochs)
print(f"{PLDAccountant(value_discretization_interval=1e-4).compose(shuffled).get_epsilon(del
```

The arithmetic holds up, since the claim's parameters give 1.72 at its δ. But that δ is a hundred times the rule's 10^{-7}, at which the run gives 2.84. And the loader voids both numbers, which assume Poisson sampling. What the code ran is certified only at 36.45, for zero-out neighbours, so the review asks for a Poisson loader and a δ of 10^{-7}.

31.8. Checking an accountant

An accountant is code, so test it on known answers. With sampling rate $q = 1$, each step is a plain Gaussian mechanism, whose composition is exact (f-DP and Gaussian DP). Ten such steps at noise multiplier 1 are exactly $(17.8566, 10^{-5})$-DP, so an upper bound must lie above 17.8566 and a PLD bracket must contain it. Also, ε must never fall as steps grow or rise as noise grows.

Then compare two maintained libraries, with pinned versions. This snippet recomputes Table 31.1 with Google's `dp_accounting`.

```
import dp_accounting
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
run = dp_accounting.SelfComposedDpEvent(step, 10_000)  # q = 0.01, sigma = 1, 10,000 steps
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)
rdp = RdpAccountant(orders=list(range(2, 65))).compose(run)  # integer orders only
print(f"{pld.get_epsilon(1e-5):.4f} {rdp.get_epsilon(1e-5):.4f}")  # 6.1877 6.7194
print(f"{pld.get_epsilon(1e-6):.4f} {rdp.get_epsilon(1e-6):.4f}")  # 6.9074 7.4869
```

Each PLD value lies inside the table's bracket, and each RDP value sits just above the table's, which also tries fractional orders. When libraries disagree, look for three causes. RDP values that differ by an amount depending only on the order, 0.3612 at order 10, come from different conversions (RDP conversions). PLD brackets that do not overlap mean a bug. And a factor of two often means different neighbouring relations.

31.9. Cheat sheet

Table 31.5.: Reporting and reviewing an ε claim.

Question	Default	Check
Accountant for DP-SGD	PLD or PRV, upper value	RDP lies above it; never report the CLT
δ	$1/n^{1.1}$ rounded down, n counted in the claim's unit	Also report ε at $1/n^2$
Fixed-size batches	Poisson accountant at $\sigma/2$ (add/remove); replace-one: see Subsampling	Poisson batch sizes vary from step to step
Shuffle and partition	One Gaussian step per epoch, zero-out neighbours	Better, switch to Opacus's Poisson loader

31.10. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): matching the accounting to the pipeline, and what to report.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST SP 800-226, 2025): what an evaluator will ask of your claim.

Derivations. Every result here is derived step by step in the full edition's chapter.

32. Calibrate and account a DP-SGD run

32.1. Why this matters

An **accountant** turns a training run's settings into its privacy level ε under differential privacy (DP). Before the run it finds the noise that meets your target, and after it the ε the run spent. Both can go wrong silently. The noise may be calibrated with one accountant and reported with another, the loader may not sample as the accountant assumes, or a restart may reset the step count. None changes the loss curve, but each makes the claim false.

This chapter does the accounting, and trains nothing, for a run of stochastic gradient descent (SGD) with DP, called DP-SGD, on the Adult census data from the UCI Machine Learning Repository. It trains on 30,000 people in batches of 1,000 for ten epochs, aiming at $\varepsilon = 4$, and Train a model with DP-SGD trains it.

The script practitioner_calibrate.py runs the seven steps below in one to two minutes, with Opacus 1.6.0 and `dp-accounting` 0.6.0. Its defaults are Part 5's run. Options such as `--n`, `--batch`, `--epochs` and `--epsilon` set your own, as the second command does for MNIST, and only the planning table stays fixed.

```
uv run python code/part04-privacy-accounting/practitioner_calibrate.py
uv run python code/part04-privacy-accounting/practitioner_calibrate.py --n 60000 --batch 256
```

32.2. Fix the privacy unit, n and δ

The guarantee protects exactly one record, so first decide what a record is. In Adult each row is one person. If one person could own many rows, the number of records n would have to count people, and training would need the methods of User-level differential privacy.

The slack δ is roughly the chance that the guarantee fails. The book's rule sets it to at most $1/n^{1.1}$, rounded down to a power of ten, well below $1/n$ (Choosing δ).

```
def delta_for(n: int) -> float:
    """The book's default delta: at most 1/n^1.1, rounded down to a power of ten."""
    return 10.0 ** math.floor(math.log10(n**-1.1))
```

```
Adult rows: 48,842; Part 5's training split: n=30,000
delta_for(30,000) = 1e-05
delta_for(60,000) = 1e-06
delta_for(569) = 1e-04
```

Here n counts the people of the training split, not all of Adult's rows, so δ is 10^{-5}. Set the validation rows aside first, because rows that are not trained on are neither in n nor protected. The lines for MNIST and scikit-learn's breast cancer data show that δ must be recomputed for each dataset.

32.3. Choose the sampler

The accountants here assume **Poisson sampling**, in which each record joins each batch independently with probability q, the sampling rate, so batches vary in size. Other samplers cost far more privacy (Section 32.6). With an expected batch size B, the rate is $q = B/n$. A run of E epochs then takes E/q steps, rounded down, which the output calls T.

```
Poisson: B=1000, E=10: q=0.033333, T=300
Opacus DPDataLoader.from_data_loader: q=1/30=0.033333, 30 steps an epoch, 300 in 10 epochs
chosen run uses the loader's q and steps; get_noise_multiplier at that q counts int(E/q)=300
```

The second line is what training will run, because Opacus replaces your loader with a Poisson `DPDataLoader`. Its rate is one over the original loader's number of batches, $\lceil n/B \rceil$, with that many steps per epoch. Here B divides n, so the two lines agree. When it does not, training runs the loader's numbers. So the script takes q and T from the loader, and the third line checks that Opacus's calibration counts the same steps. This step has gone wrong if the training code builds its own loader with `shuffle=True` and a fixed `batch_size`. The sign is that all batches in the logs have the same size (Amplification by subsampling).

32.4. Plan the noise

Next come the batch size, the epochs and the noise multiplier σ. Training clips each record's gradient to length at most C, the clipping norm, and σ is the noise's standard deviation as a multiple of C. Given the target and the sampler, an accountant finds the smallest σ that meets them, so calibrate with the accountant you will report (The inverse problem). The script uses tight accountants from two libraries, which check each other (Numerical accounting). Opacus's `get_noise_multiplier` searches with its privacy loss random variable (PRV) accountant to within 0.01 in ε. Google's `dp_accounting` searches with its privacy loss distribution (PLD) accountant to within 0.01 in σ.

```
def sigma_opacus(epsilon: float, delta: float, q: float, epochs: int) -> float:
    """Opacus: the sigma whose PRV epsilon after `epochs` is at most the target (to 0.01 in
    return get_noise_multiplier(
        target_epsilon=epsilon, target_delta=delta, sample_rate=q, epochs=epochs, accountant
    )


def sigma_dp_accounting(epsilon: float, delta: float, q: float, steps: int) -> float:
    """dp_accounting: the same search with a PLD accountant (to 0.01 in sigma)."""
```

```
    def training_run(sigma: float) -> DpEvent:
        return SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)

    def accountant() -> PLDAccountant:
        return PLDAccountant(value_discretization_interval=PLD_GRID)

    return calibrate_dp_mechanism(accountant, training_run, epsilon, delta, LowerEndpointAnd
```

Pass `accountant="prv"` explicitly, because `get_noise_multiplier` defaults to the Rényi differential privacy (RDP) accountant, a valid but looser bound that asks for more noise. In `dp_accounting`, `LowerEndpointAndGuess(0.3, 1.0)` starts the search at a lower end of 0.3, so lower that number if the call raises an error.

The table uses a PLD grid of 10^{-3}, the step to which the accountant rounds the privacy loss, so each search takes about a second. Averaging over a batch of B divides the noise on the sum by B. So the last column, σ/B times 1,000, is the noise on the averaged gradient per unit of clipping norm.

```
dataset             n  delta     B   E  eps      T  sigma PRV  sigma PLD  sigma/B x1000
Adult           30000  1e-05   100  10    4   3000       0.62       0.62          6.16
Adult           30000  1e-05   300  10    4   1000       0.74       0.74          2.45
Adult           30000  1e-05  1000  10    4    300       0.97       0.98          0.97
Adult           30000  1e-05  1000  10    8    300       0.72       0.73          0.72
Adult           30000  1e-05  1000  20    4    600       1.16       1.16          1.16
```

The block shows 5 of the script's 18 Adult cells. At $\varepsilon = 4$ and ten epochs, a batch of 1,000 needs more noise than a batch of 100, with σ at 0.97 instead of 0.62. Yet that noise, averaged over ten times as many records, is smaller on the average, 0.97 instead of 6.16. Twenty epochs instead of ten raise σ to 1.16.

The table alone cannot choose the batch, because a larger batch lowers the noise on the average but also the number of steps. So learn the steps your model needs from a non-private run or public data. Then take the largest batch that leaves those steps and fits in memory (Batch size at a fixed budget), which for Part 5 is 1,000.

Small datasets change the picture, as a row from the end of the table shows.

```
breast cancer     569  1e-04    64  20    1    177       4.96       4.91         77.51
largest gap between PRV and PLD sigma: 1.6% (Adult, B=300, E=10, eps=8)
```

With 569 records, a batch of 64 is over a tenth of the data, so sampling amplifies little. The noise on the average is then 77.51 at $\varepsilon = 1$, so small datasets need a large ε, public pretraining, or another method.

The block's last line is the libraries' check on each other. Across all 24 cells their σ differ by at most 1.6%. A gap above about 3% means the two calls got different inputs, such as a batch size passed as a rate, epochs passed as steps, or another δ.

32.5. Check ε with a second accountant

Train with the calibrated σ unrounded, as Part 5 does with Opacus's 0.9729. If you round it, round up to the three decimals the statement prints, because less noise means a larger ε. Before training, compute the planned run's ε with every accountant at hand, three from Opacus and two from `dp_accounting`.

```
def crosscheck_epsilon(sigma: float, q: float, steps: int, delta: float) -> dict[str, float]
    """Epsilon of one planned run by five accountants (GDP is the trap: it is not an upper b
    epsilons: dict[str, float] = {}
    for name, opacus_accountant in [("Opacus RDP", RDPAccountant()), ("Opacus PRV", PRVAccou
        opacus_accountant.history = [(sigma, q, steps)]  # what `steps` calls of .step() wou
        epsilons[name] = opacus_accountant.get_epsilon(delta)
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
    epsilons["dp_accounting RDP"] = RdpAccountant().compose(run).get_epsilon(delta)
    epsilons["dp_accounting PLD"] = PLDAccountant().compose(run).get_epsilon(delta)
    gdp = GaussianAccountant()
    gdp.history = [(sigma, q, steps)]
    epsilons["Opacus GDP"] = gdp.get_epsilon(delta)
    return epsilons
```

```
chosen cell: Adult, B=1000, E=10, eps=4: sigma=0.972900 (Opacus PRV, as Part 5)
Opacus RDP         eps=4.50
Opacus PRV         eps=3.99
dp_accounting RDP  eps=4.50
dp_accounting PLD  eps=3.98
Opacus GDP         eps=3.34
PLD <= target: True; RDP >= PLD: True; |PRV - PLD|/PLD < 3%: True
crosscheck: pass
PLD bracket (grid 1e-4): lower 3.9656 (optimistic), upper 3.9806 (pessimistic); width 0.0150
```

The two tight accountants, PRV and the PLD, agree at 3.99 and 3.98, and both meet the target. The two RDP implementations agree at 4.50, above the target, as a looser bound should, and Opacus's warnings about RDP orders concern only that value. The trap is the Gaussian differential privacy (GDP) value of 3.34. It comes from a central limit theorem (CLT), which approximates the run rather than bounding it, and it lands below the certified values, so never report it.

The pass line runs three checks. The PLD value must be at most the target, which fails if σ was rounded down or the steps differ from the plan. RDP must lie at or above the PLD, up to its grid error, and PRV must agree with the PLD within 3%. If either of these fails, the accountants were given different runs.

The last line is the PLD's error bar. Rounding each step's privacy loss up to the grid gives the reported 3.9806, and rounding it down gives an optimistic 3.9656, so the true ε lies between them. Claim the upper value and report the lower one as its error bar (Which accountant). The bracket widens with the steps, so a long run needs a finer grid.

32.6. If the loader is not Poisson

Not every training loop samples by the Poisson scheme, so price the alternatives before you train. The script accounts the chosen run, with its loader's steps, under the four samplers of Amplification by subsampling.

```
Poisson (PLD)                                eps=3.98
fixed size, add/remove, sigma/2 (PLD)        eps=23.03
fixed size, replace-one (RDP)                eps=52.60
shuffle, zero-out, no amplification (PLD)    eps=18.51
PLDAccountant supports sampling without replacement: add/remove False, replace-one False
```

Fixed-size random batches cost more, because a record added to a full batch pushes another out, so the clipped sum can move by twice the clipping norm C. Under add/remove neighbours, the Poisson accountant at half the noise multiplier covers them, which turns 3.98 into 23.03. Under replace-one neighbours only RDP accounts them, at $\sigma/2$ too, since the PLD accountant cannot (last line). Its 52.60 protects a record's value rather than its presence, so it does not compare with the add/remove values.

Shuffling once per epoch and cutting the data into batches gets no amplification, so all that is certain is one Gaussian step per epoch, 18.51 here. That holds for zero-out neighbours, which differ by one record's gradient being replaced by zero, and the statement must name that relation. A step's sum then moves by at most C, as under add/remove, so the value compares with the add/remove ones. It beats fixed-size batches here, but on the 100-epoch run of Report and review an ε claim the order reverses. So the fix for a non-Poisson loader is the loader: keep the Poisson loader that Opacus's `make_private` installs.

32.7. Train, stop early and resume

During training, Opacus's optimizer calls `accountant.step` once per noisy update, which is once per logical batch, even when `BatchMemoryManager` splits it into smaller physical ones. The script simulates the run's 300 calls and asks for ε after each epoch.

```
def train_epochs(
    accountant: PRVAccountant, sigma: float, q: float, steps_per_epoch: int, epochs: int, de
) -> list[float]:
    """Step the accountant once per logical batch for `epochs` epochs; epsilon at the end of
    epsilons: list[float] = []
    for _ in range(epochs):
        for _ in range(steps_per_epoch):
            accountant.step(noise_multiplier=sigma, sample_rate=q)  # what the optimizer's s
        epsilons.append(float(accountant.get_epsilon(delta)))
    return epsilons
```

Suppose the validation loss stalls and the run stops after six of its ten planned epochs. Stopping early is safe, because the stopped run releases a prefix of the planned run, which the planned

guarantee covers. But you still report the planned maximum. The ε at the stopping point is not a valid claim, because the decision to stop depended on what the run released. Only a privacy filter running from the first step could certify it (Adaptive composition). Log ε every epoch anyway, because it must never decrease, so a drop means the accountant was reset.

Training past the maximum is not safe. If you add epochs after seeing the model, the number of steps depends on what the run released, so the longer run's ε is not a valid claim either. An extension needs a new plan, accounted from the first step. So plan the longest run you might want, and treat stopping sooner as an early stop.

A job that stops and restarts must carry the accountant with it. Save its `state_dict` with the model and load it before the first new step, as Opacus's `PrivacyEngine.save_checkpoint` and `load_checkpoint` do.

```
state = accountant.state_dict()  # saved with the model; PrivacyEngine.save_checkpoint store
# ... the job stops, and later restarts ...
restarted = PRVAccountant()
restarted.load_state_dict(state)  # without this, epsilon would restart from zero
resumed = train_epochs(restarted, sigma, q, steps_per_epoch, epochs - stop, delta)
steps_recorded = sum(steps for _, _, steps in restarted.history)
```

```
steps per epoch: 30; planned: 10 epochs = 300 steps
epoch  1: eps=1.70
epoch  2: eps=2.09
epoch  4: eps=2.69
epoch  6: eps=3.18
epsilon never decreases over 10 epochs: True
stopped early at epoch 6: eps=3.18; report the planned maximum: eps=3.99
after resuming at epoch 6: eps at epoch 10=3.99; uninterrupted run: eps=3.99; equal: True
steps recorded: 300; planned (the loader's 10 x 30): 300; equal: True
```

The resumed run ends at 3.99, like a run that never stopped, with all 300 steps in its history. A fresh accountant would have counted only the last four epochs and reported 2.69.

32.8. Write the statement

The last step writes the claim with every input a reviewer needs to recompute it. The script fills the template from the run's parameters, and rounds up the PLD's ε at the rule's δ and at $1/n^2$, where n is the training set's size.

```
def privacy_statement(
    unit: str, dataset: str, n: int, batch: int, epochs: int, sigma: float, clip: float, del
) -> str:
    """The filled template of the reporting page: every input a reviewer needs to recompute
    q, _, steps = opacus_loader_plan(n, batch, epochs)  # the steps the Poisson loader runs
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
```

```
    epsilon = round_up(PLDAccountant().compose(run).get_epsilon(delta))
    epsilon_tiny_delta = round_up(PLDAccountant().compose(run).get_epsilon(1 / n**2))
    return f"""\
Unit: one {unit} (a row of {dataset}); neighbouring datasets differ by adding or removing on
Data: n = {n:,}; Poisson sampling with q = {q:.6f} (B = {batch}); T = {steps:,} steps ({epoc
Mechanism: DP-SGD, clipping norm C = {clip}, noise multiplier sigma = {sigma:.3f}
Accountant: PLD, dp-accounting {version("dp-accounting")}; {calibration}
Guarantee: (epsilon = {epsilon:.2f}, delta = {delta:.0e}); epsilon = {epsilon_tiny_delta:.2f
Not covered: hyperparameter tuning runs (see Part 5), and any model released before this one
```

```
Unit: one person (a row of Adult); neighbouring datasets differ by adding or removing one pe
Data: n = 30,000; Poisson sampling with q = 0.033333 (B = 1000); T = 300 steps (10 epochs)
Mechanism: DP-SGD, clipping norm C = 1.0, noise multiplier sigma = 0.973
Accountant: PLD, dp-accounting 0.6.0; calibrated with Opacus 1.6.0 (PRV)
Guarantee: (epsilon = 3.99, delta = 1e-05); epsilon = 6.30 at delta = 1/n^2 = 1.1e-09
Not covered: hyperparameter tuning runs (see Part 5), and any model released before this one
```

The guarantee reads 3.99 because the PLD value printed as 3.98 above is rounded up here, not to the nearest value. The unit, dataset and clipping norm come from options, because the accountant never sees them. The last line matters most to a reviewer, because it names privacy spent outside this ε. Tuning on the private data adds its own cost (Hyperparameter tuning), and so does every earlier model trained on the same people.

A published statement also gives what the script cannot know (The statement). That is the setting, the PLD's grid and bracket, and the data outside the run, such as pretrained weights and the unprotected validation rows.

32.9. Checklist

Table 32.1.: The steps of calibrating and accounting a DP-SGD run.

Step	Default	Check
Unit, n and δ	One person; n from the training split, validation rows excluded; $\delta \leq 1/n^{1.1}$, rounded down	n counts people, not rows; δ recomputed for each dataset
Sampler	Poisson; q and T from Opacus's loader, $\lceil n/B \rceil$ steps an epoch	Calibration counts the same steps; batch sizes vary
Plan the noise	Calibrate with PRV or PLD, the accountant you report	Two libraries agree within about 3%

Step	Default	Check
Cross-check	Train at the calibrated σ, or round it up	PLD at most the target; RDP above PLD; GDP never reported; PLD bracket reported
Other samplers	Keep Opacus's Poisson loader	No `shuffle=True` loader with fixed batches in the training code
Train and resume	One accountant step per logical batch; save the accountant's state; any extension inside the plan	ε never decreases; steps recorded equal the loader's steps
Statement	ε at the planned maximum, rounded up, at δ and $1/n^2$	Every input to recompute it; tuning, earlier models, pretraining, preprocessing and validation named

32.10. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): planning a run's privacy budget, and what to report.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): why fixed-size and shuffled batches need their own accounting.

Derivations. The results this workflow relies on are derived in the full edition's chapters Amplification by subsampling in the full edition, Numerical accounting in the full edition and Choosing an accountant in the full edition.

33. Amplification by subsampling

33.1. Why this matters

Most private models are trained with stochastic gradient descent (SGD) made to satisfy differential privacy (DP), a method called DP-SGD. Each step clips every record's gradient to a maximum length, the **clipping norm**, sums the clipped gradients and adds Gaussian noise. The noise's standard deviation is the clipping norm times the **noise multiplier** σ. If every record paid a full Gaussian step's cost each time, ten thousand steps at $\sigma = 1$ would have a privacy level ε of 5425.5 at $\delta = 10^{-5}$ (Rényi DP), no guarantee at all.

DP-SGD escapes this cost because each step looks only at a small random batch. Write q for the **sampling rate**, the probability that a given record lands in the batch. An attacker must then first guess whether the record was used at all, and that uncertainty is worth a lot. This **amplification by subsampling** depends on exactly how the batch was drawn. So the accountant, the code that adds up a run's privacy cost, must assume the sampler the training code runs. Getting that wrong is the largest single error in DP-SGD accounting, far larger than the choice of accountant (Section 33.4).

33.2. One step under Poisson sampling

The simplest sampler is **Poisson sampling**, in which each record enters the batch by its own coin flip, with probability q, independently of every other record. It goes with **add/remove** neighbours, two datasets of which one has an extra record. For this sampler and these neighbours, Balle, Barthe and Gaboardi (2018) give the best possible one-step result. Suppose a step is (ε, δ)-DP when it sees every record, where δ is the small additive slack of Part 3. Then the same step run on a Poisson sample is (ε', δ')-DP with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{33.1}$$

Small and large ε behave very differently. For small ε the amplified cost is about $q\varepsilon$, so a 1% sample makes a step about a hundred times cheaper, and a 1-DP step costs 0.0170. For large ε the amplified cost is about

$$\varepsilon' \approx \varepsilon - \ln(1/q), \tag{33.2}$$

so sampling only subtracts a fixed amount, and a 10-DP step on the same sample still costs 5.3993. The switch happens where ε is near $\ln(1/q)$. So amplification helps most when each step is already fairly private, which is why DP-SGD takes many cheap steps.

The result has two conditions. Each record's coin must be independent of the others, so that every record is drawn with probability exactly q whatever else is in the batch. And the batch

must stay secret, because an attacker who reads the list of drawn records, say in a log, knows whether the target was used.

33.3. Match the sampler to the accountant

Most training code does not draw Poisson batches. Some draws a fresh random batch of a fixed size B from the n records at each step. Much more shuffles the data once per epoch and cuts it into consecutive batches, which is called **shuffle and partition**.

Each sampler needs its own accounting under each neighbouring relation. Under **replace-one**, the datasets have the same size and differ in one record's value, and under **zero-out**, the target's gradient is set to zero. Table 33.1 gives DP-SGD's accounting as events of Google's `dp_accounting` library, which a Rényi differential privacy (RDP) or privacy loss distribution (PLD) accountant then composes.

Table 33.1.: What to account for under each sampler and neighbouring relation in DP-SGD.

Sampler	Neighbours	What to account for	`dp_accounting` step
Poisson, rate q	add/remove	A Poisson accountant with q and σ	`PoissonSampledDpEvent(q, GaussianDpEvent(σ))`
Poisson, rate q	replace-one	The same event, at σ, in a PLD accountant built with `NeighboringRelation.REPLACE_ONE`, which models the swap itself	`PoissonSampledDpEvent(q, GaussianDpEvent(σ))`
Fixed size B of n	add/remove	A Poisson accountant with rate B/n and noise multiplier $\sigma/2$	`PoissonSampledDpEvent(B/n, GaussianDpEvent(σ/2))`
Fixed size B of n	replace-one	Sampling without replacement at noise multiplier $\sigma/2$, in the RDP accountant only	`SampledWithoutReplacementDpEver B, GaussianDpEvent(σ/2))`
Shuffle and partition	zero-out	No amplification: each epoch is one full Gaussian release	`GaussianDpEvent(σ)` once per epoch

The factor of two for fixed-size batches comes from the gradient sum. Under Poisson sampling, adding or removing a record moves the clipped sum by at most the clipping norm. A fixed-size batch has no room for an extra record, so a drawn record takes another's place, and the sum can move twice as far (Lebeda et al. 2025). Twice the shift with the same noise is the same as the original shift with half the noise multiplier. Replace-one swaps a record by definition, so its shift doubles too, and there only the RDP accountant of `dp_accounting` covers fixed-size batches.

Shuffle and partition is worse, because the source of amplification is missing. Within an epoch the record is in exactly one batch and is never absent, so the attacker only has to find which batch held it. Chua et al. (2024) build a dataset in which the target's gradient points against all the others, so that its batch's sum stands out. On it, at $q = 0.01$ and $\sigma = 1$, one shuffled epoch provably has an ε of at least 4.06, while a Poisson accountant reports at most 1.21. So a Poisson number for shuffled training can be provably false. The honest bound is the table's last row, one full Gaussian release per epoch, since each epoch uses the record exactly once.

So read the data loader before you trust a reported ε. A PyTorch `DataLoader` with `shuffle=True` and a `batch_size` is shuffle and partition. Opacus replaces the loader with a Poisson one by default, since `PrivacyEngine.make_private` takes `poisson_sampling=True` unless you turn it off. Its `batch_sampler` is then a `UniformWithReplacementSampler`, which despite its name flips one coin per record, so batch sizes vary from step to step, a quick check in the training log. When the largest batches do not fit in memory, Opacus's `BatchMemoryManager` splits each into pieces that fit, and the optimizer and the accountant still count one step per sampled batch (Train a model with DP-SGD). If you cannot use Poisson sampling, report under the matching row of Table 33.1, and name the sampler next to ε.

33.4. Accounting each sampler

DP-SGD is never accounted step by step with Equation 33.1. That route splits δ into a tiny share per step, and a tiny δ forces a large per-step ε, which sampling barely reduces, as for the 10-DP step above. On 10,000 steps it gives 24379.55 with advanced composition, worse even than 5425.5 without sampling. An RDP accountant instead adds the steps' Rényi curves order by order and converts the total to (ε, δ) once, which gives 6.71 (Rényi DP). A PLD accountant is tighter still (Numerical accounting).

The snippet below accounts that run under four rows of Table 33.1, with 50,000 records in batches of 500, so $q = 0.01$. It uses $\delta = 10^{-5}$ to compare with the rest of the part, not the 10^{-6} a report would use (Report and review an ε claim).

```
from dp_accounting import GaussianDpEvent, NeighboringRelation, PoissonSampledDpEvent
from dp_accounting import SampledWithoutReplacementDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

n, batch, sigma, steps, delta = 50_000, 500, 1.0, 10_000, 1e-5  # q = 0.01, 100 epochs
poisson = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma)), step
fixed_size = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma / 2)
shuffled = SelfComposedDpEvent(GaussianDpEvent(sigma), 100)  # one Gaussian release per epoc
pld = [PLDAccountant(value_discretization_interval=1e-3).compose(e) for e in (poisson, fixed
print(" ".join(f"{accountant.get_epsilon(delta):.2f}" for accountant in pld))  # 6.19 43.37
replace_one = RdpAccountant(neighboring_relation=NeighboringRelation.REPLACE_ONE)
swap = SampledWithoutReplacementDpEvent(n, batch, GaussianDpEvent(sigma / 2))  # replace-one
replace_one.compose(SelfComposedDpEvent(swap, steps))
print(f"{replace_one.get_epsilon(delta):.2f}")  # 118.73
```

Poisson sampling costs 6.19. With the same noise, fixed-size batches cost 43.37 under add/remove neighbours, about seven times more, and shuffling costs 91.82 under zero-out. The replace-one value, 118.73, protects a record's value rather than its presence, so compare it only with other replace-one numbers. So the sampler moves ε sevenfold or more, while the accountant moves it by under 10%, from 6.71 by RDP to 6.19 by PLD.

Before you compose an unusual event, call `PLDAccountant().supports(event)`, which returns `False`, for example, for a `SampledWithoutReplacementDpEvent` under either relation.

33.5. Batch size, epochs and noise

Amplification also shapes how you plan a run. Write E for the number of epochs, so that a run takes E/q steps at sampling rate q. For small q and a large noise multiplier, the run's Rényi curve, its cost at each order α, is then roughly

$$\varepsilon(\alpha) \approx \frac{E\,q\,\alpha}{2\sigma^2}. \tag{33.3}$$

So at a fixed number of epochs a smaller sampling rate costs less privacy, because each step costs about q^2 while the number of steps grows only as $1/q$. Use this only for planning, never as the reported number, because at high orders the true curve rises far above it (from about order 9 at $\sigma = 1$ and $q = 0.01$).

Privacy is not the only cost, because the noise lands on the averaged gradient. With C for the clipping norm, the noise on the sum of B records has standard deviation σC, so the noise on their average has $\sigma C/B$. So compare plans by σ/B, not by σ. The planning table of Calibrate and account a DP-SGD run shows why. At a fixed ε, a ten times larger batch needs a larger σ, yet its σ/B is six times smaller.

Plan in this order, then. First fix the batch size B and the epochs E that your accuracy needs. They set $q = B/n$ and the number of steps, where n is the size of the dataset you actually train on, after any filtering. Then let the accountant solve for σ, naming the numerical accountant in Opacus's `get_noise_multiplier`, whose default is RDP (Calibrate and account a DP-SGD run).

33.6. Cheat sheet

Table 33.2.: Which accounting to use for each sampler.

Situation	What to use	Notes
One step, Poisson, add/remove	$\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$	About $q\varepsilon$ for small ε
DP-SGD, Poisson loader	PLD or RDP accountant with q and σ	6.19 by PLD for the running example
DP-SGD, fixed-size batches, add/remove	Poisson accountant with B/n and $\sigma/2$	43.37, about seven times the Poisson value

Situation	What to use	Notes
DP-SGD, fixed-size batches, replace-one	`SampledWithoutReplacementDpEvent` at $\sigma/2$ in the RDP accountant	Not comparable with add/remove numbers
DP-SGD, shuffle and partition	One Gaussian release per epoch, under zero-out	A Poisson number does not apply; better, switch the loader to Poisson sampling

Rules of thumb

- Account for the sampler you run, check that its batches vary in size, and state the neighbouring relation next to ε. ## References
- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018): amplification for each sampler and neighbouring relation.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size batches and other accounting pitfalls.

Derivations. Every result here is derived step by step in the full edition's chapter.

34. Rényi differential privacy

34.1. Why this matters

Training a model runs thousands of private steps, and you report one differential privacy (DP) guarantee for all of them: a privacy level ε with a small slack δ. Adding up the steps' pairs with the advanced composition of Part 3 (Composition) is far too pessimistic. Take 1000 steps of full-batch noisy gradient descent, each adding Gaussian noise with a standard deviation 20 times the most one record can move the gradient. At $\delta = 10^{-5}$, advanced composition gives $\varepsilon = 68.01$, nine times the exact 7.51, so a run calibrated to it adds more noise than it needs.

Rényi differential privacy (RDP) fixes this by keeping a whole curve of privacy costs for each step instead of one point. The curves of the steps add, with no δ spent along the way, and you convert the total into (ε, δ) once, at the end. Even with Mironov's original conversion (Section 34.6), the same run then costs 8.84, only 18% above the exact value. Opacus and Google's `dp_accounting` both ship an RDP accountant.

34.2. The RDP curve

A DP guarantee compares the outputs of a mechanism on two neighbouring datasets, which differ in one person's record. RDP measures how far apart those two output distributions are with the **Rényi divergence** of Divergences. That divergence has a dial, the **order** α, any number above 1. A small order looks at the typical output, and a large order weighs more heavily the rare outputs that are much likelier on one dataset than on the other.

A mechanism is (α, ε)**-RDP** if, at order α, the divergence is at most ε for every pair of neighbours. The smallest such ε at each order traces out the mechanism's **RDP curve** $\varepsilon(\alpha)$, and a higher curve means less privacy. The curve never decreases with the order. At order infinity it is the largest privacy loss of any output, which is what pure DP, the case $\delta = 0$, bounds. You never compute the divergence yourself, because accountants know the curves of the standard mechanisms on a grid of orders.

34.3. RDP curves of the basic mechanisms

To account for a mechanism you need its curve, and the basic mechanisms have one in closed form (Mironov 2017). For the **Gaussian mechanism**, write σ for its noise multiplier, the noise standard deviation in units of the ℓ_2 sensitivity, which is the most one record can move the answer. Its curve is the straight line

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{34.1}$$

Table 34.1 gives the curves of the Gaussian, Laplace and randomized-response mechanisms at a few orders, each at sensitivity 1.

Table 34.1.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, scale $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, which reports a bit truthfully with probability 0.75	0.7340	0.8473	1.0666	1.0940	1.0986

The Laplace and randomized-response curves level off at their pure DP level. The Gaussian curve grows without limit, so the Gaussian mechanism is not ε-DP for any ε, but an accountant only converts at finite orders, where the curve is finite. These curves are for steps on the whole dataset. The private form of stochastic gradient descent (SGD), called DP-SGD, runs each step on a random batch, whose much smaller curves libraries compute for you (Amplification by subsampling).

34.4. Composition adds curves

Training runs thousands of mechanisms in sequence, each of which may depend on what the earlier ones released, as a gradient step depends on the current model. RDP composes them by adding their curves, order by order. Write ε_i for the curve of the i-th step. Then a whole run of k steps has a curve ε_M with

$$\varepsilon_M(\alpha) \le \sum_{i=1}^{k} \varepsilon_i(\alpha) \qquad \text{for every order } \alpha. \tag{34.2}$$

Different mechanisms mix freely, because you only add numbers at the same order.

Two conditions come with the sum. First, every curve must describe the same neighbouring relation, such as add/remove or replace-one, and the same unit of privacy, such as one record or one user. Second, each step's curve must be fixed before the run. Steps may adapt to earlier outputs, but if the noise level or the number of steps is chosen from the data, the curves depend on the data too. Stopping when a validation metric looks good is such a choice, and needs Adaptive composition instead.

The sum is simplest for Gaussian steps, because their curve is the straight line $\alpha/(2\sigma^2)$. Adding k such lines multiplies the slope by k, which gives the line of one step with noise multiplier $\sigma/\sqrt{k}$. So k Gaussian steps cost as much privacy as one step with $\sqrt{k}$ times less noise, which is bad news for training. If 10,000 steps with noise multiplier 1 each used the whole dataset, they would cost as much as one step with noise multiplier $1/\sqrt{10{,}000} = 0.01$, whose ε at $\delta = 10^{-5}$ is 5425.5, no guarantee at all. DP-SGD escapes this by running each step on a small random sample. This part's running example is such a run: 10,000 steps with noise multiplier 1, each on a random 1% of the data.

34.5. Post-processing and group privacy

Post-processing is free in RDP. Any computation on the output that does not look at the data again leaves the curve no larger. So releasing the trained model or its predictions costs nothing beyond the training run.

Groups are harder: when one user contributes several records, you need a guarantee for all of them at once. RDP's generic group rule is loose, so for Gaussian noise use the group's sensitivity instead. A group of m records moves the answer by at most m times as much as one record, so the curve becomes

$$\varepsilon_{\text{group}}(\alpha) = \frac{m^2\alpha}{2\sigma^2}, \tag{34.3}$$

which is m^2 times the single-record curve. Ten records per user cost a hundred times as much. The fix is to clip each user's whole contribution to one clipping norm and account per user (User-level privacy).

34.6. Compose, then convert once

After composing, you hold a curve, but you report an (ε, δ) pair. Both libraries turn the curve into a pair with the **improved conversion** (Balle et al. 2020, and independently Canonne, Kamath and Steinke 2020). From the curve's value $\varepsilon(\alpha)$ at one order and a target δ, it gives the privacy level

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}. \tag{34.4}$$

You report the smallest ε' over the grid of orders, or 0 if that is negative, and libraries call the order that gave it the best order.

Older papers and code use Mironov's original conversion, which gives larger numbers from the same curve, 7.43 instead of 6.71 on the DP-SGD running example. The **moments accountant** of the original DP-SGD paper (Abadi et al. 2016) is the RDP curve in a rescaled form, converted with Mironov's formula. So its ε is loose but not wrong. The exact value is lower still (From Rényi DP to (ε, δ) and to tests).

34.7. Check the order grid

A conversion can only use the orders on the accountant's grid, so the grid must contain the best order, which moves with the run. Long runs with little noise want orders close to 1, short runs with much noise want large ones. For the noisy gradient descent at the start of the chapter, it falls from 128 after one step to 4 after 1000.

The default grid of Opacus, `RDPAccountant.DEFAULT_ALPHAS`, stops at order 63, while that of `dp_accounting`, `DEFAULT_RDP_ORDERS`, goes on to 1024. A best order at either end of the grid means the true optimum lies outside it, so pass a longer list, with `alphas=[...]` in Opacus or `orders=[...]` in `dp_accounting`. Opacus warns when this happens, but with `dp_accounting` you compare the returned order with the ends of the list yourself.

34.8. Accounting with a library

In practice you describe each step to a library and let it compose and convert. In `dp_accounting` a DP-SGD step is a `GaussianDpEvent` inside a `PoissonSampledDpEvent`, which samples each record independently, with probability 0.01 in the running example. This snippet composes the 10,000 steps and prints ε with the best order.

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
accountant = dp_accounting.rdp.RdpAccountant()
accountant.compose(step, count=10_000)
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7128 4.1
```

In Opacus the accountant is attached to the optimizer, which calls `step` after every noisy update. Making those calls by hand gives the same ε to the fourth decimal, at the same order, 4.1, which is well inside both grids.

```
from opacus.accountants import RDPAccountant

accountant = RDPAccountant()
for _ in range(10_000):  # what the optimizer does at each step of training
    accountant.step(noise_multiplier=1.0, sample_rate=0.01)
eps, order = accountant.get_privacy_spent(delta=1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7127 4.1
```

Opacus's defaults mix two accountants, so name the one you want. In Opacus 1.6.0, `PrivacyEngine` defaults to `"prv"`, a numerical accountant, and `accountant="rdp"` selects this class. But `get_noise_multiplier`, which solves for σ, defaults to RDP. Both give valid upper bounds, but different ones: the numerical accountant gives 6.19 for the running example.

Two more checks catch most mistakes. The event must match the sampler your data loader runs, because shuffled fixed-size batches get none of a Poisson event's amplification (Amplification by

subsampling). And a manual call to `step` on top of the optimizer's own charges each update twice.

34.9. Cheat sheet

Table 34.2.: The results of this chapter.

Result	Formula	Notes
RDP curve	$\varepsilon(\alpha)$ over orders $\alpha > 1$	Never decreases; $\varepsilon(\infty)$ is pure DP
Gaussian, noise multiplier σ	$\varepsilon(\alpha) = \alpha/(2\sigma^2)$	σ in units of the ℓ_2 sensitivity
Composition	$\varepsilon_M(\alpha) \le \sum_i \varepsilon_i(\alpha)$	Same relation and unit; curves fixed in advance
Group of m, Gaussian	$m^2\alpha/(2\sigma^2)$	The generic group rule is loose
Improved conversion	$\varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta+\ln\alpha}{\alpha-1}$, best α	What Opacus and `dp_accounting` use

34.10. References

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the curves of the basic mechanisms and group privacy.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.

Derivations. Every result here is derived step by step in the full edition's chapter.

35. From Rényi DP to (ε, δ) and to tests

35.1. What it is

An accountant that tracks Rényi differential privacy (RDP) ends a run holding a curve, a privacy level $\varepsilon(\alpha)$ at each order α above 1 (Rényi differential privacy). A report needs a differential privacy (DP) guarantee instead, a privacy level ε' with a small slack δ. A conversion turns each point of the curve into such a pair, and you keep the smallest ε'.

There are two conversions in use. The original one, **Mironov's conversion** (Mironov 2017), gives at each order

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{35.1}$$

The **improved conversion**, found independently by three groups in 2020, gives

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}. \tag{35.2}$$

Both are valid, but at the same order the improved one is always lower, by an amount that depends only on the order: 0.3612 at order 10, and more at smaller orders. If the improved value comes out negative, report 0.

For example, one Gaussian release with noise multiplier 1 has the curve $\alpha/2$. At $\delta = 10^{-5}$ its exact value is 4.377, while Mironov's conversion gives 5.299 and the improved one 4.729.

35.2. When you meet it

You meet the conversion whenever you compare an RDP number with someone else's. Opacus 1.6.0 and `dp_accounting` 0.6.0 use the improved conversion. Older papers and code use Mironov's, including the moments accountant of the original paper on the private form of stochastic gradient descent (SGD), called DP-SGD. On this part's running example, 10,000 DP-SGD steps on a random 1% of the data with noise multiplier 1, Mironov's conversion gives 7.43 and the improved one 6.71.

Because the saving depends only on the order, two RDP accountants that report values about 0.36 apart for one run, with a best order near 10, differ in their conversion. Recompute both with one library before comparing them.

35.3. How to use it

In `dp_accounting`, `get_epsilon_and_optimal_order` converts at the best order of the grid and returns both, so you never type the formula in. One curve answers every δ, so `get_epsilon` can report several pairs at no extra privacy cost. This snippet converts the single Gaussian release above.

```
from dp_accounting import GaussianDpEvent
from dp_accounting.rdp import RdpAccountant

accountant = RdpAccountant()
accountant.compose(GaussianDpEvent(1.0))  # one release, noise multiplier 1
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 4.7285 5.4
print(f"{accountant.get_epsilon(1e-8):.4f}")  # 6.0864
```

A δ a thousand times smaller raises ε only from 4.7285 to 6.0864, because δ enters the conversion through a logarithm. The accountant picks the best order from its fixed grid, and short runs with much noise can want orders above 63, where the Opacus default grid stops. So whenever the returned order sits at an end of the grid, extend the grid and convert again (Rényi differential privacy).

35.4. What goes wrong

No conversion closes the whole gap to the exact value. The curve is only a summary, and other mechanisms with the same curve have a larger exact ε. The improved conversion sits 5 to 10% above the exact value for the plain Gaussian, and can sit further above for the subsampled steps of DP-SGD. A privacy loss distribution (PLD) accountant keeps the whole distribution instead, and reaches the exact 4.377 for the single release (Numerical accounting).

The conversion also works in one direction only, because a guarantee with δ above 0 gives no RDP curve. A mechanism that publishes your record with probability δ is $(0, \delta)$-DP, yet its Rényi divergence is infinite. So never feed an (ε, δ) pair into an RDP accountant, and take the curve from the mechanism behind it instead.

Converting also throws away what the curve says about attacks. Take a membership attack on the single release above, run at a false-positive rate of 10^{-3}. The curve caps its true-positive rate at 2.4910×10^{-2}. The pair at $\varepsilon = 4.729$ only caps it at 0.1131 (f-DP and Gaussian differential privacy).

35.5. References

- Balle, Barthe, Gaboardi, Hsu & Sato, “Hypothesis Testing Interpretations and Rényi Differential Privacy” (AISTATS 2020): the improved conversion, and what an RDP curve says about tests.

Derivations. Every result here is derived step by step in the full edition's chapter.

36. Numerical accounting with privacy loss distributions

36.1. What it is

A privacy loss distribution (PLD) accountant computes the privacy level ε of a whole differential privacy (DP) run numerically. Take a dataset and a neighbour that differs in one record, and run the mechanism on the first. The **privacy loss** L of that run is the logarithm of how much likelier the output is on the first dataset than on the second. Because the output is random, L is too, and its distribution is the PLD. Only outputs whose loss exceeds ε add to the slack δ, and with $(\cdot)_+$ turning a negative value into 0, the smallest slack is

$$\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+]. \tag{36.1}$$

When steps draw their noise independently, their losses add up, so the PLD of a run is the convolution of the steps' PLDs. All the error comes from storing each PLD on a grid of losses, with a spacing h between grid points. Rounding every loss up can only raise δ, so the ε it gives is an upper bound, the **pessimistic** side. Rounding down gives a lower bound, the optimistic side. Each rounding moves a step's loss by at most h. So after a number T of steps the two values are at most Th apart, a width known in advance.

36.2. When you meet it

You meet a PLD accountant every time you train with Opacus, whose default `"prv"` is the privacy loss random variable (PRV) accountant of Gopi, Lee and Wutschitz (2021). It rounds each loss to the nearest grid point, so the errors partly cancel, then adds a proven error bound and reports the upper value. Google's `dp_accounting` has `PLDAccountant`, which by default rounds up with **connect the dots**, the tightest pessimistic rounding a grid allows.

Table 36.1 compares a PLD with Rényi differential privacy (RDP) and with the central limit theorem (CLT) estimate of Gaussian differential privacy (GDP). The run is the part's example of private stochastic gradient descent (SGD), DP-SGD, with sampling rate 0.01 and noise multiplier 1.

Table 36.1.: ε at $\delta = 10^{-5}$, for the running example. The PLD columns round up and down on the grid $h = 10^{-5}$, without connect the dots.

Steps	PLD upper	PLD lower	RDP	GDP CLT
1	0.1995	0.1994	0.9555	0.0368

Steps	PLD upper	PLD lower	RDP	GDP CLT
1000	1.8332	1.8232	2.1014	1.6177
10000	6.2377	6.1377	6.7127	6.0071

RDP gives a valid but loose bound (Rényi DP). The CLT estimate falls below the PLD's lower bound at every length, so it claims a guarantee the run lacks (f-DP and Gaussian DP).

36.3. How to use it

In `dp_accounting` you describe the run as an event, compose it, and read off ε at your δ. The method `get_delta` runs the other way, from a claimed ε to a bound on its δ.

```
from dp_accounting import GaussianDpEvent, PoissonSampledDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from opacus.accountants import PRVAccountant

run = SelfComposedDpEvent(PoissonSampledDpEvent(0.01, GaussianDpEvent(1.0)), 10_000)
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)  # h, 10x the table's
print(f"{pld.get_epsilon(1e-5):.4f}")  # 6.1877
print(f"{pld.get_delta(6.24):.1e}")  # 8.5e-06

prv = PRVAccountant()
prv.history = [(1.0, 0.01, 10_000)]  # (noise multiplier, sample rate, steps)
print(f"{prv.get_epsilon(delta=1e-5):.4f}")  # 6.1980
```

Both answers are upper bounds inside the bracket of Table 36.1, so you can report either. Connect the dots reaches 6.1877 on a grid ten times coarser than the table's. And `get_delta` lets a reviewer confirm the claim $(6.24, 10^{-5})$, since it bounds the run's slack at that ε by 8.5×10^{-6}.

Each library has one accuracy setting. In `dp_accounting` it is the grid spacing `value_discretization_interval` and a tenfold finer grid narrows the bracket tenfold for about ten times the memory. In Opacus it is `eps_error`, the error allowed in ε. The error grows with the steps, so set either for your longest run.

36.4. What goes wrong

The commonest mistake is reporting the wrong side of the bracket. Only the pessimistic value, or the PRV accountant's upper value, is a guarantee. The optimistic value and the bracket's midpoint are estimates.

Under add-or-remove neighbours, a subsampled step has a different PLD for adding a record than for removing one, and neither is larger in general. So an accountant must compose both and report the larger, as `dp_accounting` does by default. Opacus's PRV accountant models only

removal, the larger direction in the running example. Both assume Poisson sampling, which shuffled or fixed-size batches break (Amplification by subsampling).

A run whose steps or noise are chosen from the outputs needs a filter, which enforces a budget fixed in advance (Adaptive composition).

36.5. References

- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, with explicit error bounds.

Derivations. Every result here is derived step by step in the full edition's chapter.

37. f-DP and Gaussian differential privacy

37.1. What it is

Every differential privacy (DP) guarantee limits how well an attacker can tell whether the data held a target's record. The attacker runs a hypothesis test, so the fullest guarantee is a **trade-off curve**, the lowest false negative rate any test can reach at each false positive rate. A mechanism is f**-DP** when its curve never falls below a given curve f (Dong, Roth and Su 2022). A privacy level ε with its slack δ only summarizes that curve.

For Gaussian noise the curve has one parameter. Write σ for the noise multiplier, the noise's standard deviation in units of the query's ℓ_2 sensitivity. In those units the attacker must tell a standard normal draw from one shifted by a distance μ equal to $1/\sigma$. So the mechanism satisfies Gaussian differential privacy (GDP) with parameter μ, written μ**-GDP**. Write Φ for the standard normal distribution function and Φ^{-1} for its inverse. No membership attack at a false positive rate α then reaches a true positive rate above

$$1 - \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{37.1}$$

37.2. When you meet it

You meet GDP when deciding whether a guarantee is enough, and as a trap in Opacus (Section 37.4). Table 37.1 reads μ as the best true positive rate (TPR) of an attack at a given false positive rate (FPR), whatever else the attacker knows.

Table 37.1.: Reading μ-GDP as an attack and as (ε, δ).

μ	Best TPR at 0.1% FPR	At 1%	At 5%	ε at $\delta = 10^{-5}$
0.5	0.0048	0.0339	0.1261	1.9931
1	0.0183	0.0924	0.2595	4.3772
2	0.1378	0.3721	0.6388	9.9973

For example, at a false positive rate of 1%, no attacker on a 1-GDP release detects the record in more than 9.24% of the trials where it is present. That is the ceiling for any membership inference attack, and what a privacy audit tests. Dong et al. suggest that $\mu \leq 0.5$ is a reasonable guarantee and that $\mu \geq 6$ promises almost nothing, so report μ next to a Gaussian pipeline's (ε, δ).

37.3. How to use it

Gaussian releases compose exactly within GDP, even when each one depends on the earlier ones. Write μ_i for the parameter of the i-th release. The run is then μ-GDP with

$$\mu = \sqrt{\mu_1^2 + \mu_2^2 + \cdots + \mu_k^2}, \tag{37.2}$$

so the quantity that adds up is μ^2. No accountant does better for releases on the whole dataset, such as analytics queries and full-batch noisy gradient descent. So keep a ledger of μ^2 and convert once, at the end. For example, four 1-GDP releases compose to 2-GDP, which is $(9.9973, 10^{-5})$-DP. Converting each release first, with a quarter of the slack, and adding the four values of ε gives 18.7607, almost twice as much.

The snippet computes the attack reading and converts both ways. The `dp_accounting` functions take a noise multiplier, so for μ-GDP you pass $1/\mu$.

```
from statistics import NormalDist

import dp_accounting

N = NormalDist()


def attack_power(mu: float, fpr: float) -> float:
    """Best true positive rate of any membership attack on mu-GDP at a false positive rate."
    return 1 - N.cdf(N.inv_cdf(1 - fpr) - mu)


print(" ".join(f"{attack_power(mu, 0.001):.4f}" for mu in (0.5, 1.0, 2.0)))  # 0.0048 0.0183
print(f"{dp_accounting.get_epsilon_gaussian(1.0, 1e-5):.4f}")  # 4.3772
print(f"{dp_accounting.get_sigma_gaussian(1.0, 1e-5):.4f}")  # 3.7306
```

The second print converts 1-GDP to $(4.3772, 10^{-5})$-DP. The third finds the noise multiplier of a single $(1, 10^{-5})$-DP release. Both use the exact curve, which asks for less noise than the classical formula of the Gaussian mechanism.

37.4. What goes wrong

Opacus offers `accountant="gdp"` for the private form of stochastic gradient descent (SGD), DP-SGD. It estimates μ with a central limit theorem (CLT), by which many steps that each leak little compose to nearly a Gaussian curve. For a number T of steps, each sampling a record with probability q, Bu et al. (2020) estimate

$$\mu \approx q\sqrt{T(e^{1/\sigma^2} - 1)}. \tag{37.3}$$

It is an approximation, not a bound, and it errs on the unsafe side, because a small δ is decided by rare, large losses, whose tails a CLT fits poorly. On the part's running example of 10,000 steps,

the CLT gives 6.0071, but a numerical accountant proves that the true ε is at least 6.1377. So report with Opacus's default numerical accountant, `"prv"`.

A mix of Laplace and Gaussian releases needs a numerical accountant, because a curve that is not Gaussian takes the composition out of GDP. Gaussian pipelines without subsampling, though, are best accounted in GDP, since even the best conversion from a Rényi DP curve loses ground. For 1-GDP at $\varepsilon = 1$, it certifies a δ of 0.2149 against the exact 0.1269.

37.5. References

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022).

Derivations. Every result here is derived step by step in the full edition's chapter.

38. Adaptive composition: filters, odometers and individual accounting

38.1. What it is

The composition theorems of differential privacy (DP) assume that each step's privacy parameter, and the number of steps, are fixed before the run (Composition). Real runs often choose them from the outputs, as when training stops on the validation loss. Plugging the realized values into the theorems afterwards is then invalid (Rogers et al. 2016). This **plug-in mistake** can make the bound fail about three times as often as its δ claims.

A **privacy filter** fixes a total budget in advance and refuses any step that would exceed it, so the run keeps the budget's guarantee whatever the stopping rule. A **privacy odometer**, for open-ended analyses, reports the spend so far instead, valid at every step but at a higher ε.

38.2. When you meet it

Early stopping with a planned maximum needs neither. You fix the noise, sampling rate and maximum number of steps in advance, then stop when the validation loss stalls. **Report the ε of the planned maximum, not of the steps you ran.** Those steps are a prefix of the planned run, so the planned guarantee covers them. If 100 planned full-batch Gaussian steps with noise multiplier 10 are $(5.299, 10^{-5})$-DP, a run that stops at step 60 reports that.

The rule needs three conditions. The stopping signal must come from the released models or from held-out data, so that stopping is post-processing. Training past the maximum needs a new plan, accounted from the first step. And a resumed run must restore its accountant (the calibration workflow).

You meet filters when the noise is changed during training in response to the outputs, and in analytics services, such as OpenDP's `make_privacy_filter` or Tumult Analytics' `Session`.

38.3. How to use it

The Rényi filter of Feldman and Zrnic (2021) works in Rényi differential privacy (RDP) at one order, written α and fixed in advance. It charges each step its spend at that order (Rényi DP), which may depend on earlier outputs but must be known before the step runs. It refuses a step

that would take the total past a budget B. To aim at a target privacy level ε_g with slack δ, run the conversion from RDP backwards, which sets

$$B = \varepsilon_g - \frac{\ln(1/\delta)}{\alpha - 1}. \tag{38.1}$$

A Gaussian step with a noise multiplier, written σ, spends $\alpha/(2\sigma^2)$ at order α. At order 5.8, the best fit for the early-stopping target, the budget is $B = 2.900$. That lasts 100 full-batch steps at $\sigma = 10$. Suppose training plateaus at step 60 and you halve the noise. Each later step then costs four times as much, so the budget left for 40 steps buys 10 more, and the run still meets the target.

OpenDP's filter works in zero-concentrated DP (zCDP). Write ρ for its single budget number, which bounds the RDP spend at every order at once (Concentrated DP).

```
import opendp.prelude as dp

dp.enable_features("contrib")
domain, metric = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
odometer = dp.c.make_fully_adaptive_composition(domain, metric, dp.zero_concentrated_diverge
queryable = dp.c.make_privacy_filter(odometer, d_in=1.0, d_out=0.5)(0.0)  # rho = 0.5, i.e.

answered = 0
try:
    for _ in range(1000):  # sigma = 10: rho = 1 / (2 sigma^2) = 0.005 each
        queryable(dp.m.make_gaussian(domain, metric, scale=10.0))
        answered += 1
except dp.OpenDPException:  # the release that would exceed the budget
    pass
print(answered)  # 99
```

OpenDP lets through 99 releases, not 100, because it rounds each charge up for floating-point safety. For pure-DP queries, the **Whitehouse filter** (Whitehouse et al. 2023) fits many more small queries than a plain sum of their ε.

38.4. What goes wrong

An odometer costs more than a filter. The **Ville odometer** converts the RDP spend at the best of a grid of orders fixed in advance, It pays for that late choice by dividing δ by the grid's size. On the 100-step run above, that raises ε from 5.299 to 6.253.

Individual accounting charges each training record by its own clipped gradient norm, since a small gradient moves the model less, and drops a record once its spend would pass the budget. In the full edition's simulation, the median record ends at an individual ε of 3.17, against 5.299 for the worst case. But that number comes from realized norms, so it is an estimate, not a guarantee, and it is sensitive, since a small one says the model fits that record well. And the records dropped first, with large gradients, are often those the model serves worst (Disparate impact).

Choosing hyperparameters from several runs is a separate cost (Hyperparameter tuning).

38.5. References

- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023).
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021).

Derivations. Every result here is derived step by step in the full edition's chapter.

39. Concentrated differential privacy

39.1. What it is

Zero-concentrated differential privacy (zCDP) keeps a differential privacy (DP) budget as one number that adds up. It builds on the curve of Rényi differential privacy (RDP), which bounds how far apart a mechanism's outputs on neighbouring datasets can be at each order α. A mechanism is ρ-**zCDP** when that curve lies under the straight line $\rho\alpha$, so ρ is a slope. Composition adds the slopes, even when each step is chosen after seeing the earlier outputs.

The Gaussian mechanism's curve is exactly such a line. Write σ for the standard deviation of its noise. The query f has an ℓ_2 sensitivity $\Delta_2 f$, the most one record can move it in Euclidean distance. One release then costs

$$\rho = \frac{(\Delta_2 f)^2}{2\sigma^2}. \tag{39.1}$$

So a release with noise multiplier 1, where $\sigma = \Delta_2 f$, costs $\rho = 0.5$, and doubling the noise cuts ρ to a quarter.

39.2. When you meet it

You meet zCDP in releases that add Gaussian noise to the whole dataset, such as noisy counts or the marginals behind a synthetic dataset. The 2020 United States Census used it with the discrete Gaussian mechanism (Discrete and floating-point-safe mechanisms), and Tumult Analytics and OpenDP keep budgets in ρ.

39.3. How to use it

Keep a ledger, a running list of releases, and charge each its ρ from Table 39.1.

Table 39.1.: What each kind of step costs in a zCDP ledger.

Step	Charge in ρ	Notes
Gaussian, sensitivity $\Delta_2 f$, noise σ	$(\Delta_2 f)^2/(2\sigma^2)$	Exact
Any pure ε-DP step	$\varepsilon^2/2$, or $\varepsilon^2/8$ for an exponential mechanism	Above the plain ε once $\varepsilon \geq 2$: keep such a step in RDP

Step	Charge in ρ	Notes
An (ε, δ)-DP step	$\varepsilon^2/2$, plus δ in a second column that also adds	**Approximate zCDP**
Poisson-subsampled Gaussian	$(\Delta_2 f)^2/(2\sigma^2)$, as without sampling	No amplification

Add the charges and convert the total once, at the end, to a privacy level ε and a small slack δ. Then you pay the conversion's slack only once. An all-Gaussian total ρ costs the same as one Gaussian release with noise multiplier $1/\sqrt{2\rho}$, so `get_epsilon_gaussian` converts it exactly. The snippet does this, then calibrates a $(1, 10^{-5})$-DP release.

```
from math import sqrt

import dp_accounting

rho = 1 / (2 * 1.0**2)  # one Gaussian release, noise multiplier 1
print(f"{rho:.1f} {dp_accounting.get_epsilon_gaussian(1 / sqrt(2 * rho), 1e-5):.4f}")  # 0.5
sigma = dp_accounting.get_sigma_gaussian(1.0, 1e-5)  # one release that must be (1, 1e-5)-DP
budget = 1 / (2 * sigma**2)
quarter = budget / 4  # four releases share it
print(f"{budget:.5f} {sigma:.3f} {1 / sqrt(2 * quarter):.3f}")  # 0.03593 3.731 7.461
```

So the budget for $(1, 10^{-5})$ is $\rho = 0.03593$. A single query meets it with noise multiplier 3.731, so four queries sharing it each get a quarter and need twice the noise. Split the budget in ρ, which adds exactly, never in ε. A histogram in which each person sits in one cell is charged once, not once per cell (Parallel composition).

A ledger with steps that are not Gaussian has no exact conversion. The simple conversion of Bun and Steinke (2016) gives

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{39.2}$$

It is safe but loose, overshooting the exact Gaussian value by 21% at $\rho = 0.5$. The improved conversion of Canonne, Kamath and Steinke (2020) overshoots by only 8%, and the RDP accountant in `dp_accounting` applies it. So use it, or the exact one for an all-Gaussian ledger, and name it in the report.

39.4. What goes wrong

The commonest error is reading ρ as ε. A Gaussian release at $\rho = 0.5$ is $(4.3772, 10^{-5})$-DP, so reading it as $\varepsilon = 0.5$ badly overrates the protection. And ρ covers only what passed through the noise. The Census Bureau released some counts exactly, and such **invariants** sit outside the guarantee, so list them next to the budget.

Subsampled steps belong outside zCDP, which gives them no amplification. On this part's running example, a run of the private form of stochastic gradient descent (SGD), DP-SGD, zCDP reports $\varepsilon = 5475.8$ at $\delta = 10^{-5}$ where RDP gives 6.719. So keep a pipeline that includes DP-SGD in RDP

or a numerical accountant, where a ρ-zCDP release joins without loss as a `GaussianDpEvent` with noise multiplier $1/\sqrt{2\rho}$.

Several records of one user cost more, because a group of m records costs $m^2\rho$. Four records in a 0.1-zCDP release cost $\rho = 1.6$.

A budget never renews, because next year's releases about the same people compose with this year's, so fix the period and its total in advance (Release and governance). And keep a long ledger in ρ, not in diffprivlib's `BudgetAccountant`, which adds ε by plain composition (Keeping a budget).

39.5. References

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): the definition.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the improved conversion.

Derivations. Every result here is derived step by step in the full edition's chapter.

40. Amplification by shuffling and by iteration

40.1. What it is

In the shuffle model, each of n users randomizes their own data, so each report satisfies local differential privacy (DP) at a level ε_0. A **shuffler** strips identities and mixes the order, so each report hides among the others. The bag of reports then satisfies central DP at a much smaller ε, which falls like $\sqrt{\ln(1/\delta)/n}$ for a slack δ that must be above 0. All results here are for replace-one neighbours.

For binary randomized response, the analyst learns only the count of 1-reports, so the shuffled guarantee can be computed exactly. For any other local randomizer, use the clone bound of Feldman, McMillan and Talwar (2021). With probability $e^{-\varepsilon_0}$, another user's report looks exactly like one you could have sent, and such **clones** hide yours. The **numeric clone bound** computes this exactly, while the paper's closed form is looser and needs enough users.

40.2. When you meet it

Shuffling appears in telemetry systems such as Prochlo (Bittau et al. 2017), where the decision is how large ε_0 can be. No maintained library accounts for it, so use the full edition's shuffling_iteration.py.

Two other things share the name but get none of this amplification. A data loader that shuffles the training set once per epoch is a sampler (Match the sampler to the accountant), Federated clients that join rounds when available pass through no shuffler either, so account them without amplification or with matrix mechanisms.

You also meet amplification by iteration, where later training noise protects earlier records, in papers that release only the final model (Section 40.4).

40.3. How to use it

Work backwards from the target: compute the numeric clone bound for a few values of ε_0, and keep the largest that meets it.

Table 40.1.: Central ε at $\delta = 10^{-6}$ after shuffling the reports of n users. "n/a" marks too few users for the closed form.

n	ε_0	Exact, randomized response	Clones, numeric	Clones, closed form
1,000	4	1.6910	3.9900	n/a
10,000	2	0.0870	0.1550	0.5009
100,000	2	0.0247	0.0450	0.1862
100,000	4	0.0847	0.1698	0.5346

The choice of bound can decide feasibility. For 100,000 users and a target of 0.2, the closed form allows ε_0 up to 2 but the numeric bound allows 4, so deploy the numeric bound.

Before reporting a shuffled ε, check the bound's assumptions, because if one fails, such as a leaked order or a colluding shuffler, only ε_0 remains.

- **The order is uniformly random and hidden,** with no metadata such as arrival time or network address.
- **The shuffler does not collude with the analyst.**
- **Only honest users count in n,** since colluders reveal their reports to the analyst.
- **The count of users is the number of reports shuffled,** so with several batches it is the smallest batch.
- **Each user sends one report per batch,** or the reports compose like any other releases.

40.4. What goes wrong

Shortcuts fail where users are few or ε_0 is large. For 1,000 users at $\varepsilon_0 = 4$, the closed form says nothing, and a normal approximation of the count of 1-reports gives 0.9667 against the exact 1.6910.

Amplification by iteration covers one pass of noisy stochastic gradient descent (SGD) that keeps the weights in a fixed convex set (Feldman et al. 2018). It needs two conditions. Only the final model may be released, and every step must be a contraction, never moving two sets of weights further apart. That takes a convex, smooth loss and a small enough step. Even then the gain depends on the record. Over 10,000 records with noise multiplier 1, the middle record gets $\varepsilon = 0.0707$ at $\delta = 10^{-5}$, but the last gains nothing and gets 5.2985, as one Gaussian step does. So report the worst record's value.

Deep learning breaks both conditions. Checkpoints, logged models and a best checkpoint picked from several all release more than the final model, and non-convex losses are not contractions. Annamalai (2024) builds one whose final model leaks as much as all the intermediate models together. So claim amplification by iteration only for convex models, such as logistic regression, that release just their final weights.

40.5. References

- Feldman, McMillan & Talwar, “Hiding Among the Clones” (FOCS 2021): the clone bound and how to compute it.

Derivations. Every result here is derived step by step in the full edition’s chapter.

Part V.

Differential privacy in machine learning

This part is for anyone who trains, ships or reviews a model under differential privacy (DP). It builds on the Gaussian mechanism and composition from Part 3 and an accountant from Part 4. Its audits reuse the membership-inference attacks of Part 2.

Every private training method has two halves. The privacy half, the guarantee ε, is computed by an accountant, while the utility half, accuracy, can only be measured. The main private algorithm is stochastic gradient descent (SGD) with clipped per-example gradients and Gaussian noise, called DP-SGD. Its accountant needs just three settings: the noise multiplier, the sampling rate and the number of steps. So every page asks whether the ε is correct and whether the accuracy at that ε is good enough.

Three pages are workflows, each a whole job done with real libraries on public data. Train a model with DP-SGD trains a classifier on the Adult census data with Opacus. Fine-tune a language model privately with LoRA adapts a small public language model to a private text and checks it for memorized secrets. Release and governance recomputes the guarantee, writes it down, maps it to the law and lists what to check before a model ships.

The other pages explain the decisions those jobs make, in the order a project meets them. A short page on private empirical risk minimization for convex models leads to DP-SGD. Then come hyperparameter tuning, the privacy unit in User-level differential privacy and the budget in Choosing ε. Public data and private fine-tuning explains why a public starting model helps so much. Disparate impact and Auditing private training are the checks before release. Three reference pages cover rarer methods: private prediction, correlated noise across steps and adaptive data analysis.

Every number on these pages comes from a script you can run. At the end, you can train a private model, defend its settings and its ε, audit it, and write its release statement.

41. Private empirical risk minimization

41.1. What it is

Most of machine learning is **empirical risk minimization (ERM)**, which picks the weights θ with the least average loss over a training set. Write D for that set and n for its number of records. A ridge penalty usually keeps the weights small. Its strength is a number λ, and training minimizes

$$\mathcal{L}(\theta; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2} \|\theta\|^2, \tag{41.1}$$

where $\ell(\theta; z)$ is the loss of the weights on one record z. Differential privacy (DP) asks that replacing one record barely change the released weights. So noise must enter the trained weights, the objective or each gradient step.

Output perturbation trains as usual and adds Gaussian noise to the final weights, with a small slack δ. The noise is scaled to the most the trained weights θ^* can move between two datasets D and D' that differ in one record. For a convex loss, the ridge limits that move to

$$\|\theta^*(D) - \theta^*(D')\| \le \frac{2L}{\lambda n}. \tag{41.2}$$

Here the constant L bounds the length of each record's loss gradient. It is 1 for logistic regression with labels ± 1 and features of length at most 1.

Objective perturbation adds a random linear term to the objective and minimizes the result exactly. Part of the privacy budget ε pays for the loss's curvature, which is at most a constant c, one quarter for the logistic loss. The noise is scaled to what is left,

$$\varepsilon' = \varepsilon - 2 \ln \Big(1 + \frac{c}{n\lambda} \Big). \tag{41.3}$$

The subtracted term grows as $n\lambda$ shrinks, so with little data the ridge must grow to keep ε' positive.

Gradient perturbation adds noise to every gradient step instead. With random batches and clipped per-record gradients, it becomes noisy stochastic gradient descent (SGD), DP-SGD. It is the only option for a model that is not convex.

41.2. When you meet it

You meet these methods for linear models, such as a linear probe on features from a public pretrained network (Public data and private fine-tuning). Objective perturbation is usually the

most accurate, because the objective's full curvature damps its noise, while only the ridge damps output perturbation's. So use output perturbation only when training is a black box.

Whether you have enough data depends on n and ε. On the training objective they matter only through their product, so halving ε costs as much as halving the data. On new data, with d features, privacy adds less error than sampling does once n is well above $d\ln(1/\delta)/\varepsilon^2$.

41.3. How to use it

IBM's diffprivlib offers objective perturbation as `LogisticRegression(epsilon=..., data_norm=...)`, with `data_norm` bounding each row's length. Its version 0.6.6 fails with scikit-learn 1.8, so the snippet calls the mechanism it wraps, `mechanisms.Vector`, with the curvature bound c as `function_sensitivity`. The breast-cancer data has no public data dictionary to take feature bounds from. So the snippet treats 500 rows as private and 69 as a public stand-in. It clips the private rows to the stand-in's log-transformed ranges and maps them to $[-1, 1]$. A constant 1 replaces the intercept, and dividing by $\sqrt{31}$ keeps each row's length at most 1.

```
import numpy as np
from diffprivlib.mechanisms import Vector
from scipy.optimize import minimize
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
public, private = np.log1p(data.data[:69]), np.log1p(data.data[69:])  # 69 stand-in rows fix
lo, hi = public.min(axis=0), public.max(axis=0)
X = np.hstack([2 * (np.clip(private, lo, hi) - lo) / (hi - lo) - 1, np.ones((500, 1))]) / np
y = 2 * data.target[69:] - 1
Xt, yt, n = X[::2], y[::2], 250  # even rows train, odd rows test
lam = 1 / n  # diffprivlib's default ridge, fixed in advance

def objective(w):  # J(w) and its gradient
    m = yt * (Xt @ w)
    return np.logaddexp(0, -m).mean() + lam / 2 * w @ w, lam * w - Xt.T @ (yt / (1 + np.exp(

def accuracy(eps, seed):  # eps = inf adds no noise
    vector = Vector(epsilon=eps, function_sensitivity=0.25, dimension=31, alpha=n * lam, n=n
    w = minimize(vector.randomise(objective), np.zeros(31), jac=True, options={"gtol": 1e-9}
    return np.mean(np.sign(X[1::2] @ w) == y[1::2])

print(" ".join(f"{np.mean([accuracy(e, s) for s in range(20)]):.3f}" for e in [1, 4, np.inf]
```

Averaged over 20 noise draws, test accuracy is 0.783 at $\varepsilon = 1$ and 0.946 at $\varepsilon = 4$, against 0.964 without noise. With only 250 training records, accuracy comes close only at $\varepsilon = 4$. Nothing was tuned on the private rows, so the guarantee is pure DP for the 250 training rows, against replacing one record. The 69 stand-in rows, whose ranges the code prints, get none.

41.4. What goes wrong

The most common failure is a bound taken from the data. Left unset, `data_norm` is computed from the training rows with a `PrivacyLeakWarning`, and the guarantee is gone. A range table computed from the same rows, like breast cancer's, is no better, because each minimum and maximum is one record's value. So take every bound from public knowledge or public rows, and clip the private rows to it.

The second failure is the solver, because output and objective perturbation assume the exact minimizer. A solver that stops where the gradient still has length g adds $2g/\lambda$ to the sensitivity. That is small beside the exact sensitivity $2L/(\lambda n)$ only when g is well below L/n. So set the tolerance there, and check the final gradient rather than the number of steps.

The third failure is an unpenalized weight, because the sensitivity bound needs the ridge's curvature in every direction. Scikit-learn's loss, which diffprivlib reuses, leaves the intercept out of the ridge, hence the snippet's constant feature. Fix λ by a rule or on public data, since tuning it on private data is a search that must be paid for (Hyperparameter tuning). Finally, these guarantees assume replaced records, unlike the add/remove accountants of later chapters. The two do not convert freely, so state the relation next to every ε.

Derivations. Every result here is derived step by step in the full edition's chapter.

42. DP-SGD

42.1. Why this matters

Nearly every deep-learning system that satisfies differential privacy (DP) runs one algorithm. It is stochastic gradient descent (SGD) with clipped per-record gradients and Gaussian noise at every step, called DP-SGD (Abadi et al. 2016). Clipping limits how far one record moves the model at each step, and the noise hides that move. An **accountant** adds up the cost of the steps into the guarantee's ε, with a small additive slack δ.

The accountant sees only the noise, the sampling rate (the chance that a record joins a batch) and the number of steps, never the model. So those three decide ε, and the clipping norm and learning rate can be tuned for accuracy alone. Most failures come from code that breaks an assumption of the privacy proof, so the checks matter as much as the settings. Train a model with DP-SGD does the whole job in Opacus.

42.2. The algorithm and its guarantee

A training set holds n records. The gradient of one record's loss is its **per-example gradient**, written g_i for record i. Each step of DP-SGD does five things.

1. Sample a batch. Each record enters independently with probability q, the **sampling rate**, which is called Poisson sampling.
2. Compute the per-example gradient of each record in the batch.
3. Clip each gradient longer than the **clipping norm** C to length C.
4. Sum the clipped gradients, and add Gaussian noise with standard deviation σC to each coordinate, where the **noise multiplier** σ measures the noise in units of C.
5. Divide the noisy sum by the expected batch size $B = qn$, a fixed public number, and step against it, scaled by the **learning rate** η.

Write θ_t for the weights at step t and S_t for its batch. With Gaussian noise ξ_t drawn afresh on each coordinate, the update is

$$\theta_{t+1} = \theta_t - \frac{\eta}{B}\Big(\sum_{i \in S_t} g_i \min\Big(1, \frac{C}{\|g_i\|}\Big) + \xi_t \Big), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I). \tag{42.1}$$

Releasing every intermediate set of weights then satisfies (ε, δ)-DP for **add/remove neighbours**, datasets that differ by one record (Amplification by subsampling).

This part's running example (Section 42.7) trains a small network on 10,000 synthetic records. It runs 10,000 steps with $\sigma = 1$, and each record joins a batch with probability $q = 0.01$. For this run, the Rényi differential privacy (RDP) accountant, a standard and slightly loose upper bound,

gives $\varepsilon = 6.71$ at $\delta = 10^{-5}$. That δ follows the book's default of $1/n^{1.1}$ rounded down to a power of ten (Choosing ε). To pick σ for a target ε, let the accountant you will report solve for it (Train a model with DP-SGD).

42.3. The implementation checklist

The guarantee rests on four properties of your code, and code that breaks one still trains and reports as before.

First, one record must move the clipped sum by at most C. Batch normalization breaks this, because its statistics let one record change every other record's gradient. So private models use group or layer normalization, which works on each record alone. With `strict=False`, Opacus's validator lists the layers to replace instead of raising an error.

```
from opacus.validators import ModuleValidator
from torchvision.models import resnet18

model = resnet18(weights=None)
print(len(ModuleValidator.validate(model, strict=False)))  # 20
model = ModuleValidator.fix(model)
print(len(ModuleValidator.validate(model, strict=False)))  # 0
```

Also fix C in advance, from public data or a private estimate. A C read off the current batch makes the noise depend on the data.

Second, the accountant must assume the sampler the code actually runs (Amplification by subsampling). The Poisson accountant's saving comes from each record's chance of being left out of a batch. A loader that shuffles once per epoch and cuts fixed batches puts every record in exactly one batch per epoch. That chance is then gone, and the true ε can be far above the Poisson number.

Fixed-size batches drawn afresh at every step can still use a Poisson accountant, at the batch's share of the data and half your noise multiplier. The halving is needed because a record that enters a fixed-size batch pushes another out, so the sum can move by $2C$. It is expensive, raising the running example's ε from 6.71 to 47.42, and it holds only for add/remove neighbours.

Third, divide the noisy sum by the public number B, never by the actual batch size. The actual size is one larger when the protected record is drawn, so dividing by it reveals whether it was.

Fourth, the accountant must count every step that touched the data, including those of runs discarded in tuning.

Beyond the proof, state the neighbouring relation next to every ε, and make data-dependent preprocessing private too. The guarantee covers one record, so a person with many records needs user-level privacy. For a model you release, turn on Opacus's `secure_mode=True`, because floating-point sampling is not the exact Gaussian noise of the proof (Discrete and floating-point-safe mechanisms).

42.4. Clipping and the learning rate

Clipping must happen per record. PyTorch's `torch.nn.utils.clip_grad_norm_` clips a whole batch's gradient, which bounds no single record and so gives no privacy at any threshold.

The clipping norm C does not change ε, but it costs accuracy either way. A small C clips more gradients, so records with large gradients count for less, and they are often rare records (Disparate impact). A large C means more noise, since the noise is σC.

When every gradient is clipped, each has length exactly C and the noise is proportional to C too. So the step depends only on the product ηC of the learning rate and the clipping norm. Clip hard, and tune that one number. On the running example, holding ηC at 0.25 keeps test accuracy at 0.9599 or above for every C up to 0.3. Holding η at 0.5 instead, $C = 10$ drops it to 0.8347.

Noise and clipping also move the best learning rate. Noise lowers it, as on the logistic problem of Optimization, while clipping raises it, because clipped gradients are shorter. The two do not cancel predictably, so search the step again whenever σ or C changes, and count the search's runs in ε.

Gradient norms also drift. The running example's median per-example norm falls from 3.592 to 0.006 in a thousand steps. So a C that clips everything early clips only a minority later. Adaptive clipping follows the drift by moving C toward a quantile of the norms, measured by a small private count. Opacus offers it with `clipping="adaptive"` and five more parameters. Because the count spends part of the budget, the gradient gets more noise at the same σ.

42.5. Batch size at a fixed budget

Averaging the noisy sum over B records divides the added noise by B, so larger batches dilute it. At a fixed budget and number of epochs, a larger batch means fewer steps. The noise multiplier must then rise to keep ε, but more slowly than B does. For a small sampling rate q and a σ well above 1, the accountant's cost over a fixed number of epochs depends on q/σ^2. So σ only has to grow like $\sqrt{q}$. Since $q = B/n$, with n the number of records, the noise on the averaged gradient falls with the batch size,

$$\sigma \propto \sqrt{\frac{B}{n}}, \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{Bn}}. \tag{42.2}$$

So choose the largest batch your hardware allows, with a larger learning rate for the fewer steps. When it does not fit in memory, Opacus's `BatchMemoryManager` splits the batch the sampler draws into physical batches that fit. It still steps once per sampled batch, so the accounting stays the same.

42.6. Making it fast

Computed one record at a time, DP-SGD can run tens or hundreds of times slower than ordinary training. Two fixes cost nothing. Vectorized per-example gradients, from `torch.func.vmap` or Opacus's hooks, compute the same clipped sum in parallel. **Ghost clipping** also saves memory,

because it gets each record's gradient norm from each layer's inputs and errors without the gradient itself. Opacus turns it on with `grad_sample_mode="ghost"`.

The third fix, microbatching, costs accuracy. It clips the average gradient of each group of m records. One record can change its group's clipped average by up to $2C$, so the noise doubles. The update then averages over B/m groups rather than B records, so the noise on each coordinate of the averaged update becomes

$$2m \cdot \frac{\sigma C}{B}. \tag{42.3}$$

On the running example at $\sigma = 1$, groups of 4 cut test accuracy from 0.9589 to 0.9336. So use microbatching only as a last resort.

42.7. What privacy costs in accuracy

Table 42.1 trains the running example, a network with one hidden layer of 64 tanh units, at four budgets with C and η both 0.5.

Table 42.1.: Test accuracy of the running example under DP-SGD at four budgets, with $\delta = 10^{-5}$ and the RDP accountant.

	Noise multiplier	Test accuracy (mean ± standard deviation over 3 runs)
$\varepsilon = 0.5$	7.72	0.8209 ± 0.0180
$\varepsilon = 1$	4.13	0.9175 ± 0.0161
$\varepsilon = 4$	1.36	0.9548 ± 0.0019
$\varepsilon = 8$	0.917	0.9604 ± 0.0019
Clipped, no noise	0	0.9649 ± 0.0009
Non-private SGD (1,000 steps)	—	0.9610 ± 0.0005

Clipping without noise scores slightly above non-private SGD here, so the whole loss comes from the noise, and it grows steeply as ε shrinks. Going from $\varepsilon = 8$ to 4 costs half a point, while going from 1 to 0.5 costs almost 10. The noise also grows in total length with the square root of the number of trained weights, while the clipped signal stays at most C. So a smaller model or public pretraining often helps.

42.8. Cheat sheet

Table 42.2.: DP-SGD starting settings and checks.

Setting	Start with	Check
δ	$1/n^{1.1}$ rounded down to a power of ten	n counts people

Setting	Start with	Check
Batch size	256 to 4,096, as large as fits	Noise on the average is $\sigma C/B$
Epochs	Fixed before calibrating; Adult uses 10	σ recomputed if they change
Noise multiplier	From the accountant you report	A second accountant gives the same ε
Clipping norm	1, or the median norm on public data	Per record, never `clip_grad_norm_`
Optimizer, learning rate	SGD with momentum 0.9; several times the non-private learning rate (ten on Adult), chosen on public data	Retuned when σ or C changes
Sampler	Poisson	Fixed-size batches: account at $\sigma/2$ (add/remove only)
Report	ε, δ, neighbours, accountant	Every run counted

Rules of thumb

- Fix ε, δ and the number of epochs first, since they are the contract. Then choose the batch, let the accountant set σ, and tune the learning rate last.
- Find the ε below which accuracy collapses on your task before you promise a budget.

42.9. References

- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): what to clip, how to tune and what to report.

Derivations. Every result here is derived step by step in the full edition's chapter.

43. Train a model with DP-SGD

43.1. Why this matters

You have a PyTorch training loop on sensitive data, and it must become private. The algorithm is stochastic gradient descent (SGD) with differential privacy (DP), called DP-SGD, which clips each record's gradient and adds noise at every step (DP-SGD). The library is Opacus, and the hard part is not the call that wraps the model but the dozen decisions around it. Most are inputs to the guarantee. A mistake in any of them leaves code that trains as before but reports a false privacy level ε.

This chapter does the whole job on Adult, a census survey of 48,842 people from the UCI Machine Learning Repository. The task is to predict whether a person earns more than $50,000 a year. The script practitioner_dp_sgd.py runs every step except tuning, which practitioner_tuning.py does. Each runs in one to two minutes without a GPU.

```
uv run python code/part05-dp-ml/practitioner_dp_sgd.py
uv run python code/part05-dp-ml/practitioner_tuning.py
```

43.2. Fix the privacy unit, n and δ

The first decision is what one record is, because the guarantee protects exactly that (User-level differential privacy). In Adult each row is one person, which rests on how the survey was collected, since Adult has no person identifier. When your data has one, count distinct identifiers, not rows, and give a person with many rows user-level training.

The script splits the rows with seed 0 into 30,000 private training rows and 16,842 test rows. The other 2,000, the stand-in rows from here on, play the part of public data. The script chooses the clipping norm (the length each record's gradient is cut to) and the learning rate on them, and never trains on them. Since they come from the same survey, this is an assumption for the demonstration. In production the role belongs to genuinely public data, and without any, tuning costs privacy (Section 43.10).

Next comes δ, the small additive slack the guarantee allows on top of ε. It depends only on the number of people n in the private training set. The book's default is $\delta \leq 1/n^{1.1}$, rounded down to a power of ten, which keeps it safely below $1/n$ (Choosing ε).

```
def delta_for(n: int) -> float:
    """The book's default: delta <= 1/n^1.1, rounded down to a power of ten."""
    return 10.0 ** math.floor(math.log10(n**-1.1))
```

For the 30,000 private training rows, not the 48,842 of the whole dataset, the script prints $\delta = 10^{-5}$. The usual mistake is quoting a δ above $1/n$.

43.3. Preprocess from public facts

Every statistic computed from the private rows is itself a release. The accountant, which adds up the privacy cost of training, does not count it. A scaler fitted on the training set leaks each column's mean and spread. An encoder fitted on it leaks which categories occur, even a country only one person comes from.

So the script takes its category lists from the dataset's documentation. It scales each number by bounds the survey publishes, such as ages from 17 to 90. Capital gains and losses go on a log scale first, and a missing value becomes its own category.

```
def features(rows: pl.DataFrame) -> Array:
    """One-hot categories from the documented lists, numbers scaled by public bounds."""
    rows = rows.with_columns(pl.col("capital-gain", "capital-loss").log1p())
    numeric = [
        np.clip((rows[name].to_numpy() - low) / (high - low), 0.0, 1.0) for name, (low, high
    ]
    encoder = OneHotEncoder(categories=list(CATEGORIES.values()), sparse_output=False)
    text = rows.select(pl.col(list(CATEGORIES)).fill_null("?")).to_numpy()
    encoder.fit(text[:1])  # the categories are given, so fitting learns nothing from the ro
    return np.hstack([np.column_stack(numeric), encoder.transform(text)])
```

This gives 91 features, and the model is a small network with one hidden layer of 64 units and 6,018 weights in all. To check this step in someone else's code, search for every `fit` call and ask what data it sees.

43.4. Train the baselines

A private accuracy means little on its own, so train three baselines with the same model, batch size, epochs and optimizer. The non-private model shows the best you can hope for. A model with clipping and no noise separates the cost of clipping from the cost of noise. Always predicting the majority class shows the floor, where a private model whose training collapsed lands.

```
non-private: test accuracy 0.847
clip only (C=1, no noise): test accuracy 0.845
always '<=50K': test accuracy 0.759
```

Clipping alone costs little here. Had clip-only accuracy been far below non-private, no privacy budget would fix it, because the problem would be the clipping norm or the learning rate.

43.5. Check the model

Opacus computes one gradient per record, and some layers break that. Batch normalization is the common case, because its statistics mix the records of a batch, so one record changes every other record's gradient. The validator lists such layers before you train.

```
errors = ModuleValidator.validate(model, strict=False)
print(f"validator errors: {len(errors)}")
```

Here it finds none. For a model with such layers, `ModuleValidator.fix` replaces batch normalization with group normalization (DP-SGD shows it on a ResNet).

43.6. Choose the clipping norm, batch and epochs

These settings are fixed before the private data is touched, because choosing them on it costs privacy. The usual guide for the clipping norm C is the typical per-example gradient norm on public data. So the script measures it on the stand-in rows.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every weight p
norms = stacked_norms(wrapped)
```

```
at initialisation: median norm 1.444  90th percentile 1.538
fraction clipped at C=1: 1.000
```

The median rule would give a C of about 1.4. The script keeps the usual default $C = 1$, just below the median, so every gradient is clipped at the start. That costs nothing, because when every gradient is clipped, only the learning rate times C matters (DP-SGD).

The batch should be as large as you can afford. The noise on the averaged gradient is $\sigma C/B$. Here B is the expected batch size, and the noise multiplier σ is the noise's standard deviation in units of C. The size is only expected because every batch is drawn at random (Section 43.8).

With $B = 1{,}000$, Opacus sets the sampling rate q, the chance that a row joins a batch, to one over the number of batches per epoch. That is $B/n = 0.0333$ for the $n = 30{,}000$ training rows, but not when B does not divide n, so plan with Opacus's value. Ten epochs of $1/q$ steps each make 300 steps, and the epochs must be fixed now, since the noise is calibrated to them.

The optimizer is SGD with momentum 0.9. The script picks its learning rate on the stand-in rows too. It trains a non-private model on 1,500 of them at six rates and scores each on the other 500.

```
  lr=0.05: 0.814  lr=0.1: 0.816  lr=0.2: 0.826  lr=0.5: 0.792  lr=1: 0.812  lr=2: 0.768
non-private learning rate 0.2; DP-SGD learning rate 2 (x10)
```

DP-SGD then uses ten times the winning rate. That factor is a rule of thumb, not a measurement. Private runs usually need a step several times the non-private one, because clipped gradients are shorter (DP-SGD). No private row has been touched, so this choice costs no privacy.

43.7. Calibrate the noise

Fix the target ε, the slack δ, the sampling rate q and the number of epochs. Then let the accountant solve for the smallest noise multiplier that meets them. Use the accountant you will report, here the privacy loss random variable (PRV) accountant, which computes ε almost exactly. Calibrating with a looser accountant pays for its overestimate of ε with extra noise (Calibrate and account a DP-SGD run).

```
sigma = get_noise_multiplier(
    target_epsilon=epsilon,
    target_delta=delta,
    sample_rate=sample_rate,
    epochs=epochs,
    accountant="prv",
)
```

For $\varepsilon = 4$ at $\delta = 10^{-5}$ it prints `sigma=0.973`. The result holds only for the epochs and sampling rate you gave it. If you later train longer or change the batch size, recompute σ before training. Otherwise report the larger ε of what actually ran.

43.8. Train

The privacy engine wraps the model, the optimizer and the data loader. Its optimizer clips each record's gradient and adds the noise. Its loader draws Poisson batches, in which each row joins each batch independently with probability q, which is the sampling the accountant assumes. The script's own loader shuffles and cuts fixed batches, so `make_private` must replace it for that assumption to hold.

```
engine = PrivacyEngine(accountant="prv")
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
model, optimizer, data_loader = engine.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=data_loader,
    noise_multiplier=sigma,
    max_grad_norm=CLIP,
)  # Poisson sampling is the default: each row joins each batch with probability q
```

The training loop is an ordinary one inside `BatchMemoryManager`. It splits each logical batch of about 1,000 rows into physical batches of at most 500 that fit in memory. The optimizer still steps once per logical batch, so the accounting does not change.

```
with BatchMemoryManager(
    data_loader=data_loader, max_physical_batch_size=PHYSICAL_BATCH, optimizer=optimizer
) as batches:
    for x, y in batches:  # Opacus splits each logical batch; it steps once per logical batc
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
```

```
steps=300  q=0.0333
epsilon (Opacus PRV) = 3.99
test accuracy at epsilon=4: 0.846
```

The script leaves `secure_mode` off, so that its noise is seeded and its numbers repeat, and its record says so. For a model you release, pass `secure_mode=True` to `PrivacyEngine`. It draws the noise from a cryptographically secure generator and guards against the floating-point flaws of ordinary Gaussian samples (Discrete and floating-point-safe mechanisms).

43.9. Cross-check ε

An ε from one library is a claim about that library's arithmetic. So recompute it with Google's `dp_accounting` from the noise multiplier, sampling rate and step count of your plan. Opacus counts the steps it actually took. Its privacy loss distribution (PLD) accountant is almost exact, like PRV, but uses a different numerical method, so the two should agree to within rounding. Its Rényi differential privacy (RDP) accountant, an older and slightly loose bound, runs alongside.

```
step = dp_accounting.PoissonSampledDpEvent(sample_rate, dp_accounting.GaussianDpEvent(sigma)
training = dp_accounting.SelfComposedDpEvent(step, steps)
epsilon_pld = dp_accounting.pld.PLDAccountant().compose(training).get_epsilon(delta)
epsilon_rdp = dp_accounting.rdp.RdpAccountant().compose(training).get_epsilon(delta)
```

```
epsilon PRV (Opacus) = 3.99
epsilon PLD (dp_accounting) = 3.98
epsilon RDP (dp_accounting) = 4.50
```

The PRV and PLD values agree. RDP gives 4.50 for the same run, a valid but looser bound, so a statement must name its accountant. If the two tight accountants disagree by more than rounding, the run did not do what the plan says.

43.10. Tune honestly

The default is the one this run used: choose every hyperparameter on public data, or on a public stand-in, and tuning costs nothing. When no public data exists, tuning runs on the private rows.

Then every private run that influences the release costs privacy, including the runs you discard (Hyperparameter tuning).

The tuning script measures that cost for six learning rates at a total ε of 4, against a single untuned run. A grid runs all six rates and pays for each by composition. A random search runs a Poisson number of models with mean 4, each at a rate drawn from the six, and releases only the best. Only RDP handles a random number of runs, so the script uses it throughout. That is why its single-run σ is larger than the PRV value above.

```
def random_search(sigma: float) -> dp_accounting.DpEvent:
    """A Poisson number of runs with mean 4 (shape=inf selects the Poisson case); only the b
    return dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(sigma), mean=MEAN_RUNS, sha


def sigma_for(strategy: Callable[[float], dp_accounting.DpEvent], epsilon: float, delta: flo
    """The smallest noise multiplier per run that keeps the whole strategy within (epsilon,

    The search starts at 0.1 and tries 1 first.
    """
    return dp_accounting.calibrate_dp_mechanism(
        dp_accounting.rdp.RdpAccountant, strategy, epsilon, delta, dp_accounting.LowerEndpoi
    )
```

```
one run, no tuning: sigma=1.029
grid of 6, all runs counted: sigma=1.816
random search, Poisson mean 4 runs: sigma=1.256
naive grid of 6 at sigma=1.029: reports epsilon=4.00
naive grid, true epsilon of all 6 runs = 9.84
```

The naive grid is the common mistake. It trains six models at the single-run noise, reports the ε of one, and actually spends 9.84. The random search pays for its honesty with a modest rise in σ, from 1.029 to 1.256.

With seed 0, the search drew only two runs, both at the learning rate 0.05. Its model reaches 0.828 on the test set, below the 0.846 of the run tuned on public data. A search with few runs can miss. Do not redraw until you like the answer, because the guarantee depends on the randomness of the draw. So spend private runs on tuning only when no public data can stand in.

43.11. Measure utility and groups

Train at several budgets with everything else fixed, and compare against the baselines. Fix the budget you release beforehand, as the script does with $\varepsilon = 4$. These runs are scored on private rows, so choosing ε from them would use private data that no ε counts (Choosing ε).

Table 43.1.: Test accuracy on Adult against the privacy budget, with $\delta = 10^{-5}$ and one seed.

Run	Noise multiplier	Test accuracy
Non-private	—	0.847
Clipped, no noise	0	0.845
$\varepsilon = 8$	0.722	0.846
$\varepsilon = 4$	0.973	0.846
$\varepsilon = 2$	1.431	0.845
$\varepsilon = 1$	2.383	0.843
$\varepsilon = 0.25$	8.125	0.793
Always “≤50K”	—	0.759

Privacy costs almost nothing on Adult down to $\varepsilon = 1$, and only at $\varepsilon = 0.25$ does accuracy drop, by about five points. Above that, the differences are a thousandth or two on a single seed, too small to read as an effect.

Adult is easy. Its training set is large, so a batch of 1,000 still has a small sampling rate and divides the noise by 1,000. Its model is small, and the noise's total length grows with the square root of the number of trained weights. So expect a steeper curve with millions of weights trained from scratch, a few thousand records, a rare class or many more epochs. The running example of DP-SGD already loses several points at $\varepsilon = 1$, and public pretraining is the usual remedy (Public data and private fine-tuning).

An average can hide a group that pays. So check accuracy by group with exact (Clopper–Pearson) binomial intervals, which stay valid for small groups (Disparate impact).

```
in_group = sex == group
correct, total = int(right[in_group].sum()), int(in_group.sum())
interval = binomtest(correct, total).proportion_ci(confidence_level=0.95, method="exact")
```

```
non-private Female: 0.924 [0.917, 0.931]  (5,186/5,613)
non-private Male: 0.808 [0.800, 0.815]  (9,071/11,229)
DP Female: 0.922 [0.915, 0.929]  (5,177/5,613)
DP Male: 0.807 [0.800, 0.815]  (9,065/11,229)
```

At $\varepsilon = 4$ neither group pays a measurable price, since each private interval overlaps its non-private one almost entirely. The gap between the groups exists without privacy too, and it is the baseline a private model must be compared against.

43.12. When it goes wrong

Table 43.2 maps the common symptoms to their usual cause and fix.

Table 43.2.: Common DP-SGD failures and their fixes.

Symptom	Likely cause	What to do
The validator reports errors	Batch normalization or an unsupported custom layer	`ModuleValidator.fix`, then build the optimizer on the fixed model's weights and retrain the baseline
"Model needs to be in training mode"	A model last used for evaluation	`model.train()` before `make_private`
Out of memory	Per-example gradients for a large physical batch	Lower `max_physical_batch_size`, or use ghost clipping: pass `grad_sample_mode="ghost"` and `criterion=` to `make_private`, which then returns the model, optimizer, loss function and loader, and train with that loss
Ten times slower than usual	Per-example gradients for every weight	Train fewer weights: freeze a public backbone, use adapters
Accuracy near the majority class	Noise too large for the signal	Check $\sigma C/B$; raise the batch, or train fewer epochs so that the accountant needs a smaller σ; start from a public model
ε looks too good	A shuffling loader with `poisson_sampling=False`, uncounted tuning runs, or epochs changed after calibration	Check that the loader is Opacus's `DPDataLoader`, as the record's sampler does; `engine.get_epsilon` and the cross-check report the same ε either way. Recount runs and epochs

Accuracy and privacy fail independently. A run whose ε is wrong usually trains normally, which is why the cross-check and the audit of Auditing private training exist.

43.13. Record the run

A reviewer can only check a claim whose inputs are written down, so the script ends by writing a record of the run. It holds the unit, n and δ, the accountant's three inputs and its name, the library versions, and the three values of ε. It reads the sampler from the loader Opacus returned and `secure_mode` from the engine, so it cannot claim Poisson sampling the code did not do. It also says how each setting was chosen and whether that choice is counted in ε. Release and governance turns the record into a statement.

43.14. Checklist

Table 43.3.: The steps of training with DP-SGD, their defaults and their checks.

Step	Default	Check
Unit, n, δ	Person; $\delta \leq 1/n^{1.1}$ rounded down	n counts people; $\delta < 1/n$
Preprocessing	Public categories and bounds	No `fit` on private rows
Baselines	Non-private, clip-only, majority class	Clip-only close to non-private
Model	No batch normalization	`ModuleValidator.validate` returns nothing
Clipping, batch, epochs	C near or below the median norm on public data; large B; epochs fixed	Chosen without private data
Noise	`get_noise_multiplier` with the reported accountant	Epochs and q match the run
Training	Poisson sampling, `BatchMemoryManager`, `secure_mode=True` for release	The loader is Opacus's
Cross-check	PLD from `dp_accounting`	Agrees with PRV within rounding
Tuning	On public data; else random search with a Poisson number of runs	Every private run counted
Utility and groups	Several budgets; per-group exact intervals	Compared with the baselines
Record	Every parameter and version written	ε recomputable from it

43.15. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): the decisions of this chapter, with what to report.
- The Opacus documentation and tutorials (opacus.ai): the privacy engine, `BatchMemoryManager`, ghost clipping and the validator.

Derivations. This workflow draws on the full edition's chapters on DP-SGD in the full edition, hyperparameter tuning in the full edition, disparate impact in the full edition and choosing ε in the full edition.

44. Hyperparameter tuning and private selection

44.1. Why this matters

Stochastic gradient descent (SGD) with clipped gradients and Gaussian noise satisfies differential privacy (DP) at the level its accountant reports for one run. It is called DP-SGD. But nobody trains once. You try several learning rates, score each model and ship the best, and every run and every score touched private data. So the privacy level ε you report must pay for the search too.

Releasing only the best of a fixed number of runs can cost as much as releasing all of them. On the Adult census data, a grid of six runs that each meet $\varepsilon = 4$ really costs 9.84. If instead you draw the *number* of runs at random, from a distribution fixed in advance, the whole search costs little more than one run.

44.2. The procedure and the report

The cheapest tuning is the tuning you don't do on private data. Runs on public or proxy data cost nothing, and neither do published settings for similar tasks. Fix a large batch, the number of epochs and a small clipping norm in advance. With the clipping norm below the typical gradient norm, only its product with the learning rate matters, so often only that product needs a search (DP-SGD).

If you must tune on private data, fix the search before you touch the data. Write down the candidate set, the distribution of the number of runs (Section 44.4), the score and the validation set. Trying "a few more" later, or restarting a disappointing search, makes the search depend on the data, which no accountant covers. The score must be private as well (Section 44.5).

Then account for the whole search. For two or three candidates, charge for every run (Section 44.3), and for more, use random search with a random number of runs (Section 44.4). Either way, count every run that touched the private data, including crashed and debugging runs and evaluations for early stopping.

Finally, log the candidate set, the drawn number of runs and each run's settings and score. Then report two numbers. The ε of the released run says what the model alone would leak if its hyperparameters had been public. The ε of the whole procedure is what the release guarantees. An audit cannot check it, because an audit tests only the released model. With the book's default slack for 30,000 records, $\delta = 10^{-5}$ (Choosing ε), the Adult search of Section 44.6 reports as follows.

> DP-SGD ran for 300 steps with Poisson sampling at rate 1/30. It clipped gradients to norm 1 and added noise with multiplier 1.256, so one run satisfies $(2.73, 10^{-5})$-DP. The learning rate was chosen by random search over $\{0.05, 0.1, 0.2, 0.5, 1, 2\}$. The

number of runs was drawn from a Poisson distribution with mean 4, and each run was scored on 2,000 validation records treated as public. The whole procedure satisfies $(4.00, 10^{-5})$-DP for add/remove neighbours, datasets that differ by one record. It was computed with `dp_accounting` from the Rényi bound of Papernot & Steinke (2022).

44.3. Paying for every run

A common argument says that choosing among eight learning rates reveals at most three bits. It is wrong, because the release is a whole private run, picked because of how it came out. If each run says only "good" or "bad", the best is bad only when every run was bad. So a bad best reveals all of them, which in the worst case is as much as releasing every run. Even tuning without privacy and then training once privately leaks. The winning regularization weight can change when a few outliers are added, and so reveal whether they were in the data (Membership inference).

The safe answer is to charge for every run, because the best run is computed from all the runs' outputs (Composition). For DP-SGD, a grid of k runs, each of T steps, costs exactly as much as one run of kT steps. One run of the synthetic running example of DP-SGD costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$. Ten such runs cost 27.23, about four times one run rather than ten, because the conversion to ε pays its δ cost only once.

44.4. A random number of runs

The way out is to draw the number of runs K at random, independently of the data. A search whose runs all came out bad then most likely drew few runs, which says little about the data. So the whole search costs about one run, plus a term that grows with the logarithm of the expected number of runs. For DP-SGD, use the version of Papernot & Steinke (2022), which works with the Rényi differential privacy (RDP) that DP-SGD's accountants already compute (Rényi DP).

To use it, fix the candidate set and the distribution of K, either Poisson or a truncated negative binomial with a shape parameter ν. That family includes the geometric distribution, $\nu = 1$, which is the **random stopping** of Liu & Talwar, and the logarithmic one, $\nu = 0$. Then draw K, run that many candidates drawn at random from the set, and release the best one by a private score. To get the cost from Google's `dp_accounting`, wrap the event of one run in a `RepeatAndSelectDpEvent`. Give it ν as its `shape` (infinite for Poisson), and compose it in an `RdpAccountant`.

NumPy draws K with `rng.poisson(mean)`, `rng.geometric(1 / mean)` or `rng.logseries(p)`. The last takes p rather than a mean, so solve its mean $-p/((1-p)\ln(1-p))$ for p.

Table 44.1.: The ε at $\delta = 10^{-5}$ of a search over runs of the running example (one run: 6.71), from the full edition's chapter.

Expected runs	10	100	1,000
Composition	27.23	145.17	1035.93
Poisson	13.67	67.77	302.57

Expected runs	10	100	1,000
Geometric ($\nu = 1$)	11.31	14.32	16.76
Logarithmic ($\nu = 0$)	10.08	11.95	13.46

Table 44.1 shows how the choice plays out. The geometric and logarithmic searches barely grow from 10 to 1,000 runs, while composition grows almost linearly. A smaller ν gives a cheaper bound, but the search also stops after one run more often. The Poisson bound needs runs that are already very private, which a run at $\varepsilon = 6.71$ is not, so here it grows almost like composition. A random number of runs may also miss the best candidate. With eight candidates and a geometric number of runs with mean eight, it never tries the best one with probability 0.4667.

The theorems have three conditions, and each is easy to break. First, each run must draw its candidate independently from a fixed distribution, as random search does. Bayesian optimization picks each candidate from earlier scores, so it is not covered and needs adaptive composition. Second, the number of runs must be drawn independently of the data. Third, the score that ranks the runs must be private and counted in each run's cost.

44.5. Selecting by a noisy validation score

Sometimes the candidates are fixed in advance, such as public pretrained checkpoints, and only the choice among them uses private data. That is a **selection** problem, which Selection solves with the exponential mechanism, a random choice weighted toward high scores. Score each candidate by its number of correct predictions, which one record changes by at most 1. An accuracy divides by a private set size. The mechanism's shortfall from the best count then grows only with the logarithm of the number of candidates.

For the random-number bounds, add Gaussian noise to each run's count of correct predictions, since an exact validation accuracy leaves the validation records unprotected. With noise of standard deviation 20, one noisy count costs $\varepsilon = 0.182$. If the validation set is disjoint from the training set, each record pays only one of the two costs. So the event for one run is the costlier of its training and its count, not their sum. Otherwise compose both into one event. On the running example the counts are the cheaper, so they add nothing to the total.

44.6. Tuning on Adult

The script practitioner_tuning.py tunes the learning rate of a small network on 30,000 Adult census records, with 300 steps per run. It asks the accountant for the noise multiplier per run that each strategy can afford.

Table 44.2.: Tuning the learning rate on Adult at a total of $\varepsilon = 4$, $\delta = 10^{-5}$, with `dp_accounting`'s RDP accountant.

Strategy	Noise multiplier per run	Total ε
One run, no tuning	1.029	4.00

Strategy	Noise multiplier per run	Total ε
Grid of 6 at the noise of one run, one run's ε reported	1.029	9.84 (reported as 4.00)
Grid of 6, every run counted	1.816	4.00
Random search, Poisson number of runs with mean 4	1.256	4.00

The second row is the reporting bug of Section 44.3. This snippet recomputes it, together with the numbers of the report in Section 44.2.

```
import dp_accounting


def one_run(sigma: float) -> dp_accounting.DpEvent:
    step = dp_accounting.PoissonSampledDpEvent(1 / 30, dp_accounting.GaussianDpEvent(sigma))
    return dp_accounting.SelfComposedDpEvent(step, 300)


def epsilon(event: dp_accounting.DpEvent) -> float:
    return dp_accounting.rdp.RdpAccountant().compose(event).get_epsilon(1e-5)


search = dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(1.256), mean=4, shape=float("
naive_grid = dp_accounting.SelfComposedDpEvent(one_run(1.029), 6)
print(f"{epsilon(one_run(1.256)):.2f}")  # 2.73
print(f"{epsilon(search):.2f}")  # 4.00
print(f"{epsilon(naive_grid):.2f}")  # 9.84
```

With seed 0, the search drew only two runs, both at the smallest learning rate, and its model reaches 0.828 on the test set. The run of Train a model with DP-SGD, tuned for free on public stand-in rows, reaches 0.846, so a search with few runs can miss.

44.7. Cheat sheet

Table 44.3.: Which tuning strategy for which search.

Situation	Strategy	Cost
Two or three candidates	Fixed grid, composition	k runs of T steps = one run of kT steps

Situation	Strategy	Cost
Many candidates	Random search; compute Poisson, geometric and logarithmic for your per-run noise and mean, and take the cheapest	Poisson wins only for very private runs; at one run of 6.71, logarithmic is cheapest at every size in Table 44.1, but it stops after one run more often
Fixed candidates	Exponential mechanism on correct counts	Shortfall grows with the log of the candidate count
Trained candidates, disjoint validation set	Exponential mechanism or noisy counts	The larger of training and selection costs

Rules of thumb

- Tune on public or proxy data when you can. Otherwise fix the search before touching the private data, and never extend it.
- Report the whole search's ε, never the selected run's alone.

44.8. References

- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022): the Poisson and truncated negative binomial bounds for DP-SGD.
- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019): random stopping and its bound of three times one run.

Derivations. Every result here is derived step by step in the full edition's chapter.

45. User-level differential privacy

45.1. Why this matters

Every differential privacy (DP) guarantee in this part so far protects one record. Stochastic gradient descent (SGD) with clipping and noise, this part's DP-SGD, gives each record of its running example a privacy loss ε of 6.71. But a messaging user contributes hundreds of messages, and together a person's records can reveal far more than any one of them.

A DP guarantee protects whatever differs between two neighbouring datasets, so **the neighbouring relation sets the privacy unit**. So this chapter gives the running example's 10,000 synthetic records to 1,000 users, very unevenly. The median user holds 4 records and the largest 348.

45.2. The privacy unit and δ

Take a mechanism M, and call two datasets neighbours, D and D', when they differ by adding or removing all the records of one user. The mechanism is **user-level** (ε, δ)-DP, with a small slack δ, if every such pair and every set S of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{45.1}$$

Example-level DP is the same inequality for datasets that differ in one record. So every property and accountant of Part 4 holds for user-level DP unchanged. What changes is the sensitivity, because one user can move the computation with all of their records at once.

The right unit, such as a record, a user, a household or a device, answers the question "whose participation do I want to hide?". Report it with every ε (Report and review an ε claim). The unit also sets the slack, because the book's default δ (Choosing ε) now counts users. For 1,000 users it gives 10^{-4} (Section 45.8), so this chapter's 10^{-5} is stricter than it needs to be.

The textbook answer for a person with k records is **group privacy** (Post-processing, group privacy and composition). An example-level (ε, δ)-DP mechanism is also user-level DP for users with at most k records, at level $k\varepsilon$ and with slack

$$\delta_k = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{45.2}$$

The fraction grows like $e^{(k-1)\varepsilon}$, so a record-level δ must never be read as a user's. Holding a user's slack at 10^{-5} needs a much smaller per-record δ, and so a larger per-record ε. On the running example a user with two records therefore gets $\varepsilon = 18.61$, not twice 6.71. For ten records no per-record δ is small enough.

45.3. What an example-level model gives a user

Group privacy treats DP-SGD as a black box, but you know how a user's records enter each step.

Each step includes every record independently with the sampling rate q. So it sees some record of a user with k records with probability about kq. While kq is small, two of their records rarely share a batch. So **a user with k records looks like one record sampled at rate kq**. So you can screen a user's exposure by rerunning the record-level accountant at rate kq (Section 45.8). An exact accountant for a user with k records also exists (Charles et al. 2024).

Table 45.1.: The user-level ε at $\delta = 10^{-5}$ of the running example's example-level DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps) for a user with k records, from the full edition's Rényi accountant (6.72 at one record comes from its coarser grid of orders).

Records of the user k	1	2	5	10	50
Exact user accountant	6.72	15.54	52.36	146.20	2161.43
One record at rate kq	6.71	15.32	50.11	134.50	1673.08

Ten records already cost 146.20, no protection to speak of, yet the exact accountant beats group privacy for every user with more than one record. So never use group privacy for DP-SGD.

The rate-kq screen stays within 8% of the exact value up to ten records, but always a little low. It ignores steps that draw several of the user's records together. So report the exact value when you have an implementation. The libraries this book uses offer only the screen. Then report its value, say it can be a few percent low, and keep kq small.

Treat the screen as the guarantee, not as a pessimistic bound, because a person's similar records do push the gradient partly the same way.

45.4. User-level DP-SGD

The better route makes the user the unit that DP-SGD samples, clips and accounts for (McMahan et al. 2018). Federated learning does this with a device as the user. Starting from weights chosen without looking at the data, each step does five things.

1. **Sample users.** Put each user in the batch independently with probability q, now the user sampling rate.
2. **Compute one vector per user.** From that user's records alone, compute their average gradient, or the change in weights after a few training steps on their records.
3. **Clip per user.** Rescale that vector to length at most the clipping norm C.
4. **Add noise.** Sum the clipped vectors and add Gaussian noise of standard deviation σC, for a noise multiplier σ.
5. **Step.** Divide by the expected number of sampled users and take a gradient step.

One user moves the sum by at most C, however many records they have. So the guarantee is user-level DP with exactly the accountant of example-level DP-SGD, run at the user sampling rate.

That has two consequences. First, **you pay for having fewer units**. On a shorter schedule of the running example, 1,000 steps with about 100 units per batch, sampling records needs $\sigma = 0.778$ for $\varepsilon = 4$. Sampling the ten times fewer users needs a ten times higher rate for the same batch, and so $\sigma = 3.78$. Second, **records inside a user are free**, since they can be averaged, used for several training steps or subsampled without changing ε.

Carry a user identifier through the pipeline to sample by. Split training and test data by user, so that no user's records sit on both sides. Opacus clips each row of a batch, so for user-level training you write a data loader that yields users and clip each user's vector yourself. Three things break the guarantee:

1. A user's vector depends on other users' data, for example through normalization statistics computed over all users.
2. One person is spread over several accounts, so the unit is the account and the person is again a group.
3. Users are iterated in a fixed shuffled order instead of Poisson-sampled, which is a different mechanism that this accountant does not cover (Matrix factorization).

45.5. Contribution bounding

A pipeline built around records can still protect users. **Contribution bounding** keeps at most k records of each user, the cap, and runs example-level DP-SGD on what remains. The result is user-level DP with the ε the user accountant gives for k records. The condition is that each user's kept records are chosen without looking at other users' data, for example at random. Otherwise adding one user could change which records other users keep, and neighbours would differ by more than one user's records. To calibrate the noise for a cap, solve for σ with `dp_accounting.calibrate_dp_mechanism` at rate kq (Train a model with DP-SGD).

```
import polars as pl

rows = pl.DataFrame({"user": ["a", "a", "a", "a", "b", "c", "c"], "x": [1, 2, 3, 4, 5, 6, 7]
k = 2  # the cap, fixed from public knowledge of the population
capped = rows.sample(fraction=1.0, shuffle=True, seed=0).group_by("user").head(k)
print(capped.height)  # 5
print(capped.group_by("user").len()["len"].max())  # 2
```

Choosing the cap trades noise against bias (Amin et al. 2019). A larger cap keeps more data but needs more noise, while a smaller cap discards records and reweights the data toward light users. On the running example at a user-level $\varepsilon = 4$, a cap of fifty records instead of one multiplies the noise by 5.8. Accuracy falls from 0.8917 to 0.6401.

So take the cap from public knowledge of the population. Or choose it privately and count the cost, for example with a private quantile of records per user (Beyond global sensitivity). Trying caps on test accuracy spends privacy the accountant does not see.

45.6. Heterogeneous users

User-level training also changes the objective. Averaging per user weights users equally, while example-level training weights them by their number of records (Disparate impact). If heavy users should count for more, sum a user's gradients before clipping instead of averaging them. Their influence then rises with their records until clipping caps the sum at C, and ε does not change.

45.7. Which strategy

Table 45.2 compares three routes on the running example, each with 1,000 steps and about 100 units per batch. Strategy (a) runs example-level DP-SGD on all records and accounts for the heaviest user.

Table 45.2.: Test accuracy at the same user-level budget, $\delta = 10^{-5}$, one run each. The non-private model reaches 0.9612.

User-level ε	(a) All records, $k = 348$	(b) User-level DP-SGD	(c) At most 2 records per user
1	0.5318	0.8065	0.7084
2	0.5582	0.9028	0.8374
4	0.5713	0.9347	0.8810
8	0.5478	0.9442	0.9333

Keeping the record as the unit fails. Strategy (a) stays near chance on this two-class task, because the heaviest user forces a noise multiplier between 70.3 and 449 on everyone. User-level DP-SGD is the best at every budget, and its cost is compute, since each step processes all the records of about 100 users. The cap is the fallback when the pipeline must stay record-based. How far it trails depends on how records are spread over users, so measure it on your data.

45.8. Code

The rate-kq screen needs only a record-level accountant, such as Google's `dp_accounting`. This snippet screens users with one and two records, and computes the default δ for 1,000 users.

```
import math

import dp_accounting


def screen(k: int, q: float = 0.01, sigma: float = 1.0, steps: int = 10_000) -> float:
    """The epsilon of a user with k records, screened as one record sampled at rate k * q."""
    step = dp_accounting.PoissonSampledDpEvent(k * q, dp_accounting.GaussianDpEvent(sigma))
    accountant = dp_accounting.rdp.RdpAccountant()
    accountant.compose(dp_accounting.SelfComposedDpEvent(step, steps))
```

```
    return accountant.get_epsilon(1e-5)


print(f"{screen(1):.2f}")  # 6.71
print(f"{screen(2):.2f}")  # 15.35
print(10 ** math.floor(math.log10(1_000**-1.1)))  # 0.0001
```

Its 15.35 is a little above the table's 15.32, because the two accountants use different Rényi orders. So quote the value you ran and name its accountant. With $k = 1$ and q the user sampling rate, the same code accounts for user-level DP-SGD.

45.9. Cheat sheet

Table 45.3.: Which route to a user-level guarantee.

Situation	Use	What you get
Example-level model already trained	User accountant, or the rate-kq screen	A user's ε grows about linearly in k, then faster for large kq
Records can be grouped by user	User-level DP-SGD	DP-SGD's accountant over users
Pipeline must stay record-based	Contribution bounding with a public cap	User accountant for k records; loses data
Heavy users should count more	Sum before clipping instead of averaging	Same ε, different objective

Rules of thumb

- Name the privacy unit with every ε, and set δ from the number of users.
- Never use group privacy for DP-SGD; screen a user at rate kq instead.

45.10. References

- Charles et al., "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024): the tight user accountant, and sampling records against sampling users.
- Chua et al., "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024): contribution bounding against user-level DP-SGD.

Derivations. Every result here is derived step by step in the full edition's chapter.

46. Choosing ε

46.1. Why this matters

Every model trained with differential privacy (DP) ships with a privacy budget ε. The chapter on private stochastic gradient descent (SGD), DP-SGD, showed what each budget costs in accuracy. This chapter adds what it means for the people in the data, without which ε gets chosen by habit ("everyone uses 8").

An attacker who wants to know whether a record was used for training runs a membership test. The test catches real members, its true positives, and wrongly flags non-members, its false positives. The budget ε and its small slack δ cap the first rate in terms of the second (The definition of differential privacy). But each training run also has its own curve of the best rates an attacker can reach, which usually lies far below the cap. To choose ε well, compute that curve, read it at the false positive rate of your threat, and set it beside the accuracy each budget buys.

46.2. Justifying a choice

A defensible choice says what the budget protects before it picks the number, in six steps.

1. **Unit and period.** Say whether one unit is a record, a user or a device, and whether neighbouring datasets differ by adding or removing one unit or by replacing one. Say which runs the budget covers, since every tuning run spends budget too (Hyperparameter tuning).
2. **Threat.** Say whether the concern is membership, reconstruction or attribute inference, and what the attacker knows and sees. This sets the false positive rate at which you read the curve, a low one for an attacker who must rarely accuse a non-member.
3. **Whole guarantee.** Choose δ by the book's default (Section 46.5), then compute the trade-off curve (Section 46.3) and read it there.
4. **Evidence.** The curve is a theorem about an algorithm, not about your code, so run an audit that could refute it (Auditing private training).
5. **Utility.** Put accuracy against ε next to the risk, and pick the smallest budget past the bend, where accuracy stops improving. Draw it on public or proxy data, because private runs that choose ε are a search whose cost the reported ε leaves out.
6. **Comparison.** Only now compare with deployments (Section 46.6) and with the tiers in the rules of thumb.

Adult, a public census survey, shows step 5 on a real run. Train a model with DP-SGD trains a small network on 30,000 of its people at five budgets, each with $\delta = 10^{-5}$. Table 46.1 sets their test accuracies beside a measure of risk from Section 46.3. It is the largest true positive rate (TPR) that any membership test can reach at a false positive rate (FPR) of 10^{-3}.

Table 46.1.: Accuracy and worst-case membership risk on Adult, one seed.

ε	Noise multiplier	Test accuracy	Largest TPR at FPR 10^{-3}
0.25	8.125	0.793	0.0013
1	2.383	0.843	0.0023
2	1.431	0.845	0.0046
4	0.973	0.846	0.0123
8	0.722	0.846	0.0445
Non-private	—	0.847	—

Accuracy rises by five points from $\varepsilon = 0.25$ to 1, then stays within a few thousandths of the non-private model, which is noise on one seed. The risk keeps growing, faster with each doubling of the budget, from 0.0023 at $\varepsilon = 1$ to 0.0445 at $\varepsilon = 8$. So on Adult the bend sits at $\varepsilon = 1$, and a budget of 1 or 2 is the defensible choice. These runs were scored on private rows, so the table only illustrates step 5, and a real choice reads it from a public stand-in. On a harder task, a larger model or a smaller dataset, expect the bend further right.

46.3. The pair and the whole curve

A membership test says "member" or "non-member" about one target record. Its TPR is the chance that it says "member" when the target was in the training set, and its FPR the chance when it was not. Every test against an (ε, δ)-DP mechanism satisfies

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{46.1}$$

The middle term is the one that binds at low FPR, and the last applies the same cap to the members a test misses. Attacks are judged at a low FPR, because an attack that rarely flags a non-member identifies people with confidence (Membership inference). Yet at $\varepsilon = 8$ and an FPR of 10^{-3}, the pair caps the TPR at 0.9997, which constrains almost nothing.

A real training run satisfies DP for a whole range of pairs, one $\delta(\varepsilon)$ for each ε, because its accountant can trade ε for δ. Each pair gives a line, the middle term of Equation 46.1, and every test must stay under all of these lines at once. The best test reaches the lowest line exactly, which is called the **trade-off curve**:

$$\mathrm{TPR}^{*}(\mathrm{FPR}) = \min_{\varepsilon}\left(e^{\varepsilon}\,\mathrm{FPR} + \delta(\varepsilon)\right). \tag{46.2}$$

For DP-SGD, a privacy loss distribution (PLD) accountant computes this curve. It can round the privacy loss up or down, and the two curves it gets bracket the true one (Numerical accounting).

Table 46.2 compares the pair with the curve for this part's running example from DP-SGD, a synthetic run sampled at rate 0.01 for 10,000 steps.

Table 46.2.: Largest TPR of a membership test against the running example at $\delta = 10^{-5}$, from the pair and from the curve.

ε	Noise multiplier	Pair, FPR 10^{-3}	Curve, FPR 10^{-3}	Curve, FPR 10^{-2}
4	1.36	0.05461	[0.01215, 0.01312]	[0.06739, 0.0724]
8	0.917	0.9997	[0.05555, 0.0589]	[0.1987, 0.2092]

So a budget of 8 means little as a bare pair, while its curve caps the TPR at 0.0589 at an FPR of 10^{-3}.

With Google's `dp_accounting`, build the run's PLD, ask it for δ on a grid of ε, and take the lowest line. The snippet's first print, for the running example at $\varepsilon = 4$, falls inside the bracket of Table 46.2. Its second uses Adult's sampling rate of 1/30 and 300 steps, and gives the last column of Table 46.1.

```
import numpy as np
from dp_accounting.pld import privacy_loss_distribution as pld

EPS_GRID = np.linspace(0.0, 20.0, 2001)


def tpr_at(sigma: float, q: float, steps: int, fpr: float) -> float:
    run = pld.from_gaussian_mechanism(sigma, sampling_prob=q).self_compose(steps)
    deltas = np.array(run.get_delta_for_epsilon(EPS_GRID))
    return float(np.min(np.exp(EPS_GRID) * fpr + deltas))


print(f"{tpr_at(1.36, 0.01, 10_000, 1e-3):.4f}")  # 0.0126
adult = [tpr_at(sigma, 1 / 30, 300, 1e-3) for sigma in (8.125, 2.383, 1.431, 0.973, 0.722)]
print(" ".join(f"{tpr:.4f}" for tpr in adult))  # 0.0013 0.0023 0.0046 0.0123 0.0445
```

The library's default PLD rounds the loss up, so its curve is a valid upper bound. A Rényi differential privacy (RDP) accountant gives a valid but looser curve. The Gaussian differential privacy (GDP) approximation is quick but not a bound, so use it to explain and the PLD to certify.

The curve is also a worst case, for an attacker who sees every intermediate model. A typical record attacked through the final model alone is much safer. It bounds reconstruction too, since a guess among m equally likely records is a membership test at an FPR of $1/m$ (Reconstruction attacks).

46.4. What an attacker can believe

Most people ask how sure an attacker can become that a given person was in the data. Write p for the attacker's **prior**, its probability of "member" before it sees the model. A privacy loss of at

most ε means that no output is more than e^{ε} times as likely with the person's record as without it. So the attacker's belief afterwards is at most

$$\Pr[\text{member} \mid \text{output}] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{46.3}$$

So DP bounds the change of belief, not the belief itself.

An attacker who starts at 1% can end at 35.55% when $\varepsilon = 4$. That is the worst output, though, and in DP-SGD the privacy loss of the model actually released is usually far below ε.

Base rates matter as much as the loss. An attacker who accuses every person its test flags as a member is right with probability

$$\Pr[\text{member} \mid \text{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}. \tag{46.4}$$

Take the running example's $\varepsilon = 8$ run and an attacker with a 1% prior who flags at an FPR of 10^{-2}. The curve caps its TPR there at 0.2092 (Table 46.2), so at most 0.1744 of its flags are right, and five out of six are false. An insider with a prior of 0.5 would be right far more often, so a statement about beliefs must name the attacker.

46.5. How small δ must be

A deliberately bad mechanism shows what δ permits. It publishes each of n records in full, independently with probability δ. It is $(0, \delta)$-DP, because adding or removing one record changes the output only when that record's own coin comes up. Yet it publishes at least one record with probability

$$1 - (1-\delta)^n \approx 1 - e^{-n\delta}. \tag{46.5}$$

For 10,000 records at $\delta = 1/n$, that is 0.6321, so $n\delta$ must be below 1. Here n counts privacy units, which means people when one person has many records (User-level differential privacy).

This book's default is $\delta \le 1/n^{1.1}$, rounded down to a power of ten, which keeps $n\delta$ below 1 for every n. It gives 10^{-5} for both of this page's runs, and the `delta_for` function of Train a model with DP-SGD computes it. When n is just above a power of ten, as with Adult's 30,000 people, the default is not far below $1/n$. NIST SP 800-226, the US guidelines for evaluating DP claims, calls a δ above 10^{-5} suspicious unless justified.

For DP-SGD a smaller δ is cheap. The RDP accountant that set the running example's $\varepsilon = 4$ run puts it at only 5.209 when δ falls to 10^{-8}. So report ε at a second, smaller δ too, and never quote a δ as large as one over the number of units.

46.6. Comparing with deployed values

"Industry uses ε between 1 and 10" hides what every ε needs, its **privacy unit** and the **period** over which it is spent. The Census Bureau's 2020 redistricting data used zero-concentrated differential privacy (zCDP), which states the guarantee as one number, written ρ, that converts

to (ε, δ) pairs. Its unit was one person's record under the replace-one relation, and persons got $\rho = 2.56$, reported as $\varepsilon = 17.14$ at $\delta = 10^{-10}$.

So before comparing, convert both budgets to the same unit, neighbouring relation and δ. A zCDP parameter ρ converts to a pair by

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{46.6}$$

For the Census figures this formula gives 17.92, and the Bureau's 17.14 comes from a tighter conversion. Since a replacement is a removal followed by an addition, an add/remove guarantee (ε, δ) implies only the replace-one guarantee $(2\varepsilon, (1 + e^{\varepsilon})\delta)$ (The definition of differential privacy). And a budget stated per day adds up over time by composition, to 365 a year at $\varepsilon = 1$ per day.

So it would be wrong to call the running example's $\varepsilon = 8$ run "less than half the Census budget". At the Census's $\delta = 10^{-10}$ it is at 11.627, and under the Census's replace-one relation that can double.

46.7. Cheat sheet

Table 46.3.: Which tool answers which question when choosing ε.

Question	Tool	Notes
What does DP-SGD allow?	The trade-off curve, Equation 46.2	Certify with the PLD, not with GDP
How sure can an attacker become?	Equation 46.3 and Equation 46.4	Name the prior; base rates dominate
How small a δ?	$\delta \leq 1/n^{1.1}$, rounded down to a power of ten	n counts privacy units
Comparing with a deployment?	Equation 46.6; replace-one doubles ε	Same unit and δ

Rules of thumb

- A budget of at most 1 is strong for any mechanism. A budget between 1 and 10 needs DP-SGD's curve as justification, and a budget above 10 needs the curve and an audit (Ponomareva et al. 2023).
- Never quote ε without its unit, δ and period.

46.8. References

- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): privacy units, δ, and the hazards of large budgets.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): tiers and reporting.

Derivations. Every result here is derived step by step in the full edition's chapter.

47. Public data and private fine-tuning

47.1. Why this matters

The differential privacy (DP) version of stochastic gradient descent (SGD), DP-SGD, loses much accuracy when it trains a model from random weights. The loss is large at a privacy loss ε of 1 or below. The most accurate private models avoid it by starting from a model **pretrained** without privacy on a large public dataset. They only **fine-tune** it privately on the sensitive data (Tramèr & Boneh 2021).

Fine-tuning only a small part of the model often helps further. DP-SGD adds noise to every weight it trains, while the useful signal does not grow with their number. So fewer weights mean less noise for the same signal. The workflow Fine-tune a language model privately with LoRA does the job end to end.

47.2. How many weights to train

Which weights you train leaves ε unchanged, because DP-SGD's guarantee needs only that one record moves the clipped sum by at most the clipping norm C. The choice changes only how much noise lands on the model. Anything else computed only from public data, such as the starting weights, is also free, if no choice in it looked at the private data. So a tokenizer or normalization statistics fitted on public data are free, while fitted on the private data they leak.

The noise grows with the number d of trainable weights. Write σ for the noise multiplier and B for the expected batch size, expected because DP-SGD's Poisson sampling draws batches of random size. Then the noise on the averaged gradient has length about

$$\text{noise length} \approx \frac{\sigma C \sqrt{d}}{B}. \tag{47.1}$$

The signal, the average clipped gradient, is at most C long whatever d is. It can even get longer when fewer weights train, because clipping then cuts each record's gradient less.

Take the running example of DP-SGD, a small network with 3,329 weights. It is pretrained on a similar public task and then fine-tuned on the 10,000 private records. At $\varepsilon = 1$, the noise is 92.0 times longer than the signal at the pretrained weights when all weights train. It is only 6.7 times longer when the 65 weights of the last layer train. That gap is wider than the formula predicts, because the signal is longer when only the last layer trains (0.0250 against 0.0130).

Counting weights overstates the cost of training them all, though. Noise does harm only along the directions in which the loss is curved, because a step along a flat direction barely changes the loss. Near a pretrained model few directions are curved (Li, Liu et al. 2022). So on the running example, full fine-tuning trains 51 times as many weights as the last layer, but its noise should

raise the loss only 7.5 times as much. A larger learning rate lets the noise into more of the nearly flat directions, so private fine-tuning wants smaller steps than training from scratch.

Training fewer weights has its own cost, a **bias**, because no private data can move the model along a frozen direction that the task needs.

47.3. Linear probing first

The simplest option freezes everything but the last layer and trains a linear classifier on the frozen features, which is **linear probing**. Because the features are fixed, the probe's loss is convex. So linear probing turns private deep learning back into Private empirical risk minimization, which makes it the safe first baseline.

A private probe's error has two parts. One is the error that privacy adds to convex empirical risk minimization (ERM), which grows with the number d of features, and the other is the **feature gap**. That gap is how far the best linear model on the features falls short of the best model of any kind. No number of records or size of ε closes it. For example, the running example's probe without privacy, its ceiling, has a test accuracy of 0.9536, while a non-private network trained from scratch reaches 0.9606. The private probe stays within 0.006 of that ceiling, but never passes it.

So measure the probe's ceiling first, by fitting it without privacy on labelled public data that resemble the task. If it already meets your target, stop there. To train the probe privately, feed the frozen features to diffprivlib's `LogisticRegression` (Private empirical risk minimization), or run DP-SGD on the one layer.

47.4. Low-rank adapters and full fine-tuning

When the probe's ceiling is below your target, only changing the earlier layers can lift it. **Low-rank adaptation** (LoRA, Hu et al. 2022) freezes each pretrained matrix and learns a small correction to it. The correction is the product of two thin matrices that each have r columns, where r is the rank. A matrix with d_{in} inputs and d_{out} outputs has $d_{\text{in}}d_{\text{out}}$ weights, while its adapter has only

$$d = r(d_{\text{in}} + d_{\text{out}}). \tag{47.2}$$

The adapter comes from Hugging Face's `peft` library. This snippet puts a rank-4 adapter on the first layer of a network shaped like the running example, and trains its last layer in full.

```
import torch
from peft import LoraConfig, get_peft_model

model = torch.nn.Sequential(torch.nn.Linear(50, 64), torch.nn.Tanh(), torch.nn.Linear(64, 1)
print(sum(p.numel() for p in model.parameters()))  # 3329
config = LoraConfig(r=4, lora_alpha=8, target_modules=["0"], modules_to_save=["2"], lora_dro
adapted = get_peft_model(model, config)  # freezes every pretrained weight
print(sum(p.numel() for p in adapted.parameters() if p.requires_grad))  # 521
```

In the snippet, `target_modules` and `modules_to_save` name layers by their index in the `Sequential`, and `lora_alpha` scales the correction. It trains 521 weights, the adapter's $4 \times (50 + 64) = 456$ and the last layer's 65. The run in Section 47.5 treats the first layer's biases as a 51st input, which gives 525. Opacus's `PrivacyEngine` then computes per-example gradients only for the parameters that require them, so the frozen weights cost nothing. Call `model.train()` before `make_private`, since Opacus rejects a model left in evaluation mode, as a pretrained model often is after you measure it.

The adapter's second factor starts at zero, so training begins exactly at the pretrained model. The first factor starts random, and that draw must not depend on the private data.

Yu et al. (2022) showed that private fine-tuning of language models with LoRA matches or beats private full fine-tuning, at a fraction of the compute and memory. And since each adapter factor is an ordinary linear layer, the ghost clipping of DP-SGD applies to it. Still, full fine-tuning is not ruled out. Li, Tramèr et al. (2022) matched the adapters with it, using large batches and a learning rate tuned for private training.

47.5. Which method at which budget

The numbers in this section come from the full edition's synthetic running example. No practitioner script reproduces them, so treat them as one task's evidence. DP-SGD fine-tunes the last layer, a rank-4 adapter plus the last layer, or all weights, against a model trained privately from scratch. Each method's learning rate was chosen at $\varepsilon = 1$ on half of the private test set, and accuracy comes from the other half. That choice looked at private data without paying, so the table shows each method at its best, not what a real run could claim (Hyperparameter tuning).

Table 47.1.: Test accuracy (mean of two runs), $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights d	3,329	3,329	65	525
Learning rate	0.25	0.1	0.3	0.1
$\varepsilon = 0.5$	0.8589	0.9395	0.9478	0.9464
$\varepsilon = 1$	0.9246	0.9599	0.9497	0.9616
$\varepsilon = 2$	0.9518	0.9663	0.9519	0.9653
$\varepsilon = 8$	0.9611	0.9677	0.9514	0.9655

Here pretraining is worth about a factor of eight in ε, since the adapter at $\varepsilon = 1$ matches training from scratch at $\varepsilon = 8$.

At $\varepsilon = 0.5$ the last layer is best, because its noise is 7.2 times shorter than full fine-tuning's, and its ceiling does not yet bind. From $\varepsilon = 1$ on, it is stuck near its ceiling while the others keep improving. The adapter is never more than 0.0022 below the best method, so it is the robust middle choice here. That ranking depends on the task, and a public task further from the private one can reverse it even when the probe's ceiling looks fine. So rerun the comparison on a public stand-in for your task before you commit.

The learning rate matters as much as the method, since a step three times too large costs full fine-tuning 1.3 percentage points of accuracy. The adapter's rank matters little, since at $\varepsilon = 1$ ranks 1, 4 and 16 differ by about the run-to-run spread.

47.6. What public data does not buy

Whatever the public data are, training on the private data is (ε, δ)-DP, where δ is the small additive slack the guarantee allows. The public data get no guarantee of any kind. Non-private pretraining can memorize individual records (Memorization and training-data extraction), and the fine-tuned model inherits whatever the pretrained weights encode (Tramèr, Kamath & Carlini 2024).

So a person in both datasets is protected only in the private copy. A membership attack against the pretrained model (Membership inference) has no bound at all. Web-scale "public" datasets contain posts, profiles and leaked documents, so private notes or emails may well overlap with them. For text, also mind the privacy unit. One person's documents span many training sequences, so a guarantee per sequence protects far less than a person (User-level differential privacy).

Check three things before you train and when you report.

1. Check that the public data resemble the private task, and deduplicate the private set against them. Deduplication protects copies, not people, since exact matching misses paraphrases.
2. Choose the public model and the hyperparameters on public data. Otherwise every private run costs privacy (Hyperparameter tuning).
3. Name the public data and the overlap check, and state that the guarantee covers the private data given the pretrained model.

47.7. Cheat sheet

Table 47.2.: Where to start when fine-tuning privately; the ranking depends on the task.

Situation	Train	Why
Tiny budget, or the probe meets the target	Last layer (linear probe)	Fewest weights; convex
Moderate budget, features not quite right	Low-rank adapter plus last layer	Never far from the best
Larger budget, large batches, tuned learning rate	All weights	Noise costs only along curved directions

Rule of thumb. Measure the linear probe's ceiling before anything else. Then use large batches and a smaller learning rate than for training from scratch. Tune the learning rate with DP-SGD runs at the target noise on public data. A non-private sweep does not show how the noise moves the best step.

47.8. References

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021): linear probing on good features.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022), and Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): full and adapter-based private fine-tuning.

Derivations. Every result here is derived step by step in the full edition's chapter.

48. Fine-tune a language model privately with LoRA

48.1. Why this matters

You have a pretrained language model and private text, such as support tickets or clinical notes. You want the model to learn from that text without being able to repeat it. Yet a model fine-tuned without privacy can write out a phone number or a password that was often in its training text (Memorization and training-data extraction). The fix is to fine-tune with the differential privacy (DP) version of stochastic gradient descent (SGD), called DP-SGD. It costs least when it trains only a small adapter on the frozen model, which is often as accurate as training every weight (Public data and private fine-tuning). Here the adapter is low-rank adaptation (LoRA), from Hugging Face's `peft` library, and Opacus does the private training.

The model is Pythia-14m, a public language model of 14 million parameters from EleutherAI. The private text is Tiny Shakespeare, a million characters of Shakespeare's plays. The script fine-tunes the model twice, once without privacy and once at a privacy loss ε of 8 for each sequence of training text. It first plants made-up secrets, called canaries, in the text, so that you can see what each model memorized. The script practitioner_lora.py runs in a few minutes without a GPU.

```
uv run python code/part05-dp-ml/practitioner_lora.py
```

48.2. Fix the privacy unit, n and δ

The guarantee protects one unit, so decide first what the unit is. A language model trains on fixed-length sequences of tokens, the word pieces its tokenizer cuts text into. The script cuts the text into sequences of 32 tokens, about one or two lines of verse, and makes each sequence the unit. To keep the run short, it uses only the first 1,024 sequences for training and the last 128 as a held-out set.

```
sequences of 32 tokens: 10,632  training n=1,024  held out 128
delta=1e-04 (privacy unit: one training sequence)
```

The slack δ is the small additive term the guarantee allows beside ε. The book's default for n units is $\delta \leq 1/n^{1.1}$, rounded down to a power of ten (Choosing ε). For 1,024 sequences that is 10^{-4}.

A sequence is the right unit for this demonstration, but rarely in production. One person's document spans many sequences, and one user writes many documents, so a guarantee per sequence protects a person much less than it seems. In production, make the unit a document or a user, and bound how many sequences each contributes. Then set δ from the number of users (User-level differential privacy).

48.3. Plant canaries

To check memorization later, you need secrets whose presence you control. The script writes **canaries** into the training text. Each is a sentence such as "the pin of Alice is 8 6 5 2 3", whose last five digits are the secret. There are four groups of three canaries, inserted 1, 4, 16 and 64 times each. Three more, the controls, are drawn the same way but never inserted, so they show what an unseen secret scores in the audit of Section 48.9.

```
def insert_canaries(
    train: torch.Tensor, canaries: list[Canary], tokenizer: PreTrainedTokenizerBase, rng: np
) -> torch.Tensor:
    """Write each copy of each canary over the start of a different training sequence."""
    train = train.clone()
    total = sum(c.copies for c in canaries)
    rows = iter(rng.choice(len(train), size=total, replace=False).tolist())
    for canary in canaries:
        tokens = torch.tensor(tokenizer(canary.text)["input_ids"])
        for _ in range(canary.copies):
            train[next(rows), : len(tokens)] = tokens
    return train
```

Each copy overwrites the start of a different training sequence, so the 255 copies take up a quarter of the training set. That is far denser than secrets in real data, and deliberately so, because it lets a model this small memorize within a short run. The audit then has a positive control, a model known to leak.

```
canary sequences: 255 (3 canaries each at (1, 4, 16, 64) copies)
tokens per secret: [5]
candidates: 2,000  exposure cap 10.97 bits  (full space 16.61)
```

The digits are separated by spaces so that every possible secret is exactly five tokens long, which the second line confirms over 1,000 random secrets.

48.4. Load a public model and tokenizer

Everything not trained privately must come from outside the private data. That includes the tokenizer, whose vocabulary would leak rare words if it were built from the private text. So the weights and the tokenizer both come from the same public checkpoint, Pythia-14m, pinned to one revision. The script loads the model in float32, because `transformers` loads some checkpoints in float16, which is far slower on a CPU.

The guarantee then covers the private text given the pretrained model, and says nothing about the model's own training data. Here that caveat is real, because Pythia was pretrained on the Pile, which includes Project Gutenberg's books and so very likely Shakespeare's plays. That is harmless here, but with real data, check for overlap with the public model's training data (Public data and private fine-tuning).

48.5. Choose what to train

DP-SGD adds noise to every weight it trains. So the noise on the averaged gradient grows with the square root of the number of trained weights, but the clipped signal does not (Public data and private fine-tuning). That makes the choice of trainable weights one of the two main levers on accuracy, with the learning rate. LoRA freezes each pretrained matrix and learns a correction to it, the product of two thin factors whose width is the rank.

```
attention_and_mlp = ["query_key_value", "dense", "dense_h_to_4h", "dense_4h_to_h"]  # every
config = LoraConfig(r=rank, lora_alpha=2 * rank, target_modules=attention_and_mlp, lora_drop
lora_model = get_peft_model(model, config)  # freezes the pretrained weights
trainable_count = sum(p.numel() for p in lora_model.parameters() if p.requires_grad)
total_count = sum(p.numel() for p in lora_model.parameters())
print(f"trainable parameters: LoRA {trainable_count:,}  full fine-tuning {total_count - trai
```

```
full model parameters: 14,067,712
trainable parameters: LoRA 98,304  full fine-tuning 14,067,712
```

By default the adapters have rank 8 and sit on every linear layer, in attention and in the feed-forward blocks. They train 98,304 weights, against about 14 million that full fine-tuning would train. `lora_alpha` scales the adapters' correction, set to the usual twice the rank. For a real task, choose the rank and the layers on public data, because a choice made from private runs costs privacy (Hyperparameter tuning).

48.6. Make the model Opacus-compatible

Opacus needs one gradient per sequence, and layers that mix the records of a batch, such as batch normalization, make that impossible. Opacus's `ModuleValidator.validate` lists such layers before you train.

```
validator errors: 0
```

Pythia has none, and only the adapters train in any case. They are ordinary linear layers, which Opacus's default hooks handle, so the script passes `grad_sample_mode="hooks"`. Opacus computes per-sequence gradients only for the weights that require gradients, so the 14 million frozen weights add no cost. Call `model.train()` before wrapping a model you have just evaluated, since Opacus rejects one in evaluation mode. After training, the hooks stay attached and slow down everything the model does next. So the script calls `remove_hooks()` before it evaluates the private model.

48.7. Calibrate and train

Fix the target ε, δ, the sampling rate and the number of epochs, and let the accountant find the noise. The script asks for $\varepsilon = 8$ over 3 epochs with an expected batch of 32 sequences. The

sampling rate q, the share of the data in an average batch, is then 32/1,024, so the 3 epochs take 96 steps. The accountant is the privacy loss random variable (PRV) accountant, which computes ε almost exactly (Report and review an ε claim).

```
privacy_engine = PrivacyEngine(accountant="prv")
private_model, optimizer, loader = privacy_engine.make_private_with_epsilon(
    module=model,
    optimizer=optimizer,
    data_loader=loader,  # replaced by Poisson sampling with rate BATCH / n
    target_epsilon=epsilon,
    target_delta=delta,
    epochs=EPOCHS,
    max_grad_norm=MAX_GRAD_NORM,
    grad_sample_mode="hooks",
)
print(f"noise multiplier: {optimizer.noise_multiplier:.3f}")
```

The training loop is an ordinary one. Opacus's loader draws Poisson batches, in which each sequence joins each batch independently with probability q.

```
for (batch,) in loader:  # each batch holds a random number of sequences
    optimizer.zero_grad()
    loss = private_model(input_ids=batch, labels=batch).loss
    loss.backward()  # Opacus's hooks keep one gradient per sequence
    optimizer.step()  # clips each one to MAX_GRAD_NORM, sums, adds noise
    steps += 1
```

```
noise multiplier: 0.553
sample rate q=0.03125  steps=96  noise multiplier=0.553
epsilon (Opacus PRV)=7.99  epsilon (dp_accounting PLD)=7.98  at delta=1e-04
```

The noise multiplier is the noise's standard deviation in units of the clipping norm, `MAX_GRAD_NORM`, which the script sets to 1. The last line cross-checks the claimed ε of 8 with the privacy loss distribution (PLD) accountant of Google's `dp_accounting`. The two values agree to within rounding.

Both runs use AdamW at a learning rate of 10^{-3}, and the private run differs only by clipping, noise and Poisson sampling. That learning rate, the rank and the epochs were chosen in a few non-private trial runs on the same text. A real run must choose them on similar public text, such as other public-domain plays, or pay for the trials in ε (Hyperparameter tuning).

Two more settings change for a real run. With more data, use batches in the thousands, not 32. Opacus's `BatchMemoryManager` splits them into physical batches that fit in memory (Train a model with DP-SGD). And a model you release needs `secure_mode=True` in `PrivacyEngine`, whose cryptographically secure noise cannot be seeded, which is why the script leaves it off.

48.8. Evaluate

A language model is judged by how well it predicts held-out text. The script measures the mean cross-entropy per token on the 128 held-out sequences, and its exponential, the **perplexity**. A perplexity of 159.3 means the model is as unsure as if it chose among about 159 equally likely tokens. So lower is better.

```
held-out loss pretrained: 5.306 nats/token  perplexity 201.6
held-out loss LoRA non-private: 5.071 nats/token  perplexity 159.3
held-out loss LoRA DP eps=8: 5.044 nats/token  perplexity 155.1
```

Both fine-tuned models improve on the pretrained one, and the private model is no worse than the non-private one. On one seed, its slightly lower perplexity means only that DP costs nothing measurable here. Both gains are modest, because a quarter of the training sequences begin with canaries rather than Shakespeare.

Do not expect privacy to be this cheap on every task. This run is short, trains few weights, and uses text that is very likely in the pretraining data already. With a harder task or a smaller ε, compare the private model with both baselines at several budgets, as Train a model with DP-SGD does.

48.9. Check memorization

The canaries show what perplexity cannot, whether the model will repeat a secret. The script measures each canary in two ways, first by extraction. It gives the model the prefix "the pin of Alice is" and lets it write the likeliest next token five times. Then it checks whether the result is the secret.

```
@torch.no_grad()
def greedy_completion(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str) ->
    """The five tokens the model writes after the prefix when it always picks the likeliest
    ids = tokenizer(prefix, return_tensors="pt")["input_ids"]
    for _ in range(5):
        next_token = model(input_ids=ids).logits[:, -1].argmax(dim=-1, keepdim=True)
        ids = torch.cat([ids, next_token], dim=1)
    return tokenizer.batch_decode(ids[:, -5:])[0].strip()
```

The second is the **exposure** of Memorization and training-data extraction. It ranks the canary's secret among random secrets of the same format by how likely the model finds each, and turns the rank into bits. The script's likelihood is the summed log-probability of the five digits. Each bit halves the number of guesses an attacker needs.

```
@torch.no_grad()
def secret_scores(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str, secrets
    """Summed log-probability of each secret's five digit tokens after the prefix."""
```

```
    prefix_ids = tokenizer(prefix)["input_ids"]
    rows = [prefix_ids + tokenizer(" " + s)["input_ids"] for s in secrets]
    ids = torch.tensor(rows)  # every secret is five tokens, so the rows have equal length
    logits = model(input_ids=ids[:, :-1], logits_to_keep=5).logits  # the predictions of the
    logprobs = logits.log_softmax(dim=-1).gather(-1, ids[:, -5:, None])
    return logprobs.sum(dim=(1, 2)).double().numpy()


def canary_exposure(
    model: PeftModel, tokenizer: PreTrainedTokenizerBase, canary: Canary, candidates: list[s
) -> tuple[float, float]:
    """The exposure of the canary's secret among the candidates, measured and extrapolated p
    candidate_scores = scores_batched(model, tokenizer, canary.prefix, candidates)
    score = float(scores_batched(model, tokenizer, canary.prefix, [canary.secret])[0])
    return exposure(score, candidate_scores), extrapolated_exposure(score, candidate_scores,
```

No library installed with this book computes exposure, so `exposure` and `extrapolated_exposure` come from the book's helper module practitioner_privacy_tests.py, after TensorFlow Privacy's versions. With 2,000 random candidates, a canary that beats them all reaches a cap of 10.97 bits. A secret the model has never seen has a random rank and averages about 1.44 bits ($\log_2 e$). Each line below gives a group's mean exposure, the same past the cap, and how many of its three secrets greedy decoding wrote out.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
DP 1 copies: mean exposure 0.9 bits  extrapolated 0.9  extracted 0/3
DP 4 copies: mean exposure 0.6 bits  extrapolated 0.6  extracted 0/3
DP 16 copies: mean exposure 4.2 bits  extrapolated 4.3  extracted 0/3
DP 64 copies: mean exposure 4.0 bits  extrapolated 3.9  extracted 0/3
DP controls: mean exposure 2.2 bits  extrapolated 2.2  extracted 0/3
```

The non-private model, the positive control, leaks. Exposure rises with the number of copies, from 2.0 bits for one copy to the cap for 64. The model writes out all three 64-copy secrets. The extrapolated column sees past the cap by fitting a skew-normal distribution, a normal with one heavier tail, to the candidates' scores. It puts the 64-copy secrets near the 16.61 bits of the whole space.

The 4-copy canaries are the instructive case. Two of the three reach 6.9 and 8.2 bits, yet none is extracted, so exposure detects memorization that extraction alone would miss.

Nothing can be extracted from the private model. Its canaries with 1 or 4 copies score no higher than its controls. Their 2.2 bits are within the scatter of a mean over three secrets. Still, DP limits the effect without erasing it, since the canaries with 16 and 64 copies still average about 4 bits.

The guarantee allows that, because $\varepsilon = 8$ protects one sequence, while a canary with many copies is many sequences. Group privacy bounds a group of units at about their number times ε. So 16 copies get 16×8, far too much to promise anything (User-level differential privacy). A secret that a user repeats across many documents is the same case, one more reason to make the unit the user.

Two cautions apply. Because the canaries are so dense, a real non-private model may leak less than this one, and a real audit should plant fewer copies. And three canaries per group is a small sample. For a fuller assessment before release, run the attacks of Pre-release privacy risk assessment. For a statistical lower bound on ε, run the one-run audit of Auditing private training.

48.10. Variants

Three variants matter for real models: ghost clipping, LoRA on GPT-2, and packages that wrap the whole loop. The block below shows them with pinned versions, and it was not run.

```
# opacus==1.6.0: ghost clipping, when per-sequence gradients do not fit in memory
model, optimizer, criterion, loader = privacy_engine.make_private_with_epsilon(
    module=model, optimizer=optimizer, data_loader=loader, criterion=torch.nn.CrossEntropyLo
    target_epsilon=8.0, target_delta=1e-4, epochs=3, max_grad_norm=1.0, grad_sample_mode="gh
)
logits = model(input_ids=batch).logits[:, :-1]                # predict each next token
loss = criterion(logits.flatten(0, 1), batch[:, 1:].flatten(), shape=logits.shape)
loss.backward()                                               # norms first, then the clipp

# peft==0.21.2: GPT-2 stores its attention as Conv1D, with the weight transposed
LoraConfig(r=8, lora_alpha=16, target_modules=["c_attn"], fan_in_fan_out=True)

# dp-transformers==1.0.0 (Microsoft): Opacus inside the Hugging Face Trainer
# private-transformers (Li et al.): install from its GitHub repository, pinned to a commit
```

Ghost clipping gets each sequence's gradient norm without storing the gradient, so memory no longer holds one gradient per sequence (DP-SGD).

48.11. Report

The release statement names the unit, here one sequence of 32 tokens, with the number of units n and the slack δ. It names the accountant and its three inputs, which are the noise multiplier, the sampling rate and the number of steps. It says that the guarantee covers the private text given the pretrained model, named with its revision. It says how the hyperparameters were chosen, here by non-private trials on the private text, which a real release could not claim for free. Attach the canary audit as evidence, with its caveats. Release and governance assembles all of this into a record and a statement.

What ships is the adapter. With `--save-adapter FOLDER`, the script saves it after `remove_hooks()` and loads it back onto the same pinned checkpoint.

```
private.save_pretrained(path)  # only the adapter weights and config, after Opacus's hooks a
base_model = AutoModelForCausalLM.from_pretrained(folder, dtype=torch.float32)  # the same p
reloaded = PeftModel.from_pretrained(base_model, path)
```

The folder holds only the adapter, so the statement names the base checkpoint and revision it fits. The guarantee covers the adapter and anything computed from it.

48.12. Checklist

Table 48.1.: The steps of fine-tuning a language model privately, their defaults and their checks.

Step	Default	Check
Unit, n, δ	Document or user in production; $\delta \leq 1/n^{1.1}$ rounded down	n counts units, not tokens or lines
Canaries	A few per group, inserted 1 to many times, plus controls	Controls near 1.44 bits, allowing for the scatter of a few
Public model and tokenizer	One public checkpoint, pinned revision, float32	Nothing fitted on private text; overlap checked
What to train	LoRA, rank 8, on every linear layer	Trainable count printed; chosen on public data
Opacus	Validator; hooks; `remove_hooks()` after training	No validator errors
Calibrate and train	PRV accountant; Poisson sampling; large batches	PLD agrees with PRV
Evaluate	Held-out perplexity against the pretrained and non-private models	Private model beats the pretrained one
Memorization	Exposure and greedy extraction, non-private and private	Nothing extracted; one-copy canaries no higher than the controls
Report and save	Unit, ε, δ, accountant, public model, tuning; adapter saved after `remove_hooks()`	Statement names what is not covered and the base revision

48.13. References

- Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): LoRA and other adapters with DP-SGD, and their accuracy.

- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022): full fine-tuning with ghost clipping, and the hyperparameters that make it work.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries and exposure.

Derivations. This workflow draws on the full edition's chapters on public data and private fine-tuning in the full edition, DP-SGD in the full edition and memorization and training-data extraction in the full edition.

49. Private prediction: PATE and in-context learning

49.1. What it is

Private prediction applies differential privacy (DP) to each *answer* instead of to the model. You split the private records into disjoint shards and train an ordinary model, a **teacher**, on each one. The teachers stay secret. Each query gets the winner of their vote after noise is added to the counts, private at a level ε with a small slack δ. A record lives in one shard, so it can move one teacher's vote to another label. That changes the counts by at most $\sqrt{2}$ in Euclidean length.

GNMax adds Gaussian noise with a standard deviation σ to each count and releases only the label with the largest noisy count. In Rényi differential privacy (RDP), each answer costs a straight line of slope $1/\sigma^2$ in the order α, and the slopes add (Rényi DP). So write $\rho(\varepsilon, \delta)$ for the largest slope that still converts to your target. A fixed number Q of answers, even adaptively chosen, then needs the noise

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{49.1}$$

For example, at $\delta = 10^{-5}$ a budget of $\varepsilon = 1$ allows a slope of 0.0306, so 1,000 answers need $\sigma = 181$.

49.2. When you meet it

You meet private prediction when you cannot train a private model. The common case is a large language model (LLM) behind an interface. Its prompt holds private records, and its answers leak them. The fix splits the records into disjoint groups, prompts the model once per group, and releases the GNMax label (Duan et al. 2023). At $\varepsilon = 1$, Wu et al. (2024) answered 10,000 sentiment queries this way as accurately as one non-private prompt.

Otherwise the choice is between private answers and a private model, whose accuracy is fixed. The vote's noise grows with $\sqrt{Q}$, so answers win up to a **break-even** number of queries, and the model wins beyond it. On the full edition's 10,000-record synthetic task, the vote beats noisy stochastic gradient descent (SGD), DP-SGD, only on the first 18 queries at $\varepsilon = 1$. So compare the two at your budget and expected number of queries, with the DP-SGD baseline of Train a model with DP-SGD.

PATE (private aggregation of teacher ensembles) spends the budget once. It labels public, unlabelled inputs by GNMax and trains a new model, the **student**, on them. Only the student is released, at exactly the cost of its labels, so PATE needs public inputs from the private data's distribution. Papernot et al.'s MNIST student comes within a percentage point of non-private accuracy, but at a data-dependent ε (Section 49.4). On the synthetic task, DP-SGD beats the best student at every budget.

49.3. How to use it

`dp_accounting` accounts for the vote. One GNMax answer is a `GaussianDpEvent` whose noise multiplier, the noise over the sensitivity, is $\sigma/\sqrt{2}$.

```
import numpy as np
from dp_accounting import GaussianDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

sigma, queries = 181.0, 1000  # noise per count, answers to release
event = SelfComposedDpEvent(GaussianDpEvent(sigma / np.sqrt(2)), queries)  # sensitivity sqr
rdp = RdpAccountant().compose(event).get_epsilon(1e-5)
pld = PLDAccountant().compose(event).get_epsilon(1e-5)
print(f"{rdp:.3f} {pld:.3f}")  # 0.999 0.914
```

The RDP accountant confirms $\varepsilon = 0.999$. The privacy loss distribution (PLD) accountant, exact for Gaussian noise, gives 0.914, so solving for σ with it gives less noise. To solve, pass a function from σ to this event to `dp_accounting.calibrate_dp_mechanism`. For many answers the PLD is much slower, and RDP is the fallback. When each query draws a fresh Poisson sample of records, as in Wu et al., wrap each event in a `PoissonSampledDpEvent`.

49.4. What goes wrong

The guarantee needs each record in exactly one shard. So assign shards by a hash of the record's or the user's identifier. Never assign them by position in a file, where one inserted record shifts every later shard. Release the label only, and fix Q in advance.

Noise this large wipes out close votes. With two labels and a gap g between the counts, the vote returns the plurality with probability $\Phi(g/(\sqrt{2}\sigma))$, where Φ is the standard normal distribution function.

For example, the noise for 1,000 answers at $\varepsilon = 4$ is $\sigma = 51.8$. Then a lopsided vote of 98 to 2 is right with probability 0.9051, but a close vote of 52 to 48 only about half the time. So measure the noiseless vote first, since no budget beats it. Adding teachers widens the gaps, since σ does not depend on their number, until the shards get too small to train on.

Finally, beware of a data-dependent ε. When teachers agree, a sharper analysis charges less, but that smaller ε is computed from the votes and leaks them. Until it is released with noise, it is a measurement, not a guarantee.

Confident-GNMax answers only queries on which a noisy check finds agreement, the cheap and accurate ones.

49.5. References

- Papernot et al., "Scalable Private Learning with PATE" (ICLR 2018): GNMax, its data-dependent analysis and Confident-GNMax.
- Wu et al., "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024): subsampled voting over prompts.

Derivations. Every result here is derived step by step in the full edition's chapter.

50. DP-FTRL and matrix factorization

50.1. What it is

Gradient descent without momentum adds up its steps. So its weights after each step depend on the sum of all clipped gradients so far, a **prefix sum**. Stochastic gradient descent (SGD) with differential privacy (DP), DP-SGD, adds fresh noise at every step. So the noise in the prefix sums grows like a random walk. Matrix mechanisms correlate the noise across steps, so that each step's noise partly cancels earlier noise in the sums.

Stack the gradient sums of the steps as the rows of a matrix x. The prefix sums are then Ax, where A has ones on and below the diagonal. The **matrix factorization mechanism** writes A as a decoder L times an encoder R. It adds a matrix of Gaussian noise, written z, to the encoded gradients and decodes the result,

$$\hat{S} = L(Rx + z) = Ax + Lz. \tag{50.1}$$

A record enters only through the column of R for its step, so privacy depends on the longest column of the encoder, its sensitivity. Accuracy depends on the decoder, whose row t carries the noise into the t-th prefix sum.

The first such method ran the follow the regularized leader (FTRL) optimizer on sums from a binary tree. It is called DP-FTRL (Kairouz et al. 2021). Today's usual factorization is the **square-root factorization**, which uses the lower-triangular square root of A for both factors. At equal privacy over one epoch of 100 steps, it leaves about a quarter of independent noise's standard deviation in the final sum.

50.2. When you meet it

You meet matrix mechanisms when your pipeline cannot honestly claim Poisson sampling, the random batches that DP-SGD's accountant assumes. Fixed batches cut from a shuffled order fail it. So do data read in arrival order, and phones in federated learning that check in when they can. Accounting them as Poisson understates ε (Amplification by subsampling), while a matrix mechanism needs no sampling.

Without sampling, independent noise loses much accuracy, and correlated noise wins most of it back. On the full edition's synthetic task, trained in a fixed order for ten epochs at $\varepsilon = 2$, independent noise reaches a test accuracy of 0.8641. The banded square root of Section 50.4 reaches 0.9491, against 0.9493 for DP-SGD with Poisson sampling. So if you can implement Poisson sampling, use DP-SGD with Opacus. If you cannot, use a banded matrix mechanism, as Google did for its Gboard keyboard models (Xu et al. 2023).

50.3. How to use it

Google's JAX Privacy implements banded matrix factorization. The block below was not run, since JAX is not in this book's environment. It cycles through 100 fixed groups of records, one per step, so each record takes part once every 100 steps.

```
# jax-privacy==2.0.0 (not run)
from jax_privacy.experimental.execution_plan import BandMFExecutionPlanConfig

config = BandMFExecutionPlanConfig.default(
    num_bands=100,       # at most the 100 steps between a record's turns
    iterations=1000,     # ten epochs of 100 steps
    epsilon=2.0,
    delta=1e-5,
    sampling_prob=1.0,   # all of the step's group joins: no amplification claimed
    l2_clip_norm=1.0,
)
plan = config.plan       # calibrates the noise multiplier with dp_accounting
```

The training loop draws each batch from `plan.batch_selection_strategy` and sums its clipped gradients with `plan.clipped_grad`. Then `plan.noise_addition_transform` adds the noise before the optimizer sees the sum. A `dp_accounting` accountant turns `plan.dp_event` into the ε you report. The module is marked experimental, so pin the version.

50.4. What goes wrong

The pitfalls come from records that take part more than once. Over E epochs a record enters through E columns of the encoder. The square root's columns for nearby steps are nonzero in the same rows, so they add up and the sensitivity grows faster than $\sqrt{E}$. **Banding** keeps a number b of the encoder's diagonals, the main one and those just below it. It needs every two participations of a record to be at least b steps apart, a condition called b**-min-separation**. The record's columns then share no rows, so the sensitivity is at most $\sqrt{E}$ times the longest column, as for independent noise.

So the data order is part of the mechanism. Fix it before training, independently of the data, and keep the same permutation every epoch. A new shuffle each epoch can put a record at the end of one epoch and the start of the next, which breaks the banded bound.

The neighbouring relation changes the numbers too. The analysis here leaves a neighbour's slot empty, so a step moves by at most the clipping norm. If a neighbour replaces the record with another instead, every sensitivity doubles, so state the relation next to every ε.

`dp_accounting` 0.6.0 enforces this. Its tree-aggregation event works only in a Rényi differential privacy (RDP) accountant built with `NeighboringRelation.REPLACE_SPECIAL`, where a neighbour replaces the record with an empty one. Under add/remove, DP-SGD's default, it raises `UnsupportedEventError`, and the privacy loss distribution accountant rejects it too. So DP-FTRL and DP-SGD runs cannot share one add/remove ledger.

50.5. References

- Choquette-Choo et al., "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023): banding, min-separation and amplification.

Derivations. Every result here is derived step by step in the full edition's chapter.

51. Disparate impact

51.1. Why this matters

A private model's test accuracy is an average over everyone, so it cannot show which people lose the accuracy that privacy costs. Bagdasaryan, Poursaeed & Shmatikov (2019) trained networks with differential privacy (DP), using the private form of stochastic gradient descent (SGD) called DP-SGD. They found that the lost accuracy fell mostly on under-represented groups, and called this the **disparate impact** of DP.

So a model that is a point worse on average can be ten points worse for one group, often a small group that privacy should protect. Before you ship a private model, measure its cost for each group against a non-private baseline, with a confidence interval on every number.

51.2. Evaluating a private model per group

Decide which groups matter before you look at any results, so that the results cannot steer the choice. Then train a non-private baseline with the same data and recipe. A gap between groups may exist without privacy. DP is responsible only for how far each group's own accuracy drops when training becomes private.

Write acc_g for a model's accuracy on the test records of group g. Private training is random, so its accuracy is averaged over several training runs, written $\mathbb{E}$. The **privacy cost** π_g is the accuracy the group loses when the same data and hyperparameters are trained privately,

$$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]. \tag{51.1}$$

DP has disparate impact when groups have different privacy costs.

Every accuracy needs an interval. Each test record is classified correctly or not, so a group's accuracy is a binomial proportion. Its exact Clopper–Pearson interval contains the true accuracy in at least 95% of test samples (Hypothesis testing).

The gap between two groups needs its own interval. Write $\hat{p}_g$ for group g's measured accuracy and m_g for its number of test records. For two groups a and b measured on disjoint test records, the variances of the two estimates add. So for large counts an approximate 95% interval for the gap is

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}. \tag{51.2}$$

These intervals are wider than most people expect. Suppose a test split holds only 100 records of a minority, and 88 of them are classified correctly. The majority gets 1,812 of its 1,900 records

right, an accuracy of 0.954, so the measured gap is 0.074. SciPy gives the minority's interval, and Equation 51.2 gives the gap's.

```
from math import sqrt

from scipy.stats import binomtest

minority = binomtest(88, 100).proportion_ci(confidence_level=0.95, method="exact")
print(f"{minority.low:.3f} {minority.high:.3f}")  # 0.800 0.936

gap, se = 0.954 - 0.880, sqrt(0.954 * 0.046 / 1_900 + 0.880 * 0.120 / 100)
print(f"{gap - 1.96 * se:.3f} {gap + 1.96 * se:.3f}")  # 0.010 0.138
```

So the gap is real, since its interval excludes zero, but it could be anywhere from one point to fourteen. An interval separates a 5-point gap from zero only with a few hundred test records from the group, and a 1-point gap only with thousands.

The privacy cost needs a different interval. Both models are scored on the same test records, so their accuracies are not independent and Equation 51.2 does not apply. Pair them instead. Among group g's m_g test records, write u for the number only the non-private model gets right, and v for those only the private one gets right. One run's cost is then $(u - v)/m_g$, and for large counts an approximate 95% interval is

$$\frac{u - v}{m_g} \pm \frac{1.96}{m_g} \sqrt{u + v - \frac{(u - v)^2}{m_g}}. \tag{51.3}$$

Training adds a second source of variation, which no test set removes. A small group's accuracy varies far more between training runs than a large group's, so a one-run comparison of two recipes can reverse. Train each recipe with several seeds, and report the mean cost and its spread.

The evaluation can also leak, because per-group accuracies on private test data are statistics of that data. To publish them, add noise to each group's count of correct predictions and charge it to a budget of its own (Release statistics with a library).

51.3. Disparate impact, measured

The script practitioner_dp_sgd.py of Train a model with DP-SGD trains a small network on the Adult census data to predict whether a person's income exceeds $50K. It trains once without privacy and once with DP-SGD at a privacy level ε of 4 and a slack δ of 10^{-5}. It scores both on the test rows of each `sex`. For each group, it counts the correct predictions, marked in the boolean array `right`, and gets the exact interval from SciPy.

```
in_group = sex == group
correct, total = int(right[in_group].sum()), int(in_group.sum())
interval = binomtest(correct, total).proportion_ci(confidence_level=0.95, method="exact")
```

Its seed-0 run, `uv run python code/part05-dp-ml/practitioner_dp_sgd.py`, gives Table 51.1.

Table 51.1.: Accuracy on Adult by sex, with exact 95% intervals, for one training run of each model.

Group	Test records	Non-private accuracy	Accuracy at $\varepsilon = 4$	Privacy cost
Female	5,613	0.924 [0.917, 0.931]	0.922 [0.915, 0.929]	+0.002
Male	11,229	0.808 [0.800, 0.815]	0.807 [0.800, 0.815]	+0.001

Here DP has no measurable disparate impact. Each group loses at most 0.002 in one private run, well inside its intervals, which overlap almost entirely. The large gap between the groups is there without privacy too, so DP is not responsible for it. Adult is easy, because both groups are large and the whole model loses almost nothing at this budget.

A small group tells a different story. The full edition's running example is a synthetic task with 10,000 training records, labelled by a fixed random network. Of these, 500 form a minority whose inputs come from a different region and follow a different labelling rule (Table 51.2).

Table 51.2.: Test accuracy of each group of the full edition's running example with a 5% minority, averaged over 3 runs ($\delta = 10^{-5}$, noise multiplier σ). Costs are fractions of accuracy, and a negative cost is a gain.

Training	Majority accuracy	Minority accuracy	Majority's privacy cost	Minority's privacy cost
Non-private SGD	0.9529	0.9411	—	—
Clipping only, $\sigma = 0$	0.9581	0.9007	−0.0051	0.0404
$\varepsilon = 4$, $\sigma = 1.36$	0.9470	0.8477	0.0059	0.0934

The minority pays almost all of the cost. At $\varepsilon = 4$ the majority loses 0.59 points, while the minority loses 9.34. The cost also depends steeply on the group's share of the data. At the same budget, a minority with 1% of the training records loses 34.88 points, and one with 20% loses only 2.80. So check every small group, whatever the overall accuracy says.

51.4. Why small groups pay: clipping and noise

DP-SGD loses accuracy through two steps, clipping and noise, and both treat every record alike. The clipping row of Table 51.2 clips without adding noise, so it gives no privacy at all, yet the minority already loses 4.04 points there. So clipping by itself treats the groups unequally.

Clipping caps every record's pull on the update at the clipping norm C, so it cuts back the groups with long gradients hardest. A group's gradients are long where the model fits it badly, as it does a rare pattern the rest of the data does not teach. So training gives such a group less weight than its share of the records.

The noise is the same for every group, while a group's signal grows with its size. So a small group's signal is the weakest against the noise, and its privacy cost grows roughly like one over its share of the training records. The two causes compound, because the noise acts on the weight that clipping has already shrunk. A group whose gradients clipping halves pays like a group half its size.

To see which cause hurts your model, measure per-example gradient norms by group. Opacus's `GradSampleModule`, the wrapper DP-SGD training already uses, keeps one gradient per row in each parameter's `grad_sample`. So one backward pass gives the norms.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every weight p
norms = stacked_norms(wrapped)
```

Group the norms by each row's group label, kept beside the batch but not fed to the model. Log each group's median norm and fraction clipped as training goes on, because the groups can separate late. At initialization on Adult, the median norm is 1.444, so $C = 1$ clips every row, whatever its group.

51.5. Mitigations and what they cost

At a fixed budget, a change to the training recipe only moves the cost between the groups or between clipping and noise (Table 51.3). Only the last row, which changes the data, the starting model or the budget, reduces it.

Table 51.3.: Mitigations for disparate impact, and when to use them.

Fix	What happens	Use it?
Upweight the group's loss, or use a smaller clipping norm	Clipping cuts an upweighted gradient back to C, and a smaller norm clips more of the group	No
Larger clipping norm, with learning rate times C held fixed	Less clipping, but more noise relative to short gradients	Rarely, since it did not help the running example

Fix	What happens	Use it?
Clipping norm set to the largest gradient seen in the data	Breaks the privacy proof, which assumes C is fixed in advance	Only if chosen privately and charged
Oversample the group	Its records' ε is set by their higher sampling rate, so σ must rise	Only with that accounting
Global scaling with an adaptive bound (Esipova et al. 2023), one shared scale for every gradient	Keeps the update's direction and needs no group labels	Worth a try
More data from the group, a public pretrained model (Public data and private fine-tuning), or a larger ε	Raises the group's signal or lowers the noise	The fixes that work

Oversampling costs the group's privacy, because a record's guarantee depends on its own chance of being sampled (Amplification by subsampling). So run the accountant at the oversampled rate. In the running example, sampling the minority four times as often at $\varepsilon = 4$ raises the noise multiplier from 1.36 to 4.71. The gap between the groups' accuracies then narrows from 9.93 points to 3.99, but by **levelling down**, since the majority loses 8.01 points.

Some of the cost cannot be avoided by any algorithm. Learning a small group's own rule means estimating it privately from that group's few records. Private estimation from few records has an error floor that no mechanism beats (Lower bounds).

51.6. Cheat sheet

Table 51.4.: The decisions of this chapter.

Question	Answer	Conditions
Does DP hurt this group?	Its privacy cost, Equation 51.1, against the non-private baseline	Same data and recipe, averaged over runs
How big will the cost be?	Negligible for large groups at moderate ε, roughly one over the group's share for small ones	Grows as ε falls

Rules of thumb

- Report accuracy and privacy cost per group, with intervals, next to the non-private model's.
- Never report the gap alone, because levelling down closes it too.

51.7. References

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019): the observation.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023): global scaling.

Derivations. Every result here is derived step by step in the full edition's chapter.

52. Auditing private training

52.1. Why this matters

When you train with differential privacy (DP), the accountant turns your configuration into a privacy level ε at a slack δ. That number is a proof about an algorithm, but what ships is code. A bug in the noise, the clipping or the sampling makes the model less private but barely changes its accuracy. Accuracy is flat across a wide range of noise levels (the training workflow), so the usual tests never notice the bug.

An **audit** checks the code by attacking it. It plants records called **canaries**, trains with and without them, and counts how often a membership test guesses right. DP caps how often any test can be right. So a test that is right too often proves that the code is less private than claimed. A test that is rarely right proves nothing, since a weak attack also fails against a leaky model.

52.2. Known bugs, and which audit catches them

These bugs break the private form of stochastic gradient descent (SGD), DP-SGD.

- **The variance passed where the standard deviation is expected,** or **noise scaled for the mean but added to the sum,** which adds the batch size times too little noise. A unit test in continuous integration (CI) catches both (Section 52.3).
- **Clipping applied after averaging, or noise from a weak seed.** Nasr et al. (2023) found both in real code with per-step audits like the CI test.
- **The accountant told the wrong numbers,** such as a record count that includes augmented copies, which makes ε look smaller than it is. Only an end-to-end audit can catch it, and only if the error is large.
- **Batches shuffled but accounted as Poisson sampling,** which can make the true ε several times the reported one (Amplification by subsampling). No audit here exercises the sampling, so check the loader itself.

Audits catch errors only in the unsafe direction, so code that adds too much noise passes them all.

52.3. A CI test for the noise and the clipping

The cheapest strong audit tests one step of the optimizer, many times. A **Dirac canary** is a per-example gradient that is zero except on one coordinate, where every other record's gradient is zero. Put it into every step of one run and leave it out of another. The reading x_t is step t's noisy

gradient sum on that coordinate, divided by the clipping norm C. The canary is longer than C, so a correct optimizer clips it to length C and it adds exactly 1 to the reading. The optimizer also adds noise of standard deviation σC, where σ is the noise multiplier, so each reading follows

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1, \sigma^2) \text{ with it.} \tag{52.1}$$

A threshold test says "canary in" whenever a reading exceeds a threshold. Its **true positive rate (TPR)** is how often it says so when the canary was there. Its **false positive rate (FPR)** is how often it says so when the canary was not.

For two normal distributions a distance μ apart, every threshold gives $\Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}) = \mu$, where Φ is the standard normal distribution function. So measured rates bound the true separation μ, which a correct optimizer keeps at $1/\sigma$:

$$\mu \geq \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}). \tag{52.2}$$

The test plugs in the low end of the TPR's Clopper–Pearson interval and the high end of the FPR's. Each end is taken at level $\beta/2$, so the bound holds with probability $1 - \beta$ (Hypothesis testing). If this bound exceeds $1/\sigma$, the code adds less noise than it claims.

The script practitioner_audit.py runs the test on Opacus's own `DPOptimizer`. It writes per-example gradients into `grad_sample` by hand, as Opacus's hooks would, and reads the noisy sum that `optimizer.step()` leaves in `grad`.

```
def canary_coordinate(sigma: float, planted_bug: str, with_canary: bool, steps: int) -> Arra
    """Run the optimizer for `steps` steps; return the canary coordinate of each noisy sum,
    layer = torch.nn.Linear(DIM_CI, 1, bias=False)
    optimizer = make_optimizer(layer, sigma, planted_bug)
    observations = np.empty(steps)
    for t in range(steps):
        setattr(layer.weight, "grad_sample", batch_grad_samples(with_canary))  # as Opacus's
        optimizer.step()  # clip each row to C, sum, add N(0, σ²C²)
        assert layer.weight.grad is not None
        observations[t] = layer.weight.grad[0, CANARY_COORD].item() / CLIP_CI
        optimizer.zero_grad()
    return observations
```

The canary's gradient is ten times C, so the test also exercises clipping. The threshold is chosen on the first half of the steps and the rates are counted on the second half. The test also checks that the canary-free readings have a standard deviation within 10% of σ, since with no noise every reading is 0. The command `uv run python code/part05-dp-ml/practitioner_audit.py` tests three optimizers at $\sigma = 0.7$ and a batch of $B = 64$ records. They are the correct one, one with the variance bug and one with no noise. The variance bug removes noise only because $\sigma^2 < \sigma$ when σ is below 1, and above 1 it would pass.

```
sigma=0.7  C=1.0  B=64  steps per run=2000  claim mu=1/sigma=1.429
correct: noise std 0.687 (claim 0.7)  FP 206/1000  TP 710/1000
correct: mu_lo=1.201  pass
variance for std: noise std 0.481 (claim 0.7)  FP 206/1000  TP 862/1000
```

```
variance for std: mu_lo=1.722  FAIL
no noise: noise std 0.000 (claim 0.7)  FP 0/1000  TP 1000/1000
no noise: mu_lo=5.360  FAIL
```

The correct optimizer's bound on μ is 1.201, below its claimed 1.429, so it passes. The variance bug's bound is 1.722 and the missing noise's is 5.360, so both fail. The test takes seconds, so run it on every commit.

Three things go wrong in practice.

First, a correct optimizer still fails with probability β, here 5%, so an unseeded test fails one build in twenty. Fix the seed, or use a β of 10^{-3} with more steps. Second, the script builds its own optimizer, so it tests Opacus, not your call to it. To test your code, have your setup function return the optimizer it makes, and drive that one. Third, the test never draws a batch or calls the accountant. So the same suite should assert that the loader is Opacus's `DPDataLoader` and that a second library's ε matches yours (Train a model with DP-SGD).

52.4. From an attack to a lower bound

For a whole training run, the attack is a membership test on a planted canary. A **trial** is one training run, with or without the canary. Under (ε, δ)-DP, every such test satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \le e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{52.3}$$

Solving these for ε turns measured rates into a lower bound on it. Write $\overline{\mathrm{FPR}}$ for the one-sided Clopper–Pearson upper bound on the FPR at level $\beta/2$, and $\underline{\mathrm{TPR}}$ for the TPR's lower bound. With probability at least $1 - \beta$, the code is then not (ε, δ)-DP, at the claimed δ, for any ε below

$$\varepsilon_{\mathrm{lo}} = \max\left\{ \ln \frac{\underline{\mathrm{TPR}} - \delta}{\overline{\mathrm{FPR}}},\ \ln \frac{1 - \overline{\mathrm{FPR}} - \delta}{1 - \underline{\mathrm{TPR}}} \right\}. \tag{52.4}$$

The bound needs independent trials. It also needs a threshold chosen on other trials than those it counts, because a threshold tuned on them inflates it.

No library installed with this book computes the bound, so the script imports `eps_lower_bound` from the book's helper module, practitioner_privacy_tests.py. Suppose an attack catches the canary in 500 of 1,000 runs that include it, and raises one false alarm in 1,000 runs without it.

```
for false_pos, true_pos in ((1, 500), (0, 1_000)):  # out of 1,000 runs without and with the
    eps_lo = eps_lower_bound(false_pos, true_pos, 1_000, DELTA)
    print(f"FP {false_pos}/1000  TP {true_pos}/1000: certified eps lower bound {eps_lo:.2f}"
```

```
FP 1/1000  TP 500/1000: certified eps lower bound 4.43
FP 0/1000  TP 1000/1000: certified eps lower bound 5.60
```

The raw rates alone would suggest 6.21, but confidence is expensive, and the certified bound is only 4.43. Even a perfect attack certifies only 5.60 from 1,000 trials per hypothesis. So do not expect an end-to-end audit to certify all of a large claim.

52.5. A one-run audit of the trained model

Thousands of runs are out of reach for a real model, so Steinke, Nasr & Jagielski (2023) get many trials from one run. Plant a number m of canaries, include each one by its own fair coin, train once and score every canary. Then fix a list length r. Guess "in" for the $r/2$ highest scores and "out" for the $r/2$ lowest, and make no guess about the rest. Write W for the number of correct guesses.

Under ε-DP, no attack guesses one canary's coin right with a probability above the limit p_ε given below. So the count W is no more likely to reach any value v than a binomial count with that success rate is:

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}. \tag{52.5}$$

An observed count that this binomial reaches with probability below β rules out that ε, and the certified bound is the largest ε ruled out. Under (ε, δ)-DP, the paper adds at most $2m\delta$ to that probability. The coins must be independent and fair. Several list lengths need a Bonferroni correction, which runs each test at β divided by their number.

The script runs it on the released pipeline of Train a model with DP-SGD on the Adult census data. It adds only the canaries, 1,000 held-out rows with flipped labels. The coins put 537 of them in the training set, and the release's noise multiplier gives $\varepsilon = 3.91$ for this larger set at $\delta = 10^{-5}$. A non-private model uses the release's baseline recipe, and a memorizing control runs non-private Adam for 60 epochs with small batches. Each canary is scored by minus its loss, and Table 52.1 gives each model's best of six list lengths from 20 to 1,000.

Table 52.1.: One-run audit of the released pipeline on Adult with 1,000 mislabeled canaries, at 95% overall confidence and $\delta = 10^{-5}$.

Training	Mean canary loss, in / out	Correct guesses	Certified ε
DP-SGD, release recipe	4.105 / 4.006	10 of 20	0.00
Non-private, release recipe	2.240 / 2.222	8 of 20	0.00
Memorizing control	1.994 / 2.992	75 of 100	0.53

Both release-recipe models certify nothing, and the reason is the recipe, not privacy. Ten epochs with large batches barely fit the flipped labels, so included canaries have the same loss as excluded ones, with or without noise. Only the memorizing control separates its canaries. Even it certifies just 0.53 for a model with no finite ε, because each canary is a single trial. A perfect list of all 1,000 guesses would certify at most 5.27.

So use the one-run audit to expose a bug, when a DP model certifies more than its claim, never as evidence of privacy. Report it with its threat model, a black-box attacker who sees only the final model. Also report a control that shows the attack can detect leakage on your data.

Stronger black-box attacks, such as the likelihood ratio attack (LiRA) of Pre-release privacy risk assessment, train **shadow models**. These are copies of the model, trained with its exact

recipe on similar data. Run them before release and keep their results with the release documents (Release and governance). LiRA's TPR at a low FPR shows what a realistic attacker learns. But its (model, record) pairs share models, so they are not independent trials, and give no certified bound.

52.6. Cheat sheet

Table 52.2.: Which audit answers which question, and its rule.

Question	Audit and cost	Rule	Blind to
Does the code clip and add the noise it claims?	CI test, seconds	Fail if the bound on μ, Equation 52.2, exceeds $1/\sigma$	Sampling and accounting bugs
Is the claimed ε right, accounting included?	End-to-end audit, thousands of runs	Equation 52.4, Clopper–Pearson at $\beta/2$	Bugs smaller than the bound's shortfall
Does a trained model leak planted records?	One-run audit, one extra run	Equation 52.5 plus $2m\delta$, Bonferroni	Most bugs, since its bounds are low
What can a realistic attacker learn?	Shadow-model attack (LiRA)	TPR at a low FPR; no certified bound	The most the code could leak

52.7. References

- Nasr et al., "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023): Dirac canaries, per-step audits and real bugs.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023): the one-run audit.

Derivations. Every result here is derived step by step in the full edition's chapter.

53. Adaptive data analysis

53.1. What it is

When you tune on a held-out validation set, you read the scores and try new settings, so each question depends on earlier answers. Such **adaptive** reuse overfits the holdout set, whose scores then overstate how the model does on new data.

Each such question is a **statistical query**, which averages a per-record answer between 0 and 1 over the n records you hold. A classifier's accuracy is an example. Its population value is the same average over new data. Suppose the records are drawn independently and a number k of queries are fixed before any answer is seen. Then the largest gap between the two values is on average at most

$$\sqrt{\frac{\ln(2k)}{2n}}. \tag{53.1}$$

The number of queries enters through a logarithm, so it hardly matters.

Adaptive queries break this, because a later query can be built to fit the sample's own noise. Blum & Hardt's boosting attack does so on a leaderboard whose labels are coin flips, where nothing can beat 0.5 on new data. Yet after 1,000 submissions on as many holdout records, it reports 0.7182.

Differential privacy (DP) is the remedy, because answers that reveal little about any record cannot be used to overfit it. Let a mechanism's answers be on average within a distance α of the empirical values. Let the whole exchange be DP at a privacy level ε, with a small slack δ, for samples that differ in one replaced record. The **transfer theorem** then bounds the expected largest error against the population by

$$\alpha + e^{\varepsilon} - 1 + \delta, \tag{53.2}$$

whatever the analyst does with the answers. For small ε the middle term is about ε, so the bound is useful only well below $\varepsilon = 1$. It protects the validity of the answers, not the people in the holdout set.

53.2. When you meet it

You meet adaptivity when a validation set, a test set or a leaderboard answers more than one round of questions, as in hyperparameter search. A round is a batch of questions fixed before any of its answers is seen. The bounds below grow with the number of rounds. Table 53.1 gives the choice for each situation.

Table 53.1.: How to reuse a holdout set, by situation.

Situation	What to do
A final evaluation	Keep a test set and use it once
Few adaptive rounds, plenty of data	Give each round its own slice of data
Many adaptive rounds, scarce data	Answer through noise, or use Thresholdout
A search that is private in the holdout set	Nothing more, since the transfer theorem covers the winner
A public leaderboard	Show a new score only when it beats the best by a margin (Blum & Hardt's Ladder)

53.3. How to use it

The simplest fix is **sample splitting**, which answers each of r rounds on its own n/r records. The expected largest error is then at most $\sqrt{r \ln(2k)/(2n)}$ for k queries in all, so the data you need grow in proportion to r. Take 10,000 validation records and a search over 1,000 configurations, each chosen from the earlier scores, so $r = k = 1{,}000$. Splitting then gives a bound of 0.6165, useless for scores between 0 and 1. A bound of 0.05 would need 1,520,181 records, where fixed queries would need 1,521.

Answering every query with Gaussian noise, under the transfer theorem, needs records that grow only like $\sqrt{k}$. Its constants are large, though, so splitting stays cheaper up to a few hundred adaptive queries.

Thresholdout (Dwork et al. 2015) answers from a training set while the training value agrees with the holdout value. Such answers reveal nothing about the holdout set. When they differ by more than a noisy threshold, it records a **detection** and returns a noisy holdout value. So its privacy cost grows with the number of detections, not of queries. But its formal ε in the full edition's experiment is about 29, and no mainstream library ships it. So treat it as a brake, not a guarantee.

53.4. What goes wrong

The common failure is not an attack but a long search that picks each setting from earlier validation scores. A fixed grid scored once, as in scikit-learn's `GridSearchCV`, is a single round, so Equation 53.1 bounds its winner's overstatement. So count rounds, and report how many a validation number has been through. Practice is often milder than the worst case, but the risk is real when records are few and rounds many.

53.5. References

- Dwork et al., "The Reusable Holdout" (*Science* 2015): Thresholdout.
- Blum & Hardt, "The Ladder" (ICML 2015): the boosting attack and a leaderboard that resists it.

Derivations. Every result here is derived step by step in the full edition's chapter.

54. Release and governance

54.1. Why this matters

A model trained with differential privacy (DP) is not finished when training stops. Someone else, a privacy reviewer or a data protection officer, decides whether it may ship. They will ask what the guarantee covers, whether its number is right and what it means under the law. None of them can rerun your training. So the claim must travel with everything needed to check it.

This chapter packages a small network for release, trained on 30,000 people from Adult with the private form of stochastic gradient descent (SGD), called DP-SGD. It is the model of Train a model with DP-SGD, whose target privacy level ε was 4. The script practitioner_release.py runs every computed step in a few seconds. It starts from the run record that the training script writes, passed with `--record PATH`, or else from a built-in copy of the book's record.

```
uv run python code/part05-dp-ml/practitioner_release.py
```

54.2. Assemble the run record

The record must hold what the accountant needs. An accountant is the code that turns a run's settings into ε (Report and review an ε claim). It reads four of them, the noise multiplier σ, the sampling rate q, the number of steps and the slack δ. It also takes four things for granted, so the record states those too.

They are the privacy unit, here one person, and the neighbouring relation, adding or removing a unit or replacing one. The others are the sampler, how the code drew each batch, and that no other use of the private data went uncounted. The training script reads the sampler from the loader Opacus returned rather than typing it, and records the seed and whether Opacus's `secure_mode` was on.

The record also gives n, the number of units, and the library versions. It says how the hyperparameters were chosen and whether any preprocessing saw private data. In the excerpt, `data` is the training script's record and `RELEASE_CONTEXT` holds the release owner's additions.

```
merged = RELEASE_CONTEXT | data  # the training record, plus what the release owner adds
rec = load_record(merged)
for key, value in asdict(rec).items():
    print(f"{key}: {value}")
problems = record_problems(merged)  # missing fields, sampler not Poisson, delta above 1/n^1
limitations = record_limitations(merged)  # stated, so not a problem, but the statement must
print(f"record problems: {len(problems)}")
```

```
for problem in problems:
    print(f"  - {problem}")
print(f"record limitations: {len(limitations)}")
for limitation in limitations:
    print(f"  - {limitation}")
```

```
unit: person (one row of Adult)
neighbours: add/remove
n: 30000
delta: 1e-05
sigma: 0.972900390625
sample_rate: 0.03333333333333333
sampler: poisson
secure_mode: False
seed: 0
steps: 300
...
record problems: 0
record limitations: 1
  - secure_mode=False: Opacus drew the noise from a seeded, non-cryptographic generator so t
```

The record's δ follows the book's default of $\delta \leq 1/n^{1.1}$ rounded down to a power of ten, 10^{-5} for Adult (Choosing ε). Its three values of ε come from the training script's cross-check. Opacus's privacy loss random variable (PRV) accountant gave the first. The privacy loss distribution (PLD) and looser Rényi differential privacy (RDP) accountants of Google's `dp_accounting` gave the others.

The tuning entry matters most to a reviewer. Every run that helps choose a hyperparameter looks at the data, and the final run's ε leaves those runs out. Here non-private runs chose the learning rate on 2,000 held-out Adult rows that play the part of public data. And $\varepsilon = 4$ was fixed before any private run. Those 2,000 people come from the same survey and get no guarantee, so the statement names them.

With that stated, the run's ε is also the ε of the whole procedure. If you tune on private data, report both, as in Hyperparameter tuning.

A record problem stops the release, such as a missing field, a sampler that is not Poisson, a δ above $1/n^{1.1}$ or unreported tuning. The one limitation here is `secure_mode`, left off so that the book's numbers repeat. The statement discloses it, and a model you ship is retrained with it on.

54.3. Recompute the guarantee

Next, recompute ε from σ, q and the number of steps alone, with a different library from the one that trained. A recomputation inside the training code would reuse any wrong number. The default is the PLD accountant of `dp_accounting`, which rounds the privacy loss up so that its ε is never too small. Read it at the record's δ and at one a hundred times smaller. Then read the

trade-off curve, the largest true positive rate (TPR) that any membership test can reach at each false positive rate (FPR) (Choosing ε).

```
pld = privacy_loss_distribution.from_gaussian_mechanism(sigma, sampling_prob=sample_rate).se
eps_pld = pld.get_epsilon_for_delta(delta)
eps_pld_small = pld.get_epsilon_for_delta(delta / 100)
eps_rdp = rdp_epsilon(sigma, sample_rate, steps, delta)
eps_rdp_small = rdp_epsilon(sigma, sample_rate, steps, delta / 100)
tpr = {fpr: tpr_bound(pld, fpr) for fpr in FPRS}  # largest TPR of any membership test
```

```
eps PLD at delta=1e-05: 3.981
eps RDP at delta=1e-05: 4.503
eps PLD at delta=1e-07: 5.191
eps RDP at delta=1e-07: 5.736
TPR bound at FPR 1e-03: 0.0123 (PLD curve)  0.0536 (pair eps, delta only)
TPR bound at FPR 1e-02: 0.0652 (PLD curve)  0.5355 (pair eps, delta only)
record's PRV eps: reported 3.99, recomputed 3.98: consistent
```

The recomputed PLD value, 3.981, agrees with the training run's PRV value, and RDP's 4.503 is higher because RDP is looser. Report the tight value with its accountant's name, and never set one accountant's ε against a budget computed by another. The curve gives a number a non-specialist can use. At an FPR of 10^{-3}, no membership test can reach a TPR above 0.0123, while the pair $(3.98, 10^{-5})$ on its own would allow 0.0536.

The script flags a reported ε more than 0.05 below the recomputation, a claim that is too good. The subtler failure is a recomputation that agrees on wrong inputs. For example, the code shuffles the data into fixed batches, but the accountant assumes Poisson sampling (Amplification by subsampling). No number shows that, so the record's sampler, read from the loader, is what catches it.

54.4. Attach the evidence

An accountant proves a theorem about an algorithm, not about the code that runs it. So the release also needs evidence about the code and the model. Attach each piece in Table 54.1, and say which attacker each one models. The curve covers the strongest attacker, who sees every intermediate model, while the audit and the attack cover only the attacks they run.

Table 54.1.: The evidence attached to a DP model release, and its limits.

Evidence	Source	On the Adult run	What it cannot show
Independent ε	This chapter's recomputation	3.981 against the reported 3.99	That the sampler matches the code

Evidence	Source	On the Adult run	What it cannot show
Clipping and noise test	Continuous integration (CI) test of Auditing private training	Passes on Opacus's optimizer; fails with the variance passed as the standard deviation, or with no noise	Sampling or accounting bugs
One-run audit	Auditing private training: the released pipeline retrained with 1,000 planted canaries	A lower bound on ε of 0.00 for DP-SGD and for the non-private baseline, against 0.53 for a memorizing control	That the model is private; a low bound proves nothing
Black-box attack	Likelihood ratio attack of Pre-release privacy risk assessment	Not run on this model; say so in the evidence	Whether a stronger attacker does better
Groups	Per-group accuracy with exact 95% intervals (Disparate impact)	Women 0.922 [0.915, 0.929], men 0.807 [0.800, 0.815]	Groups too small to measure

The audits can only refute a claim. A one-run bound above the claimed 4 would prove a bug. But 0.00 shows only that this attack found nothing, as it also found nothing in the non-private baseline. The audit also tests the released training code, not the released weights.

The group accuracies come from the 16,842 Adult test rows, which are private too. To publish them, add noise to each group's count of correct predictions, and charge it to its own budget (Release statistics with a library). Or measure them on a test set that is not private.

54.5. Write the statement

The statement goes in the model card's privacy section. The script's `format_statement` builds it from the record, so no number is typed by hand. The block below omits lines that repeat the record.

```
Guarantee: (3.98, 1e-05)-DP per person (one row of Adult), add/remove neighbours,
  for the whole training run on Adult (UCI 2), 30,000 training rows (n = 30,000).
  Also (5.19, 1e-07)-DP. Any membership test has TPR at most 0.0123 at FPR 1e-3.
Mechanism: DP-SGD, poisson sampling q = 0.0333, noise multiplier 0.9729,
  clipping norm 1, 300 steps (10 epochs).
Accounting: prv, Opacus 1.6.0; recomputed with dp_accounting PLD (RDP gives 4.50).
...
```

```
Noise generator: Opacus secure_mode=False, seed 0.
...
Not covered: data outside the training set, API outputs that use other private data, other r
```

Below the statement, add the evidence of Table 54.1, each item with the attacker it models. The last line deserves the most care. The guarantee covers the weights and anything computed from them alone, because post-processing cannot weaken DP. An example is the logits that the deployed application programming interface (API) returns. An output that also uses other private data, such as a retrieval index, is a new release. So is a second model trained on the same people.

54.6. Map the claim to regulation

This section is not legal advice, since every answer depends on facts beyond the model.

In the European Union, Recital 26 of the General Data Protection Regulation (GDPR) exempts anonymous information. It asks about the means of identification "reasonably likely to be used". Pseudonymised data, which only information kept separately links to a person, remain personal data under Article 4(5).

For AI models, Opinion 28/2024 of the European Data Protection Board (EDPB) sets a test. A model is anonymous only if the likelihood of extracting personal data about its training set is insignificant. Each claim is assessed case by case. The opinion names DP and attack tests, such as membership inference, as measures to weigh. It warns that a test shows resistance only to the attacks tested. The trade-off curve speaks to extraction as well as membership, because it caps reconstruction too (Choosing ε).

Table 54.2.: What DP evidence answers, and leaves open, for the legal questions a release meets.

Legal question	What DP evidence supports	What DP does not answer
Is the model anonymous (GDPR Recital 26; EDPB Opinion 28/2024)?	A bound on what any attacker can learn about one person's presence or record: the TPR cap and the audits	Whether the bound is "insignificant"; the training data stay personal data
Is this only pseudonymisation (GDPR Art. 4(5))?	DP relies on no separately kept key that could re-link the data to people	Removing identifiers before training is pseudonymisation at most
What goes in the risk assessment (GDPR Art. 35)?	The statement, the curve, the audits and the group table	Necessity and proportionality (Art. 35(7)(b)), and the lawful basis (Art. 6)
Is the risk of identification very small (HIPAA §164.514(b)(1))?	A quantitative bound that an expert can cite, with documented methods	Whether it is "very small"; the expert's determination itself

Legal question	What DP evidence supports	What DP does not answer
Is the information deidentified (California Consumer Privacy Act, Cal. Civ. Code §1798.140(m))?	A bound on how much one person's record can change what is inferred	Whether that meets the standard, since DP does not stop inferences from population patterns; the business's own duties
Does the data governance meet the EU AI Act (Art. 10)?	Documented data handling and per-group evaluation	Data quality, representativeness and bias, which DP can worsen

Training still processes personal data, so the GDPR applies to it, lawful basis included. Article 35 requires a data protection impact assessment (DPIA) before processing likely to result in a high risk. The DPIA is the natural home for the statement. In the United States, the safe harbor route of the Health Insurance Portability and Accountability Act (HIPAA) removes listed identifiers (§164.514(b)(2)), which DP does not do. Its expert determination route can cite DP evidence but not be replaced by it. California's law, as amended by the California Privacy Rights Act, adds duties against re-identification for the business.

None of these texts says that DP makes a model anonymous by itself. A DPIA entry built from this chapter's evidence can read as follows, with the bracketed parts filled in by the controller and counsel.

> The model was trained on personal data under [lawful basis, Art. 6]. Training used DP-SGD with a guarantee of $(3.98, 10^{-5})$-DP per person, add/remove neighbours, for the whole training run. No membership test against the released weights can exceed a TPR of 0.0123 at an FPR of 10^{-3}, and the same curve caps reconstruction. The code passed a CI test of its noise and clipping, and a one-run audit of the released pipeline found nothing, while a memorizing control certified 0.53. [A shadow-model attack was / was not run.] These tests show resistance only to the attacks run. DP does not remove identifiers and is not pseudonymisation. Whether the bound makes the model anonymous is for [counsel], and we make no claim of automatic anonymity. Not covered: the 2,000 stand-in rows, the test rows, outputs that use other private data, and later model versions.

54.7. Review someone else's claim

Run the same steps in reverse when someone else hands you a model. Ask for the record of Section 54.2, and refuse a claim that cannot be recomputed. Recompute ε from the claim's own parameters, and check its δ against the number of units. Report and review an ε claim gives the checks on the accountant and the sampler.

The script reviews a claim made up for this book, with every flaw a review should catch. It claims $\varepsilon = 1$ at $\delta = 10^{-3}$ for a model trained on 50,000 records, over 5,000 steps with noise multiplier 0.8. Its batches of 256 were shuffled and of fixed size but are accounted as Poisson sampling. And the claim states no unit, no clipping norm and no tuning.

```
def review(claim: dict[str, Any]) -> list[str]:
    """Recompute a claim from its own parameters and list what does not hold up."""
    findings = record_problems(claim)  # includes a sampler that is missing or not Poisson
    sigma, q, steps, delta = claim["sigma"], claim["sample_rate"], claim["steps"], claim["de
    pld = privacy_loss_distribution.from_gaussian_mechanism(sigma, sampling_prob=q).self_com
    recomputed = {"PLD": pld.get_epsilon_for_delta(delta)}
    if "rdp" in str(claim.get("accountant")).lower():  # also the accountant the claim names
        recomputed["RDP, the accountant named,"] = rdp_epsilon(sigma, q, steps, delta)
    for name, eps in recomputed.items():
        if eps > claim["epsilon"] + 0.05:
            findings.append(f"eps={claim['epsilon']} does not follow: {name} gives {eps:.2f}
    delta_ok = delta_for(claim["n"])
    findings.append(f"at delta={delta_ok:.0e} (1/n^1.1) PLD gives eps={pld.get_epsilon_for_d
    return findings
```

```
claim: Our model is trained with differential privacy, epsilon = 1.
findings: 10
  - missing field: unit
  - missing field: neighbours
  - missing field: max_grad_norm
  - missing field: secure_mode
  - sampler 'shuffled fixed-size batches' is not Poisson subsampling: Poisson-subsampled acc
  - delta=1e-03 is above 1/n^1.1 (use 1e-06)
  - tuning not reported: the cost of choosing hyperparameters is unknown
  - eps=1.0 does not follow: PLD gives 2.17 at its delta
  - eps=1.0 does not follow: RDP, the accountant named, gives 2.55 at its delta
  - at delta=1e-06 (1/n^1.1) PLD gives eps=3.75
```

By the vendor's own parameters the claimed 1 is 2.17, and the RDP accountant it names gives 2.55. At a δ the book accepts it is 3.75. Even that is too low, because shuffled fixed-size batches get no amplification by Poisson subsampling. The true ε can be far above all these numbers.

The script cannot see a wrong unit, since it takes n as given. If the records are messages and a customer sends many, screen the per-customer ε with User-level differential privacy yourself. Send the findings back with a request for a corrected record.

54.8. After release

The guarantee is fixed at release, but every retraining on the same people is a new release that composes with the old ones. For identical runs, compose the PLD of one run with itself once per version, `pld.self_compose(steps * versions)`. State the budget per model version and in total over a fixed **period**. If the total over a schedule is too large, calibrate each version to a share of it.

Deletion requests need their own answer, because DP deletes nothing. It says the model is about as likely as one trained without the person, but their row is still in the data. So honour a request

by removing the row, from the stand-in and test rows too, and retraining on a fixed schedule. Whether that is fast enough for the GDPR's Article 17 is a question for counsel.

Give each model version its own record and statement, and name in advance the incidents that reopen a release. A failing CI test, an audit bound above the claim, or a bug in the sampler or the accountant needs a corrected statement or a retrained model.

54.9. Checklist

Table 54.3.: The release checklist: each step, its default and its check.

Step	Default	Check
Run record	Unit, n, δ, neighbours, sampler read from the loader, σ, q, steps, clipping norm, accountant and version, `secure_mode`, seed, tuning, preprocessing, public data	`record problems: 0`; `secure_mode` on for the shipped model
δ	$\delta \leq 1/n^{1.1}$, rounded down to a power of ten	n counts units, not rows
Tuning	Choices on public data, or every private run counted	Run and procedure ε both reported
Recomputed ε	PLD in `dp_accounting`, at δ and $\delta/100$, and TPR from the curve	Reported value not below the recomputation
Audits	CI test; one-run audit of the released pipeline with a memorizing control; shadow-model attack	CI passes on the released commit; no bound above the claim; the control detects leakage; any attack not run is named
Groups	Per-group accuracy with exact intervals	Test set not private, or its release accounted
Statement	Generated from the record	Every number recomputable; "not covered" filled in
Regulation	DPIA entry mapping the evidence to each legal question	Reviewed by counsel; no claim of automatic anonymity
Third-party claim	Recompute from its own parameters	Unit, sampler, δ and tuning all stated
After release	Budget per period; retraining and deletion schedule; incident triggers	Versions composed; each has its own record

54.10. References

- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): what to ask of any DP claim.
- European Data Protection Board, *Opinion 28/2024 on certain data protection aspects related to the processing of personal data in the context of AI models* (December 2024): when a model can count as anonymous.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): its reporting section lists what a privacy statement contains.

Derivations. This workflow has no counterpart in the full edition.

A. Glossary

Acronyms used in the book and the pages that use them.

AI Artificial intelligence. Used in Release and governance.

API Application programming interface. Used in Threat models and privacy games, Pre-release privacy risk assessment, Release and governance.

ART Adversarial Robustness Toolbox. Used in Threat models and privacy games, Membership inference, Attribute inference and model inversion, Pre-release privacy risk assessment.

AUC Area under the curve. Used in Hypothesis testing, Membership inference, Pre-release privacy risk assessment.

CDF Cumulative distribution function. Used in The Laplace and Gaussian mechanisms.

CI Continuous integration. Used in Auditing private training, Release and governance.

CLT Central limit theorem. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy.

CPU Central processing unit. Used in Pre-release privacy risk assessment, Fine-tune a language model privately with LoRA.

DLG Deep Leakage from Gradients. Used in Gradient inversion.

DP Differential privacy. Used in Property inference, Gradient inversion, Pre-release privacy risk assessment, Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Query release and private multiplicative weights, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Private mean estimation, Release statistics with a library, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing ε, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Adaptive data analysis, Release and governance.

DPIA Data protection impact assessment. Used in Release and governance.

EMD Earth mover's distance. Used in Syntactic anonymity and how it breaks.

ERM Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Private empirical risk minimization, Public data and private fine-tuning.

FL Federated learning. Used in Gradient inversion.

FNR False negative rate. Used in Hypothesis testing.

FPR False positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing ε, Auditing private training, Release and governance.

FTRL Follow the regularized leader. Used in Choosing an accountant, DP-FTRL and matrix factorization.

GDP Gaussian differential privacy. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Choosing ε.

GDPR General Data Protection Regulation. Used in Linkage and re-identification, Release and governance.

GPU Graphics processing unit. Used in Train a model with DP-SGD, Fine-tune a language model privately with LoRA.

HIPAA Health Insurance Portability and Accountability Act. Used in Release and governance.

HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

HMAC Hash-based message authentication code. Used in Cryptography basics, Linkage and re-identification.

KL Kullback–Leibler. Used in Divergences.

LLM Large language model. Used in Private prediction: PATE and in-context learning.

LR Likelihood ratio. Used in Hypothesis testing.

ML Machine learning. Used in Membership inference, Choosing an accountant, Calibrate and account a DP-SGD run, DP-SGD, Train a model with DP-SGD, Choosing ε, Release and governance.

MWEM Multiplicative weights exponential mechanism. Used in Query release and private multiplicative weights.

PATE Private aggregation of teacher ensembles. Used in Beyond global sensitivity, Private prediction: PATE and in-context learning.

PII Personally identifiable information. Used in Memorization and training-data extraction.

PIN Personal identification number. Used in Information theory.

PLD Privacy loss distribution. Used in Advanced and optimal composition, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Choosing ε, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, Release and governance.

PRV Privacy loss random variable. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Fine-tune a language model privately with LoRA, Release and governance.

RDP Rényi differential privacy. Used in Advanced and optimal composition, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, Choosing ε, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Release and governance.

RMSE Root-mean-square error. Used in The Laplace and Gaussian mechanisms, Private mean estimation.

ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and privacy

games, Membership inference.

SGD Stochastic gradient descent. Used in Optimization, Algorithmic stability and generalization, Pre-release privacy risk assessment, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing ε, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Release and governance.

SQL Structured Query Language. Used in Release statistics with a library.

TPR True positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing ε, Auditing private training, Release and governance.

TV Total variation. Used in Divergences, Hypothesis testing, Syntactic anonymity and how it breaks.

XOR Exclusive or. Used in Cryptography basics.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification.

B. Index

AboveThreshold, 100, 104
adaptive composition, 139, 141, 176
advanced composition, 79, 97, 139–141
amplification by subsampling, 139, 159, 243
approximate differential privacy, 127
approximate zCDP, 178
attribute inference, 31, 44, 76, 212
auxiliary knowledge, 32

basic composition, 90, 97
binomial distribution, 8, 18, 206
black-box attack, 111, 254

canary, 33, 36, 51, 71, 76, 77, 229, 230, 246, 247, 254
central limit theorem, 141, 145, 168, 169
Chernoff bound, 8
clipping, 100, 111, 126, 127, 142, 188, 192, 200, 201, 208, 211, 230, 239–241, 243, 254, 256, 258
Clopper–Pearson interval, 8, 16, 18, 247
composition, 66, 79, 85, 90, 94, 97, 114, 139–141, 154, 159, 164, 170, 176, 184, 203, 205
composition attack, 66

deduplication, 220
Deep Leakage from Gradients, 56
differential privacy, 1, 31, 32, 51, 55, 79, 85, 90, 95, 96, 105, 110, 120, 127, 137, 139, 140, 145, 154, 164, 166, 170, 173, 176, 179, 184, 192, 201, 206, 211, 216, 221, 229, 230, 242, 247, 259
discrete Gaussian distribution, 115, 179
DP-SGD, 71, 76, 139–142, 145, 153, 156, 158, 159, 164, 184, 191, 192, 200–202, 206, 208, 210, 211, 216, 229, 246, 254, 256

empirical risk minimization, 184
entropy, 21, 56, 65
exponential mechanism, 100, 104, 108, 177, 206

f-DP, 140
frequency oracle, 120
fully adaptive composition, 176

Gaussian differential privacy, 139–141, 154, 168, 169, 171, 173, 216
Gaussian mechanism, 79, 96, 114, 184
generalization gap, 41
global sensitivity, 79, 114
gradient inversion, 31, 55, 76
group privacy, 79, 90, 94, 114, 164, 211

homomorphic encryption, 28
honest-but-curious, 32

imputation baseline, 44
individual privacy accounting, 176

k-anonymity, 65, 66

l-diversity, 65, 66
Laplace distribution, 3
Laplace mechanism, 95
likelihood ratio attack, 41, 71, 76, 77, 247, 254
Lipschitz, 22
loss attack, 41, 76, 77

membership inference, 31, 36, 41, 77, 184
meta-classifier, 71
min-entropy, 21
model inversion, 44, 71
moments accountant, 164, 192

Narayanan–Shmatikov attack, 61
neighbouring datasets, 79, 90, 91, 94, 139, 156, 159, 212

optimal composition, 97

parallel composition, 90
permutation invariance, 46
Poisson distribution, 141–143, 145, 153, 154, 156, 158, 159, 178, 188, 192, 200–203, 205, 206, 209, 229, 243
Poisson sampling, 141–143, 159, 188, 201, 202, 229, 243
post-processing, 79, 89, 90, 94, 114
privacy filter, 98
privacy game, 31, 36
privacy loss, 139–141
privacy loss distribution, 139–142, 144, 145, 153, 154, 156, 158, 168, 169, 201, 216, 229, 258
privacy loss random variable, 141
privacy profile, 96, 98
property inference, 31, 46, 76
PRV accountant, 170, 229
pure differential privacy, 84, 95, 141, 164

quantitative information flow, 21
quasi-identifier, 31, 61, 71, 76

randomized response, 120, 141, 161, 181
RDP curve, 161, 164, 166
re-identification, 77, 84
reconstruction attack, 31, 69
report-noisy-max, 100, 104
ROC curve, 16, 18, 36
Rényi differential privacy, 139, 140, 145, 154, 156, 158, 159, 161, 164, 166, 168, 169, 177, 191, 204
Rényi filter, 141, 176

sampling without replacement, 143, 145, 156, 158, 159, 192
secure aggregation, 56, 71
sensitivity, 79, 94, 95, 111, 114, 115, 141, 161, 164, 177
shadow model, 31, 41, 71, 247, 256, 258
shuffle model, 120, 141, 181
sparse vector technique, 94, 104, 105
stochastic gradient descent, 71, 76, 139–142, 145, 153, 156, 158, 159, 164, 184, 191, 192, 200–202, 206, 208, 210, 211, 216, 229, 239, 246, 254, 256
strong convexity, 22
subsample-and-aggregate, 111
subsampled Gaussian mechanism, 178

t-closeness, 66
threat model, 1, 31, 32, 76, 77
trade-off function, 3, 17, 212, 216

unary encoding, 120
unit of privacy, 79, 84, 94, 143, 208, 210, 211

variance, 3, 5, 7, 41, 126, 127, 243, 254

zero-concentrated differential privacy, 110, 136, 141, 177, 178, 216